

Membuat Corpus untuk RAG

Onno W. Purbo
onno@indo.net.id
onno@itts.ac.id
Twitter onnowpurbo

Institut Teknologi Tangerang Selatan (ITTS)
Diskominfo Tangerang Selatan
OnnoCenter
2026

Lisensi: Karya ini dilisensikan di bawah **Creative Commons Attribution (CC BY) 4.0 International**. Anda bebas menggunakan, membagikan, dan mengadaptasi karya ini untuk tujuan apa pun, termasuk komersial, dengan tetap mencantumkan atribusi yang layak kepada penulis.

Disclaimer: Buku ini disusun untuk tujuan edukasi, dokumentasi, dan refleksi publik berdasarkan sumber-sumber yang dianggap kredibel pada saat penulisan. Penulis telah berupaya menjaga akurasi, namun sangat mungkin masih terdapat kekurangan atau kekeliruan; karena itu, buku ini tidak dimaksudkan sebagai satu-satunya rujukan yang final, dan penulis terbuka terhadap koreksi serta masukan yang membangun

Kata Pengantar

Alhamdulillah, puji syukur kita panjatkan ke hadirat Allah SWT. Buku ini lahir dari satu gagasan yang sangat sederhana, tetapi sering dilupakan: dalam dunia RAG, **corpus sering jauh lebih penting daripada sekadar model besar**. Model yang sangat canggih tetap bisa memberi jawaban yang lemah jika dokumen yang dibacanya buruk, labelnya lemah, dan *chunking*-nya dilakukan tanpa disiplin. Sebaliknya, model yang lebih kecil sering jauh lebih berguna jika diberi pengetahuan yang bersih, rapi, dan siap dicari.

Karena itu, banyak sistem RAG gagal bukan karena idenya salah, melainkan karena fondasinya rapuh. Dokumen mentah langsung diunggah. Metadata kosong. Label tidak konsisten. Teks dipotong asal-asalan. Akibatnya, AI tampak pintar, tetapi jawabannya sering tidak benar-benar berpijak pada sumber. Buku ini ditulis untuk melawan kebiasaan itu. Fokusnya bukan mengejar model paling besar, tetapi membangun **corpus yang sehat, terstruktur, dan bisa dipercaya**.

Lebih dari itu, buku ini mengajak kita bergerak dari sekadar “**memakai AI**” menjadi “**menguasai pengetahuan sendiri**.” Dengan membangun corpus sendiri, kita tidak hanya menjadi pengguna teknologi, tetapi mulai memiliki kendali atas dokumen, retrieval, dan kualitas jawaban yang dihasilkan.

Semoga buku ini bermanfaat, mudah dipraktikkan, dan membantu pembaca membangun **RAG yang benar-benar bekerja**.

Jakarta, April 2026

Penulis

Executive Summary

Buku ini berangkat dari satu kenyataan yang sering diabaikan dalam euforia AI: **yang paling menentukan kualitas RAG bukan terutama model besar, tetapi kualitas corpus**. Banyak implementasi gagal bukan karena *LLM*-nya lemah, melainkan karena dokumen yang dimasukkan buruk, metadata kosong, label lemah, struktur kacau, dan *chunking* dilakukan tanpa disiplin. Akibatnya, sistem tampak pintar di permukaan, tetapi rapuh dalam praktik: jawaban meleset, sumber tidak jelas, konteks hilang, dan halusinasi muncul dengan wajah yang meyakinkan. Karena itu, buku ini memindahkan pusat perhatian dari “model apa yang dipakai” ke pertanyaan yang jauh lebih strategis: **bagaimana membangun pengetahuan yang siap dicari, siap diuji, dan siap dipakai dengan benar**.

Secara praktis, buku ini memandu pembaca membangun fondasi RAG yang sehat dengan pendekatan *hands-on*: menyiapkan ekosistem **Ubuntu 26.04 + Ollama + Open WebUI**, menulis dokumen manual yang ramah retrieval, mengubah textbook PDF menjadi dokumen kerja yang bersih, memberi metadata dan label yang konsisten, membangun corpus untuk tiga dunia berbeda—*knowledge/text*, log server, dan programming—lalu menentukan strategi *chunking*, *embedding*, *vector database*, dan evaluasi retrieval yang tepat. Pembaca tidak hanya diajak “menggunakan AI”, tetapi dilatih untuk **menguasai pengetahuan sendiri**: memahami sumber, mengatur struktur, mengendalikan retrieval, dan menilai kualitas jawaban dengan bukti. Di sinilah nilai strategis buku ini: ia mengubah RAG dari sekadar demonstrasi teknologi menjadi **infrastruktur pengetahuan yang bisa dipercaya**.

Buku ini juga menegaskan perbedaan penting antara masalah pengetahuan dan masalah perilaku model. Bila persoalannya adalah fakta, kutipan sumber, data yang cepat berubah, dan kebutuhan pembaruan murah, maka **RAG sering sudah cukup**. Bila persoalannya adalah format keluaran, gaya yang harus konsisten, atau *reasoning* domain yang sangat khas, barulah PEFT seperti **LoRA**, **QLoRA**, atau **DoRA** layak dipertimbangkan. Dengan cara pandang ini, pembaca diajak mengambil keputusan arsitektur secara matang, bukan ikut tren. Pesan akhirnya tegas: **bangsa atau organisasi yang ingin berdaulat di era AI tidak cukup hanya memakai model orang lain; ia harus mampu membangun, merawat, dan menguasai corpus pengetahuannya sendiri**. Di situlah RAG yang sesungguhnya lahir—bukan dari kemewahan model, tetapi dari disiplin pengetahuan.

Daftar Isi

Kata Pengantar.....	3
Executive Summary.....	4
Daftar Isi.....	5
Cara Menggunakan Buku Ini.....	16
Bab 1. Corpus adalah Fondasi RAG, Bukan Sekadar “Upload File”	18
1.1 Mengapa banyak RAG gagal walau model tampak pintar.....	18
1.2 Bedakan: raw document, clean document, corpus, embedding, index, retrieval.....	19
1.3 Kritik terhadap pendekatan “unggah semua file lalu berharap AI paham”.....	20
1.4 Mengapa Open WebUI RAG tetap bergantung pada mutu dokumen.....	21
1.5 Kapan RAG cukup, kapan mulai perlu FT/PEFT.....	22
1.6 Peta besar buku: dari dokumen mentah menjadi knowledge base yang bisa diuji....	23
Penutup Bab.....	24
Bab 2. Menyiapkan Ekosistem Ubuntu 26.04 + Ollama + Open WebUI.....	25
2.1 Catatan status Ubuntu 26.04 dan disiplin version pinning.....	25
2.2 Instalasi dasar host: update sistem, Python, uv/pip, venv, git, build tools.....	26
Langkah 1. Update sistem.....	26
Langkah 2. Pasang paket dasar host.....	27
Langkah 3. Verifikasi Python.....	27
Langkah 4. Pasang uv untuk manajemen Python yang lebih cepat.....	27
Langkah 5. Buat virtual environment.....	28
2.3 Instalasi Ollama di Linux.....	28
Opsi A. Cara cepat.....	29
Opsi B. Cara disiplin dengan version pinning.....	29
Menjalankan Ollama sebagai service.....	29
Uji tarik model.....	30
2.4 Instalasi Open WebUI dengan Docker / Docker Compose.....	31
Langkah 1. Pasang Docker Engine resmi.....	31
Langkah 2. Tambahkan user ke grup docker.....	32
Langkah 3. Siapkan docker-compose.yml.....	32
Langkah 4. Jalankan container.....	33
2.5 Menghubungkan Open WebUI ke Ollama.....	33
Uji konektivitas dari host.....	33
Uji konektivitas dari container.....	33
2.6 Struktur folder proyek corpus yang rapi.....	34
How-to cepat membuat struktur.....	35
2.7 Sanity check: model jalan, UI hidup, dokumen bisa diunggah.....	36
Cek 1. Ollama aktif.....	36
Cek 2. Open WebUI aktif.....	36
Cek 3. Model tampil di Open WebUI.....	36
Cek 4. Dokumen bisa diunggah ke Knowledge.....	36
Cek 5. Tanya pertanyaan sederhana.....	37
2.8 Troubleshooting awal: izin file, memori, embedding, reindex.....	37

Masalah 1. Docker hidup, tetapi Open WebUI tidak bisa simpan data.....	37
Masalah 2. Open WebUI hidup, tetapi model Ollama tidak muncul.....	37
Masalah 3. Memori habis.....	38
Masalah 4. Hasil retrieval jelek setelah ganti model embedding.....	38
Masalah 5. Salah mode retrieval.....	38
Masalah 6. Tag main berubah perilaku.....	38
How-to ringkas: dari nol sampai siap dipakai.....	39
Penutup Bab.....	39
Bab 3. Memilih Model Embedding dan Vector Database yang Waras.....	40
3.1 Peran embedding model dalam kualitas retrieval.....	40
How-to cepat menguji kualitas embedding.....	41
3.2 Dense, sparse, dan hybrid retrieval secara praktis.....	41
Contoh kasus sederhana.....	42
How-to berpikir praktis.....	42
3.3 Kriteria memilih model embedding.....	43
a. Multilingual.....	43
b. Code-aware.....	43
c. Dokumen teknis.....	43
d. Kecepatan.....	44
e. Memori.....	44
f. Lisensi.....	44
How-to uji model embedding secara sederhana.....	44
3.4 Kriteria memilih vector database.....	45
Chroma untuk mulai cepat.....	45
LanceDB untuk lokal tertanam dan hybrid/full-text.....	45
Qdrant untuk filtering dan arsitektur client-server.....	46
Ringkasan keputusan praktis.....	46
How-to memilih dengan kepala dingin.....	46
Open-WebUI default vector database.....	47
Daftar Vector Database yang Didukung.....	47
Cara Mengganti Vector Database.....	48
3.5 Metadata wajib yang harus ikut tersimpan.....	48
Contoh metadata dalam YAML.....	49
Golden Rule.....	49
3.6 Kapan perlu reranker.....	49
Kapan reranker layak dipakai?.....	49
Logika yang sehat.....	50
How-to berpikir sederhana.....	50
3.7 Dampak pergantian model embedding: kapan wajib reindex.....	50
Kapan wajib reindex?.....	50
Kapan mungkin tidak wajib penuh, tetapi tetap perlu ditinjau?.....	50
Gejala Anda butuh reindex.....	50
How-to praktis.....	51
3.8 Checklist keputusan sebelum ingest besar-besaran.....	51

How-to mini sebelum ingest.....	52
Penutup Bab.....	52
Bab 4. Teknik Membuat DOC untuk RAG Manual, Pakai Tangan, Tanpa Aplikasi.....	53
4.1 Mengapa menulis manual tetap penting di era AI.....	53
4.2 Format paling aman: Markdown + YAML front matter.....	54
How-to membuat dokumen Markdown + YAML dari nol.....	55
4.3 Golden Rule dokumen RAG-ready.....	56
Satu topik utama per bagian.....	56
Judul jelas.....	57
Paragraf pendek.....	57
Daftar tertib.....	57
Tabel seperlunya.....	57
Code block utuh.....	58
How-to memeriksa metadata secara cepat.....	59
4.5 Template dokumen manual.....	60
a. Knowledge note.....	60
b. SOP.....	61
c. FAQ.....	61
d. Catatan teknis.....	62
e. Tutorial singkat.....	62
4.6 Contoh dokumen buruk vs dokumen siap RAG.....	62
Dokumen buruk.....	62
Dokumen siap RAG.....	63
4.7 Teknik memberi penanda penting langsung di dokumen.....	64
Penutup Bab.....	67
Bab 5. Mengubah Textbook PDF Menjadi DOC untuk RAG.....	68
5.1 Bedakan PDF teks, PDF scan, dan PDF campuran.....	68
How-to mengenali PDF secara cepat.....	69
5.2 Alur kerja: PDF → Markdown / JSON / TXT → DOC bersih.....	69
How-to memilih format output.....	70
5.3 Ekstraksi dengan PyMuPDF4LLM.....	70
Instalasi dasar.....	70
Ekstraksi sederhana ke Markdown.....	71
Ekstraksi per halaman atau page chunks.....	71
How-to kerja yang disarankan.....	72
5.4 Membersihkan hasil ekstraksi.....	72
Header/footer.....	72
Nomor halaman.....	72
Catatan kaki.....	72
Tabel rusak.....	72
Pecahan paragraf.....	73
Caption gambar.....	73
How-to membersihkan cepat dengan skrip sederhana.....	73
5.5 Menjaga struktur bab, subbab, referensi, dan daftar isi.....	74

How-to memulihkan heading.....	74
5.6 Memecah textbook menjadi dokumen yang sehat untuk retrieval.....	75
How-to memecah berdasarkan heading utama.....	75
5.7 Validasi manual sebelum masuk Open WebUI.....	76
How-to validasi cepat.....	76
5.8 Studi kasus: satu bab buku menjadi corpus siap pakai.....	77
Tahap 1. Ekstrak PDF ke Markdown.....	77
Tahap 2. Bersihkan hasil ekstraksi.....	77
Tahap 3. Ambil hanya bab yang dibutuhkan.....	77
Tahap 4. Tambahkan metadata YAML.....	77
Tahap 5. Rapikan isi.....	78
Tahap 6. Simpan ke folder corpus bersih.....	78
Tahap 7. Uji retrieval kecil.....	78
Penutup Bab.....	78
Bab 6. Teknik Melabeli Informasi / Knowledge di Dalam DOC.....	80
6.1 Mengapa label sering lebih berharga daripada model yang lebih besar.....	80
6.2 Bedakan: metadata, tag, label, kategori, skema domain.....	81
Contoh sederhana.....	82
6.3 Label minimum yang saya rekomendasikan.....	83
Template label minimum.....	84
6.4 Teknik manual labeling yang konsisten.....	85
How-to membuat kamus label.....	85
Step by step labeling manual.....	86
Validasi cepat dengan CLI.....	86
6.5 Menambah importance, authority, dan freshness.....	86
Contoh metadata dengan skor tambahan.....	87
Catatan penting.....	87
6.6 Versioning dan jejak revisi.....	88
Yang minimal harus di-version.....	88
Cara praktis dengan Git.....	88
6.7 Anti-kekacauan: label terlalu banyak vs label terlalu miskin.....	89
Tanda Anda terlalu banyak label.....	89
Tanda Anda terlalu miskin label.....	89
6.8 Checklist mutu sebelum di-embed.....	90
How-to cek cepat dengan skrip kecil.....	90
Penutup Bab.....	91
Bab 7. Membangun Corpus untuk Tiga Dunia: Knowledge/Text, Log Server, dan Programming.....	92
7.1 Corpus untuk knowledge / text.....	92
How-to: membangun corpus knowledge/text dari nol.....	93
7.2 Corpus untuk log server.....	94
How-to: langkah awal corpus log.....	95
7.3 Teknik melabel attack / non-attack / suspicious / unknown.....	96
How-to: aturan labeling sederhana.....	97

7.4 Medan log yang penting.....	98
Contoh format log yang sehat untuk corpus.....	98
How-to: normalisasi field log dengan Python.....	99
7.5 Corpus untuk programming.....	100
Mengapa programming corpus berbeda.....	100
How-to: struktur corpus programming.....	100
7.6 Apa yang berbeda antar-domain.....	102
7.7 Latihan: bangun 3 corpus berbeda dari nol.....	103
Latihan 1. Corpus knowledge.....	103
Latihan 2. Corpus log.....	103
Latihan 3. Corpus programming.....	104
How-to membuat kerangka latihan.....	104
Uji kecil setelah corpus jadi.....	104
Penutup Bab.....	105
Bab 8. Chunking Strategies: Recursive, Semantic, dan Hierarchical.....	106
8.1 Mengapa chunking adalah jantung retrieval.....	106
8.2 Recursive chunking: kapan dipakai, ukuran awal, overlap, jebakan.....	107
Kapan dipakai.....	107
Ukuran awal.....	107
Overlap.....	108
Jebakan.....	108
How-to sederhana.....	108
8.3 Semantic chunking: pecah berdasarkan perubahan makna.....	109
Kapan dipakai.....	109
Kapan lebih unggul dari fixed chunk.....	109
Biaya dan kompleksitas.....	109
How-to mental model.....	110
CLI contoh untuk eksperimen.....	110
8.4 Hierarchical chunking: parent-child, bab-subbab-paragraf.....	110
Parent-child.....	110
Kapan dipakai.....	111
Kelebihan.....	111
Kekurangan.....	111
8.5 Chunking untuk textbook.....	111
Strategi sehat.....	112
How-to praktis.....	112
8.6 Chunking untuk log.....	112
Kapan satu event per chunk.....	112
Kapan jendela waktu lebih tepat.....	112
Risiko terbesar.....	113
8.7 Chunking untuk code dan API docs.....	113
Untuk source code.....	113
Untuk README.....	113
Untuk API docs.....	114

Risiko retrieval.....	114
8.8 Kapan perlu context enrichment, parent document retrieval, dan sentence window.....	114
Sentence window retrieval.....	114
Parent document retriever.....	114
Context enrichment.....	115
Kapan perlu.....	115
Kapan belum perlu.....	115
8.9 Mengukur hasil chunking: relevansi, fragmentasi, konteks hilang, duplikasi.....	115
Relevansi.....	115
Fragmentasi.....	115
Konteks hilang.....	115
Duplikasi.....	116
How-to uji cepat.....	116
8.10 Template keputusan pemilihan strategi.....	117
Gunakan recursive chunking bila:.....	117
Gunakan semantic chunking bila:.....	117
Gunakan hierarchical chunking bila:.....	117
Tambahkan sentence window bila:.....	117
Tambahkan parent document retrieval bila:.....	117
Pertimbangkan reranking bila:.....	117
Checklist keputusan cepat.....	118
Penutup Bab.....	118
Bab 9. Menyiapkan RAG di Open WebUI: Ingest → Embed → Retrieve → Evaluate...	119
9.1 Workspace, Knowledge, dan cara berpikir Open WebUI.....	119
How-to: memeriksa bahwa Open WebUI siap dipakai.....	120
9.2 Membuat knowledge base dari corpus yang sudah dibersihkan.....	120
Langkah praktis.....	120
Contoh struktur sebelum ingest.....	121
How-to: langkah kerja sederhana.....	121
Checklist awal.....	121
9.3 Kapan memakai Focused Retrieval, kapan Full Context.....	122
Pelajaran penting.....	123
Cara memilih cepat.....	123
9.4 Pengaturan dokumen, embedding, dan reindex.....	123
Langkah kerja yang sehat.....	123
How-to: kebiasaan administrasi minimum.....	123
Cek log bila ada masalah.....	124
9.5 Strategi query yang sehat.....	124
How-to: siapkan file query uji.....	125
9.6 Hybrid retrieval, metadata filtering, dan reranking.....	126
9.7 Evaluasi minimum: recall@k, MRR, groundedness, ketepatan jawaban, jejak sumber	127
Cara berpikir minimum.....	127

9.8 Prompt evaluasi retrieval dan audit bukti.....	128
Contoh prompt audit bukti.....	128
Contoh prompt evaluasi groundedness.....	128
9.9 Kesalahan umum Open WebUI RAG dan cara menghindarinya.....	129
How-to: langkah cepat saat hasil terasa jelek.....	129
9.10 Mini project: satu knowledge base yang benar-benar berguna.....	130
Tema proyek.....	130
Langkah kerja.....	130
Struktur sederhana.....	130
Jalankan evaluasi retrieval lokal.....	131
Uji di Open WebUI.....	131
Penutup Bab.....	131
Bab 10. Hybrid RAG + FT: Kapan RAG Cukup, Kapan PEFT Wajib.....	132
10.1 Empat keputusan arsitektur.....	132
Cara berpikir cepat.....	132
10.2 Kapan RAG cukup.....	133
Contoh kasus RAG sudah cukup.....	134
How-to keputusan cepat.....	134
10.3 Kapan PEFT wajib mulai dipertimbangkan.....	134
10.4 LoRA, QLoRA, DoRA dalam bahasa praktis.....	135
Ringkas dalam bahasa lapangan.....	136
How-to mental model.....	136
10.5 Menyiapkan dataset training dari corpus.....	136
Bentuk dataset yang masuk akal.....	136
How-to membangun dataset awal.....	137
CLI contoh membuat dataset ringkas.....	137
10.6 Kapan fine-tune embedder lebih masuk akal daripada fine-tune LLM.....	138
Ringkasan sederhana.....	138
How-to diagnosis praktis.....	138
10.7 Teknik synthetic data: kapan berguna, kapan berbahaya.....	139
Kapan berguna.....	139
Kapan berbahaya.....	139
Aturan sehat.....	139
How-to membuat sintetik secara hati-hati.....	139
10.8 Penutup: corpus sebagai infrastruktur pengetahuan jangka panjang.....	140
Ringkasan keputusan praktis.....	140
CLI penutup yang sehat.....	141
Lampiran A. Checklist Instalasi Cepat Ubuntu 26.04 + Ollama + Open WebUI.....	142
A.1 Prinsip sebelum mulai.....	142
A.2 Checklist cepat.....	142
1. Verifikasi sistem operasi.....	142
2. Update sistem.....	143
3. Pasang paket dasar host.....	143
4. Verifikasi Python.....	143

5. Instal Docker Engine + Compose plugin.....	144
6. Tambahkan user ke grup docker.....	145
7. Instal Ollama di Linux.....	145
8. Aktifkan Ollama sebagai service.....	146
9. Bila perlu, set OLLAMA_HOST.....	147
10. Tarik satu model uji.....	147
11. Siapkan folder proyek.....	148
12. Buat docker-compose.yml untuk Open WebUI.....	148
13. Jalankan Open WebUI.....	149
14. Cek UI Open WebUI.....	149
15. Uji koneksi Open WebUI ke Ollama.....	149
16. Uji knowledge pertama.....	150
A.3 Checklist sanity check akhir.....	150
A.4 Perintah cek cepat yang paling berguna.....	151
A.5 Masalah umum dan solusi cepat.....	151
A.6 Catatan penutup.....	152
Lampiran B. Template DOC Markdown + YAML untuk RAG.....	153
B.1 Aturan minimum template.....	153
B.2 Template umum paling aman.....	153
B.3 Arti field YAML yang paling penting.....	155
B.4 Template untuk knowledge note.....	156
B.5 Template untuk SOP.....	157
B.6 Template untuk FAQ.....	158
B.7 Template untuk catatan teknis.....	159
B.8 Template untuk tutorial singkat.....	160
B.9 Template untuk dokumen log yang sudah diperkaya.....	161
B.10 Template untuk dokumen programming.....	162
B.11 Aturan isi dokumen yang disarankan.....	163
B.12 Howto membuat dokumen baru dengan cepat.....	163
B.13 Cek metadata minimum dengan CLI.....	164
B.14 Template ringkas paling minimum.....	165
Penutup Lampiran.....	165
Lampiran C. Template Labeling Log ATTACK / NON-ATTACK / SUSPICIOUS / UNKNOWN.....	166
C.1 Definisi empat label utama.....	166
ATTACK.....	166
NON-ATTACK.....	167
SUSPICIOUS.....	167
UNKNOWN.....	167
C.2 Template skema label minimum.....	168
C.3 Field minimum yang saya rekomendasikan.....	168
C.4 Template JSON paling dasar.....	169
C.5 Template YAML untuk anotasi manual.....	169
C.6 Template CSV untuk pekerjaan cepat.....	170

C.7 Contoh kasus dan label yang disarankan.....	170
Kasus 1. Login gagal satu kali.....	170
Kasus 2. Banyak login gagal beruntun dari IP sama.....	170
Kasus 3. Health check internal.....	170
Kasus 4. Akses ke endpoint sensitif.....	170
Kasus 5. Log tidak lengkap.....	171
C.8 Aturan keputusan cepat.....	171
C.9 Template keputusan manual per event.....	172
C.10 Howto kerja labeling log yang sehat.....	173
Langkah 1. Simpan log mentah.....	173
Langkah 2. Ambil contoh event kecil.....	173
Langkah 3. Parse field penting.....	173
Langkah 4. Beri label manual.....	173
Langkah 5. Tambahkan confidence.....	173
Langkah 6. Simpan hasil ke file baru.....	174
Langkah 7. Cek hasil.....	174
C.11 Template batch labeling sederhana.....	175
C.12 Checklist mutu sebelum log di-embed.....	176
C.13 Anti-kesalahan umum.....	177
C.14 Template ringkas paling minimum.....	177
Penutup Lampiran.....	177
Lampiran D. Contoh alur PDF → Markdown → cleaning → ingest.....	178
D.1 Struktur folder kerja.....	178
D.2 Letakkan PDF mentah.....	179
D.3 Instal library ekstraksi.....	179
D.4 Tahap 1 — PDF → Markdown.....	180
D.5 Tahap 2 — inspeksi awal hasil ekstraksi.....	180
D.6 Tahap 3 — cleaning dasar.....	181
D.7 Tahap 4 — inspeksi hasil bersih.....	182
D.8 Tahap 5 — pecah per bab atau per subbab.....	183
D.9 Tahap 6 — tambah YAML metadata.....	184
D.10 Tahap 7 — validasi cepat sebelum ingest.....	185
D.11 Tahap 8 — salin ke folder corpus bersih.....	186
D.12 Tahap 9 — ingest pilot ke vector store.....	186
D.13 Tahap 10 — evaluasi retrieval cepat.....	187
D.14 Tahap 11 — unggah ke Open WebUI.....	188
D.15 Alur ringkas satu halaman.....	188
D.16 Contoh file akhir yang siap masuk corpus.....	189
D.17 Kesalahan umum yang harus dihindari.....	189
Penutup Lampiran.....	190
Lampiran E. Checklist Evaluasi Kualitas Corpus.....	191
E.1 Prinsip dasar evaluasi corpus.....	191
E.2 Checklist tingkat 1 — kualitas dokumen sumber.....	191
A. Struktur dokumen.....	191

B. Kebersihan isi.....	192
C. Kelayakan isi.....	192
Cek cepat dengan CLI.....	192
E.3 Checklist tingkat 2 — kualitas metadata.....	193
Pertanyaan evaluasi metadata.....	193
Cek cepat dengan CLI.....	193
E.4 Checklist tingkat 3 — kualitas label.....	194
Untuk corpus umum.....	194
Untuk corpus log.....	194
Pertanyaan evaluasi label.....	194
Cek cepat nilai label log.....	194
E.5 Checklist tingkat 4 — kualitas chunking.....	195
Checklist chunking.....	195
Tanda chunking buruk.....	195
Uji cepat dengan script yang sudah dibuat.....	195
E.6 Checklist tingkat 5 — kualitas retrieval.....	196
Metrik minimum yang disarankan.....	196
Pertanyaan evaluasi retrieval.....	196
Cek retrieval minimum.....	196
E.7 Checklist kualitas per domain.....	197
A. Corpus knowledge/text.....	197
B. Corpus log.....	197
C. Corpus programming.....	197
E.8 Tabel penilaian cepat corpus.....	198
E.9 Lembar audit mini sebelum ingest.....	199
E.10 Howto audit corpus dari nol.....	200
Langkah 1. Audit dokumen.....	200
Langkah 2. Audit metadata.....	200
Langkah 3. Audit label.....	200
Langkah 4. Audit chunking.....	200
Langkah 5. Audit retrieval.....	200
E.11 CLI audit cepat yang sangat berguna.....	201
E.12 Tanda corpus sudah cukup sehat.....	201
Tanda corpus belum sehat.....	201
E.13 Golden Rule.....	202
Penutup Lampiran.....	202
Lampiran F. Source Code ingest_pilot.py.....	203
Source Code ingest_pilot.py.....	203
Dependensi yang dibutuhkan.....	215
Contoh struktur file input.....	216
Contoh menjalankan.....	216
Lampiran G. Source Code eval_retrieval.py.....	217
Contoh file query uji.....	217
Source code eval_retrieval.py.....	218

Dependensi.....	225
Contoh menjalankan.....	226
Evaluasi LanceDB.....	226
Evaluasi Chroma.....	226
Contoh output.....	226
Pasangan workflow yang sehat.....	227
Lampiran H. Source Code clean_markdown.py.....	228
Contoh pakai.....	228
Bersihkan satu file.....	228
Mode check saja, tanpa menulis file.....	228
Bersihkan semua file .md dalam satu folder.....	228
Source code clean_markdown.py.....	229
Apa yang dibersihkan oleh script ini.....	236
Contoh 1: check saja.....	237
Contoh 2: bersihkan satu file.....	237
Contoh 3: bersihkan satu folder.....	237
Catatan penggunaan yang sehat.....	238
Tambahan kecil yang bisa diajarkan di buku.....	238
Lampiran I. test_embedding_retrieval.py.....	239
Fungsi script.....	239
Format file query uji.....	240
Source code test_embedding_retrieval.py.....	240
Dependensi.....	252
Contoh menjalankan.....	252
Uji dua model.....	252
Uji tiga model dan simpan laporan JSON.....	252
Contoh output.....	253
Cara membaca hasilnya.....	254
Urutan kerja yang sehat.....	254
Lampiran J. compare_chunking.py.....	255
Contoh file query uji.....	255
Source code compare_chunking.py.....	256
Dependensi.....	269
Contoh menjalankan.....	269
Bandingkan tiga ukuran chunk dengan mode default heading.....	269
Bandingkan mode heading dan paragraph.....	269
Simpan hasil JSON.....	270
Contoh output.....	270
Cara membaca hasilnya.....	271
Aturan praktis yang bisa diajarkan di buku.....	271
Workflow sehat setelah script ini.....	272
Glossary.....	273
Referensi.....	278
Tentang Penulis.....	281

Cara Menggunakan Buku Ini

Buku ini dirancang agar pembaca tidak hanya memahami konsep, tetapi **langsung bisa membangun corpus yang sehat untuk RAG**. Karena itu, cara terbaik memakai buku ini bukan membacanya seperti teori dari awal sampai akhir tanpa praktik, melainkan menjadikannya **panduan kerja**. Setiap bab membangun fondasi untuk bab berikutnya. Jika urutannya diikuti dengan benar, pembaca akan bergerak dari dokumen mentah menuju *knowledge base* yang benar-benar bisa diuji dan dipakai.

Jalur baca tercepat yang saya sarankan adalah:

DOC manual → PDF → labeling → chunking → Open WebUI → evaluasi → keputusan RAG vs PEFT

Artinya, mulailah dari kemampuan paling dasar tetapi paling penting: **menulis dan merapikan dokumen sendiri**. Setelah itu, pelajari cara mengubah PDF menjadi dokumen kerja yang bersih. Berikutnya, beri label dan metadata yang konsisten. Sesudah itu, pelajari *chunking* agar dokumen dipotong dalam unit makna yang sehat. Barulah setelah fondasi itu kuat, masuk ke **Open WebUI** untuk membangun *knowledge base*, menguji retrieval, lalu mengevaluasi hasilnya. Di tahap paling akhir, pembaca baru diajak mengambil keputusan yang lebih strategis: **apakah RAG sudah cukup, atau kebutuhan mulai mengarah ke PEFT** seperti LoRA, QLoRA, atau DoRA.

Untuk pembaca yang ingin **langsung praktik**, saya sarankan urutan berikut:

- baca **Bab 1–4** untuk memahami fondasi dokumen dan corpus;
- lanjut ke **Bab 5–8** untuk PDF, labeling, tiga jenis corpus, dan *chunking*;
- masuk ke **Bab 9** untuk implementasi di Open WebUI;
- tutup dengan **Bab 10** untuk keputusan arsitektur RAG vs PEFT.

Untuk pembaca yang sudah teknis dan ingin cepat sampai ke sistem berjalan, Anda bisa mulai dari:

- **Bab 2** untuk ekosistem Ubuntu 26.04 + Ollama + Open WebUI,
- lalu **Bab 4–9** untuk membangun corpus dan menguji retrieval,
- dan kembali ke **Bab 1** bila ingin memperkuat cara berpikir konseptualnya.

Buku ini juga menyediakan banyak **contoh CLI, template dokumen, dan latihan**. Gunakan semuanya secara aktif. Jangan hanya dibaca. Jalankan perintah CLI di mesin Anda, salin template Markdown + YAML, isi dengan dokumen Anda sendiri, lalu kerjakan latihan-latihannya. Di situlah nilai buku ini muncul. Contoh CLI membantu Anda melihat alur kerja nyata. Template dokumen membantu Anda menulis corpus yang konsisten. Latihan membantu Anda membuktikan apakah retrieval benar-benar sehat, bukan sekadar terasa pintar.

Cara paling efektif memakai buku ini adalah:

- jalankan contoh CLI sambil membaca
- modifikasi template dengan kasus Anda sendiri
- kerjakan latihan kecil sebelum pindah ke bab berikutnya
- uji retrieval dengan query nyata, bukan query buatan
- catat apa yang berhasil dan apa yang gagal

Kalau buku ini dipakai dengan cara itu, maka pembaca tidak hanya selesai membaca. Pembaca akan sampai pada sesuatu yang jauh lebih berharga: **punya corpus sendiri, punya alur kerja sendiri, dan mulai menguasai pengetahuan sendiri di era AI.**

Bab 1. Corpus adalah Fondasi RAG, Bukan Sekadar “Upload File”

Banyak orang masuk ke dunia *Retrieval-Augmented Generation* dengan harapan sederhana: unggah PDF, sambungkan model, lalu AI akan otomatis menjadi pintar, akurat, dan tahu konteks. Harapan ini terdengar masuk akal, tetapi di lapangan sering berakhir mengecewakan. Model memang terlihat fasih, jawabannya rapi, kalimatnya meyakinkan, tetapi sumbernya salah, konteksnya meleset, potongan dokumen yang diambil tidak relevan, atau lebih parah lagi: AI menjawab dengan percaya diri padahal tidak benar. Di sinilah letak pelajaran pertama buku ini: **RAG bukan sulap**, dan **corpus bukan tumpukan file**. RAG hanya akan sebaik bahan yang kita siapkan untuknya. Kalau dokumen berantakan, metadata kosong, struktur kacau, *chunking* serampangan, dan retrieval tidak diuji, maka model paling bagus pun hanya akan menebak dengan gaya yang lebih meyakinkan. Itulah sebabnya buku ini tidak dimulai dari model, tidak dimulai dari GPU, dan tidak dimulai dari prompt, melainkan dari **corpus**.

Secara konseptual, RAG hadir untuk mengatasi kelemahan dasar *large language model*: model dapat menghasilkan bahasa yang lancar, tetapi tidak otomatis memiliki akses ke fakta terbaru, dokumen internal, SOP lokal, log server, atau basis pengetahuan yang sangat spesifik. Karena itu, RAG menambahkan mekanisme **retrieval**: sistem mencari potongan informasi yang relevan dari sumber eksternal, lalu memasukkannya ke konteks model sebelum jawaban dibangkitkan. Dalam bahasa sederhana, model tidak hanya “mengingat”, tetapi juga “mencari sebelum menjawab”. Literatur yang Anda lampirkan konsisten menekankan bahwa RAG menjadi penting justru karena ia memberi **factual grounding**, jejak sumber, kemampuan pembaruan dinamis, dan transparansi yang lebih baik dibanding sekadar mengandalkan memori parametris model.

1.1 Mengapa banyak RAG gagal walau model tampak pintar

Kegagalan RAG yang paling sering bukan karena model terlalu kecil. Bukan pula karena *embedding model* salah pilih sejak awal. Akar masalahnya jauh lebih mendasar: **bahan masukannya buruk**. Dokumen yang diunggah sering masih mentah. PDF hasil scan bercampur dengan PDF teks. Header, footer, nomor halaman, catatan kaki, dan tabel rusak ikut terbawa. Satu file berisi terlalu banyak topik. Nama dokumen ambigu. Metadata kosong. Paragraf terlalu panjang. Potongan kode tercampur dengan narasi. Log server tidak dinormalisasi. Lalu semua itu dilempar ke sistem dan diharapkan menjadi “knowledge base”. Hasilnya bisa ditebak: retrieval menjadi lemah, potongan yang diambil tidak tepat, dan model membangun jawaban di atas konteks yang buruk. **Masalahnya bukan AI tidak cerdas; masalahnya kita memberi makanan yang buruk.**

Di banyak implementasi, orang juga salah kaprah dengan mengira bahwa retrieval otomatis berarti kebenaran otomatis. Padahal retrieval hanya tahap mencari kandidat konteks. Ia belum menjamin bahwa konteks yang diambil adalah yang terbaik. Di sinilah *chunking*, metadata, *reranking*, dan evaluasi menjadi penentu. Literatur pendukung yang Anda lampirkan menunjukkan bahwa RAG modern berkembang dari *naive RAG* ke *advanced RAG* dan bahkan *modular RAG* justru karena bentuk dasar RAG punya banyak keterbatasan: *chunk* terlalu kasar, pencarian terlalu dangkal, query tidak diubah, sumber tidak diprioritaskan, dan sistem tidak punya mekanisme validasi. Jadi, ketika banyak sistem RAG gagal, kita sebetulnya sedang melihat kegagalan desain pipeline, bukan sekadar kegagalan model.

Ada pula satu ironi penting. Semakin fasih model, semakin mudah pengguna tertipu. Jawaban yang halus sering dianggap benar. Padahal LLM pada dasarnya adalah mesin prediksi token, bukan mesin kebenaran. Karena itu, tanpa konteks yang baik, model bisa terdengar sangat pintar saat sesungguhnya hanya sedang merangkai kemungkinan bahasa yang paling masuk akal. RAG membantu mengurangi risiko itu dengan memberi model akses ke data eksternal yang relevan dan bisa ditelusuri. Namun, kalau corpus buruk, retrieval buruk, atau sumber tak jelas, maka yang terjadi bukan hilangnya halusinasi, melainkan **halusinasi yang memakai topeng dokumen**.

1.2 Bedakan: *raw document*, *clean document*, *corpus*, *embedding*, *index*, *retrieval*

Agar pembahasan buku ini tidak kabur, kita perlu mendisiplinkan istilah. Banyak orang mencampuradukkan dokumen, vektor, index, dan corpus seolah semuanya sama. Padahal tidak.

Pertama, *raw document*. Ini adalah bahan mentah. Bisa berupa PDF buku, file DOCX, log mentah, README, kode program, wiki internal, atau dump teks apa adanya. Pada tahap ini, dokumen biasanya masih kotor, belum distandarisasi, belum dipilah per topik, dan belum tentu layak langsung dipakai dalam RAG.

Kedua, *clean document*. Ini adalah dokumen yang sudah dibersihkan. Header/footer dibuang. Nomor halaman yang mengganggu dihapus. Bagian yang duplikat dipotong. Struktur judul diperjelas. Kode dipisah dari narasi. Metadata dasar ditambahkan. Dalam banyak kasus, transformasi dari *raw document* ke *clean document* justru menentukan 50% kualitas akhir sistem.

Ketiga, *corpus*. Inilah inti buku ini. Corpus bukan satu file. Corpus adalah **kumpulan dokumen yang sengaja disusun** untuk suatu tujuan retrieval. Ia memiliki struktur, kebijakan kebersihan, aturan labeling, metadata, versi, dan cakupan domain yang jelas. Corpus yang baik bukan hanya bisa dicari, tetapi juga bisa dipertanggungjawabkan.

Keempat, *embedding*. Embedding adalah representasi numerik dari potongan teks agar makna semantisnya bisa dicari dan dibandingkan. Dengan embedding, sistem tidak hanya mencari kata yang persis sama, tetapi juga kedekatan makna. Itulah sebabnya embedding menjadi lapisan penting dalam RAG modern.

Kelima, *index*. Index adalah struktur yang membantu pencarian menjadi cepat dan terarah. Dalam praktik, index bisa berupa vector index, keyword index, atau bentuk lain yang mempercepat pengambilan konteks relevan. Tanpa index yang baik, retrieval akan lambat, mahal, atau ngawur. Literatur yang Anda lampirkan juga menekankan bahwa indexing meningkatkan kecepatan, ketepatan, dan *traceability* sistem.

Keenam, *retrieval*. Retrieval adalah proses mengambil potongan konteks yang dianggap paling relevan terhadap pertanyaan pengguna. Jadi retrieval bukan modelnya, bukan vektornya, bukan dokumennya, melainkan proses pencarian konteks itu sendiri. Retrieval yang baik harus bisa diuji: apakah *chunk* yang diambil benar-benar relevan, cukup lengkap, tidak terlalu pendek, tidak terlalu panjang, dan berasal dari sumber yang layak dipercaya.

Kalau enam istilah ini sudah jelas, maka kita bisa melihat rantai kerjanya dengan lebih tenang: **dokumen mentah dibersihkan menjadi dokumen sehat; dokumen sehat disusun menjadi corpus; corpus dipecah dan diubah menjadi embedding; embedding disimpan dan diindex; lalu retrieval mengambil potongan yang relevan untuk dipakai model menjawab.** Di sinilah kelihatan mengapa buku ini memilih memulai dari corpus, bukan dari prompt. Prompt tanpa corpus yang sehat hanya akan memperindah kekacauan.

1.3 Kritik terhadap pendekatan “unggah semua file lalu berharap AI paham”

Pendekatan “unggah semua file” biasanya lahir dari dua ilusi. Ilusi pertama: semakin banyak dokumen, semakin pintar sistem. Ilusi kedua: model modern otomatis bisa memilah mana yang penting dan mana yang sampah. Keduanya keliru. Dalam praktik, menumpuk file mentah justru sering memperburuk kualitas jawaban. File duplikat memperbanyak kebisingan. Dokumen lama bercampur dengan versi baru. Potongan yang sangat umum mengalahkan potongan yang spesifik. Tabel yang rusak ikut ter-*embed*. Kode program tanpa konteks fungsi ikut bercampur dengan deskripsi bisnis. Log normal bercampur dengan log serangan tanpa label. Akibatnya, sistem tidak kekurangan data. Ia justru **kelebihan sampah**.

Masalah lain adalah **ketiadaan niat desain**. Corpus yang sehat itu dibangun, bukan sekadar dikumpulkan. Ia butuh keputusan: domain apa yang masuk, format minimum apa yang diterima, metadata apa yang wajib ada, dokumen apa yang dibuang, versi mana yang dianggap resmi, bagaimana teks dipotong, bagaimana kualitas retrieval diuji. Kalau semua file diperlakukan setara, sistem kehilangan hirarki. Padahal tidak semua teks punya bobot sama. SOP final berbeda nilainya dengan draft. README resmi berbeda nilainya dengan komentar sementara. Dokumen terbaru berbeda nilainya dengan catatan usang. Jadi, persoalan utamanya bukan cuma banyaknya file, tetapi **tidak adanya tata kelola pengetahuan**.

Dalam banyak proyek, saya justru menyarankan kebalikan dari kebiasaan umum: **mulailah dari corpus kecil tetapi bersih**. Lebih baik 50 dokumen yang jelas, terlabel, dan teruji daripada 5.000 file yang membuat retrieval kacau. Prinsip ini penting terutama untuk mahasiswa dan praktisi yang ingin membangun sistem lokal diatas **Ollama** dan **Open WebUI**. Keduanya sangat berguna untuk eksperimen dan implementasi lokal, tetapi kualitas hasil tetap akan ditentukan oleh kebersihan basis pengetahuan yang dimasukkan. Ollama sendiri menyediakan jalur Linux yang sederhana untuk menjalankan model lokal, sementara Open WebUI menempatkan *Knowledge* sebagai tempat menyimpan dokumen dan koleksi yang bisa dicari, dibaca, dan dipakai model untuk menjawab. Artinya, platformnya sudah tersedia; pertanyaannya tinggal satu: **apakah dokumen yang Anda masukkan memang pantas dijadikan knowledge?** (docs.ollama.com) ([Ollama Docs](#))

1.4 Mengapa Open WebUI RAG tetap bergantung pada mutu dokumen

Open WebUI memudahkan banyak hal, tetapi ia bukan mesin mukjizat. Dokumentasi resminya menjelaskan bahwa *Knowledge* adalah tempat AI mencari, membaca, dan bernalar atas dokumen yang kita unggah. Ia mendukung dua mode penting: **Focused Retrieval**, yaitu mode RAG untuk mencari *chunk* paling relevan dari koleksi besar, dan **Full Context**, yaitu mode yang menyuntikkan seluruh isi dokumen ketika presisi atas dokumen utuh lebih penting daripada pencarian potongan. Ini sangat berguna, tetapi juga memberi pelajaran tegas: keberhasilan sistem sangat bergantung pada **jenis dokumen** dan **cara kita memakainya**. Dokumen pendek yang selalu relevan mungkin cocok untuk *Full Context*. Koleksi besar dengan banyak topik jelas lebih cocok untuk *Focused Retrieval*. Salah memilih mode bisa membuat konteks terlalu sempit atau justru memenuhi jendela konteks secara tidak efisien. ([Open WebUI](#))

Dokumentasi Open WebUI juga menegaskan bahwa *Knowledge* bekerja lewat retrieval, bukan dengan menyuntikkan seluruh isi dokumen secara membabi buta. Karena itu, dokumen yang buruk akan tetap buruk walau masuk ke sistem yang bagus. PDF yang tidak rapi akan menghasilkan *chunk* yang jelek. Metadata yang kosong akan melemahkan filter. Struktur yang kacau akan mempersulit pencarian semantik. Bahkan ketika *hybrid search* dan *reranking* tersedia, semuanya tetap bergantung pada kualitas bahan yang dicari. Sistem bisa membantu mencari, tetapi tidak bisa memperbaiki niat editorial yang tidak pernah dikerjakan. ([Open WebUI](#))

Ada hal praktis lain yang penting untuk dimasukkan ke buku ini. Open WebUI mendukung berbagai vector database, *hybrid search* BM25 + vector search, *cross-encoder reranking*, dan beberapa mesin ekstraksi file. Itu terdengar canggih, tetapi justru harus dibaca secara dewasa: **fitur banyak tidak menghapus kebutuhan akan corpus yang bersih**. Fitur-fitur itu bekerja sebagai pengungkit. Mereka memperbesar kualitas yang sudah ada. Kalau basisnya buruk, pengungkit itu hanya memperbesar kekacauan. Kalau basisnya baik, pengungkit itu mempercepat kualitas. Karena itu, Bab 1 ini sengaja menanamkan disiplin dasar sebelum pembaca terlalu cepat tergoda bermain dengan setelan lanjutan. ([Open WebUI](#))

1.5 Kapan RAG cukup, kapan mulai perlu FT/PEFT

Pertanyaan ini sangat penting karena banyak orang terburu-buru ingin *fine-tuning*, padahal masalahnya belum tentu ada pada model. Literatur yang Anda lampirkan cukup jelas: **RAG unggul ketika pengetahuan bersifat dinamis, perlu transparansi sumber, dan harus mudah diperbarui**. Sebaliknya, *fine-tuning* lebih tepat ketika kita ingin mengubah perilaku model, gaya penulisan, kosa kata domain, atau kemampuan khusus pada format tertentu. Dengan kata lain, RAG itu kuat untuk **fakta dan sumber**, sedangkan *fine-tuning* kuat untuk **perilaku dan kebiasaan model**.

Maka aturan praktisnya bisa dibuat sederhana.

RAG biasanya cukup bila:

- data sering berubah;
- kita perlu jejak sumber;
- tugas utamanya *question answering*, pencarian pengetahuan, FAQ, penjelasan SOP, atau penelusuran dokumen;
- biaya update harus murah;
- kita ingin meminimalkan halusinasi lewat konteks yang bisa dicek.

FT/PEFT mulai layak dipertimbangkan bila:

- gaya keluaran harus konsisten;
- format jawaban sangat khusus;
- model perlu memahami kebiasaan domain tertentu secara lebih dalam;
- kita ingin model lebih nyaman bekerja dengan format seperti kode, tabel, XML, atau pola bahasa yang sangat khas;
- tugas bukan sekadar mencari fakta, tetapi juga memerlukan kebiasaan respons yang stabil.

Di titik ini, penting sekali menekankan bahwa **RAG dan FT bukan musuh**. Keduanya bisa saling melengkapi. Bahkan beberapa sumber Anda secara eksplisit menyebut bahwa ada kasus di mana **RAG + fine-tuning** memberi hasil terbaik: RAG menangani pengetahuan yang dinamis, sementara *fine-tuning* membantu model lebih mahir memahami konteks domain atau format keluaran tertentu. Dalam bahasa yang lebih praktis untuk pembaca buku ini: **jangan fine-tune karena tren; fine-tune**

ketika corpus yang baik dan retrieval yang sehat masih belum cukup menjawab kebutuhan perilaku model.

Karena buku ini memakai kerangka **Ollama + Open WebUI** sebagai stack utama, keputusan yang paling waras bagi banyak pembaca adalah memulai dari **RAG yang sehat terlebih dahulu**. Bangun corpus yang baik. Uji retrieval. Merapikan metadata. Benahi *chunking*. Pastikan model benar-benar memakai konteks. Setelah itu baru tanyakan: masalah yang tersisa ada di pengetahuan, atau ada di perilaku model? Kalau masalahnya pengetahuan, benahi corpus. Kalau masalahnya perilaku, baru diskusikan **LoRA, QLoRA, atau DoRA** di bab-bab berikutnya.

1.6 Peta besar buku: dari dokumen mentah menjadi *knowledge base* yang bisa diuji

Buku ini disusun dengan satu filosofi yang sangat sederhana: **AI yang berguna harus bisa dibuktikan**. Karena itu, perjalanan buku tidak dimulai dari model yang paling besar, melainkan dari bahan yang paling nyata: dokumen. Mula-mula kita akan belajar membedakan dokumen mentah dan dokumen sehat. Lalu kita masuk ke teknik membuat dokumen manual untuk RAG, termasuk format yang sederhana tetapi disiplin. Setelah itu kita bahas cara mengubah textbook PDF menjadi dokumen yang benar-benar bisa dipakai, bukan sekadar teks hasil ekstraksi yang berantakan. Berikutnya, kita masuk ke jantung kualitas corpus: metadata, label, domain, dan struktur. Dari sana kita akan membangun corpus untuk tiga dunia yang sangat berbeda—*knowledge/text*, log server, dan programming—karena masing-masing menuntut strategi yang berbeda.

Sesudah fondasi corpus kuat, barulah kita membahas *chunking: recursive, semantic, dan hierarchical*. Ini penting karena retrieval yang buruk sering berasal dari *chunk* yang salah bentuk. Setelah itu, kita masuk ke tahap yang lebih operasional: menyiapkan RAG di **Open WebUI**, memahami kapan memakai **Focused Retrieval** dan kapan **Full Context**, serta bagaimana menguji apakah retrieval benar-benar bekerja. Di tahap ini, pembaca tidak hanya diminta “percaya” pada AI, tetapi diajak memeriksa jejak bukti, *groundedness*, dan kualitas konteks yang diambil. ([Open WebUI](#))

Akhirnya, di bagian yang lebih maju, buku ini membawa pembaca ke pertanyaan arsitektur: kapan RAG sudah cukup, kapan perlu *reranker*, kapan *fine-tuned embedder* masuk akal, dan kapan kita harus mulai berpikir tentang **PEFT** seperti **LoRA, QLoRA, atau DoRA**. Dengan alur ini, pembaca tidak hanya belajar “cara memakai tool”, tetapi belajar **cara berpikir sebagai perancang corpus**. Itu tujuan sebenarnya. Bukan sekadar membangun chatbot yang bisa menjawab, tetapi membangun **basis pengetahuan yang bisa diuji, diperbaiki, dan dimiliki sendiri**.

Penutup Bab

Kalau ada satu gagasan yang harus dibawa pembaca dari bab pembuka ini, maka gagasan itu adalah ini: **RAG yang baik tidak dimulai dari model, melainkan dari disiplin terhadap pengetahuan.** File mentah bukan corpus. *Embedding* bukan pengganti kurasi. *Vector database* bukan pengganti struktur. Open WebUI bukan pengganti editor yang teliti. Dan model lokal di Ollama bukan pengganti akal sehat. Yang membuat sistem benar-benar berguna adalah keputusan-keputusan kecil tetapi penting: dokumen mana yang dipilih, bagaimana dibersihkan, bagaimana diberi label, bagaimana dipotong, bagaimana diuji, dan bagaimana sumbernya ditelusuri. Di bab-bab berikutnya, kita akan mengerjakan semua itu langkah demi langkah—agar pembaca tidak hanya “punya RAG”, tapi **punya corpus yang bekerja.**

Bab 2. Menyiapkan Ekosistem Ubuntu 26.04 + Ollama + Open WebUI

Bab ini adalah fondasi yang sering diremehkan. Banyak orang ingin cepat sampai ke tahap *upload file*, *embedding*, *query*, dan jawaban AI yang tampak cerdas. Masalahnya, kalau fondasi lingkungan kerjanya berantakan, semua tahap berikutnya ikut rapuh. Python saling tabrak versi. Container tidak stabil. Volume hilang. Model tidak terbaca. Open WebUI hidup, tetapi gagal bicara ke Ollama. Atau lebih buruk: semua tampak jalan, tetapi hasilnya tidak reproduisibel. Karena itu, bab ini sengaja bukan sekadar “cara instalasi”, melainkan **cara menyiapkan lingkungan kerja yang waras, rapi, dan tahan dipakai**. Ini sejalan dengan arah draft Anda yang menekankan bahwa *engineering corpus* perlu ditopang oleh ekosistem yang stabil, bukan improvisasi sesaat.

Dalam konteks buku ini, kita memakai tiga lapisan sederhana namun kuat. **Ubuntu 26.04** sebagai sistem operasi utama. **Ollama** sebagai mesin inferensi model lokal. **Open WebUI** sebagai antarmuka dan *knowledge workspace* untuk RAG. Dokumentasi Open WebUI saat ini memang menempatkan fitur *Knowledge* sebagai tempat dokumen menjadi bisa dicari oleh AI, dengan dua mode utama, **Focused Retrieval** dan **Full Context**. Sementara itu, dokumentasi Ollama resmi menyediakan instalasi Linux, *systemd service*, dan cara memasang versi spesifik. Jadi, secara arsitektur, stack ini masuk akal untuk pembaca yang ingin membangun RAG lokal tanpa bergantung pada layanan proprietary. (docs.openwebui.com) (docs.ollama.com)

2.1 Catatan status Ubuntu 26.04 dan disiplin *version pinning*

Kita perlu jujur sejak awal. Per dokumentasi resmi Ubuntu, **Ubuntu 26.04 LTS masih ditandai “currently in development”**, dengan catatan rilis menampilkan **tanggal rilis 23 April 2026**. Artinya, buku ini boleh dan tepat memakai **Ubuntu 26.04** sebagai target, tetapi pembaca harus diingatkan bahwa **detail paket, nama dependensi, atau perilaku repositori dapat berubah tipis** sampai rilis final benar-benar mapan. Ini bukan kelemahan. Ini justru pelajaran penting tentang disiplin teknis: **jangan mengandalkan asumsi liar; pin versi dan catat lingkungan kerja Anda**. ([Dokumentasi Ubuntu](https://ubuntu.com/docs))

Di dunia AI lokal, *version pinning* bukan obsesi berlebihan. Ia adalah kebiasaan sehat. Open WebUI sendiri, pada dokumentasi *Quick Start*, secara eksplisit menyarankan **memakai tag versi spesifik untuk produksi**, bukan *floating tag*. Contoh yang mereka tampilkan adalah menarik image seperti **v0.8.6**, **v0.8.6-cuda**, atau **v0.8.6-ollama**. Di sisi lain, Ollama juga mendukung pemasangan versi spesifik lewat variabel **OLLAMA_VERSION**. Jadi, sejak bab ini, pembaca harus dibiasakan pada prinsip berikut: **pin versi OS semampunya, pin versi image, pin versi tool, pin versi Python package, dan simpan semua itu di proyek**. ([Open WebUI](https://openwebui.com))

Secara praktis, *version pinning* dalam buku ini berarti:

- catat rilis Ubuntu yang dipakai;
- catat versi Ollama;
- pakai tag image Open WebUI yang spesifik, jangan asal `latest`;
- simpan dependensi Python di `requirements.txt` atau `pyproject.toml`;
- bedakan jelas lingkungan **eksperimen**, **belajar**, dan **semi-produksi**.

Mengapa ini penting? Karena target kita bukan cuma “sekali jalan”, tetapi **bisa diulang minggu depan, bulan depan, dan di mesin lain**. Itulah bedanya proyek belajar yang matang dengan demo yang kebetulan hidup.

2.2 Instalasi dasar host: update sistem, Python, *uv/pip*, *venv*, *git*, *build tools*

Sebelum Ollama dan Open WebUI, host harus sehat dulu. Jangan buru-buru menarik image container kalau mesin dasarnya sendiri belum rapi.

Langkah 1. Update sistem

Mulailah dengan memperbaiki indeks paket dan paket yang sudah ada.

```
sudo apt update
sudo apt -y upgrade
sudo apt -y autoremove
```

Contoh keluaran:

```
Hit:1 http://archive.ubuntu.com/ubuntu InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not
upgraded.
```

Perintah ini terlihat sederhana, tetapi penting untuk mengurangi konflik saat pemasangan Docker, Python, atau dependensi lain. Pada dokumentasi Docker untuk Ubuntu, proses instalasi resmi juga dimulai dari pembaruan `apt` dan pemasangan paket dasar seperti `ca-certificates` dan `curl`. ([Docker Documentation](#))

Langkah 2. Pasang paket dasar host

```
sudo apt install -y \
    curl wget ca-certificates gnupg lsb-release \
    git build-essential pkg-config \
    python3 python3-pip python3-venv
```

Contoh keluaran ringkas:

```
Reading package lists... Done
Building dependency tree... Done
The following NEW packages will be installed:
    build-essential git python3-pip python3-venv ...
0 upgraded, 12 newly installed, 0 to remove.
```

Di sini ada enam fondasi yang ingin kita pastikan ada:

- **Python** untuk skrip bantu, validasi, dan eksperimen kecil;
- **pip** sebagai jalur kompatibilitas umum;
- **venv** untuk lingkungan virtual;
- **git** untuk versi dan sinkronisasi proyek;
- **build tools** seperti **build-essential** untuk dependensi yang perlu kompilasi;
- utilitas dasar seperti **curl** dan **wget**.

Langkah 3. Verifikasi Python

```
python3 --version
pip3 --version
```

Contoh keluaran:

```
Python 3.12.3
pip 24.0 from /usr/lib/python3/dist-packages/pip (python
3.12)
```

Langkah 4. Pasang uv untuk manajemen Python yang lebih cepat

Dokumentasi resmi Astral menunjukkan pemasangan **uv** di Linux dengan:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Contoh cek:

```
uv --version
```

Contoh keluaran:

```
uv 0.11.6
```

Kenapa *uv* menarik? Karena ia ringan, cepat, dan enak untuk alur kerja Python modern. Tetapi saya tetap sarankan **jangan fanatik**. *uv* bagus, **pip** tetap perlu dipahami, karena banyak lingkungan nyata masih mengandalkannya. Dokumentasi resmi *uv* memang menampilkan *standalone installer* dan juga opsi meminta versi spesifik, jadi tool ini cocok dengan disiplin *pinning* yang kita bangun sejak awal. (docs.astral.sh)

Langkah 5. Buat *virtual environment*

Dokumentasi Python resmi tetap merekomendasikan pembuatan lingkungan virtual dengan modul **venv** menggunakan pola `python -m venv /path/to/new/virtual/environment`. Untuk proyek buku ini, saya sarankan satu kebiasaan yang sangat mudah diingat: **satu proyek, satu venv**. ([Python documentation](https://docs.python.org/3/library/venv.html))

```
mkdir -p ~/rag-lab/corpus-rag-2026
cd ~/rag-lab/corpus-rag-2026
python3 -m venv .venv
source .venv/bin/activate
```

Contoh prompt setelah aktif:

```
(.venv) onno@ubuntu:~/rag-lab/corpus-rag-2026$
```

Ini bukan hal remeh. Dengan *venv*, eksperimen kecil untuk *preprocessing*, *PDF cleaning*, validasi YAML, atau skrip cek file tidak akan mencemari sistem global.

2.3 Instalasi Ollama di Linux

Sekarang kita masuk ke mesin model lokal. Dokumentasi resmi Ollama untuk Linux menampilkan jalur paling sederhana:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Setelah itu, Ollama juga menyediakan jalur manual, jalur ARM64, jalur ROCm untuk AMD, dan yang sangat penting: **cara memasangnya sebagai layanan startup melalui systemd**. Selain itu, dokumentasi yang sama menjelaskan bahwa versi spesifik bisa dipasang dengan `OLLAMA_VERSION=<versi>`. Jadi, untuk gaya buku ini, saya sarankan dua pendekatan: **cara cepat** dan **cara disiplin**. ([Ollama Docs](https://ollama.com/docs))

Opsi A. Cara cepat

```
curl -fsSL https://ollama.com/install.sh | sh
ollama -v
```

Contoh keluaran:

```
ollama version is 0.6.5
```

Opsi B. Cara disiplin dengan *version pinning*

```
curl -fsSL https://ollama.com/install.sh |
OLLAMA_VERSION=0.6.5 sh
ollama -v
```

Contoh keluaran:

```
ollama version is 0.6.5
```

Menjalankan Ollama sebagai service

Dokumentasi Ollama merekomendasikan menambahkannya sebagai *startup service*. Intinya:

- buat user `ollama`,
- buat file `/etc/systemd/system/ollama.service`,
- jalankan `daemon-reload`,
- aktifkan service.

Perintah inti:

```
sudo useradd -r -s /bin/false -U -m -d /usr/share/ollama
ollama || true
sudo usermod -a -G ollama $USER
sudo tee /etc/systemd/system/ollama.service > /dev/null
<<'EOF'
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="PATH=/usr/bin:/usr/local/bin"
```

```
[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable ollama
sudo systemctl start ollama
sudo systemctl status ollama --no-pager
```

Contoh keluaran:

```
• ollama.service - Ollama Service
  Loaded: loaded
  (/etc/systemd/system/ollama.service; enabled)
  Active: active (running)
```

Uji tarik model

```
ollama pull qwen3:8b
ollama list
```

Contoh keluaran:

```
pulling manifest
pulling layers
verifying sha256 digest
writing manifest
success
```

NAME	ID	SIZE	MODIFIED
qwen3:8b	1a2b3c4d5e6f	5.2 GB	2 seconds ago

Di titik ini pembaca biasanya mulai bersemangat, dan itu bagus. Tetapi jangan lompat dulu ke RAG. Pastikan dulu **Ollama benar-benar hidup stabil**.

2.4 Instalasi Open WebUI dengan Docker / Docker Compose

Untuk Open WebUI, saya sarankan fokus pada **Docker Compose** karena lebih rapi, lebih mudah dirawat, dan lebih cocok untuk struktur proyek buku ini. Dokumentasi resmi Docker untuk Ubuntu menunjukkan jalur instalasi Engine yang dimulai dari penambahan GPG key resmi, repositori Docker, lalu pemasangan `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin`, dan `docker-compose-plugin`. ([Docker Documentation](#))

Langkah 1. Pasang Docker Engine resmi

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL
https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo
${UBUNTU_CODENAME:-$VERSION_CODENAME}) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
sudo systemctl status docker --no-pager
```

Contoh keluaran:

- `docker.service` - Docker Application Container Engine
Loaded: loaded
Active: active (running)

Langkah 2. Tambahkan user ke grup docker

```
sudo usermod -aG docker $USER
newgrp docker
docker version
docker compose version
```

Contoh keluaran:

```
Docker version 29.4.0, build xxxxx
Docker Compose version v2.39.4
```

Langkah 3. Siapkan docker-compose.yml

Dokumentasi Open WebUI menampilkan contoh `docker-compose.yml` sederhana dengan image `ghcr.io/open-webui/open-webui:main`. Dokumentasi yang sama juga menegaskan bahwa untuk lingkungan produksi, lebih baik **pin versi spesifik** ketimbang memakai tag mengambang. Karena itu, untuk buku ini saya sarankan contoh Compose yang sedikit lebih disiplin. ([Open WebUI](#))

Buat file `docker-compose.yml`:

```
services:
  openwebui:
    image: ghcr.io/open-webui/open-webui:v0.8.6
    container_name: open-webui
    restart: always
    ports:
      - "3000:8080"
    environment:
      - OLLAMA_BASE_URL=http://host.docker.internal:11434
    extra_hosts:
      - "host.docker.internal:host-gateway"
    volumes:
      - ./data/open-webui:/app/backend/data
```

Catatan penting:

- Saya memakai **tag versi spesifik**.
- Saya sengaja **tidak** memakai image yang dibundel dengan Ollama, karena buku ini ingin memisahkan peran **Open WebUI** dan **Ollama** secara bersih.
- `extra_hosts` membantu container mencapai layanan Ollama di host, pola yang lazim di Docker Linux.

Langkah 4. Jalankan container

```
docker compose up -d
docker ps
```

Contoh keluaran:

NAME	IMAGE	STATUS
open-webui	open-webui:v0.8.6	Up 12 seconds

Sekarang UI seharusnya bisa diakses di:

```
http://localhost:3000
```

2.5 Menghubungkan Open WebUI ke Ollama

Ini tahap kecil, tetapi sering menjadi sumber frustrasi. Open WebUI bisa berjalan baik, tetapi kosong karena tidak bisa menjangkau API Ollama.

Dokumentasi Open WebUI memperlihatkan dua pola:

- jika Ollama berjalan di server lain, pakai `OLLAMA_BASE_URL=https://example.com`;
- jika layanan berjalan di host lokal, maka koneksi perlu diarahkan ke alamat host yang bisa dicapai container. Di dokumentasi Open WebUI untuk Podman, nama DNS yang dipakai adalah `host.containers.internal`. Untuk Docker di Linux, praktik yang umum adalah `host.docker.internal` dengan `host-gateway`. ([Open WebUI](#))

Uji konektivitas dari host

```
curl http://localhost:11434/api/tags
```

Contoh keluaran:

```
{"models":[{"name":"qwen3:8b","size":5583457484}]}
```

Uji konektivitas dari container

```
docker exec -it open-webui sh -c "wget -qO-
http://host.docker.internal:11434/api/tags"
```

Contoh keluaran:

```
{"models":[{"name":"qwen3:8b","size":5583457484}]}
```

Kalau ini berhasil, berarti jembatan utama sudah beres: **UI hidup, Ollama hidup, dan keduanya saling bicara.**

2.6 Struktur folder proyek corpus yang rapi

Sekarang kita masuk ke hal yang kelihatannya sederhana, tetapi nanti menyelamatkan banyak waktu: **struktur folder**. Draft buku Anda sendiri menekankan pentingnya lingkungan yang rapi, manifest data, dan disiplin proyek. Saya sepakat penuh. Di proyek RAG, folder yang kacau akan cepat berubah menjadi hutang teknis.

Saya sarankan struktur seperti ini:

```
~/rag-lab/corpus-rag-2026/
├── docker-compose.yml
├── .env
├── .venv/
├── data/
│   ├── open-webui/
│   ├── raw/
│   ├── cleaned/
│   ├── logs/
│   ├── programming/
│   └── exports/
├── scripts/
│   ├── check_env.sh
│   ├── import_docs.py
│   └── validate_yaml.py
├── docs/
│   ├── notes/
│   ├── textbook/
│   └── sop/
├── manifests/
│   ├── corpus-manifest.yaml
│   └── sources.csv
└── README.md
```

Makna sederhananya:

- **raw/** untuk bahan mentah;
- **cleaned/** untuk dokumen yang sudah lolos pembersihan;
- **logs/** untuk corpus log server;
- **programming/** untuk code, README, API docs;
- **exports/** untuk hasil ekstraksi;
- **manifests/** untuk daftar sumber dan metadata proyek;
- **data/open-webui/** untuk volume Open WebUI.

How-to cepat membuat struktur

```
mkdir -p
~/rag-lab/corpus-rag-2026/{data/{open-webui,raw,cleaned,logs,programming,exports},scripts,docs/{notes,textbook,sop},manifests}
touch ~/rag-lab/corpus-rag-2026/{README.md,.env}
```

Contoh cek:

```
tree -L 3 ~/rag-lab/corpus-rag-2026
```

Contoh keluaran:

```
corpus-rag-2026
├── README.md
├── data
│   ├── cleaned
│   ├── exports
│   ├── logs
│   ├── open-webui
│   ├── programming
│   └── raw
├── docs
│   ├── notes
│   ├── sop
│   └── textbook
├── manifests
└── scripts
```

Mengapa ini penting? Karena sejak bab-bab berikutnya, kita akan punya dokumen manual, hasil ekstraksi PDF, log, dan dokumen programming. Kalau dari awal tidak dipisahkan, retrieval nanti ikut kacau.

2.7 *Sanity check*: model jalan, UI hidup, dokumen bisa diunggah

Ini bagian yang menyenangkan. Setelah semua terpasang, kita perlu memastikan sistem benar-benar hidup.

Cek 1. Ollama aktif

```
systemctl status ollama --no-pager
ollama list
```

Contoh keluaran:

- ollama.service - Ollama Service
Active: active (running)

NAME	ID	SIZE
qwen3:8b	1a2b3c4d5e6f	5.2 GB

Cek 2. Open WebUI aktif

```
docker ps
curl -I http://localhost:3000
```

Contoh keluaran:

```
HTTP/1.1 200 OK
server: uvicorn
content-type: text/html; charset=utf-8
```

Cek 3. Model tampil di Open WebUI

Masuk ke browser:

- buka <http://localhost:3000>
- login / buat akun admin pertama
- buka pengaturan koneksi model
- pastikan model dari Ollama terlihat

Cek 4. Dokumen bisa diunggah ke *Knowledge*

Dokumentasi Open WebUI menjelaskan bahwa *Knowledge* membuat dokumen bisa dicari oleh AI, dengan **Focused Retrieval** sebagai default untuk koleksi besar, dan **Full Context** untuk dokumen singkat yang selalu relevan. Gunakan satu file kecil dulu, misalnya [README.md](#) atau satu catatan teknis pendek. Jangan langsung unggah ratusan file. ([Open WebUI](#))

Cek 5. Tanya pertanyaan sederhana

Contoh:

- “Apa isi utama dokumen yang saya unggah?”
- “Sebutkan tiga poin penting dari file ini.”
- “Dari sumber mana jawaban ini diambil?”

Kalau semua berjalan, pembaca akan merasakan kemenangan kecil yang penting: **bukan cuma instalasi sukses, tetapi alur end-to-end mulai hidup.**

2.8 Troubleshooting awal: izin file, memori, embedding, reindex

Tidak ada bab instalasi yang jujur tanpa bagian masalah umum. Di dunia nyata, justru di sinilah pembaca merasa ditolong.

Masalah 1. Docker hidup, tetapi Open WebUI tidak bisa simpan data

Gejala:

- UI bisa dibuka;
- tetapi upload gagal;
- data hilang setelah container dihapus.

Cek:

```
ls -ld ./data/open-webui
docker logs open-webui --tail 50
```

Solusi awal:

- pastikan volume diarahkan ke folder yang benar;
- pastikan user Docker punya izin tulis;
- jangan simpan semua hal di folder acak.

Masalah 2. Open WebUI hidup, tetapi model Ollama tidak muncul

Gejala:

- UI normal;
- tetapi daftar model kosong.

Cek:

```
curl http://localhost:11434/api/tags
docker exec -it open-webui sh -c "wget -qO-
http://host.docker.internal:11434/api/tags"
```

Kalau perintah pertama sukses dan yang kedua gagal, berarti masalahnya jaringan container ke host, bukan Ollama.

Masalah 3. Memori habis

Gejala:

- model lambat sekali;
- proses mati mendadak;
- container restart sendiri;
- mesin mulai swap berat.

Solusi praktis:

- mulai dari model yang lebih kecil;
- jangan langsung pakai dokumen raksasa;
- tutup proses lain;
- pahami bahwa RAG yang baik bukan berarti model terbesar.

Masalah 4. Hasil retrieval jelek setelah ganti model embedding

Di dunia Open WebUI, perubahan konfigurasi retrieval dan embedding dapat membuat knowledge base lama tidak lagi konsisten secara semantik. Draft Anda sendiri juga sudah mengingatkan bahwa perubahan model embedding dapat menuntut reindex. Jadi, biasakan satu aturan sederhana: kalau embedding model berubah, anggap index lama perlu ditinjau ulang, dan sering kali perlu dibangun ulang.

Masalah 5. Salah mode retrieval

Dokumentasi Open WebUI menegaskan:

- Focused Retrieval cocok untuk koleksi besar dan hanya mengambil chunk yang relevan;
- Full Context menyuntikkan seluruh isi dokumen dan bisa menghabiskan context window jika file terlalu besar. ([Open WebUI](#))

Jadi:

- untuk SOP pendek, panduan gaya, atau file singkat yang selalu relevan, Full Context bisa masuk akal;
- untuk ratusan dokumen atau textbook besar, Focused Retrieval jauh lebih waras.

Masalah 6. Tag main berubah perilaku

Gejala:

- minggu lalu jalan;
- minggu ini perilaku berubah tipis.

Penyebab paling umum: Anda memakai image mengambang seperti :main atau :latest. Dokumentasi Open WebUI sendiri menyarankan pin versi spesifik untuk produksi. Jadi solusi paling elegan bukan marah ke tool, melainkan disiplin sejak awal. ([Open WebUI](#))

How-to ringkas: dari nol sampai siap dipakai

Agar bab ini mudah dipraktekkan mahasiswa dan praktisi, berikut alur singkat yang bisa diikuti dalam satu sesi kerja:

- **Tahap 1. Rapikan host**
jalankan `apt update`, pasang Python, `git`, `venv`, dan *build tools*.
- **Tahap 2. Pasang uv dan siapkan proyek**
buat folder proyek, aktifkan `.venv`, dan catat versi tool.
- **Tahap 3. Pasang Ollama**
instal, aktifkan sebagai service, tarik satu model kecil-menengah.
- **Tahap 4. Pasang Docker**
gunakan repositori resmi Docker, lalu pasang Compose plugin.
- **Tahap 5. Jalankan Open WebUI**
pakai `docker-compose.yml`, pin tag image, mount volume persistent.
- **Tahap 6. Sambungkan Open WebUI ke Ollama**
pastikan API Ollama bisa dijangkau dari container.
- **Tahap 7. Lakukan sanity check**
model muncul, UI hidup, file kecil bisa diunggah, jawaban keluar.
- **Tahap 8. Catat lingkungan**
simpan versi Ubuntu, Ollama, image Open WebUI, dan struktur folder.

Penutup Bab

Bab ini mengajarkan satu pelajaran yang sering diabaikan di dunia AI: **fondasi yang tenang lebih penting daripada demo yang heboh**. Ubuntu 26.04 memberi landasan yang modern, tetapi perlu dihadapi dengan disiplin karena status rilisnya masih dekat fase pengembangan. Ollama memberi inferensi lokal yang sederhana, tetapi tetap butuh pengelolaan service dan model yang rapi. Open WebUI memberi antarmuka RAG yang nyaman, tetapi tetap bergantung pada konektivitas, mode retrieval, dan volume data yang sehat. Jika bab ini dikerjakan dengan sabar, pembaca akan sampai ke titik yang sangat penting: **mesin sudah hidup, lingkungan sudah stabil, dan kita siap masuk ke pekerjaan yang lebih bermakna—membangun corpus yang benar-benar layak dipakai**.

Bab 3. Memilih Model *Embedding* dan *Vector Database* yang Waras

Di dunia RAG, banyak orang terlalu cepat fokus ke model generatif. Mereka sibuk membandingkan LLM A melawan LLM B, ukuran parameter, atau gaya jawaban. Padahal, untuk pertanyaan seperti “dokumen mana yang diambil”, “potongan mana yang paling relevan”, “mengapa jawaban ini meleset”, atau “kenapa AI salah kutip”, akar masalahnya justru sering ada di lapisan yang lebih sunyi: ***embedding model***, ***retrieval strategy***, metadata, dan ***vector database***. Bab ini sengaja membahas lapisan itu, karena di sinilah kualitas factual grounding banyak ditentukan. Literatur pendukung Anda juga menekankan hal yang sama: *chunking*, *embedding*, dan *retrieval logic* adalah tiga langkah yang mengubah kumpulan dokumen statis menjadi *knowledge base* yang hidup.

Secara praktis, ***embedding model*** bertugas mengubah potongan teks menjadi vektor numerik agar kemiripan makna bisa dicari. ***Vector database*** bertugas menyimpan vektor itu bersama metadata dan mengembalikannya dengan cepat saat ada pertanyaan. Open WebUI lalu duduk di atas lapisan ini untuk memberi antarmuka *Knowledge/RAG*, sementara Ollama menjadi mesin model lokal untuk menghasilkan jawaban. Dokumentasi Open WebUI sendiri menegaskan bahwa kualitas *embedding* langsung memengaruhi apa yang berhasil diambil, dan setelah model *embedding* diganti, dokumen yang sudah ada perlu di-*reindex*. ([Open WebUI](#))

3.1 Peran *embedding model* dalam kualitas retrieval

Mari kita mulai dari pertanyaan paling penting: **mengapa *embedding model* begitu menentukan?** Karena RAG modern tidak mencari teks hanya lewat kecocokan kata. Ia mencoba mencari **kedekatan makna**. Dua kalimat bisa memakai kata yang berbeda tetapi membahas hal yang sama. Sebaliknya, dua kalimat bisa memakai kata yang sama tetapi maknanya berbeda. Di sinilah *embedding* bekerja: ia memetakan teks ke ruang vektor agar kemiripan semantik bisa dihitung. Sentence Transformers menjelaskan secara resmi bahwa pustaka ini digunakan untuk menghitung *embeddings*, menghitung skor kemiripan dengan *Cross-Encoder/reranker*, dan juga membangkitkan *sparse embeddings*. ([Sbert](#))

Artinya sederhana tetapi penting. Kalau *embedding model* Anda lemah terhadap Bahasa Indonesia, dokumen Indonesia akan lebih mudah salah ambil. Kalau ia buruk untuk kode, maka *README*, API docs, dan komentar kode akan mudah tenggelam dalam noise. Kalau ia lemah pada dokumen teknis, manual sistem dan SOP akan tampak “jauh” padahal sesungguhnya relevan. Itulah sebabnya pemilihan *embedding model* bukan urusan kosmetik, tetapi **keputusan arsitektur**. Draft Anda juga sudah memberi arah ini dengan menyarankan *embedding model* yang dipilih berdasarkan domain, bahasa, dan konteks penggunaan, bukan sekadar tren.

Dalam praktik, gejala *embedding* yang buruk biasanya tampak seperti ini:

- pertanyaan tepat, tetapi konteks yang diambil meleset;
- dokumen relevan muncul di bawah hasil yang kurang penting;
- dokumen dengan kata kunci mirip menang atas dokumen yang maknanya benar;
- jawaban terasa “nyambung sedikit”, tetapi tidak benar-benar grounded.

Pelajaran pentingnya: sebelum menyalahkan LLM, periksa dulu *embedding*-nya.

How-to cepat menguji kualitas *embedding*

Buat tiga pertanyaan kecil terhadap corpus yang sama:

- satu pertanyaan faktual;
- satu pertanyaan paraphrase;
- satu pertanyaan yang memakai istilah domain.

Lalu lihat:

- apakah dokumen yang benar muncul di *top-k*;
- apakah potongan yang muncul masih utuh maknanya;
- apakah sumber yang muncul terlalu umum.

Kalau hasil retrieval sudah salah di tahap ini, mengganti prompt hampir pasti tidak akan menyelamatkan sistem.

3.2 *Dense, sparse, dan hybrid retrieval* secara praktis

Di lapangan, ada tiga cara berpikir retrieval yang perlu dipahami pembaca buku ini.

Pertama, *dense retrieval*. Ini pendekatan yang paling lazim di RAG modern. Query dan dokumen diubah menjadi vektor *dense*, lalu sistem mencari vektor yang paling dekat. Keunggulannya ada pada pencarian semantik. Sistem dapat menemukan teks yang maknanya mirip walau kata-katanya tidak persis sama.

Kedua, *sparse retrieval*. Ini pendekatan yang lebih dekat ke pencarian berbasis token atau istilah yang menonjol. Dalam praktik modern, *sparse encoder* juga bisa dipelajari oleh model, bukan sekadar BM25 klasik. Sentence Transformers memang menyediakan *Sparse Encoder* sebagai kelas resmi untuk membangkitkan *sparse embeddings*. ([Sbert](#))

Ketiga, *hybrid retrieval*. Ini gabungan *dense* dan *sparse*. Pendekatan ini sering sangat berguna untuk corpus nyata karena banyak pertanyaan memiliki dua sifat sekaligus: butuh kemiripan makna, tetapi juga butuh kata kunci yang eksplisit. LanceDB mendokumentasikan *hybrid search* sebagai gabungan vector dan *full-text search* dengan *reranking*. Chroma juga menjelaskan bahwa ia dapat menyimpan *embeddings* dengan metadata, melakukan pencarian dengan *dense* dan *sparse vectors*, serta filter metadata. Qdrant mendokumentasikan *hybrid and multi-stage queries* untuk pencarian gabungan *dense-sparse*. ([LanceDB](#))

Dalam bahasa yang lebih membumi:

- **Dense** bagus untuk *paraphrase* dan kemiripan konsep.
- **Sparse** bagus ketika kata kunci sangat menentukan.
- **Hybrid** bagus untuk dunia nyata, terutama saat corpus campuran: dokumen teknis, SOP, log, API, dan catatan manual.

Contoh kasus sederhana

Query: “bagaimana reindex knowledge base setelah ganti model embedding?”

- *Dense retrieval* mungkin menemukan dokumen yang membahas migrasi *knowledge base*, walau kata “reindex” tidak dominan.
- *Sparse retrieval* cenderung kuat jika kata “reindex” memang muncul persis.
- *Hybrid retrieval* biasanya paling sehat, karena menangkap baik istilah eksplisit maupun konteks semantik.

How-to berpikir praktis

Untuk buku ini, pola keputusan awal yang waras adalah:

- mulai dari ***dense retrieval*** bila corpus Anda masih kecil-menengah dan topiknya rapi;
- pertimbangkan ***hybrid retrieval*** saat corpus mulai heterogen atau mengandung banyak istilah teknis, nama fungsi, kode error, atau terminologi domain;
- pakai ***sparse*** sebagai penguat, bukan sebagai satu-satunya jalan, kecuali kebutuhan Anda memang dominan pada *keyword match*.

3.3 Kriteria memilih model *embedding*

Bagian ini sangat penting, karena banyak pembaca akan tergoda mencari “model terbaik 2026”. Saya sengaja menolak cara berpikir itu. **Tidak ada model terbaik secara mutlak. Yang ada adalah model yang paling cocok untuk corpus dan tujuan retrieval Anda.** Draft Anda juga sudah memberi sinyal yang benar: pemilihan model harus mengikuti kebutuhan bahasa, domain, dan konteks deployment.

a. *Multilingual*

Untuk konteks Indonesia, kemampuan *multilingual* bukan bonus, melainkan kebutuhan. Banyak corpus bercampur antara Bahasa Indonesia dan Inggris: SOP lokal, README proyek, dokumentasi upstream, log, komentar kode, dan materi kuliah. Model yang buruk di *multilingual retrieval* akan membuat hasil campuran bahasa menjadi tidak stabil. **Kalau corpus Anda bilingual atau multilingual, jangan memilih model yang hebat di Inggris saja tetapi lunglai di bahasa lain.**

b. *Code-aware*

Kalau corpus Anda memuat:

- source code,
- komentar fungsi,
- API docs,
- konfigurasi YAML,
- Docker Compose,
- README teknis,

maka ***code-aware embedding*** layak diprioritaskan. Model yang hanya bagus pada prosa umum sering gagal membedakan makna antara `docker compose up -d`, `docker build`, dan `docker exec`, padahal bagi praktisi perbedaannya sangat besar.

c. Dokumen teknis

Dokumen teknis punya ciri khas:

- istilah padat;
- struktur formal;
- daftar parameter;
- kode dan teks bercampur;
- referensi silang;
- kalimat pendek tapi bermakna besar.

Model *embedding* yang cocok untuk artikel umum belum tentu cocok untuk manual teknis. Karena itu, jangan uji model hanya dengan pertanyaan “umum”, tetapi juga dengan query teknis nyata dari domain Anda.

d. Kecepatan

Kecepatan penting pada dua tempat:

- saat **ingest** besar-besaran;
- saat query real-time.

Model terlalu berat akan membuat *embedding pipeline* lambat, *reindex* lama, dan eksperimen terasa mahal. Untuk lab pribadi dan Open WebUI lokal, model yang “cukup bagus dan cepat” sering lebih sehat daripada model yang “sedikit lebih bagus tapi menyiksa workflow”.

e. Memori

Jangan abaikan RAM/VRAM. Model *embedding* yang terlalu besar bisa terasa heroik di atas kertas, tetapi menyulitkan eksperimen lokal. Ingat pesan besar buku ini: **yang waras lebih penting daripada yang glamor**.

f. Lisensi

Karena buku ini menekankan **open source** dan kemandirian, lisensi wajib diperiksa. Jangan sampai Anda membangun corpus besar di atas komponen yang lisensinya membatasi distribusi atau penggunaan.

How-to uji model *embedding* secara sederhana

Buat set uji kecil berisi:

- 10 query Indonesia,
- 10 query Inggris,
- 10 query teknis,
- 10 query kode atau API,
- 10 query paraphrase.

Lalu ukur secara praktis:

- apakah sumber benar masuk *top-3*;
- apakah *top-1* sering meleset;
- apakah dokumen bahasa Indonesia kalah terus;
- apakah query kode salah diarahkan ke dokumen naratif.

Contoh pseudo-langkah CLI untuk eksperimen Python lokal:

```
source .venv/bin/activate
python test_embedding_retrieval.py
```

Contoh keluaran:

```
[INFO] Model: local-embed-model-A
[INFO] Queries tested: 50
[INFO] Top-1 correct: 31
[INFO] Top-3 correct: 44
[INFO] Technical queries failed: 6
[INFO] Code-aware queries failed: 9
```

Keluaran seperti ini jauh lebih berguna daripada klaim marketing.

3.4 Kriteria memilih *vector database*

Sekarang kita masuk ke penyimpanan. Di buku ini, saya sarankan tiga nama inti karena semuanya relevan untuk jalur open source dan lokal: **Chroma**, **LanceDB**, dan **Qdrant**. Draft terlampir juga sudah memusatkan pilihan ke tiga ini.

Chroma untuk mulai cepat

Chroma mendokumentasikan dirinya sebagai infrastruktur data open source untuk AI yang memberi semua kebutuhan dasar retrieval: menyimpan *embeddings* dengan metadata, pencarian *dense* dan *sparse vectors*, filter metadata, serta retrieval lintas teks dan media lain. Ini membuat Chroma cocok untuk:

- prototyping cepat;
- eksperimen lokal;
- skenario belajar;
- corpus pribadi yang belum terlalu besar. ([Chroma Docs](#))

Kapan Chroma masuk akal? Saat Anda ingin cepat bergerak, tidak ingin banyak konfigurasi, dan skala belum menuntut arsitektur terpisah yang kompleks.

LanceDB untuk lokal tertanam dan *hybrid/full-text*

LanceDB mendokumentasikan dukungan untuk metadata filtering, *pre-filtering*, dan *hybrid search* yang menggabungkan vector dan *full-text* dengan *reranking*. Ini sangat menarik untuk stack lokal karena:

- cocok untuk penyimpanan tertanam;
- enak untuk eksperimen lokal yang serius;
- punya cerita kuat untuk *hybrid retrieval*;
- baik saat Anda butuh keyword + semantic search sekaligus. ([LanceDB](#))

Kapan LanceDB masuk akal? Saat Anda ingin lokal yang lebih matang, retrieval lebih kaya, dan tetap menjaga kesederhanaan deployment.

Qdrant untuk filtering dan arsitektur *client-server*

Qdrant mendokumentasikan dirinya sebagai open-source vector search engine dengan *hybrid dense-sparse retrieval*, metadata filtering, dan *real-time updates*. Dokumentasinya soal *filtering* sangat jelas: ada kondisi tambahan yang penting justru ketika fitur objek tidak bisa seluruhnya diekspresikan dalam embedding, misalnya ketersediaan stok, lokasi, harga, atau batasan bisnis lain. Dalam dunia corpus RAG, ini bisa diterjemahkan menjadi:

- sumber dokumen;
- tanggal;
- sensitivitas;
- bahasa;
- domain;
- status validasi. ([Qdrant](#))

Kapan Qdrant masuk akal? Saat Anda sudah berpikir lebih serius tentang layanan terpisah, filtering yang kaya, *multi-user*, atau skenario yang mendekati produksi.

Ringkasan keputusan praktis

- **Chroma**: cepat mulai, ringan, cocok untuk belajar dan prototyping.
- **LanceDB**: kuat untuk lokal tertanam, *hybrid search*, dan *full-text*.
- **Qdrant**: cocok untuk arsitektur layanan, filtering kuat, dan pertumbuhan skala.

How-to memilih dengan kepala dingin

Tanyakan tiga hal ini:

- corpus Anda kecil, menengah, atau tumbuh cepat?
- Anda butuh *hybrid search* atau cukup semantic saja?
- Anda ingin satu folder lokal atau layanan terpisah?

Kalau tiga jawaban itu jernih, pilihan database biasanya ikut jernih.

Open-WebUI default vector database

Di **Open WebUI**, **vector database** yang digunakan secara **default** adalah **ChromaDB** (embedded / built-in, zero-config).

ChromaDB → Default dan paling mudah digunakan. Cocok untuk penggunaan lokal/single instance, karena otomatis tersedia tanpa setup tambahan. Namun, ada keterbatasan: tidak aman untuk multi-process/multi-replica (bisa crash karena pakai SQLite backend).

Open WebUI mendukung **9+ vector database** lainnya yang bisa diubah lewat environment variable `VECTOR_DB`.

Daftar Vector Database yang Didukung

Vector Database	Tipe	Kelebihan Utama	Rekomendasi Penggunaan
ChromaDB	Embedded (default)	Zero-config, ringan	Instalasi sederhana, testing, single user
PGVector	PostgreSQL extension	Pakai DB yang sama dengan main database	Kalau sudah pakai Postgres
Qdrant	Client-server	Cepat, filtering bagus, production-ready	Skala sedang-besar, banyak user
Milvus	Client-server	Sangat scalable, cloud-native	Koleksi data sangat besar
Elasticsearch	Full-text + vector	Hybrid search (BM25 + vector)	Butuh keyword + semantic search
OpenSearch	Sama seperti ES	Alternatif open-source Elasticsearch	Sama seperti Elasticsearch
Lainnya	(MariaDB Vector, dll)	Community contribution	Sesuai kebutuhan

Cara Mengganti Vector Database

Kamu bisa set environment variable di docker-compose.yml atau .env:

```
VECTOR_DB=qdrant
# atau pgvector, milvus, elasticsearch, dll.
QDRANT_URI=http://qdrant:6333
# atau untuk PGVector:
#
PGVECTOR_DB_URL=postgresql://user:pass@postgres:5432/openw
ebui
```

Untuk scaling/high availability, sangat disarankan pindah dari **ChromaDB default** ke **PGVector** atau **Qdrant**.

3.5 Metadata wajib yang harus ikut tersimpan

Ini bagian yang sering disepelekan, padahal justru menyelamatkan RAG dari kebodohan yang sangat mahal. **Vektor tanpa metadata hanyalah memori yang bisu**. Beberapa sumber yang Anda lampirkan juga tegas: simpan metadata seperti sumber, tanggal, tipe dokumen; gabungkan filter keyword dan semantik; perlakukan vector database sebagai komponen inti, bukan pelengkap.

Metadata minimum yang saya sarankan untuk buku ini:

- **source** — sumber dokumen;
- **title** — judul atau identitas utama;
- **doc_type** — SOP, buku, log, README, API docs, memo, kode;
- **language** — **id**, **en**, atau campuran;
- **created_at** / **updated_at** — waktu;
- **domain** — misalnya **network**, **security**, **devops**, **ai**;
- **sensitivity** — publik, internal, terbatas;
- **version** — versi dokumen;
- **author** atau **owner** — pemilik pengetahuan;
- **tags** — label topik;
- **chunk_id** dan **parent_doc_id** — untuk jejak chunk;
- **quality_flag** — draft, reviewed, approved.

Mengapa ini penting? Karena banyak hal yang tidak bisa ditangkap embedding secara penuh. Qdrant sendiri menulis bahwa *additional conditions* penting ketika tidak semua fitur objek bisa diekspresikan dalam embedding. Dalam bahasa sederhana: vektor mungkin tahu makna teks, tetapi tidak otomatis tahu bahwa dokumen ini **versi terbaru**, **dokumen resmi**, atau **khusus bahasa Indonesia**. ([Qdrant](#))

Contoh metadata dalam YAML

```
title: "Panduan Reindex Knowledge Base"
source: "internal-docs"
doc_type: "SOP"
language: "id"
domain: "rag"
sensitivity: "internal"
version: "1.2"
updated_at: "2026-04-13"
tags: ["open-webui", "embedding", "reindex"]
quality_flag: "approved"
```

Golden Rule

Jangan simpan metadata karena “nanti mungkin berguna”. Simpan metadata yang memang akan dipakai untuk filter, audit, versi, dan evaluasi.

3.6 Kapan perlu *reranker*

Reranker menjadi penting ketika retrieval awal sudah lumayan, tetapi urutannya belum cukup cerdas. Literatur pendukung Anda menjelaskan *reranking* sebagai strategi dua tahap:

1. ambil banyak chunk dengan retrieval biasa;
2. urutkan ulang chunk dengan model kedua, lalu kirim subset terbaik ke LLM.

Sentence Transformers juga mendokumentasikan *reranker models* sebagai *CrossEncoder* yang menerima pasangan teks, misalnya query dan dokumen, lalu menghasilkan satu skor relevansi. ([Sbert](#))

Kapan reranker layak dipakai?

- saat *top-k* awal berisi banyak kandidat “agak relevan”;
- saat query teknis sering menemukan dokumen benar, tetapi posisinya terlalu bawah;
- saat corpus mulai besar dan heterogen;
- saat *hybrid search* memberi banyak hasil campuran dan Anda perlu urutan yang lebih cerdas.

Kapan belum perlu?

- saat corpus masih kecil;
- saat *top-3* sudah stabil;
- saat latensi harus sangat hemat;
- saat Anda masih membereskan kualitas corpus dasar.

Logika yang sehat

Jangan pasang *reranker* untuk menutupi corpus yang berantakan. *Reranker* adalah penguat, bukan penyihir.

How-to berpikir sederhana

Ambil 20 hasil pertama dari retrieval dasar. Lalu lihat:

- apakah hasil yang benar sering ada di posisi 6–15?
- apakah query teknis sering “hampir benar” tapi kalah urutan?
- apakah *top-3* terlalu generik?

Kalau jawabannya “ya”, *reranker* layak dicoba.

3.7 Dampak pergantian model *embedding*: kapan wajib *reindex*

Ini salah satu aturan yang harus ditanamkan keras pada pembaca. Dokumentasi resmi Open WebUI sangat jelas: setelah mengganti model *embedding*, ***reindex all existing documents*** agar *embedding* baru berlaku. Dokumentasi konfigurasi environment Open WebUI juga membahas tombol **Reindex Knowledge Base** dan menegaskan bahwa proses ini memigrasikan *knowledge base* yang ada. ([Open WebUI](#))

Mengapa? Karena vektor lama dibuat dengan model lama. Saat model berubah, ruang representasi berubah. Dua kalimat yang dulu “dekat” mungkin kini memiliki geometri berbeda. Jadi, menyimpan vektor lama setelah model *embedding* diganti sering berarti retrieval Anda sekarang memakai peta lama untuk kota yang bentuknya sudah berubah.

Kapan wajib *reindex*?

- saat model *embedding* diganti;
- saat dimensi *embedding* berubah;
- saat strategi *chunking* berubah signifikan;
- saat metadata yang dipakai untuk retrieval berubah struktur;
- saat Anda pindah skema penyimpanan yang mempengaruhi representasi.

Kapan mungkin tidak wajib penuh, tetapi tetap perlu ditinjau?

- saat hanya menambah sebagian kecil dokumen baru dengan model yang sama;
- saat hanya memperbaiki metadata non-kritis tanpa mengubah vector content.

Gejala Anda butuh *reindex*

- retrieval terasa memburuk setelah ganti model;
- dokumen lama kalah oleh dokumen baru secara aneh;
- hasil tampak acak atau kehilangan relevansi semantik;
- Open WebUI menunjukkan knowledge lama, tetapi hasil query terasa “pincang”.

How-to praktis

Urutan aman:

- catat model lama;
- pilih model baru;
- backup metadata dan dokumen;
- lakukan *reindex knowledge base*;
- uji 20 query acuan yang sama;
- bandingkan *top-k* sebelum dan sesudah.

Jangan ganti model embedding lalu langsung percaya hasil. Uji dulu.

3.8 Checklist keputusan sebelum *ingest* besar-besaran

Sebelum ribuan dokumen masuk, berhenti sejenak. Banyak kekacauan RAG lahir karena orang terlalu cepat ingest. Bab ini ingin mengajarkan kebiasaan yang lebih matang.

Gunakan checklist ini:

- **Apakah domain corpus sudah jelas?**
Jangan campur semua hal tanpa batas.
- **Apakah model embedding cocok untuk bahasa corpus?**
Indonesia, Inggris, campuran, atau kode.
- **Apakah model embedding cukup cepat dan muat di mesin Anda?**
Hebat di kertas belum tentu waras di laptop.
- **Apakah lisensi model aman untuk tujuan penggunaan?**
Jangan buta lisensi.
- **Apakah retrieval awal akan dense, sparse, atau hybrid?**
Pilih berdasarkan sifat query, bukan ikut tren.
- **Apakah vector database dipilih karena alasan nyata?**
Chroma untuk cepat mulai, LanceDB untuk lokal hybrid, Qdrant untuk filtering dan layanan.
- **Apakah metadata minimum sudah ditentukan?**
Sumber, tanggal, tipe dokumen, bahasa, versi, sensitivitas.
- **Apakah Anda sudah punya 20 query uji tetap?**
Tanpa ini, Anda tidak akan tahu sistem membaik atau memburuk.
- **Apakah aturan reindex sudah jelas?**
Ganti embedding model = jangan malas reindex.
- **Apakah Anda benar-benar butuh reranker sekarang?**
Atau masalahnya masih ada pada corpus dan metadata?

How-to mini sebelum ingest

Jalankan pilot kecil dulu:

1. pilih 30–50 dokumen;
2. tetapkan metadata minimum;
3. uji satu model embedding;
4. uji satu vector store;
5. ukur *top-3* dan *top-10*;
6. baru putuskan skala lebih besar.

Contoh alur CLI sederhana:

```
source .venv/bin/activate
python ingest_pilot.py --input ./data/cleaned --limit 50
python eval_retrieval.py --queries ./tests/queries.yaml
```

Contoh keluaran:

```
[INFO] Documents ingested: 50
[INFO] Embedding model: local-embed-model-A
[INFO] Vector store: lancedb
[INFO] Top-3 hit rate: 78%
[INFO] Top-10 hit rate: 92%
[INFO] Recommendation: proceed with pilot expansion
```

Keluaran seperti ini memberi dasar keputusan yang jauh lebih jujur daripada sekadar merasa “kayaknya sudah bagus”.

Penutup Bab

Bab ini membawa kita ke satu kenyataan yang sering tidak populer, tetapi sangat penting: **RAG yang baik lahir dari keputusan yang masuk akal, bukan dari komponen yang paling mahal atau paling ramai dibicarakan.** *Embedding model* menentukan bagaimana makna dicari. *Dense*, *sparse*, dan *hybrid retrieval* menentukan cara konteks ditemukan. *Vector database* menentukan bagaimana pengetahuan disimpan, di filter, dan diambil kembali. Metadata menentukan apakah sistem bisa diaudit dan dikendalikan. *Reranker* menentukan apakah kandidat yang “lumayan” bisa naik menjadi konteks yang benar-benar relevan. Dan *reindex* menentukan apakah perubahan model tidak merusak fondasi tanpa disadari. Semua itu jauh lebih penting daripada sekadar memasang model generatif besar lalu berharap hasilnya otomatis benar. ([Open WebUI](#))

Dalam semangat buku ini, keputusan yang waras selalu lebih kuat daripada keputusan yang trendi. Mulailah dari corpus kecil, metadata rapi, retrieval terukur, dan *embedding* yang cocok. Setelah itu, baru naikan kompleksitas. Itulah jalan yang lebih lambat sedikit, tetapi jauh lebih kokoh.

Bab 4. Teknik Membuat DOC untuk RAG Manual, Pakai Tangan, Tanpa Aplikasi

Ada satu kebiasaan buruk yang mulai makin umum di era AI: orang merasa menulis manual itu kuno. Padahal untuk RAG, justru **dokumen manual yang rapi sering jauh lebih bernilai** daripada dokumen panjang yang lahir dari salin-tempel, hasil ekstraksi mentah, atau ekspor acak dari berbagai aplikasi. Dokumen manual memaksa penulis berpikir jernih: apa inti topiknya, siapa pembacanya, istilah apa yang konsisten, informasi mana yang fakta, mana yang asumsi, mana langkah, mana penjelasan, dan mana konteks. Itulah yang dibutuhkan sistem retrieval. RAG tidak butuh teks yang panjang. RAG butuh teks yang **terstruktur, konsisten, dan mudah dipotong tanpa kehilangan makna**.

Di sinilah letak kekuatan teknik manual. Saat kita menulis sendiri, kita bisa mengendalikan bentuk pengetahuan sejak awal. Kita bisa menentukan satu dokumen hanya membahas satu prosedur. Kita bisa menuliskan judul yang eksplisit. Kita bisa memisahkan fakta dari opini. Kita bisa menambah metadata yang memang berguna untuk filtering dan audit. Dengan kata lain, kita tidak sedang “membuat file”, tetapi **merancang unit pengetahuan**. Bab ini mengajarkan keterampilan itu. Sederhana, tetapi sangat menentukan.

4.1 Mengapa menulis manual tetap penting di era AI

Banyak sistem RAG gagal bukan karena model terlalu kecil, tetapi karena dokumen yang dimasukkan terlalu buruk. File hasil ekspor sering membawa banyak sampah: header, footer, elemen dekoratif, nomor halaman, paragraf pecah, judul kabur, tabel tidak jelas, dan topik bercampur. Ketika dokumen seperti itu dipotong menjadi *chunk*, hasilnya menjadi lebih rusak lagi. Lalu sistem retrieval dipaksa mencari potongan yang relevan dari bahan yang dari awal sudah tidak sehat. Akibatnya, jawaban model tampak masuk akal, tetapi konteksnya salah. Itulah sebabnya **menulis manual bukan nostalgia; menulis manual adalah cara termurah untuk menjaga kualitas retrieval**.

Menulis manual juga penting karena ia memaksa disiplin editorial. Dalam dunia operasional, kita sering menemukan kasus seperti ini:

- satu file SOP berisi lima prosedur berbeda;
- satu dokumen teknis mencampur teori, konfigurasi, dan troubleshooting;
- satu catatan rapat dijadikan sumber kebenaran final;

- satu file README memuat potongan perintah tanpa penjelasan konteks.

Untuk manusia, dokumen seperti ini masih bisa dibaca pelan-pelan. Untuk RAG, dokumen seperti ini adalah jebakan. Model retrieval tidak “mengerti niat penulis”. Ia hanya melihat teks, struktur, metadata, dan kedekatan makna. Jadi, saat penulisannya buruk, retrieval ikut kehilangan arah.

Ada alasan lain yang lebih strategis. Menulis manual memberi kita **kendali atas istilah dan domain**. Di banyak organisasi, istilah lokal sangat penting. Nama sistem, singkatan internal, label keamanan, atau penamaan tahap operasional tidak selalu ada di dokumentasi publik. Kalau dokumen manual ditulis dengan disiplin, kita sedang membangun *vocabulary bridge* antara bahasa manusia di organisasi dan bahasa yang nanti dipakai query pengguna.

4.2 Format paling aman: Markdown + YAML *front matter*

Untuk buku ini, saya sangat menyarankan format dasar berikut: **Markdown** untuk isi dokumen, dan **YAML front matter** untuk metadata di bagian paling atas. Alasannya sederhana. Markdown mudah dibaca manusia, mudah diedit, cocok untuk judul, daftar, tabel sederhana, dan *code block*. YAML cocok untuk metadata seperti judul, sumber, tanggal, domain, bahasa, label, dan tingkat sensitivitas. Dokumentasi Markdown klasik menekankan bahwa sintaksnya dibuat agar terlihat seperti maknanya, misalnya *#* untuk judul, daftar yang benar-benar tampak seperti daftar, dan penekanan dengan tanda baca sederhana. YAML 1.2.2 sendiri mendefinisikan YAML sebagai bahasa serialisasi data yang manusiawi dan terbuka. ([Daring Fireball](#))

Struktur minimalnya seperti ini:

```
---
title: "Cara Reindex Knowledge Base Open WebUI"
source: "catatan internal"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "reviewed"
topic_labels: ["open-webui", "embedding", "reindex"]
---

# Cara Reindex Knowledge Base Open WebUI
```

Dokumen ini menjelaskan kapan **reindex** diperlukan, langkah dasarnya, dan risiko jika penggantian **embedding model** dilakukan tanpa membangun ulang index.

Format ini aman karena:

- **metadata ada di satu tempat;**
- isi dokumen tetap nyaman dibaca;
- mudah di parsing dengan skrip sederhana;
- mudah dibersihkan, dipecah, dan diuji;
- cocok untuk corpus manual, catatan teknis, SOP, dan FAQ.

How-to membuat dokumen Markdown + YAML dari nol

Langkah paling sederhana:

```
mkdir -p docs/manual
nano docs/manual/reindex-openwebui.md
```

Isi file:

```
---
title: "Cara Reindex Knowledge Base Open WebUI"
source: "lab-rag"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "draft"
topic_labels: ["open-webui", "knowledge", "reindex"]
---

# Cara Reindex Knowledge Base Open WebUI

## Tujuan
Menjelaskan kapan *reindex* diperlukan dan langkah umumnya.

## Ringkasan
Lakukan *reindex* bila model *embedding* berubah atau struktur *chunking* berubah besar.

## Langkah Dasar
1. Cadangkan dokumen sumber.
2. Catat model *embedding* lama dan baru.
3. Jalankan *reindex*.
4. Uji 20 query acuan.
```

Cek hasil:

```
sed -n '1,20p' docs/manual/reindex-openwebui.md
```


Contoh keluaran:

```
---
title: "Cara Reindex Knowledge Base Open WebUI"
source: "lab-rag"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "draft"
topic_labels: ["open-webui", "knowledge", "reindex"]
---
```

Kalau pembaca sudah bisa melakukan langkah ini, berarti ia sudah memegang fondasi penting untuk bab-bab berikutnya.

4.3 Golden Rule dokumen RAG-ready

Tidak semua dokumen yang enak dibaca manusia otomatis enak untuk retrieval. Karena itu, kita perlu Golden Rule. Bukan untuk membatasi kreativitas, tetapi untuk menjaga agar dokumen tetap sehat saat di-*chunk*, di-*embed*, dan dicari ulang.

Satu topik utama per bagian

Ini aturan nomor satu. **Jangan serakah**. Satu bagian harus punya satu fokus utama. Satu file boleh memiliki beberapa sub bagian, tetapi satu subbagian sebaiknya membahas satu ide yang jelas.

Buruk:

- “Panduan Open WebUI” yang di dalamnya membahas instalasi, troubleshooting Docker, prompt, chunking, embedding, dan FAQ sekaligus.

Lebih sehat:

- satu dokumen untuk instalasi;
- satu dokumen untuk *knowledge base*;
- satu dokumen untuk *reindex*;
- satu dokumen untuk troubleshooting.

Mengapa? Karena *chunking* akan bekerja lebih baik jika setiap bagian punya koherensi internal yang kuat.

Judul jelas

Judul harus memberi tahu isi, bukan sekadar terdengar keren.

Buruk:

- “Catatan penting”
- “Beberapa hal tentang sistem”
- “Panduan lanjutan”

Bagus:

- “Cara Menambahkan Dokumen ke Knowledge Base Open WebUI”
- “Troubleshooting Ollama Tidak Terdeteksi oleh Open WebUI”
- “Struktur Metadata Minimal untuk Corpus RAG”

Judul yang jelas sangat membantu retrieval berbasis semantik maupun *keyword*.

Paragraf pendek

Paragraf yang terlalu panjang sering menyulitkan pemotongan. Untuk corpus manual, saya sarankan paragraf pendek, padat, dan satu gagasan utama per paragraf. Ini membantu manusia dan mesin sekaligus.

Daftar tertib

Kalau ada langkah, syarat, atau komponen, gunakan daftar. Markdown mendukung daftar dengan sintaks sederhana, dan itu sangat membantu struktur isi. ([Panduan Markdown](#))

Tabel seperlunya

Tabel boleh dipakai, tetapi hanya bila benar-benar membantu. Jangan memaksa semua hal ke tabel. Untuk RAG, tabel yang terlalu padat kadang lebih sulit dibaca dan dipotong dibanding daftar atau subjudul yang rapi.

Code block utuh

Kalau ada perintah CLI, potongan YAML, JSON, atau Python, jangan campur di paragraf biasa. Markdown mendukung *fenced code blocks* dengan tiga backtick, dan ini cara yang paling aman agar struktur kode tetap utuh.

([Panduan Markdown](#))

Contoh:

```
## Menarik model di Ollama

```bash
ollama pull qwen3:8b
ollama list
```

Kalau *code block* dipotong di tengah atau ditempel acak dalam paragraf, retrieval bisa mengambil potongan yang tidak utuh dan sangat menyesatkan.

```
4.4 Metadata minimal
```

Metadata adalah penjelasan singkat tentang dokumen. Ia bukan pajangan. Ia harus berguna untuk:

- filtering;
- audit;
- pemeringkatan relevansi;
- penelusuran sumber;
- pengelolaan versi.

Untuk buku ini, metadata minimal yang saya sarankan adalah:

- **judul**  
identitas utama dokumen.
- **sumber**  
asal dokumen: catatan internal, SOP resmi, materi kuliah, hasil observasi, dokumentasi proyek.
- **tanggal**  
kapan dokumen dibuat atau diperbarui.
- **domain**  
misalnya: `rag`, `security`, `network`, `python`, `devops`.
- **bahasa**  
`id`, `en`, atau kombinasi.
- **tingkat sensitivitas**  
`public`, `internal`, `restricted`.
- **kualitas**

```
`draft`, `reviewed`, `approved`.
```

- **label topik**  
daftar topik atau istilah penting.

Contoh:

```

title: "Template SOP Upload Dokumen ke Knowledge Base"
source: "tim-ai-lokal"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "approved"
topic_labels: ["sop", "knowledge-base", "open-webui",
"upload"]

```

## How-to memeriksa metadata secara cepat

Buat aturan sederhana:

- semua dokumen manual wajib punya delapan metadata ini;
- kalau satu item kosong, dokumen belum boleh di-*ingest*.

Cek cepat dengan **grep**:

```
grep -E
'^(title|source|date|domain|language|sensitivity|quality|t
opic_labels):' docs/manual/*.md
```

Contoh keluaran:

```
docs/manual/reindex-openwebui.md:title: "Cara Reindex
Knowledge Base Open WebUI"
docs/manual/reindex-openwebui.md:source: "lab-rag"
docs/manual/reindex-openwebui.md:date: "2026-04-13"
...
```

## 4.5 Template dokumen manual

Salah satu cara tercepat membangun corpus yang sehat adalah **menggunakan template**. Template menjaga konsistensi. Kita tidak mulai dari kertas kosong setiap kali.

### a. *Knowledge note*

Dokumen singkat untuk satu konsep atau satu pengetahuan inti.

```

title: "Apa itu Focused Retrieval di Open WebUI"
source: "catatan belajar"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "public"
quality: "reviewed"
topic_labels: ["focused-retrieval", "open-webui", "rag"]

Apa itu Focused Retrieval di Open WebUI

Definisi
Focused Retrieval adalah mode pencarian yang mengambil
chunk paling relevan dari koleksi dokumen.

Kapan dipakai
Gunakan untuk corpus besar yang tidak perlu dimasukkan
utuh ke konteks.

Risiko
Kalau dokumen berantakan, *chunk* yang diambil bisa
meleset.
```

## b. SOP

Untuk langkah operasional yang perlu konsisten.

```

title: "SOP Menambahkan Dokumen Baru ke Corpus"
source: "tim-rag"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "approved"
topic_labels: ["sop", "corpus", "ingest"]

SOP Menambahkan Dokumen Baru ke Corpus

Tujuan
Menjaga agar dokumen yang masuk tetap bersih dan terlabel.

Langkah
1. Cek format dokumen.
2. Tambahkan metadata minimum.
3. Pastikan satu topik utama per bagian.
4. Simpan ke folder `cleaned/`.
5. Lakukan uji kecil sebelum ingest massal.
```

## c. FAQ

Sangat bagus untuk retrieval karena pertanyaan dan jawaban sudah eksplisit.

```

title: "FAQ Open WebUI Knowledge Base"
source: "lab-rag"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "public"
quality: "reviewed"
topic_labels: ["faq", "knowledge-base", "open-webui"]

FAQ Open WebUI Knowledge Base

Pertanyaan
Kapan saya perlu *reindex*?

Jawaban
Saat model *embedding* berubah atau struktur dokumen
berubah signifikan.
```

#### d. Catatan teknis

Cocok untuk pengamatan, hasil uji, dan keputusan teknis.

#### e. Tutorial singkat

Cocok untuk *how-to* yang satu tujuan, satu hasil.

Template penting karena ia membuat corpus lebih seragam. Semakin seragam bentuk dasar, semakin mudah retrieval, filtering, dan evaluasi dilakukan.

## 4.6 Contoh dokumen buruk vs dokumen siap RAG

Mari kita lihat kontrasnya.

### Dokumen buruk

Catatan sistem

Kemarin saya coba beberapa hal untuk openwebui dan kayaknya jalan sih tapi masih ada masalah. Mungkin karena docker. Atau mungkin modelnya. Saya juga sempat lihat tentang knowledge dan file manager. Lalu saya coba upload PDF yang banyak, ada yang masuk ada yang tidak. Sepertinya harus diulang. Pokoknya nanti dicek lagi.

Masalahnya:

- judul tidak jelas;
- tidak ada metadata;
- banyak asumsi dan ketidakpastian;
- topik bercampur;
- tidak ada struktur;
- tidak ada langkah atau kesimpulan tegas.

## Dokumen siap RAG

```

title: "Catatan Uji Upload PDF ke Open WebUI"
source: "lab-rag"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "draft"
topic_labels: ["open-webui", "pdf", "upload-test"]

Catatan Uji Upload PDF ke Open WebUI

Tujuan
Menguji apakah dokumen PDF dapat diunggah dan dipakai
dalam *knowledge base*.

Hasil Utama
- PDF teks berhasil diunggah.
- PDF scan membutuhkan penanganan tambahan.
- Beberapa file besar perlu diuji ulang dengan corpus
 lebih kecil.

Catatan Masalah
- Upload berhasil, tetapi retrieval belum selalu relevan.
- Dugaan awal: struktur PDF dan *chunking* perlu
 diperiksa.

Tindak Lanjut
1. Pisahkan PDF teks dan PDF scan.
2. Bersihkan file besar sebelum ingest ulang.
3. Uji 10 query pembandingan.
```

Perbedaannya sangat terasa. Dokumen kedua lebih siap untuk manusia, lebih siap untuk retrieval, dan lebih siap untuk audit.



## 4.7 Teknik memberi penanda penting langsung di dokumen

Selain metadata di kepala dokumen, kita juga bisa memberi penanda langsung di isi. Ini membantu pembaca manusia dan juga membantu retrieval.

Beberapa teknik yang sehat:

- gunakan subjudul yang tegas seperti `## Tujuan, ## Risiko, ## Langkah, ## Catatan;`
- tandai bagian penting dengan label eksplisit, misalnya `## Risiko, ## Batasan, ## Keputusan;`
- gunakan daftar bernomor untuk prosedur;
- gunakan istilah yang konsisten, jangan berganti-ganti sebutan untuk hal yang sama;
- tandai konteks CLI, konfigurasi, dan output secara terpisah.

Contoh:

```
Risiko
Jika model *embedding* diganti tanpa *reindex*, hasil retrieval
dapat memburuk.

Langkah CLI
```bash
ollama list
docker ps

## Output yang Diharapkan:
NAME          ID          SIZE
qwen3:8b      1a2b3c4d5e6f 5.2 GB
```

Teknik ini sederhana, tetapi sangat kuat. Ia membantu *chunk* tetap bermakna ketika dipotong. Ia juga membantu *reranker* atau pembaca manusia memahami peran tiap potongan.

```
#### *How-to* memberi penanda secara konsisten
Buat kamus kecil untuk proyek Anda. Misalnya selalu pakai:
- `## Tujuan`
- `## Definisi`
- `## Langkah`
- `## Risiko`
- `## Troubleshooting`
- `## Referensi`
```

Jangan hari ini menulis `Catatan`, besok `Perhatian`, lusa `Hal penting`, padahal maksudnya sama. Konsistensi istilah adalah hadiah untuk retrieval.

```
### 4.8 Latihan: menulis 5 dokumen kecil siap *ingest*
```

Latihan ini sangat penting. Jangan hanya membaca bab ini. ****Kerjakan.**** Karena keterampilan corpus lahir dari praktik.

Target latihan: tulis ****5 dokumen kecil****, masing-masing 150–300 kata, siap masuk corpus.

```
#### Dokumen 1 – *Knowledge note*
```

```
Tema: "Apa itu *embedding* dalam RAG?"
```

```
#### Dokumen 2 – SOP
```

```
Tema: "Langkah menambahkan dokumen baru ke folder `cleaned/`."
```

```
#### Dokumen 3 – FAQ
```

```
Tema: "Kapan saya harus *reindex*?"
```

```
#### Dokumen 4 – Catatan teknis
```

```
Tema: "Masalah umum Open WebUI tidak terhubung ke Ollama."
```

```
#### Dokumen 5 – Tutorial singkat
```

```
Tema: "Cara menarik model di Ollama."
```

```
#### *Step by step* latihan
```

- buat folder latihan;
- buat lima file Markdown;
- isi semua metadata minimum;
- pakai template yang sesuai;
- cek apakah satu dokumen hanya punya satu fokus utama;
- simpan semua kode dalam **code block**;
- baca ulang dan potong paragraf yang terlalu panjang.

Perintah awal:

```
mkdir -p latihan-bab4
cd latihan-bab4
touch 01-embedding.md 02-sop-ingest.md 03-faq-reindex.md
04-catatan-koneksi.md 05-tutorial-ollama.md
ls -l
```

Contoh keluaran:

```
01-embedding.md
02-sop-ingest.md
03-faq-reindex.md
04-catatan-koneksi.md
05-tutorial-ollama.md
```

Contoh isi singkat untuk `05-tutorial-ollama.md`:

```
---
title: "Cara Menarik Model di Ollama"
source: "latihan-bab4"
date: "2026-04-13"
domain: "llm"
language: "id"
sensitivity: "public"
quality: "draft"
topic_labels: ["ollama", "model", "pull"]
---

# Cara Menarik Model di Ollama

## Tujuan
Menjelaskan langkah dasar untuk mengambil model lokal.

## Langkah
```bash
ollama pull qwen3:8b
ollama list

Output yang Diharapkan
pulling manifest
success

Catatan
Pastikan layanan Ollama aktif sebelum menjalankan perintah ini.
```

Cek semua file:

```
grep -n
'^title:|^source:|^date:|^domain:|^language:|^sensitivity:|^quality:|^topic_labels:' *.md
```

Contoh keluaran:

```
01-embedding.md:2:title: "Apa itu Embedding dalam RAG"
01-embedding.md:3:source: "latihan-bab4"
...
```

Kalau pembaca bisa menyelesaikan latihan ini, ia bukan lagi sekadar memahami teori corpus. Ia sudah mulai **membangun corpus dengan tangannya sendiri**.

## Penutup Bab

Bab ini mungkin tampak sederhana, tetapi sesungguhnya ia sangat strategis. **RAG yang baik dimulai dari dokumen yang ditulis dengan niat yang jelas.** Markdown memberi tubuh yang ringan dan mudah dibaca. YAML memberi kepala metadata yang teratur. Struktur yang disiplin membuat isi mudah dicari, mudah dipotong, dan mudah diuji. Template membuat corpus konsisten. Penanda internal membuat retrieval lebih tajam. Dan latihan manual melatih penulis untuk berpikir sebagai perancang pengetahuan, bukan sekadar pengumpul file. ([Open WebUI](#))

Dengan kata lain, sebelum AI menjadi pintar, manusianya harus terlebih dahulu **tertib menulis**. Itulah fondasi corpus yang sehat.

## Bab 5. Mengubah Textbook PDF Menjadi DOC untuk RAG

Ada kesalahan yang sangat umum di dunia RAG: orang mengira PDF adalah pengetahuan. Padahal PDF hanyalah **wadah**. Ia bisa berisi teks yang rapi, bisa juga berisi gambar hasil scan, bisa pula campuran teks, tabel, gambar, dan tata letak yang rumit. Untuk manusia, semua itu masih terlihat “seperti buku”. Untuk sistem retrieval, semua itu bisa berubah menjadi kekacauan yang halus: paragraf putus di tengah, judul hilang, tabel remuk, catatan kaki menempel ke paragraf utama, dan nomor halaman ikut terbawa seolah bagian dari isi. Karena itu, pekerjaan kita bukan “menggunggah PDF”, melainkan **mengkonversi PDF menjadi dokumen kerja yang bisa dipercaya**.

Bab ini mengajarkan satu keterampilan inti: mengubah textbook PDF menjadi **DOC bersih untuk RAG**. Kata “DOC” di sini tidak harus berarti format `.docx`. Dalam konteks buku ini, DOC berarti **dokumen kerja yang siap di-ingest**: bisa berupa Markdown, JSON, atau TXT yang sudah dibersihkan, diberi struktur, dan siap dipecah lebih lanjut.

### 5.1 Bedakan PDF teks, PDF scan, dan PDF campuran

Langkah pertama sebelum ekstraksi adalah **mendiagnosis jenis PDF**. Ini sangat penting, karena strategi penanganannya berbeda.

**PDF teks** adalah PDF yang memang berisi lapisan teks digital. Biasanya berasal dari aplikasi pengolah kata, sistem penerbitan, atau ekspor digital langsung. Jenis ini paling ramah untuk ekstraksi karena huruf-hurufnya masih ada sebagai teks, bukan gambar.

**PDF scan** adalah PDF yang pada dasarnya kumpulan gambar halaman. Secara visual ia tampak seperti buku, tetapi bagi program ekstraksi biasa, ia lebih mirip foto halaman. Pada jenis ini, kalau tidak ada lapisan OCR, hasil ekstraksi teks bisa kosong atau buruk sekali.

**PDF campuran** adalah bentuk yang paling sering bikin pusing. Beberapa halaman masih teks, beberapa halaman adalah scan, sebagian tabel masih bisa dibaca, sebagian lain hancur, ada catatan kaki, gambar, dua kolom, atau lampiran yang struktur layout-nya berbeda dari isi utama.

PyMuPDF4LLM memang mendukung **automatic detection of pages which profit from OCR** dan mendukung berbagai *OCR engines*, tetapi itu justru mengajarkan satu hal penting: **jangan memperlakukan semua PDF sama**. Bedakan dulu jenisnya, lalu baru tentukan jalur ekstraksinya. ([PyMuPDF](#))

## How-to mengenali PDF secara cepat

Cara cepat paling praktis untuk mahasiswa atau praktisi adalah gabungan tiga cek sederhana:

- buka PDF dan coba blok teks dengan mouse;
- jalankan ekstraksi teks cepat;
- lihat apakah hasilnya masuk akal atau kosong.

Contoh cek cepat dengan Python dan PyMuPDF:

```
python - <<'PY'
import fitz

pdf_path = "textbook.pdf"
doc = fitz.open(pdf_path)

for i in range(min(3, len(doc))):
 text = doc[i].get_text("text")
 print(f"--- PAGE {i+1} ---")
 print(text[:500].strip() or "[NO TEXT EXTRACTED]")
PY
```

Contoh output untuk PDF teks:

```
--- PAGE 1 ---
Chapter 1 Introduction to Retrieval-Augmented Generation
Retrieval-augmented generation (RAG) combines...
```

Contoh output untuk PDF scan:

```
--- PAGE 1 ---
[NO TEXT EXTRACTED]
```

Kalau hasil seperti ini keluar, Anda sudah punya sinyal kuat bahwa PDF tersebut perlu jalur OCR atau perlakuan khusus.

## 5.2 Alur kerja: PDF → Markdown / JSON / TXT → DOC bersih

Jangan langsung berpikir bahwa tujuan akhirnya adalah “mengambil semua isi”. Tujuan kita lebih spesifik: **mengambil isi yang benar, mempertahankan struktur seperlunya, lalu membersihkannya menjadi unit pengetahuan yang sehat.**

Alur kerja yang saya sarankan untuk buku ini adalah:

- **PDF mentah**
- ekstraksi ke **Markdown / JSON / TXT**
- pembersihan hasil ekstraksi

- pemulihan struktur bab dan subbab
- pemisahan dokumen besar menjadi unit yang lebih sehat
- validasi manual
- baru kemudian masuk ke Open WebUI atau pipeline RAG lain

Kenapa urutannya begitu? Karena **Markdown / JSON / TXT** adalah bentuk kerja yang jauh lebih mudah dibersihkan daripada PDF langsung. Dokumentasi PyMuPDF4LLM memang mendukung ketiga bentuk itu, sehingga kita bisa memilih output sesuai tujuan. Untuk textbook, saya paling menyukai **Markdown** sebagai format kerja utama karena struktur heading, daftar, tabel sederhana, dan *code block* lebih mudah dipulihkan dan dibaca manusia. JSON lebih cocok bila kita ingin memproses blok secara lebih programatik. TXT berguna untuk inspeksi kasar, tetapi paling miskin struktur. ([PyMuPDF](#))

## How-to memilih format output

Gunakan aturan sederhana ini:

- **Markdown** untuk alur editorial dan pembersihan manual
- **JSON** untuk eksperimen blok, koordinat, atau pemrosesan lanjut
- **TXT** untuk inspeksi cepat dan debugging awal

Kalau target Anda adalah textbook yang akan menjadi corpus RAG, **mulailah dari Markdown**.

## 5.3 Ekstraksi dengan PyMuPDF4LLM

Sekarang kita masuk ke jantung pekerjaan teknis. Dokumentasi resmi menyebut bahwa PyMuPDF4LLM membaca halaman file dan mengeluarkan teks dalam **Markdown**, serta mendukung **page chunks**. Ini membuatnya ideal sebagai jembatan awal dari PDF ke bentuk kerja yang bisa diproses lebih lanjut. ([PyMuPDF](#))

### Instalasi dasar

Di lingkungan virtual proyek:

```
pip install pymupdf pymupdf4llm
```

Contoh cek:

```
python - <<'PY'
import fitz
import pymupdf4llm
print("PyMuPDF OK")
print("PyMuPDF4LLM OK")
PY
```

Contoh output:

```
PyMuPDF OK
PyMuPDF4LLM OK
```

## Ekstraksi sederhana ke Markdown

Contoh paling dasar:

```
python - <<'PY'
import pymupdf4llm

pdf_path = "textbook.pdf"
md_text = pymupdf4llm.to_markdown(pdf_path)

with open("output/textbook.md", "w", encoding="utf-8") as
f:
 f.write(md_text)

print("Saved to output/textbook.md")
print(md_text[:800])
PY
```

Contoh output:

```
Saved to output/textbook.md
Chapter 1
Introduction to Retrieval-Augmented Generation
Retrieval-augmented generation (RAG) ...
```

Pada banyak kasus, hasil awal ini sudah memberi keuntungan besar:

- heading mulai muncul;
- urutan baca lebih masuk akal;
- beberapa tabel ikut terbawa dalam bentuk yang masih bisa diselamatkan.

Dokumentasi PyMuPDF4LLM juga menekankan dukungan **layout analysis**, **multi-column pages**, serta **image and vector graphics extraction**. Artinya, ia memang dirancang bukan sekadar untuk menumpuk teks mentah, melainkan untuk membantu pemulihan struktur dokumen. ([PyMuPDF](#))

## Ekstraksi per halaman atau *page chunks*

Untuk textbook besar, sering kali lebih sehat menyimpan hasil per halaman atau per bagian, bukan satu file raksasa. Dokumentasi PyMuPDF4LLM API memang menyebut dukungan **page chunks from the Markdown text**. Ini sangat berguna bila kita ingin melakukan pembersihan bertahap. ([PyMuPDF](#))



## How-to kerja yang disarankan

- ekstrak dulu ke satu Markdown utuh;
- simpan salinan mentah;
- baru buat versi kerja yang dibersihkan;
- jangan pernah mengedit satu-satunya hasil ekstraksi mentah.

Struktur folder yang sehat:

```
mkdir -p output/{raw,cleaned,logs}
cp output/textbook.md output/raw/textbook.raw.md
cp output/textbook.md output/cleaned/textbook.work.md
```

## 5.4 Membersihkan hasil ekstraksi

Di sinilah pekerjaan editor corpus benar-benar dimulai. Hasil ekstraksi awal hampir tidak pernah langsung siap pakai. Kita harus membersihkannya dengan disiplin.

### Header/footer

Header dan footer sering ikut terbawa: judul buku, nama penulis, nama bab, atau nama penerbit yang berulang di setiap halaman. Untuk manusia, ini mudah diabaikan. Untuk retrieval, ini menjadi noise yang sangat merusak karena berulang berkali-kali.

### Nomor halaman

Nomor halaman yang ikut menjadi teks dapat masuk ke *chunk* dan mengganggu retrieval. Terutama bila angka-angka itu berdiri sendiri di antara paragraf.

### Catatan kaki

Catatan kaki sering menempel ke paragraf utama, membuat alur bacaan rusak. Kadang catatan kaki memang penting, tetapi sering kali ia harus dipisahkan atau ditandai jelas.

### Tabel rusak

Dokumentasi PyMuPDF memang punya dukungan tabel dan konversi menuju *markdown-formatted tables*, tetapi hasil tabel dari PDF nyata tetap perlu diperiksa manual. Beberapa tabel akan rapi, sebagian lagi pecah kolomnya atau kehilangan konteks header. ([PyMuPDF](#))

## Pecahan paragraf

Ini salah satu penyakit paling umum. Satu paragraf pecah menjadi beberapa baris pendek hanya karena layout halaman. Kalau tidak diperbaiki, retrieval akan menghasilkan *chunk* yang miskin kohesi.

## Caption gambar

*Caption* bisa sangat berguna, tetapi harus jelas apakah ia bagian dari penjelasan utama atau hanya penanda ilustrasi. Jangan biarkan caption menempel liar ke paragraf di atas atau di bawahnya.

## How-to membersihkan cepat dengan skrip sederhana

Contoh skrip pembersihan awal Markdown:

```
python - <<'PY'
import re
from pathlib import Path

src = Path("output/raw/textbook.raw.md")
dst = Path("output/cleaned/textbook.clean.md")

text = src.read_text(encoding="utf-8")

buang nomor halaman yang berdiri sendiri
text = re.sub(r'(?m)^\s*\d+\s*$', '', text)

rapikan baris kosong berlebihan
text = re.sub(r'\n{3,}', '\n\n', text)

gabungkan baris yang pecah secara kasar
text = re.sub(r'(?<\n)\n(?:[#\-*\d])', ' ', text)

rapikan spasi
text = re.sub(r'[\t]{2,}', ' ', text)

dst.write_text(text.strip() + "\n", encoding="utf-8")
print(f"Saved cleaned file to: {dst}")
PY
```

Contoh output:

```
Saved cleaned file to: output/cleaned/textbook.clean.md
```

Ini belum sempurna, tetapi cukup kuat sebagai langkah pertama. Setelah itu tetap perlu pemeriksaan manusia.

## 5.5 Menjaga struktur bab, subbab, referensi, dan daftar isi

Salah satu kesalahan paling mahal adalah terlalu agresif membersihkan sampai struktur bukunya hilang. Untuk textbook, struktur justru sangat penting karena:

- membantu *chunking* berbasis heading;
- menjaga konteks semantik;
- memudahkan navigasi manusia;
- memperjelas batas topik.

Karena itu, saat membersihkan, pertahankan:

- **judul bab**
- **subjudul**
- **subsubjudul bila relevan**
- **bagian referensi**
- **daftar isi bila berguna untuk navigasi**
- **daftar istilah atau lampiran bila memang nanti dipakai**

Markdown sangat membantu di sini karena heading bisa ditandai jelas dengan #, ##, ###. Ini sejalan dengan filosofi Markdown yang dibuat agar struktur tulisan mudah dibaca dan mudah diolah. ([PyMuPDF](#))

### How-to memulihkan heading

Bila hasil ekstraksi sudah mengenali heading, pertahankan. Bila belum, Anda bisa memulihkannya manual.

Contoh sebelum:

```
Chapter 3 Embedding Strategies
Choosing an embedding model
```

Contoh sesudah:

```
Chapter 3 Embedding Strategies
Choosing an Embedding Model
```

Kalau textbook punya daftar isi yang bagus, saya sarankan **jangan langsung dibuang**. Simpan minimal satu versi referensi karena ia sangat membantu saat memecah dokumen besar menjadi bagian-bagian yang lebih sehat.

## 5.6 Memecah textbook menjadi dokumen yang sehat untuk retrieval

Ini salah satu keputusan paling penting. **Satu textbook utuh jarang sehat bila diperlakukan sebagai satu dokumen tunggal untuk retrieval.** Untuk RAG, textbook besar sebaiknya dipecah menjadi unit yang masuk akal.

Aturan sehatnya:

- satu bab besar bisa menjadi satu dokumen induk;
- sub bab penting bisa menjadi dokumen turunan;
- bagian referensi bisa dipisah;
- lampiran atau glosarium bisa dipisah bila sering dicari sebagai entitas sendiri.

Dokumentasi PyMuPDF tentang RAG menegaskan bahwa *chunking* penting untuk memberi konteks yang tepat pada data LLM, dan dukungan Markdown membantu *level 3 chunking*. Jadi, memecah textbook setelah struktur heading dipulihkan adalah keputusan yang sangat masuk akal. ([PyMuPDF](#))

### How-to memecah berdasarkan heading utama

Contoh skrip sederhana:

```
python - <<'PY'
from pathlib import Path
import re

src = Path("output/cleaned/textbook.clean.md")
text = src.read_text(encoding="utf-8")

parts = re.split(r'(?m)^# ', text)
outdir = Path("output/chapters")
outdir.mkdir(parents=True, exist_ok=True)

count = 0
for part in parts:
 part = part.strip()
 if not part:
 continue
 lines = part.splitlines()
 title = lines[0].strip()
 slug = re.sub(r'^a-zA-Z0-9+', '-',
title.lower()).strip('-')[:60]
 content = "# " + part + "\n"
 (outdir /
f"{count+1:02d}-{slug}.md").write_text(content,
encoding="utf-8")
 count += 1

print(f"Created {count} chapter files.")
PY
```

Contoh output:

```
Created 8 chapter files.
```

Setelah itu, tiap bab masih bisa dipecah lagi ke level subbab jika terlalu panjang.

## 5.7 Validasi manual sebelum masuk Open WebUI

Ini tahap yang tidak boleh dilewatkan. Open WebUI memang membuat dokumen menjadi **searchable by AI**, dengan **Focused Retrieval** untuk koleksi besar dan **Full Context** untuk dokumen yang perlu dimasukkan utuh. Tetapi justru karena Open WebUI bekerja di atas dokumen itu, validasi manual tetap wajib. **RAG yang jelek sering bukan salah model; sering itu salah konteks.** Dokumentasi troubleshooting Open WebUI bahkan menyebutkan secara eksplisit bahwa ketika model tampak “halusinasi” atau melewati informasi relevan, itu sering bukan salah model, melainkan masalah konteks. ([Open WebUI](#))

Checklist validasi manual yang saya sarankan:

- apakah judul bab masih benar;
- apakah subbab masih utuh;
- apakah paragraf tidak pecah aneh;
- apakah nomor halaman sudah dibuang;
- apakah catatan kaki sudah dipisahkan atau diberi bentuk jelas;
- apakah tabel penting masih bisa dibaca;
- apakah *caption* gambar tidak menempel sembarangan;
- apakah istilah domain masih konsisten;
- apakah dokumen terlalu panjang dan perlu dipecah lagi.

### How-to validasi cepat

- buka file Markdown hasil bersih;
- baca awal, tengah, dan akhir;
- lakukan pencarian untuk:
  - angka halaman berdiri sendiri
  - baris sangat pendek yang aneh
  - heading yang hilang
  - kata terpotong
- uji satu atau dua query manual di Open WebUI setelah upload.

Kalau untuk textbook besar, saya sarankan validasi **per bab**, bukan menunggu satu buku selesai total.

## 5.8 Studi kasus: satu bab buku menjadi corpus siap pakai

Mari kita buat alur konkret. Misalnya kita punya satu bab textbook PDF tentang *embedding strategies*. Targetnya: satu bab ini berubah menjadi corpus kecil yang siap dipakai untuk retrieval.

### Tahap 1. Ekstrak PDF ke Markdown

```
python - <<'PY'
import pymupdf4llm
from pathlib import Path

pdf_path = "book.pdf"
out = Path("work/book.md")
out.parent.mkdir(parents=True, exist_ok=True)

md = pymupdf4llm.to_markdown(pdf_path)
out.write_text(md, encoding="utf-8")
print(f"Saved: {out}")
PY
```

Contoh output:

```
Saved: work/book.md
```

### Tahap 2. Bersihkan hasil ekstraksi

```
python clean_markdown.py
```

Contoh output:

```
Input : work/book.md
Output : work/book.clean.md
Status : OK
```

### Tahap 3. Ambil hanya bab yang dibutuhkan

```
grep -n "^# " work/book.clean.md
```

Contoh output:

```
12:# Chapter 1 Introduction
88:# Chapter 2 Vector Stores
154:# Chapter 3 Embedding Strategies
241:# Chapter 4 Evaluation
```

Lalu potong Bab 3 menjadi file sendiri.

### Tahap 4. Tambahkan metadata YAML

```

title: "Embedding Strategies"
source: "textbook-rag-2026"
date: "2026-04-13"
```

```

domain: "rag"
language: "en"
sensitivity: "public"
quality: "reviewed"
topic_labels: ["embedding", "retrieval", "rag"]

Chapter 3 Embedding Strategies
...

```

## Tahap 5. Rapikan isi

- buang nomor halaman
- rapikan paragraf
- periksa subjudul
- periksa tabel
- tandai bagian definisi, contoh, dan catatan penting

## Tahap 6. Simpan ke folder corpus bersih

```

mkdir -p ./data/cleaned/textbook
cp work/chapter-03-embedding-strategies.md
./data/cleaned/textbook/
ls -l ./data/cleaned/textbook

```

Contoh output:

```
chapter-03-embedding-strategies.md
```

## Tahap 7. Uji retrieval kecil

Gunakan query seperti:

- “apa perbedaan embedding model untuk multilingual retrieval?”
- “kapan hybrid retrieval lebih baik?”
- “apa hubungan embedding dan vector store?”

Kalau jawaban yang muncul sudah mengambil bagian yang tepat dari bab itu, berarti textbook tidak lagi hanya menjadi PDF mati. Ia sudah berubah menjadi **pengetahuan yang bisa dicari**.

## Penutup Bab

Bab ini mengajarkan satu pelajaran yang sangat penting untuk seluruh buku: **PDF bukan corpus**. PDF hanyalah bahan mentah. Yang membuatnya berguna untuk RAG adalah proses konversi, pembersihan, pemulihan struktur, pemecahan yang sehat, dan validasi manual. PyMuPDF4LLM memberi jembatan yang sangat kuat karena mendukung output **Markdown, JSON, dan TXT**, mendukung **layout analysis, multi-column pages**, dan **page chunking**, sehingga ia sangat cocok sebagai alat awal dalam alur kerja textbook ke corpus. Tetapi, seperti seluruh pesan

besar buku ini, tool yang baik tetap tidak menggantikan disiplin editorial manusia.  
([PyMuPDF](#))

Setelah bab ini, pembaca seharusnya tidak lagi berpikir “bagaimana cara upload PDF”, melainkan **bagaimana cara mengubah PDF menjadi dokumen sehat yang layak dipercaya oleh retrieval**. Itu lompatan mental yang sangat besar.



## Bab 6. Teknik Melabeli Informasi / Knowledge di Dalam DOC

Banyak orang masuk ke dunia RAG dengan bayangan bahwa model yang lebih besar akan otomatis memberi jawaban yang lebih baik. Dalam praktik, itu sering salah arah. Model besar bisa tetap tersesat kalau corpus tidak diberi penanda yang benar. Sebaliknya, model yang lebih kecil sering bisa bekerja sangat baik bila dokumen ditulis rapi, dipotong sehat, dan **dilabeli dengan disiplin**. Itulah sebabnya bab ini penting. Kita tidak sedang membahas “aksesoris metadata”, tetapi **alat untuk membuat pengetahuan bisa dicari dengan lebih tepat, difilter dengan lebih cerdas, dan diaudit dengan lebih jujur**.

### 6.1 Mengapa label sering lebih berharga daripada model yang lebih besar

Ada alasan sederhana. *Embedding* menangkap **kedekatan makna**, tetapi tidak selalu menangkap semua hal yang penting untuk retrieval. Misalnya:

- dokumen mana yang paling baru,
- mana yang resmi dan mana yang draft,
- mana yang sensitif,
- mana yang berasal dari SOP final,
- mana yang hanya catatan eksperimen,
- mana yang tentang *attack*, mana yang *non-attack*.

Qdrant menjelaskan ini secara gamblang: filtering dibutuhkan ketika tidak semua fitur objek dapat diekspresikan dalam embedding. Chroma juga menyediakan filter metadata langsung di query. Dari sini pelajarannya jelas: **label dan metadata sering menjadi pembeda antara retrieval yang sekadar mirip makna dan retrieval yang benar-benar tepat guna**. ([Qdrant](#))

Bahan pendukung Anda juga memberi pesan serupa. Devlin menekankan praktik menyimpan metadata seperti **title, section, source, timestamp** bersama embedding, lalu mengevaluasi retrieval sebelum generation. Raieli menulis bahwa metadata memungkinkan pencarian yang lebih granular dan memberi konteks yang lebih padat daripada bila informasi itu tersebar samar di seluruh dokumen. Dengan kata lain, **label adalah cara kita mengemas makna operasional yang tidak selalu tampak di teks mentah**.

Jadi, bila seorang pembaca bertanya, “Mana yang lebih penting dulu, ganti model atau rapikan label?”, jawaban saya biasanya ini: **rapikan label dulu**. Model besar yang buta konteks tetap bisa ngawur. Model sedang dengan corpus yang sehat sering jauh lebih berguna.

## 6.2 Bedakan: metadata, tag, label, kategori, skema domain

Istilah-istilah ini sering dipakai campur-aduk. Untuk buku ini, kita perlu disiplin.

**Metadata** adalah informasi terstruktur tentang dokumen atau *chunk*. Ia menjawab pertanyaan seperti: judulnya apa, sumbernya dari mana, tanggalnya kapan, bahasanya apa, statusnya draft atau final, sensitif atau publik. Metadata biasanya disimpan sebagai pasangan *key-value*.

**Tag** adalah penanda ringan, biasanya berupa kata atau frasa pendek. Tag berguna untuk asosiasi cepat. Misalnya `open-webui`, `ollama`, `reindex`, `security`.

**Label** lebih operasional daripada tag. Label biasanya dipakai untuk keputusan atau klasifikasi yang ingin kita pakai dalam retrieval dan evaluasi. Misalnya `ATTACK`, `NON-ATTACK`, `SUSPICIOUS`, `approved`, `deprecated`, `high-priority`.

**Kategori** adalah kelompok tingkat lebih tinggi. Ia membantu pengelompokan besar seperti `knowledge-note`, `SOP`, `FAQ`, `log`, `programming`, `tutorial`.

**Skema domain** adalah aturan keseluruhan tentang label apa saja yang kita pakai dalam satu domain tertentu. Misalnya untuk log keamanan, kita punya medan `severity`, `event_type`, `outcome`, `confidence`, `source_system`. Untuk dokumen pemrograman, kita mungkin punya `language`, `repo`, `component`, `api_surface`, `stability`.

Kalau dibikin singkat:

- metadata = informasi struktur,
- tag = penanda ringan,
- label = keputusan klasifikasi,
- kategori = kelompok besar,
- skema domain = tata bahasa labeling untuk satu dunia data.

Ini penting, karena tanpa pembedaan ini orang cenderung memasukkan semuanya ke `tags`, lalu retrieval kehilangan ketegasan.

## Contoh sederhana

Dokumen:  
"Cara Reindex Knowledge Base Open WebUI"

Bisa punya struktur seperti ini:

```

title: "Cara Reindex Knowledge Base Open WebUI"
source: "lab-rag"
date: "2026-04-13"
category: "SOP"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "approved"
tags: ["open-webui", "embedding", "reindex"]
labels:
 status: "final"
 priority: "high"
 confidence: "high"
 freshness: "current"

```

Di sini:

- **title, source, date, language** = metadata
- **open-webui, embedding, reindex** = tag
- **status, priority, confidence, freshness** = label
- **SOP** = kategori
- seluruh kombinasi itu = bagian dari skema domain dokumen RAG internal

## 6.3 Label minimum yang saya rekomendasikan

Untuk buku ini, saya sarankan **label minimum** yang sederhana tetapi kuat. Jangan terlalu miskin, tetapi juga jangan terlalu ramai.

### **Pertama, topik.**

Ini menjawab dokumen ini secara utama tentang apa. Misalnya `rag`, `embedding`, `vector-db`, `security`, `python`.

### **Kedua, subtopik.**

Ini memperjelas fokus yang lebih sempit. Misalnya `reindex`, `focused-retrieval`, `metadata-filtering`, `log-parsing`.

### **Ketiga, entitas.**

Ini menangkap nama sistem, nama tool, nama layanan, nama komponen, atau istilah domain penting. Misalnya `Open WebUI`, `Ollama`, `Chroma`, `Qdrant`, `Docker`.

### **Keempat, tingkat kepercayaan.**

Apakah isi dokumen ini sudah diverifikasi, masih dugaan, hasil uji awal, atau final. Ini sangat penting untuk menghindari retrieval yang terlalu percaya diri terhadap catatan yang belum matang.

### **Kelima, sensitivitas.**

Minimal bedakan `public`, `internal`, `restricted`. Untuk banyak sistem nyata, ini bukan pilihan—ini kewajiban.

### **Keenam, sumber.**

Harus jelas dokumen ini datang dari mana: catatan internal, dokumentasi resmi, buku, SOP, hasil observasi, log sistem, dan seterusnya.

### **Ketujuh, prioritas.**

Tidak semua dokumen punya bobot sama. SOP final bisa lebih prioritas daripada catatan rapat. Patch penting bisa lebih prioritas daripada memo lama.

### **Kedelapan, freshness.**

Ini menjawab apakah dokumen masih baru, perlu ditinjau, atau sudah usang. Banyak retrieval gagal karena dokumen lama dan baru diperlakukan setara.

Beberapa referensi juga mengarah ke sini, walau memakai istilah yang sedikit berbeda di beberapa versi: topik, subtopik, entitas, prioritas, tingkat kepercayaan, sensitivitas, sumber, *freshness*, serta pada versi lain menambahkan status validasi.

## Template label minimum

```

title: "FAQ Reindex Open WebUI"
source: "tim-ai-lokal"
date: "2026-04-13"
category: "FAQ"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "reviewed"
topic: "rag"
subtopic: "reindex"
entities: ["Open WebUI", "embedding"]
confidence: "high"
priority: "medium"
freshness: "current"

```

## 6.4 Teknik manual labeling yang konsisten

Masalah besar dalam labeling bukan biasanya kurang niat. Masalahnya adalah **inkonsistensi**. Hari ini seseorang menulis `OpenWebUI`, besok `Open WebUI`, lusa `open-webui`. Hari ini `high`, besok `urgent`, lusa `critical`, padahal maksudnya sama. Akhirnya label bukan membantu retrieval, tetapi malah menambah variasi yang tidak perlu.

Karena itu, teknik manual labeling yang konsisten harus dimulai dari **kamus label**. Buat dokumen kecil yang menjelaskan:

- daftar nilai yang boleh dipakai,
- arti setiap nilai,
- kapan sebuah nilai digunakan,
- contoh penggunaannya.

### How-to membuat kamus label

Buat file `label_schema.yaml`:

```
topic:
 allowed: ["rag", "embedding", "vector-db", "security",
"python"]

confidence:
 allowed: ["low", "medium", "high"]
 meaning:
 low: "belum diverifikasi penuh"
 medium: "cukup masuk akal, perlu cek"
 high: "sudah diverifikasi"

sensitivity:
 allowed: ["public", "internal", "restricted"]

priority:
 allowed: ["low", "medium", "high"]

freshness:
 allowed: ["current", "review-soon", "stale"]
```

Cek file:

```
cat label_schema.yaml
```

Contoh output:

```
topic:
 allowed: ["rag", "embedding", "vector-db", "security",
"python"]
...
```

## Step by step labeling manual

- baca dokumen sekali tanpa melabeli;
- tentukan topik utama;
- tentukan subtopik paling spesifik;
- tandai entitas penting;
- tentukan confidence;
- tentukan sensitivity;
- tentukan priority;
- tentukan freshness;
- simpan dengan format metadata yang konsisten.

## Validasi cepat dengan CLI

Misalnya semua file `.md` ada di `./data/cleaned`:

```
grep -R
"^topic:|^subtopic:|^confidence:|^priority:|^freshness
:" ./data/cleaned
```

Contoh output:

```
./data/cleaned/reindex.md:topic: "rag"
./data/cleaned/reindex.md:subtopic: "reindex"
./data/cleaned/reindex.md:confidence: "high"
./data/cleaned/reindex.md:priority: "medium"
./data/cleaned/reindex.md:freshness: "current"
```

Kalau hasilnya sudah rapi, berarti labeling Anda mulai bisa diaudit.

## 6.5 Menambah *importance*, *authority*, dan *freshness*

Di tahap lebih matang, saya sarankan tiga skor tambahan yang sangat berguna: **importance**, **authority**, dan **freshness**.

**Importance** menjawab: seberapa penting dokumen ini untuk tugas retrieval?

Contoh:

- SOP final = tinggi
- catatan eksperimen = menengah
- memo singkat = rendah

**Authority** menjawab: seberapa resmi atau kuat otoritas sumbernya?

Contoh:

- dokumentasi resmi = tinggi
- catatan internal personal = menengah
- draft tak jelas = rendah

**Freshness** menjawab: seberapa baru dan relevan secara waktu?

Contoh:

- pembaruan minggu ini = current
- dokumen setahun lalu tapi masih valid = review-soon
- dokumen lama yang mungkin usang = stale

Ini bukan hiasan. Devlin secara eksplisit menyarankan penyimpanan metadata seperti sumber, bagian, dan timestamp bersama embedding, serta *versioning* ketika model berubah. Raieli menekankan bahwa metadata bisa memadatkan informasi penting yang kalau tidak, akan tersebar samar di dokumen. Dari sini, memberi skor semacam *authority* dan *freshness* adalah langkah yang sangat masuk akal sebagai perluasan praktis.

### Contoh metadata dengan skor tambahan

```

title: "Panduan Metadata Filtering di Chroma"
source: "docs-oss"
date: "2026-04-13"
domain: "rag"
language: "en"
sensitivity: "public"
quality: "reviewed"
topic: "vector-db"
subtopic: "metadata-filtering"
entities: ["Chroma"]
confidence: "high"
priority: "medium"
freshness: "current"
importance: 0.85
authority: 0.95

```

### Catatan penting

Disarankan skor numerik hanya dipakai kalau memang nanti dipakai dalam evaluasi atau ranking tambahan. Kalau tidak, cukup pakai label ordinal seperti **low**, **medium**, **high**.



## 6.6 Versioning dan jejak revisi

Ini bagian yang sering diabaikan, padahal sangat penting. Dokumen berubah. Label berubah. Model *embedding* berubah. Index berubah. Kalau semua itu tidak punya jejak, Anda akan cepat kehilangan kepercayaan pada corpus sendiri.

Bahan pendukung Anda sangat tegas di sini: **version your embeddings when changing models; keep migration scripts to rebuild the index deterministically**. Open WebUI juga menjelaskan bahwa setelah mengganti model *embedding*, dokumen yang sudah ada perlu **reindex** agar embedding baru benar-benar berlaku. Jadi, *versioning* bukan kemewahan; ia adalah cara menjaga corpus tetap dapat dipercaya. ([Open WebUI](#))

Yang minimal harus di-version

- dokumen sumber
- label schema
- model *embedding*
- aturan *chunking*
- hasil evaluasi retrieval
- waktu ingest / reindex

Cara praktis dengan Git

```
git init
git add .
git commit -m "Initial corpus with base labels"
```

Contoh output:

```
Initialized empty Git repository in
/home/user/rag-lab/.git/
[main (root-commit) 1a2b3c4] Initial corpus with base
labels
12 files changed, 430 insertions(+)
```

Saat skema label berubah:

```
git add label_schema.yaml data/cleaned
git commit -m "Refine confidence and freshness labels"
```

Saat model embedding berubah:

```
git add manifests/embedding-model.txt
reports/eval-before-after.json
git commit -m "Switch embedding model and trigger reindex
plan"
```

Kalau ingin lebih disiplin, buat file manifest sederhana:

```
embedding_model=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
chunking=heading:1200:150
label_schema=v1.2
last_reindex=2026-04-13
```

## 6.7 Anti-kekacauan: label terlalu banyak vs label terlalu miskin

Dua ekstrem sama-sama berbahaya.

**Label terlalu banyak** membuat sistem melelahkan. Orang tidak konsisten mengisinya. Nilainya tumpang tindih. *Schema drift* terjadi. Query filtering jadi rumit. Evaluasi jadi kabur.

**Label terlalu miskin** membuat retrieval tumpul. Dokumen penting dan tidak penting diperlakukan sama. Dokumen final dan draft bercampur. Dokumen lama dan baru tidak bisa dibedakan. Semua terasa “semantik”, tetapi kontrol praktis hilang.

Jadi, prinsip sehatnya adalah: **minimum tetapi bermakna**.

Untuk kebanyakan corpus buku ini, saya sarankan:

- metadata inti sekitar 8–12 field;
- label operasional sekitar 5–8 field;
- nilai label harus terbatas dan terdokumentasi.

Raieli menulis bahwa metadata memungkinkan pencarian lebih granular. Tetapi granular tidak berarti tak terbatas. Granular berarti tepat sasaran. Jadi, lebih baik 8 label yang benar-benar dipakai daripada 30 label yang hanya terlihat canggih di YAML.

### Tanda Anda terlalu banyak label

- orang lupa cara mengisinya;
- ada tiga field untuk hal yang sama;
- nilai label berubah-ubah penulisannya;
- field ada, tetapi tidak pernah dipakai untuk filter atau evaluasi.

### Tanda Anda terlalu miskin label

- dokumen final dan draft bercampur;
- sulit menyaring dokumen terbaru;
- tidak bisa membedakan sumber resmi dan catatan pribadi;
- retrieval sering “hampir benar” tetapi salah dokumen.

## 6.8 Checklist mutu sebelum di-embed

Sebelum dokumen masuk ke tahap embedding, saya sarankan pemeriksaan mutu singkat. Ini kebiasaan kecil yang bisa menyelamatkan banyak jam debugging.

Gunakan checklist berikut:

- Apakah judul dokumen jelas?
- Apakah metadata minimum lengkap?
- Apakah topik dan subtopik sudah tegas?
- Apakah entitas penting sudah ditandai?
- Apakah confidence masuk akal, tidak terlalu optimistis?
- Apakah sensitivity sudah benar?
- Apakah freshness sudah diperiksa?
- Apakah sumber dokumen jelas?
- Apakah label mengikuti kamus yang resmi?
- Apakah dokumen ini terlalu panjang dan sebaiknya dipecah dulu?
- Apakah ada duplikasi atau versi lama yang belum dipensiunkan?
- Apakah perubahan besar sudah dicatat di jejak revisi?

### How-to cek cepat dengan skrip kecil

```
python - <<'PY'
from pathlib import Path
import re

required = [
 "title", "source", "date", "domain", "language",
 "sensitivity", "quality", "topic", "subtopic",
 "confidence", "priority", "freshness"
]

for path in Path("./data/cleaned").rglob("*.md"):
 text = path.read_text(encoding="utf-8", errors="ignore")
 missing = []
 for key in required:
 if not re.search(rf'^(key):\s*', text,
 flags=re.MULTILINE):
 missing.append(key)
 if missing:
 print(f"[WARN] {path} missing: {' '.join(missing)}")
 else:
 print(f"[OK] {path}")
PY
```

Contoh output:

```
[OK] data/cleaned/reindex-openwebui.md
[WARN] data/cleaned/faq-knowledge.md missing: freshness,
priority
```

Kalau pembaca membiasakan langkah seperti ini, kualitas corpus akan naik jauh sebelum model generatif ikut bicara.

## Penutup Bab

Kalau ada satu pelajaran yang harus menempel dari bab ini, itu adalah: **label bukan dekorasi; label adalah alat kontrol pengetahuan**. Ia membantu retrieval memilih yang relevan, membantu filter membuang yang tidak cocok, membantu audit menelusuri sumber, membantu evaluasi membaca hasil dengan jujur, dan membantu manusia menjaga corpus tetap waras. Embedding yang bagus memang penting. Tetapi embedding tanpa label sering hanya memberi “kemiripan makna”, bukan keputusan yang benar. Itu sebabnya label sering terasa lebih kecil, tetapi dampaknya justru lebih besar. ([Qdrant](#))

Setelah bab ini, pembaca seharusnya siap masuk ke tahap berikutnya: **membangun corpus untuk tiga dunia yang sangat berbeda—knowledge/text, log server, dan programming**—dengan skema label yang lebih sadar domain.

# Bab 7. Membangun Corpus untuk Tiga Dunia: *Knowledge/Text*, Log Server, dan Programming

Kalau pada bab-bab sebelumnya kita belajar membersihkan dokumen, memberi label, dan menyiapkan ekosistem, maka di bab ini kita mulai masuk ke kenyataan lapangan: **corpus yang baik selalu bersifat domain-aware**. Anda tidak bisa mengambil aturan dari textbook lalu menempelkannya mentah-mentah ke log server. Anda juga tidak bisa memakai intuisi dari FAQ untuk membaca repository kode. RAG yang sehat justru lahir ketika kita tahu bahwa “pengetahuan” punya banyak bentuk, dan masing-masing bentuk menuntut perlakuan yang berbeda.

## 7.1 Corpus untuk *knowledge / text*

Mari mulai dari bentuk yang paling akrab: **teks pengetahuan umum**. Di sini masuk:

- buku,
- artikel,
- memo,
- SOP,
- kebijakan,
- FAQ,
- dokumen organisasi.

Sekilas domain ini tampak paling mudah, karena manusia memang terbiasa membaca narasi. Tetapi justru karena itu, banyak orang ceroboh. Mereka menggabungkan terlalu banyak topik ke satu file, membiarkan judul kabur, mencampur draft dan final, atau memasukkan memo singkat dan SOP final ke level yang sama. Akibatnya, retrieval menjadi “lumayan relevan”, tetapi tidak benar-benar tajam.

Untuk domain *knowledge/text*, tujuan utama corpus adalah **menyajikan pengetahuan yang bisa dijawab kembali secara faktual, tertelusur, dan mudah diperbarui**. Jenis pertanyaan yang muncul di sini biasanya berbentuk:

- “apa isi dokumen ini?”,
- “apa prosedur resminya?”,
- “apa perbedaan A dan B?”,
- “langkah-langkahnya apa?”,
- “dokumen mana yang menjelaskan hal ini?”

Karena pola query seperti ini, dokumen *knowledge/text* harus kuat di:

- judul yang jelas,
- struktur heading,
- metadata sumber dan tanggal,
- pernyataan tujuan,
- daftar langkah,

- definisi singkat,
- referensi silang.

Bahan pendukung Anda juga mendukung arah ini. Devlin menulis bahwa sistem RAG yang baik menyimpan metadata seperti **source, date, document type**, lalu memadukan filter keyword dan semantik agar retrieval lebih presisi. Dalam praktik Open WebUI, dokumen seperti kontrak, spesifikasi teknis, dan *research papers* memang diposisikan sebagai bahan *Knowledge* yang bisa dicari AI “by meaning, not just keywords.” ([Open WebUI](#))

### *How-to: membangun corpus knowledge/text dari nol*

Langkah yang disarankan:

- pilih satu domain kecil dulu, misalnya **rag** atau **python**;
- kumpulkan 20–50 dokumen sehat;
- bedakan dokumen final dan draft;
- tambah metadata minimum;
- pecah dokumen terlalu panjang;
- uji 10 query nyata;
- baru tambah volume.

Contoh struktur folder:

```
mkdir -p
./data/knowledge/{books,articles,memos,sop,policies,faq}
tree -L 2 ./data/knowledge
```

Contoh output:

```
./data/knowledge
├── articles
├── books
├── faq
├── memos
├── policies
└── sop
```

Contoh dokumen Markdown yang sehat:

```

title: "SOP Reindex Knowledge Base"
source: "tim-ai-lokal"
date: "2026-04-13"
category: "SOP"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "approved"
topic: "rag"
subtopic: "reindex"
entities: ["Open WebUI", "embedding"]
confidence: "high"
priority: "high"
freshness: "current"

SOP Reindex Knowledge Base

Tujuan
Menjelaskan kapan dan bagaimana melakukan *reindex*.

Langkah
1. Catat model *embedding* lama.
2. Ganti model baru.
3. Jalankan *reindex*.
4. Uji query pembandingan.
```

## 7.2 Corpus untuk log server

Sekarang kita pindah ke dunia yang sangat berbeda: **log server**. Di sini teks bukan dibuat untuk dibaca manusia secara nyaman. Ia dibuat sebagai jejak kejadian. Karena itu, log adalah domain yang lebih dekat ke *event stream* daripada narasi.

OpenTelemetry mendefinisikan *logs data model* sebagai model data dan konvensi semantik untuk merepresentasikan log dari berbagai sumber—*application log files*, *machine-generated events*, *system logs*, dan sebagainya. Itu penting, karena ia mengingatkan kita bahwa log bukan sekadar “teks acak”; log adalah **peristiwa dengan atribut**. OpenTelemetry juga secara eksplisit menempatkan unsur seperti *timestamp*, *severity*, *body*, dan *attributes* sebagai bagian penting model log, dan

dokumentasi korelasi log menegaskan bahwa *TraceId* dan *SpanId* dapat mengaitkan log dengan aktivitas atau jejak eksekusi lain. ([OpenTelemetry](#))

Karena itu, saat membangun corpus log server, fokus kita bukan gaya bahasa, melainkan:

- normalisasi event,
- konsistensi field,
- konteks waktu,
- relasi antar-kejadian,
- dan label yang membantu membedakan perilaku normal dari perilaku berisiko.

Log server sering datang dari:

- *web server*,
- *application server*,
- *database server*,
- *firewall*,
- *reverse proxy*,
- *authentication service*,
- *container runtime*,
- *system journal*.

Kalau semua itu dicampur tanpa struktur, retrieval akan rusak. Query seperti “ada brute force jam 02:00?” akan kalah oleh potongan log yang kebetulan mirip secara tekstual tetapi berbeda konteks. Karena itu, **log harus diperlakukan sebagai corpus event**, bukan corpus naratif.

### *How-to*: langkah awal corpus log

- pilih satu sumber log dulu, misalnya *nginx* atau *auth.log*;
- pisahkan log mentah dari log yang sudah diparsing;
- normalisasi field penting;
- beri label kejadian;
- tambahkan konteks waktu;
- baru uji query.

Contoh struktur:

```
mkdir -p ./data/logs/{raw,parsed,labeled}
ls -l ./data/logs
```

Contoh output:

```
labeled
parsed
raw
```



Contoh log mentah:

```
2026-04-13T02:14:22Z web01 sshd[8831]: Failed password for
invalid user admin from 203.0.113.8 port 54422 ssh2
```

Contoh bentuk yang sudah diperkaya:

```
{
 "timestamp": "2026-04-13T02:14:22Z",
 "host": "web01",
 "service": "sshd",
 "user": "admin",
 "ip": "203.0.113.8",
 "severity": "warning",
 "event": "failed_login",
 "outcome": "failure",
 "label": "suspicious"
}
```

### 7.3 Teknik melabel *attack* / *non-attack* / *suspicious* / *unknown*

Ini bagian yang sangat penting, terutama kalau corpus log nanti dipakai untuk RAG keamanan, *triage*, atau analisis insiden. Jangan terburu-buru memberi label terlalu dramatis. Banyak kejadian tampak seperti serangan, padahal hanya salah konfigurasi, *health check*, atau *scanner* internal. Sebaliknya, beberapa serangan tampak biasa-biasa saja bila hanya dilihat satu baris.

Karena itu, saya sarankan empat label kerja yang sederhana tetapi kuat:

- **attack**  
Ada indikasi kuat kejadian ini bagian dari aktivitas berbahaya atau eksploitasi.
- **non-attack**  
Kejadian ini normal, diharapkan, atau sudah jelas bukan ancaman.
- **suspicious**  
Ada gejala yang patut dicurigai, tetapi belum cukup bukti untuk menyebutnya serangan.
- **unknown**  
Data belum cukup, konteks belum cukup, atau analisis belum selesai.

Beberapa referensi menegaskan pasangan ini: *attack* / *non-attack* / *suspicious* / *unknown*, ditambah perhatian pada *false positive*, *false negative*, dan validasi manusia.

#### Prinsip penting

**Jangan memaksa semua hal menjadi *attack* atau *non-attack*.** Label *suspicious* dan *unknown* justru menyelamatkan corpus dari kepercayaan diri palsu.

## How-to: aturan labeling sederhana

Gunakan aturan berjenjang:

- apakah ada indikator eksplisit, misalnya *exploit success*, *payload known bad*, atau IOC yang tervalidasi?
- apakah ada pola berulang yang kuat, misalnya *multiple failed logins* dari satu IP dalam jendela waktu pendek?
- apakah ada konteks operasional yang menjelaskan kejadian itu sebagai normal?
- kalau tidak ada kepastian, simpan sebagai **suspicious** atau **unknown**.

Contoh YAML untuk label skema:

```
log_labels:
 allowed: ["attack", "non-attack", "suspicious",
 "unknown"]
 notes:
 attack: "indikasi kuat aktivitas berbahaya"
 non-attack: "aktivitas normal atau terverifikasi aman"
 suspicious: "ada gejala aneh, belum cukup bukti"
 unknown: "konteks belum cukup"
```

Contoh validasi sederhana:

```
grep -R '"label"' ./data/logs/labeled | head
```

Contoh output:

```
./data/logs/labeled/auth-001.json: "label": "suspicious"
./data/logs/labeled/auth-002.json: "label": "non-attack"
```

## 7.4 Medan log yang penting

Kalau corpus log ingin sehat, ada beberapa field yang hampir selalu penting. Referensi yang banyak memilih inti yang benar: **waktu, host, service, user, IP, severity, event, outcome**. Penulis akan pertahankan itu karena sudah sangat kuat.

**Waktu** adalah tulang punggung log. Tanpa waktu, Anda kehilangan urutan kejadian.

**Host** memberi tahu dari mesin mana event itu berasal.

**Service** membantu membedakan *sshd*, *nginx*, *postgres*, dan lain-lain.

**User** penting untuk konteks autentikasi dan otorisasi.

**IP** penting untuk asal lalu lintas atau target komunikasi.

**Severity** memberi sinyal awal prioritas, walau tidak boleh dipercaya buta.

**Event** adalah jenis kejadian, misalnya *failed\_login*, *http\_500*, *config\_reload*.

**Outcome** menjawab apakah kejadian itu berakhir *success*, *failure*, atau status lain.

OpenTelemetry logs data model menegaskan pentingnya *severity* dan atribut log, sementara blog resmi tentang *log deduplication* menjelaskan bahwa log yang identik dapat dipandang berbagi *resource attributes*, *body*, *attributes*, *severity*, dan *event name*—menariknya, *timestamp* dikecualikan dari identitas karena memang berubah terus. Itu memberi pelajaran yang sangat bagus untuk buku ini: **field log tidak semua punya peran yang sama**. Sebagian adalah identitas event, sebagian adalah konteks temporal. ([OpenTelemetry](#))

### Contoh format log yang sehat untuk corpus

```
{
 "timestamp": "2026-04-13T02:14:22Z",
 "host": "web01",
 "service": "sshd",
 "user": "admin",
 "ip": "203.0.113.8",
 "severity": "warning",
 "event": "failed_login",
 "outcome": "failure",
 "label": "suspicious",
 "source": "auth.log",
 "freshness": "current"
}
```

## How-to: normalisasi field log dengan Python

```
python - <<'PY'
import json

record = {
 "timestamp": "2026-04-13T02:14:22Z",
 "host": "web01",
 "service": "sshd",
 "user": "admin",
 "ip": "203.0.113.8",
 "severity": "warning",
 "event": "failed_login",
 "outcome": "failure",
 "label": "suspicious"
}

print(json.dumps(record, indent=2))
PY
```

Contoh output:

```
{
 "timestamp": "2026-04-13T02:14:22Z",
 "host": "web01",
 "service": "sshd",
 "user": "admin",
 "ip": "203.0.113.8",
 "severity": "warning",
 "event": "failed_login",
 "outcome": "failure",
 "label": "suspicious"
}
```

## 7.5 Corpus untuk programming

Sekarang kita pindah ke dunia ketiga: **programming corpus**. Ini domain yang sangat sering diremehkan. Banyak orang memasukkan source code ke RAG seolah ia hanya teks biasa. Padahal kode punya struktur yang jauh lebih ketat, istilah yang lebih padat, dan konteks yang tersebar di banyak tempat.

Dalam dunia programming, corpus yang sehat biasanya tidak hanya berisi:

- **source code**,
- tetapi juga **README**,
- **API docs**,
- **komentar**,
- **commit message**,
- **changelog**.

Disini ditambah satu lagi yang berguna yaitu: *issue summary* atau ringkasan masalah. Alasannya sederhana: kode tanpa konteks sering membuat retrieval miskin makna. README menjelaskan tujuan. API docs menjelaskan kontrak. Komentar menjelaskan niat lokal. *Commit message* menjelaskan perubahan. *Changelog* menjelaskan sejarah rilis. *Keep a Changelog* mendefinisikan *changelog* sebagai daftar perubahan penting yang terkurasi dan tersusun kronologis untuk setiap versi proyek. Itu sangat berguna untuk corpus programming karena ia memberi jawaban yang lebih manusiawi dibanding menelusuri *git log* mentah. ([Keep A Changelog](#))

### Mengapa programming corpus berbeda

Karena query di domain ini sering berbentuk:

- “fungsi ini dipakai di mana?”
- “bagian mana yang mengatur autentikasi?”
- “apa langkah instalasinya?”
- “apa perubahan sejak versi terakhir?”
- “kenapa endpoint ini gagal?”
- “apa konfigurasi minimumnya?”

Kalau corpus hanya berisi source code, retrieval bisa menemukan fungsi, tetapi sering gagal menjelaskan niat, status, atau sejarah perubahan. Itulah sebabnya saya sangat menyarankan corpus programming selalu menggabungkan **kode + dokumentasi + sejarah perubahan**.

### How-to: struktur corpus programming

```
mkdir -p
./data/programming/{src,readme,api,comments,commits,change
log}
tree -L 2 ./data/programming
```

Contoh output:

```
./data/programming
├─ api
├─ changelog
├─ comments
├─ commits
├─ readme
└─ src
```

Contoh metadata untuk file programming:

```

title: "README instalasi service RAG"
source: "repo-internal"
date: "2026-04-13"
category: "README"
domain: "programming"
language: "en"
sensitivity: "internal"
quality: "reviewed"
topic: "deployment"
subtopic: "installation"
entities: ["Docker", "Open WebUI", "Ollama"]
confidence: "high"
priority: "medium"
freshness: "current"
repo: "rag-lab"
component: "deploy"

```

## 7.6 Apa yang berbeda antar-domain

Sekarang kita rangkum perbedaan utamanya. Ini bagian yang sangat penting karena pembaca harus berhenti mencari “satu formula untuk semua”.

### **Ukuran *chunk***

Untuk *knowledge/text*, *chunk* bisa mengikuti heading dan paragraf. Untuk log, *chunk* sering perlu berbasis event atau jendela waktu. Untuk programming, *chunk* sebaiknya menghormati batas fungsi, kelas, file, atau blok API docs. Devlin menulis bahwa *chunking* yang baik harus menjaga unit makna yang utuh, bukan sekadar panjang karakter. Itu cocok sekali dengan pesan bab ini.

### **Metadata**

Pada *knowledge/text*, metadata seperti sumber, kategori, tanggal, dan kualitas sangat penting. Pada log, field seperti timestamp, host, service, IP, severity, event, outcome jauh lebih krusial. Pada programming, metadata seperti repo, komponen, bahasa, modul, versi, dan jenis dokumen lebih menentukan.

### **Pola query**

Pada *knowledge/text*, query cenderung konseptual atau prosedural. Pada log, query sering investigatif dan temporal. Pada programming, query sering sangat spesifik, kadang nama fungsi, endpoint, parameter, atau error.

### **Risiko retrieval**

Pada *knowledge/text*, risiko terbesar adalah dokumen generik mengalahkan dokumen resmi. Pada log, risiko terbesar adalah event normal terlihat jahat atau sebaliknya. Pada programming, risiko terbesar adalah potongan kode relevan secara tekstual tetapi salah konteks fungsional.

### **Kebutuhan *reranking***

Untuk corpus kecil *knowledge/text*, sering belum perlu. Untuk log yang besar dan bercampur banyak event, *reranking* bisa sangat membantu setelah *top-k* awal. Untuk programming corpus besar, *reranking* juga sering masuk akal karena query cenderung presisi dan konteksnya sempit. Devlin bahkan secara eksplisit menyarankan menambah *cross-encoder re-ranker* ketika corpus membesar, dan Open WebUI juga menyebut dukungan *cross-encoder reranking* dalam fiturnya. ([Open WebUI](#))

## 7.7 Latihan: bangun 3 corpus berbeda dari nol

Bab ini akan gagal bila hanya dibaca. Ia harus dikerjakan. Karena itu, saya sarankan latihan praktis membangun tiga corpus kecil dari nol.

### Latihan 1. Corpus *knowledge*

Target:

- 10 dokumen kecil
- campuran SOP, FAQ, memo, dan catatan teknis
- metadata lengkap
- topik jelas

Langkah:

- buat folder `./lab/knowledge`
- tulis 10 file Markdown
- beri YAML metadata
- uji 5 query

### Latihan 2. Corpus log

Target:

- 100–200 event log kecil
- dipisah `raw`, `parsed`, `labeled`
- punya label `attack/non-attack/suspicious/unknown`

Langkah:

- ambil satu sumber log
- normalisasi field inti
- buat label manual
- uji query seperti “apakah ada failed login berulang?”



## Latihan 3. Corpus programming

Target:

- 1 repo kecil
- ambil README, satu set API docs, beberapa commit message, dan beberapa file kode
- simpan dengan metadata yang konsisten

Langkah:

- pilih repo kecil
- pisahkan `src`, `readme`, `api`, `commits`, `changelog`
- beri metadata
- uji query seperti “apa langkah instalasinya?” dan “bagian mana yang mengatur autentikasi?”

### How-to membuat kerangka latihan

```
mkdir -p ./lab/{knowledge,logs,programming}
mkdir -p ./lab/logs/{raw,parsed,labeled}
mkdir -p
./lab/programming/{src,readme,api,commits,changelog}
tree -L 2 ./lab
```

Contoh output:

```
./lab
├── knowledge
├── logs
│ ├── labeled
│ ├── parsed
│ └── raw
└── programming
 ├── api
 ├── changelog
 ├── commits
 ├── readme
 └── src
```

### Uji kecil setelah corpus jadi

```
python ingest_pilot.py --input ./lab/knowledge --limit 10
--vector-store lancedb

python eval_retrieval.py --queries ./tests/knowledge.yaml
--vector-store lancedb
```

Contoh output:

```
[INFO] Documents ingested: 10
[INFO] Chunks created: 28
[INFO] Top-1 hit rate: 80.00%
[INFO] Top-5 hit rate: 100.00%
```

Lakukan hal serupa untuk log dan programming. Di titik ini, pembaca akan benar-benar merasakan bahwa **membangun corpus bukan satu pekerjaan, melainkan tiga cara berpikir yang berbeda.**

## Penutup Bab

Kalau ada satu gagasan yang harus dibawa pembaca dari bab ini, itu adalah: **RAG yang sehat selalu menghormati bentuk asli pengetahuan.** Buku, SOP, dan FAQ adalah pengetahuan naratif yang butuh struktur editorial. Log server adalah jejak event yang butuh normalisasi dan labeling yang hati-hati. Source code dan dokumentasi pemrograman adalah pengetahuan teknis yang butuh konteks, sejarah perubahan, dan struktur yang berbeda dari narasi biasa. Karena itu, corpus yang baik bukan sekadar kumpulan file, tetapi **arsitektur pengetahuan yang sadar domain.**

Dengan memahami tiga dunia ini, pembaca tidak lagi akan tergoda memakai satu resep untuk semua. Justru di sinilah kualitas seorang perancang corpus diuji: ia tahu kapan harus memakai heading, kapan harus memakai event field, kapan harus menyertakan changelog, kapan harus menambah label, dan kapan harus menambahkan *reranker*. Itulah yang membuat corpus tidak hanya bisa dicari, tetapi juga **bisa dipercaya.** ([OpenTelemetry](#))

## Bab 8. *Chunking Strategies*: Recursive, Semantic, dan Hierarchical

Kalau ada satu bagian dalam RAG yang paling sering diremehkan, itu adalah *chunking*. Banyak orang membayangkan *chunking* hanya sebagai pekerjaan memotong teks menjadi ukuran tertentu. Padahal, dalam praktik, *chunking* adalah **cara kita menentukan unit makna**. Dan retrieval hanya bisa sebaik unit makna yang kita berikan padanya. Kalau *chunk* terlalu besar, query akan tenggelam dalam konteks yang terlalu luas. Kalau terlalu kecil, makna pecah, hubungan antar-kalimat hilang, dan model diberi potongan-potongan yang terlalu tipis untuk bernalar. Jadi, *chunking* bukan soal panjang karakter semata. **Chunking adalah desain epistemik**: bagaimana pengetahuan dipecah agar tetap bisa dicari, dipahami, dan dikembalikan dengan tepat.

### 8.1 Mengapa *chunking* adalah jantung retrieval

Retrieval dalam RAG pada dasarnya tidak mengambil “dokumen” secara utuh setiap kali. Yang biasanya diambil adalah **potongan-potongan dokumen**. Potongan itulah yang menjadi konteks untuk LLM. Jadi, kalau potongan itu salah bentuk, retrieval bisa salah walau model *embedding* bagus dan vector database rapi.

Ada empat kegagalan klasik yang hampir selalu muncul dari *chunking* yang buruk.

**Pertama, potongan terlalu gemuk.**

Query spesifik tentang satu langkah kecil tenggelam di dalam paragraf panjang, lampiran, atau satu bagian bab yang memuat terlalu banyak topik.

**Kedua, potongan terlalu kurus.**

Kalimat yang relevan memang ketemu, tetapi penjelasan sebelum dan sesudahnya hilang. LLM lalu menjawab berdasarkan konteks separuh matang.

**Ketiga, batas potongan merusak makna.**

Judul berada di *chunk* sebelumnya, definisi ada di *chunk* berikutnya, contoh ada di *chunk* lain lagi. Hasilnya retrieval seolah “hampir benar”, tetapi tidak pernah benar-benar utuh.

**Keempat, duplikasi tak sehat.**

*Overlap* terlalu besar membuat potongan hampir sama terus-menerus. *Top-k* jadi penuh dengan jawaban yang pada dasarnya salinan satu sama lain.

Itulah sebabnya banyak referensi menyebut *chunking* sebagai **jantung retrieval**. Bukan aksesori, bukan optimasi kecil, tetapi pusat keputusan.

LlamaIndex secara resmi menjelaskan bahwa *node parsers* memecah dokumen menjadi *Node objects*, dan node-node itu mewarisi metadata dan atribut dokumen induknya. Ini penting karena menunjukkan bahwa *chunking* bukan hanya pemotongan teks, tetapi juga **pewarisan konteks dan hubungan**. ([LlamaIndex OSS Documentation](#))

## 8.2 *Recursive chunking*: kapan dipakai, ukuran awal, *overlap*, jebakan

Mari mulai dari bentuk yang paling mudah dipahami dan paling cocok untuk banyak kasus awal: **recursive chunking**. Ide dasarnya sederhana: kita memecah dokumen dengan aturan berlapis. Biasanya dimulai dari struktur besar seperti heading atau paragraf, lalu jika potongan masih terlalu besar, kita pecah lagi ke unit yang lebih kecil. Ini lebih sehat daripada memotong teks secara buta setiap sekian karakter.

Dalam ekosistem yang lebih praktis, pola *recursive small-to-big retrieval* juga terdokumentasi di LlamaIndex: dokumen dibagi ke *parent chunks*, lalu tiap parent dipecah lagi menjadi *child chunks* yang lebih kecil, dan *child chunks* ini ditautkan ke parent untuk retrieval berikutnya. Jadi kata “recursive” di sini bisa berarti dua hal yang saling berhubungan:

- **pemotongan bertahap**
- **retrieval bertahap dari kecil ke besar** ([LlamaIndex OSS Documentation](#))

### Kapan dipakai

Gunakan *recursive chunking* ketika:

- dokumen punya struktur heading atau paragraf yang cukup sehat;
- Anda ingin solusi awal yang sederhana dan kuat;
- domain masih didominasi *knowledge/text*;
- Anda belum butuh model atau alat pemisah semantik yang lebih canggih.

### Ukuran awal

Disarankan pembaca buku ini **tidak obsesif pada satu angka**, tetapi mulai dari rentang yang waras. Untuk teks naratif biasa, ukuran awal setara **sekitar 400–800 token** atau kira-kira **1.000–2.000 karakter** sering menjadi titik mulai yang masuk akal untuk uji pilot. Draft DOC terlampir Anda sendiri bahkan menyebut default *recursive chunking* sekitar **400–512 token** sebagai titik awal yang sehat. Itu masuk akal untuk banyak kasus karena cukup kecil untuk pencarian, tetapi belum terlalu tipis untuk kehilangan konteks.

## Overlap

*Overlap* adalah irisan antar-*chunk*. Fungsinya sederhana: menyelamatkan konteks yang berada di batas potongan. Tanpa *overlap*, definisi bisa terpotong dari contohnya, atau perintah CLI bisa terpisah dari penjelasan risiko. Dengan *overlap* terlalu besar, indeks membengkak dan duplikasi naik.

Aturan praktis saya:

- mulai dari *overlap* kecil-menengah;
- untuk teks naratif biasa, *overlap* sekitar **10–20%** dari ukuran *chunk* sering cukup;
- untuk kode atau langkah prosedural, kadang perlu sedikit lebih hati-hati bila ada blok yang rawan terbelah.

## Jebakan

Jebakan terbesar *recursive chunking* adalah **terlalu percaya diri bahwa struktur permukaan dokumen sudah sehat**. Padahal kalau heading kacau, paragraf pecah, atau PDF hasil ekstraksi masih kotor, *recursive chunking* hanya akan menurunkan kekacauan ke level yang lebih kecil.

## How-to sederhana

Misalnya Anda ingin menguji tiga konfigurasi:

- **800:100**
- **1200:150**
- **1600:200**

Gunakan skrip yang sudah kita buat sebelumnya:

```
python compare_chunking.py \
 --input ./data/cleaned \
 --queries ./tests/queries.yaml \
 --embedding-model
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v
2 \
 --chunk-configs 800:100 1200:150 1600:200 \
 --top-k 5
```

Contoh keluaran:

```
=== CHUNK CONFIG: heading:1200:150 ===
[INFO] Chunks created: 38
[INFO] Top-1 hit rate: 100.00%
[INFO] Top-5 hit rate: 100.00%
```

Ini mengajarkan kebiasaan penting: **jangan menebak-nebak ukuran chunk; uji.**

### 8.3 *Semantic chunking*: pecah berdasarkan perubahan makna

Sekarang kita masuk ke bentuk yang lebih cerdas: **semantic chunking**. Di sini dokumen tidak dipecah hanya berdasarkan ukuran, tetapi berdasarkan **perubahan makna**. LlamaIndex menjelaskan *semantic chunking* sebagai pendekatan yang secara adaptif memilih titik potong di antara kalimat menggunakan **embedding similarity**, sehingga satu *chunk* cenderung berisi kalimat-kalimat yang secara semantik masih satu keluarga. ([LlamaIndex OSS Documentation](#))

Ini sangat menarik, karena manusia memang jarang berpikir dalam batas “500 karakter”. Kita berpikir dalam unit ide. Semantic chunking mencoba meniru itu.

#### Kapan dipakai

Gunakan *semantic chunking* ketika:

- dokumen panjang tetapi transisi ide di dalamnya jelas;
- paragraf tidak selalu rapi, tetapi alur semantiknya kuat;
- Anda ingin potongan yang lebih “manusiawi”;
- query cenderung konseptual, bukan hanya *keyword-driven*.

#### Kapan lebih unggul dari *fixed chunk*

Ia biasanya lebih unggul saat dokumen:

- banyak mengandung transisi konsep,
- punya paragraf sangat panjang,
- memuat campuran penjelasan, contoh, dan argumen.

Dalam kasus seperti itu, *fixed chunk* sering memotong di tempat yang tidak alami. Semantic chunking berusaha menjaga satu ide tetap utuh.

#### Biaya dan kompleksitas

Tetapi jangan romantis. Semantic chunking juga punya harga:

- lebih mahal secara komputasi;
- lebih lambat dibanding pemotongan karakter sederhana;
- hasilnya sensitif terhadap model embedding yang dipakai;
- tidak otomatis lebih baik untuk semua domain.

Untuk log server, misalnya, semantic chunking sering kurang cocok dibanding pemotongan berbasis event. Untuk code, semantik alami kadang kalah oleh struktur sintaksis.

## How-to mental model

Bayangkan Anda punya paragraf textbook yang membahas:

1. definisi *embedding*,
2. contoh penggunaan,
3. perbandingan dengan *keyword search*.

*Fixed chunk* bisa memotong tepat di tengah contoh.

Semantic chunking cenderung berusaha menjaga definisi dan contoh yang masih berkaitan tetap satu blok.

## CLI contoh untuk eksperimen

Bila Anda memakai alat bantu seperti LlamaIndex untuk eksperimen lanjutan, semantic chunking memang tersedia sebagai modul terpisah dan juga muncul dalam dokumentasi contoh resmi. Disini disarankan semantic splitter sebagai salah satu primitif yang relevan untuk bab ini. ([LlamaIndex OSS Documentation](#))

## 8.4 Hierarchical chunking: parent-child, bab-subbab-paragraf

Sekarang kita masuk ke strategi yang menurut saya sangat penting untuk buku seperti ini: **hierarchical chunking**. LlamaIndex mendokumentasikan **HierarchicalNodeParser** sebagai cara memecah satu dokumen ke beberapa hierarki ukuran, dengan tiap node menyimpan referensi ke parent-nya. Ini sangat sesuai dengan struktur dokumen nyata seperti:

- buku → bab → subbab → paragraf
- SOP → tujuan → langkah → catatan
- kebijakan → bagian → klausul → butir
- repo → direktori → file → fungsi ([LlamaIndex OSS Documentation](#))

Mengapa ini kuat? Karena dokumen nyata memang punya **hirarki makna**. Judul bab memberi konteks makro. Subbab memberi fokus. Paragraf memberi detail. Kalau kita hanya menyimpan level paragraf, konteks makro hilang. Kalau kita hanya menyimpan level bab, detail query tenggelam.

### Parent-child

Dalam pola ini:

- *child chunks* kecil dipakai untuk pencarian presisi;
- *parent chunks* dipakai untuk pemulihan konteks yang lebih utuh.

Raieli menjelaskan teknik ini dengan sangat baik: *child chunks* di-embed untuk menjawab query spesifik, tetapi ketika *child* yang relevan ditemukan, sistem bisa mengembalikan **parent document** agar konteks lebih kaya.

## Kapan dipakai

Gunakan *hierarchical chunking* ketika:

- dokumen panjang dan berstruktur jelas;
- Anda ingin keseimbangan antara presisi dan konteks;
- textbook, SOP panjang, buku teknis, atau dokumentasi API besar menjadi sumber utama.

## Kelebihan

- lebih mudah menjaga konteks bab dan subbab;
- cocok untuk *parent document retrieval*;
- bagus untuk textbook dan dokumentasi teknis besar.

## Kekurangan

- lebih rumit dikelola;
- metadata harus lebih disiplin;
- evaluasinya lebih sulit daripada *fixed chunk* sederhana.

## 8.5 *Chunking* untuk textbook

Textbook adalah salah satu domain yang paling cocok untuk *hierarchical chunking*. Kenapa? Karena textbook punya struktur alami:

- bab
- subbab
- subsubbab
- paragraf
- contoh
- tabel
- referensi

Kalau textbook dipotong membabi buta setiap 1.000 karakter, kita sering merusak justru hal yang paling berguna: **alur penjelasan**. Satu definisi bisa terpisah dari ilustrasinya. Judul subbab bisa lepas dari isi. Daftar butir bisa pecah di tengah.

Untuk textbook, saya sarankan urutan berpikir seperti ini:

- pulihkan dulu struktur heading dari PDF hasil ekstraksi;
- simpan bab sebagai parent;
- simpan subbab sebagai child utama;
- kalau subbab masih terlalu besar, pecah lagi ke paragraf atau blok makna;
- pertahankan referensi ke parent.



## Strategi sehat

- bab = konteks besar
- subbab = unit retrieval utama
- paragraf = unit detail bila perlu
- tabel dan contoh = perlakukan hati-hati, jangan dicampur asal

## How-to praktis

Mulai dari hasil Markdown bersih, lalu pecah per heading utama:

```
grep -n "^# " output/cleaned/textbook.clean.md
grep -n "^## " output/cleaned/textbook.clean.md
```

Contoh keluaran:

```
12:# Chapter 1 Introduction
88:# Chapter 2 Vector Stores
154:# Chapter 3 Embedding Strategies
171:## Why embeddings matter
188:## Choosing an embedding model
```

Dari sini, subbab seperti **## Why embeddings matter** bisa menjadi *child chunk* yang sehat.

## 8.6 Chunking untuk log

Log tidak boleh dipotong dengan intuisi textbook. Di dunia log, **event** adalah unit utama, bukan paragraf. Karena itu, strategi *chunking* untuk log harus lebih dekat ke:

- satu event per chunk,
- atau satu jendela waktu kecil untuk kumpulan event yang memang berhubungan.

### Kapan satu event per chunk

Gunakan saat:

- event berdiri sendiri sudah cukup informatif;
- tujuan utamanya klasifikasi atau pencarian insiden spesifik;
- log memiliki field kaya seperti host, user, IP, severity, event, outcome.

### Kapan jendela waktu lebih tepat

Gunakan saat:

- satu kejadian penting baru masuk akal jika dilihat sebagai rangkaian;
- misalnya lima kali *failed login* lalu satu *success*;
- atau beberapa peristiwa dari IP yang sama dalam 2 menit.

## Risiko terbesar

Kalau log di-*chunk* terlalu besar, event berbahaya bisa tenggelam.  
Kalau terlalu kecil, pola serangan yang sebenarnya rangkaian bisa terputus.

Jadi, untuk log:

- event-centric bila analisis per baris cukup;
- time-window bila pola urutan penting.

## 8.7 *Chunking* untuk code dan API docs

Kode juga tidak boleh dipotong seperti esai. Untuk code, unit makna alami sering berupa:

- fungsi
- kelas
- method
- blok konfigurasi
- file
- endpoint documentation

LlamaIndex bahkan punya **CodeHierarchyNodeParser** yang mendeskripsikan pemisahan code menggunakan parser AST dan menambahkan metadata tentang *scope* serta relasi antarblok kode. Itu sinyal kuat bahwa code memang membutuhkan strategi yang berbeda dari teks biasa. ([LlamaIndex OSS Documentation](#))

## Untuk source code

Potong mengikuti:

- fungsi
- kelas
- file
- modul

Jangan potong di tengah fungsi bila bisa dihindari.

## Untuk README

README bisa diperlakukan seperti *knowledge/text*:

- heading
- langkah instalasi
- konfigurasi
- contoh penggunaan

## Untuk API docs

Potong mengikuti:

- endpoint
- method
- parameter group
- response schema
- contoh request/response

## Risiko retrieval

- fungsi yang benar ditemukan, tetapi tanpa komentar atau README jadi sulit dipahami;
- endpoint yang mirip tertukar;
- potongan kode diambil tanpa konteks impor, konfigurasi, atau side effects.

Jadi, untuk programming corpus, *chunking* idealnya tidak berdiri sendiri. Ia perlu dibantu metadata seperti *repo*, *component*, *language*, *endpoint*, atau *module*.

## 8.8 Kapan perlu *context enrichment*, *parent document retrieval*, dan *sentence window*

Ini bagian yang sangat penting karena ia menjembatani dua kebutuhan yang sering bertabrakan:

- **presisi tinggi dari chunk kecil**
- **konteks kaya untuk generation**

Raieli menjelaskan tiga pendekatan yang sangat berguna di sini.

### Sentence window retrieval

Setiap kalimat di-embed secara terpisah untuk presisi tinggi. Setelah satu kalimat yang tepat ditemukan, sistem memperluas konteks dengan mengambil beberapa kalimat sebelum dan sesudahnya. Ini cocok untuk dokumen yang jawabannya sering tersembunyi di satu kalimat, tetapi tetap butuh konteks sekitar.

### Parent document retriever

Sistem mencari *child chunks* kecil dulu. Setelah *child* relevan ditemukan, sistem mengambil parent document atau parent chunk yang lebih besar. Ini sangat bagus untuk textbook, kebijakan panjang, dan dokumentasi teknis besar.

## Context enrichment

Secara umum, ini berarti setelah chunk relevan ditemukan, kita menambahkan konteks sekitar agar generation tidak kehilangan kedalaman makna. Raieli menempatkan ini sebagai jawaban langsung atas masalah “loss of context in granular chunking.”

## Kapan perlu

Gunakan strategi ini ketika:

- chunk kecil memberi presisi bagus tetapi jawaban LLM terasa tipis;
- textbook sering menghasilkan jawaban setengah benar;
- query kompleks membutuhkan latar belakang tambahan;
- retrieval sering menemukan kalimat tepat, tetapi bukan konteks yang cukup.

## Kapan belum perlu

Jangan buru-buru memakai ini bila:

- corpus masih kecil;
- *recursive chunking* biasa sudah stabil;
- Anda belum punya evaluasi retrieval yang rapi.

## 8.9 Mengukur hasil *chunking*: relevansi, fragmentasi, konteks hilang, duplikasi

Bab *chunking* akan menjadi wacana kosong kalau kita tidak mengukurnya. Saya sarankan empat metrik praktis yang mudah dipahami mahasiswa dan praktisi.

### Relevansi

Apakah chunk yang terambil benar-benar menjawab query? Ini bisa dilihat dari *top-1* dan *top-k hit rate* pada query uji.

### Fragmentasi

Apakah satu ide penting tersebar ke terlalu banyak chunk kecil? Kalau ya, retrieval jadi harus beruntung untuk mendapat cukup konteks.

### Konteks hilang

Apakah jawaban tampak hampir benar tetapi kehilangan definisi, caveat, atau langkah lanjutan karena potongan terlalu sempit?

## Duplikasi

Apakah *top-k* diisi oleh chunk yang hampir sama terus-menerus karena *overlap* terlalu besar?

## How-to uji cepat

Gunakan skrip yang sudah Anda punya:

```
python compare_chunking.py \
 --input ./data/cleaned \
 --queries ./tests/queries.yaml \
 --embedding-model
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v
2 \
 --chunk-configs heading:800:100 heading:1200:150
heading:1600:200 \
 --top-k 5 \
 --verbose
```

Contoh keluaran ringkas:

PERBANDINGAN KONFIGURASI CHUNKING				
No	Config	Chunks	Top-1	Top-5
1	heading:1200:150	38	100.00%	100.00%
2	heading:800:100	51	66.67%	100.00%
3	heading:1600:200	27	66.67%	100.00%

Dari sini pembaca bisa belajar bahwa:

- chunk terlalu kecil belum tentu lebih baik;
- chunk terlalu besar belum tentu lebih buruk di *top-k*, tetapi bisa kalah di *top-1*;
- evaluasi harus berbasis query, bukan selera.

## 8.10 Template keputusan pemilihan strategi

Agar bab ini bisa dipakai, bukan hanya dibaca, saya sarankan template keputusan sederhana berikut.

Gunakan recursive chunking bila:

- dokumen cukup rapi;
- ingin mulai cepat;
- corpus dominan *knowledge/text*;
- ingin baseline yang mudah diuji.

Gunakan semantic chunking bila:

- alur ide di dokumen lebih penting daripada batas karakter;
- dokumen panjang dan transisi konsep jelas;
- ingin chunk yang lebih natural secara semantik;
- siap menerima biaya komputasi lebih tinggi.

Gunakan hierarchical chunking bila:

- dokumen punya struktur kuat: bab, subbab, paragraf;
- textbook atau dokumentasi besar menjadi sumber utama;
- ingin *parent-child retrieval*;
- ingin menyeimbangkan presisi dan konteks.

Tambahkan sentence window bila:

- jawaban sering tersembunyi di satu kalimat;
- presisi tinggi dibutuhkan;
- tetapi konteks sekitar tetap penting.

Tambahkan parent document retrieval bila:

- chunk kecil ketemu, tetapi generation terlalu miskin konteks;
- textbook dan kebijakan panjang sering dipakai;
- ingin *small-to-big retrieval*.

Pertimbangkan reranking bila:

- *top-k* sudah lumayan, tapi urutannya belum tajam;
- corpus besar dan heterogen;
- query sering teknis atau ambigu.

## Checklist keputusan cepat

- jenis dokumen saya apa: textbook, log, atau code?
- unit makna alaminya apa: paragraf, event, fungsi?
- query saya cenderung sempit atau lebar?
- konteks lebih sering hilang atau justru terlalu gemuk?
- masalah saya ada di chunking, embedding, atau ranking?
- apakah saya butuh konteks parent sesudah child ditemukan?

## Penutup Bab

Kalau ada satu hal yang harus melekat dari bab ini, itu adalah: **chunking bukan pekerjaan memotong teks; chunking adalah pekerjaan memotong makna.** *Recursive chunking* memberi baseline yang kuat dan praktis. *Semantic chunking* membantu menjaga kesatuan ide. *Hierarchical chunking* memberi jembatan antara detail kecil dan konteks besar. *Sentence window* dan *parent document retrieval* menyelamatkan sistem saat presisi dan konteks mulai saling bertabrakan. Dan evaluasi atas relevansi, fragmentasi, konteks hilang, serta duplikasi adalah cara kita menjaga agar keputusan ini tetap jujur. ([LlamaIndex OSS Documentation](#))

Dengan bab ini, pembaca seharusnya berhenti bertanya “berapa ukuran chunk yang paling bagus?” dan mulai bertanya yang lebih matang: **unit makna apa yang paling sehat untuk domain saya, dan bagaimana saya membuktikannya lewat retrieval?** Itulah pertanyaan yang benar.

## Bab 9. Menyiapkan RAG di Open WebUI: *Ingest* → *Embed* → *Retrieve* → *Evaluate*

Bab ini adalah titik di mana semua kerja sebelumnya mulai terasa nyata. Kita sudah menyiapkan Ubuntu, Ollama, Open WebUI, model *embedding*, *vector database*, teknik menulis dokumen manual, konversi PDF, pelabelan, dan *chunking*. Tetapi semua itu baru fondasi. Sekarang pertanyaannya berubah: **bagaimana menata semuanya di Open WebUI agar benar-benar menjadi RAG yang berguna?** Di sinilah banyak proyek gagal. Mereka berhenti pada tahap “UI hidup, file bisa diunggah, jawaban muncul.” Padahal sistem yang hanya “muncul jawabannya” belum tentu sistem yang **tepat, grounded, dan tertelusur**.

### 9.1 *Workspace*, *Knowledge*, dan cara berpikir Open WebUI

Sebelum menyentuh tombol apapun, pembaca perlu memahami cara berpikir Open WebUI. Dokumentasi resminya menjelaskan bahwa **Knowledge** adalah ruang tempat dokumen diunggah agar bisa dicari AI berdasarkan makna, bukan sekadar kata kunci. Dokumen-dokumen ini dapat dilampirkan ke model tertentu atau dibiarkan tetap bisa ditemukan oleh model sesuai hak akses pengguna.

Dokumentasi fitur juga menekankan bahwa Open WebUI tidak hanya melakukan pencarian pasif; dengan *native function calling*, model bisa menelusuri *knowledge bases*, membaca file, dan mensintesis jawaban dari beberapa dokumen sekaligus.

([Open WebUI](#))

Cara berpikir yang sehat adalah ini: **Open WebUI bukan tempat menyimpan file; Open WebUI adalah tempat mengoperasikan pengetahuan.** Karena itu, pembaca harus memisahkan tiga level:

- **Workspace**  
ruang kerja tempat model, file, dan basis pengetahuan dipakai dalam interaksi.
- **Knowledge**  
kumpulan dokumen yang sudah dipersiapkan agar dapat dicari.
- **Chat / Query**  
tempat retrieval dan generation bertemu untuk menjawab kebutuhan pengguna.

Kalau pembaca memahami pemisahan ini, mereka tidak akan mudah jatuh pada kesalahan umum seperti mengunggah file mentah ke chat lalu berharap hasilnya sama dengan *knowledge base* yang dirancang dengan benar.



## How-to: memeriksa bahwa Open WebUI siap dipakai

Dari host Linux, lakukan cek dasar:

```
docker ps
curl -I http://localhost:3000
curl http://localhost:11434/api/tags
```

Contoh keluaran:

```
HTTP/1.1 200 OK
server: uvicorn

{"models":[{"name":"qwen3:8b","size":5583457484}]}
```

Bila tiga cek ini sehat, berarti:

- container Open WebUI hidup,
- UI dapat diakses,
- Ollama berjalan dan model tersedia.

## 9.2 Membuat *knowledge base* dari corpus yang sudah dibersihkan

Ini bagian yang sangat menentukan. **Jangan unggah dokumen mentah.** Dokumen yang masuk ke Open WebUI seharusnya sudah melewati:

- pembersihan,
- pelabelan,
- *chunking* yang masuk akal,
- penentuan metadata minimum,
- dan validasi manual.

Dokumentasi Open WebUI menjelaskan bahwa *Knowledge* cocok untuk kontrak, spesifikasi teknis, *research papers*, dan dokumen lain yang perlu dicari berdasarkan makna. Itu artinya dokumen yang masuk harus sudah **bermartabat sebagai pengetahuan**, bukan sekadar file kebetulan ada. ([Open WebUI](#))

### Langkah praktis

Urutan kerja yang saya sarankan:

- buat satu *knowledge base* untuk satu domain, jangan campur semua tema;
- unggah corpus kecil dulu, misalnya 20–50 dokumen;
- cek apakah metadata dan isi dokumen terlihat benar;
- uji beberapa query acuan;
- baru naikan skala.

## Contoh struktur sebelum *ingest*

Misalnya folder lokal Anda seperti ini:

```
./data/cleaned/
├── rag/
│ ├── sop-reindex.md
│ ├── faq-focused-retrieval.md
│ └── embedding-strategies.md
├── logs/
│ ├── auth-log-labeled.json
│ └── nginx-errors-labeled.json
└── programming/
 ├── readme-deploy.md
 ├── api-auth.md
 └── changelog.md
```

Untuk awal, pilih satu domain dulu, misalnya **rag/**.

### *How-to*: langkah kerja sederhana

- masuk ke Open WebUI
- buka area **Knowledge**
- buat satu *knowledge base* baru, misalnya: **rag-lab-2026**
- unggah dokumen dari **./data/cleaned/rag**
- tunggu proses ekstraksi dan embedding selesai
- cek satu per satu file yang masuk

### Checklist awal

- nama *knowledge base* jelas
- dokumen tidak duplikat
- versi lama tidak ikut terbawa
- file yang sangat besar ditinjau lagi
- dokumen yang sensitif tidak salah unggah

### 9.3 Kapan memakai Focused Retrieval, kapan Full Context

Ini keputusan yang jauh lebih penting daripada kelihatannya. Dokumentasi resmi Open WebUI menyatakan dengan cukup tegas:

- **Focused Retrieval (RAG)** dipakai untuk mencari *chunks* paling relevan dari koleksi besar
- **Full Context** dipakai untuk menyuntikkan seluruh dokumen ketika ketepatan terhadap isi utuh lebih penting daripada pencarian potongan ([Open WebUI](#))

Jadi, jangan pilih mode berdasarkan selera. Pilih berdasarkan **jenis dokumen dan jenis pertanyaan**.

**Pakai Focused Retrieval** bila:

- corpus besar
- banyak dokumen
- query spesifik
- ingin efisiensi
- ingin retrieval per *chunk*

Contohnya:

- textbook yang dipecah per bab
- kumpulan SOP
- FAQ banyak dokumen
- dokumentasi teknis campuran

**Pakai Full Context** bila:

- dokumen pendek
- satu file sangat penting dan harus dibaca utuh
- risiko kehilangan konteks lebih besar daripada biaya konteks
- Anda ingin model melihat seluruh dokumen apa adanya

Contohnya:

- satu SOP singkat
- satu kebijakan final
- satu kontrak pendek
- satu README kecil yang memang selalu relevan

## Pelajaran penting

**Full Context bukan berarti lebih cerdas.** Ia hanya berarti seluruh isi file disuntikkan. Kalau file terlalu panjang, konteks bisa boros. Kalau file kotor, semua kekotorannya ikut masuk.

## Cara memilih cepat

Tanyakan:

- apakah saya punya banyak dokumen atau satu dokumen inti?
- apakah pertanyaan pengguna biasanya spesifik atau menyeluruh?
- apakah konteks yang hilang lebih berbahaya daripada konteks yang terlalu banyak?

## 9.4 Pengaturan dokumen, *embedding*, dan *reindex*

Di balik UI yang nyaman, Open WebUI tetap bekerja di atas keputusan teknis yang penting: **bagaimana dokumen diproses, model *embedding* apa yang dipakai, dan kapan basis pengetahuan harus di-*reindex*.** Dokumentasi troubleshooting RAG dan konfigurasi lingkungan sama-sama menegaskan bahwa ketika model *embedding* berubah atau pola penyimpanan tertentu bermigrasi, **Reindex Knowledge Base** perlu dijalankan agar *knowledge base* lama benar-benar mengikuti representasi baru. ([Open WebUI](#))

Ini sangat sejalan dengan pesan di bab-bab sebelumnya: **ganti model *embedding* tanpa *reindex* adalah resep retrieval yang memburuk diam-diam.**

### Langkah kerja yang sehat

- catat model *embedding* yang sedang dipakai
- catat konfigurasi *chunking*
- catat vector backend yang aktif
- ganti satu variabel pada satu waktu
- jalankan *reindex* jika memang representasi berubah
- uji query acuan sebelum dan sesudah

### *How-to*: kebiasaan administrasi minimum

Simpan file manifest seperti ini:

```
embedding_model=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
retrieval_mode=focused
chunking=heading:1200:150
last_reindex=2026-04-13
knowledge_base=rag-lab-2026
```

Kalau terjadi perubahan, buat jejak revisi di Git:

```
git add manifests/ reports/
git commit -m "Ganti embedding model dan jadwalkan reindex
knowledge base"
```

## Cek log bila ada masalah

```
docker logs open-webui --tail 100
```

Contoh keluaran:

```
INFO ... indexing file: embedding-strategies.md
WARN ... embedding dimension mismatch
INFO ... reindex required
```

## 9.5 Strategi query yang sehat

Banyak orang menilai RAG hanya dari satu atau dua pertanyaan spontan. Ini tidak cukup. Query harus dirancang sehat, karena query adalah jendela untuk menilai apakah *knowledge base* benar-benar bekerja.

Saya sarankan query dibagi minimal menjadi empat jenis:

- **faktual langsung**  
"Kapan saya harus melakukan reindex?"
- **parafrase**  
"Dalam kondisi apa basis pengetahuan perlu dibangun ulang?"
- **konseptual**  
"Apa perbedaan Focused Retrieval dan Full Context?"
- **prosedural**  
"Langkah apa saja untuk menambah dokumen ke knowledge base?"

Kalau *knowledge base* hanya bagus pada satu jenis query, itu belum sehat. Query juga sebaiknya:

- memakai istilah domain nyata
- tidak terlalu dibuat-buat
- mencerminkan kebutuhan pengguna sesungguhnya
- disimpan sebagai set uji tetap

## How-to: siapkan file query uji

Misalnya `./tests/queries.yaml`:

```
queries:
 - id: q01
 question: "Kapan saya harus melakukan reindex
knowledge base?"
 expected_titles_contains:
 - "Reindex"

 - id: q02
 question: "Apa perbedaan Focused Retrieval dan Full
Context?"
 expected_titles_contains:
 - "Focused Retrieval"

 - id: q03
 question: "Langkah menambahkan dokumen baru ke corpus
apa saja?"
 expected_titles_contains:
 - "SOP"
```

Lalu uji di pipeline lokal:

```
python eval_retrieval.py \
 --queries ./tests/queries.yaml \
 --vector-store lancedb \
 --db-path ./vectordb \
 --table-name pilot_corpus \
 --top-k 5
```

Contoh keluaran:

```
[INFO] Queries tested: 3
[INFO] Top-1 hit rate: 66.67%
[INFO] Top-5 hit rate: 100.00%
```

## 9.6 Hybrid retrieval, metadata filtering, dan reranking

Di titik tertentu, *semantic search* saja biasanya tidak cukup. Dokumentasi fitur Open WebUI menyebut dukungan **hybrid search** berbasis **BM25 + vector search** dengan **cross-encoder reranking**. Ini sangat penting untuk dunia nyata karena query sering punya dua sifat sekaligus:

- butuh kemiripan makna
- butuh kata kunci spesifik yang eksplisit ([Open WebUI](#))

**Hybrid retrieval** cocok bila:

- corpus heterogen
- ada istilah teknis, kode error, nama fungsi, nomor kebijakan
- semantic retrieval saja sering “hampir benar”

**Metadata filtering** cocok bila:

- ingin menyaring hanya dokumen **approved**
- ingin fokus ke bahasa tertentu
- ingin membatasi pada satu domain
- ingin memisahkan **public** dan **internal**

Dokumentasi Chroma dan Qdrant sama-sama mendukung ide ini: metadata filtering dipakai saat ada atribut objek yang penting tetapi tidak tertangkap utuh oleh embedding. Jadi, filter bukan lawan semantic search, melainkan pengarahnya. ([Open WebUI](#))

**Reranking** cocok bila:

- *top-k* awal sudah lumayan, tetapi urutannya belum tajam
- hasil yang benar sering muncul di posisi 4–10
- query teknis sering kalah oleh dokumen generik

**Cara berpikir praktis**

- semantic search = pencari awal
- hybrid retrieval = pencari yang lebih cerdas untuk dunia nyata
- metadata filtering = pagar pembatas
- reranking = menyusun ulang kandidat

## 9.7 Evaluasi minimum: *recall@k*, MRR, *groundedness*, ketepatan jawaban, jejak sumber

Bab ini akan tidak jujur kalau berhenti di “sudah bisa ditanya”. Kita perlu evaluasi minimum.

**Recall@k** mengukur berapa banyak dokumen relevan yang berhasil masuk ke hasil teratas. Dokumentasi Anserini menjelaskan *Recall@k* sebagai fraksi dari seluruh dokumen relevan yang ditemukan di *top-k*. Ini sangat cocok untuk menilai tahap retrieval pertama. ([Mintlify](#))

**MRR** atau *Mean Reciprocal Rank* mengukur seberapa cepat dokumen relevan pertama muncul. Ini berguna karena pengguna biasanya sangat peduli apakah hasil pertama langsung benar atau tidak. Sumber-sumber IR dan retrieval modern konsisten menempatkan MRR sebagai metrik ranking penting. ([Mintlify](#))

**Groundedness** mengukur apakah jawaban benar-benar didukung oleh konteks yang diambil. Tutorial evaluasi RAG LangSmith menjelaskan *groundedness* sebagai evaluasi antara respons dan dokumen yang diretriev, sedangkan Azure mendefinisikan *groundedness* dalam konteks RAG sebagai ukuran apakah konten model terikat pada materi sumber yang diberikan. ([docs.langchain.com](#))

**Ketepatan jawaban** berarti jawaban memang benar terhadap pertanyaan, bukan sekadar terdengar meyakinkan.

**Jejak sumber** berarti pengguna atau evaluator bisa melihat dari dokumen mana jawaban diambil.

### Cara berpikir minimum

Untuk buku ini, saya sarankan jangan terlalu banyak metrik di awal. Cukup:

- *recall@k*
- MRR
- *groundedness*
- ketepatan jawaban
- jejak sumber

Itu sudah cukup untuk membedakan sistem yang **berguna** dari sistem yang hanya **fasih**.



## 9.8 Prompt evaluasi retrieval dan audit bukti

Salah satu kebiasaan sehat dalam RAG adalah **memeriksa bukti**, bukan hanya jawaban. Karena itu, prompt evaluasi retrieval harus mendorong model atau evaluator manusia untuk:

- menyebut potongan konteks yang dipakai
- menunjukkan sumber
- menandai bagian yang belum cukup bukti
- membedakan fakta dari inferensi

### Contoh prompt audit bukti

Anda adalah evaluator retrieval.

Tugas:

1. Baca pertanyaan pengguna.
2. Baca konteks yang diretriev.
3. Nilai apakah konteks ini benar-benar cukup untuk menjawab.
4. Jika cukup, sebutkan bagian mana yang menjadi bukti.
5. Jika tidak cukup, jelaskan apa yang hilang.
6. Jangan menambah fakta dari luar konteks.

Keluaran:

- relevan / tidak relevan
- bukti utama
- risiko konteks hilang
- apakah jawaban nanti berpotensi halusinasi

### Contoh prompt evaluasi groundedness

Nilai apakah jawaban berikut sepenuhnya didukung oleh konteks retrieval.

Jawab dengan:

- grounded: ya / tidak
- alasan singkat
- kutipan atau ringkasan bukti dari konteks

Prompt seperti ini melatih pembaca untuk **mengaudit retrieval**, bukan sekadar terpukau oleh kelancaran bahasa model.

## 9.9 Kesalahan umum Open WebUI RAG dan cara menghindarnya

Dokumentasi troubleshooting RAG Open WebUI memberi satu pelajaran yang sangat bagus: ketika halusinasi atau jawaban meleset terjadi, sering kali **masalahnya ada pada konteks** ([Open WebUI](#))

Kesalahan umum yang paling sering saya lihat:

- **mengunggah dokumen mentah**  
solusi: bersihkan, labeli, dan pecah dulu
- **mencampur terlalu banyak domain ke satu knowledge base**  
solusi: pisahkan *knowledge base* per domain atau per tujuan
- **salah memilih Focused Retrieval vs Full Context**  
solusi: sesuaikan dengan ukuran dan sifat dokumen
- **mengganti embedding model tanpa reindex**  
solusi: catat perubahan, lakukan *reindex*, lalu uji ulang
- **mengandalkan semantic search saja saat query butuh kata kunci kuat**  
solusi: aktifkan *hybrid retrieval* dan filtering bila perlu
- **tidak punya query uji tetap**  
solusi: simpan set query evaluasi dan bandingkan hasil sebelum/sesudah
- **tidak memeriksa jejak sumber**  
solusi: biasakan audit bukti dan *groundedness*

*How-to*: langkah cepat saat hasil terasa jelek

- cek apakah dokumen yang benar memang ada
- cek mode retrieval yang dipakai
- cek apakah *embedding* baru belum di-*reindex*
- cek apakah *top-k* retrieval memuat dokumen yang benar
- cek apakah jawaban mengambil sumber atau hanya menebak

## 9.10 Mini project: satu *knowledge base* yang benar-benar berguna

Sekarang kita tutup bab ini dengan proyek kecil yang realistis. Targetnya bukan membangun RAG raksasa, melainkan satu *knowledge base* yang **benar-benar berguna**.

### Tema proyek

Buat *knowledge base* kecil untuk topik: **Open WebUI + Ollama + reindex + corpus RAG**

### Langkah kerja

- pilih 15–25 dokumen yang sudah bersih
- masukkan ke satu folder `./data/cleaned/rag-mini`
- pastikan metadata minimum lengkap
- buat satu *knowledge base* `rag-mini-2026`
- unggah ke Open WebUI
- pilih **Focused Retrieval**
- siapkan 10 query uji
- catat:
  - *recall@5*
  - MRR
  - *groundedness*
  - akurasi jawaban
  - jejak sumber

### Struktur sederhana

```
mkdir -p ./data/cleaned/rag-mini
cp ./data/cleaned/rag/*.md ./data/cleaned/rag-mini/
ls -l ./data/cleaned/rag-mini | head
```

### Contoh keluaran:

```
embedding-strategies.md
faq-focused-retrieval.md
sop-reindex.md
```

## Jalankan evaluasi retrieval lokal

```
python ingest_pilot.py --input ./data/cleaned/rag-mini
--limit 25 --vector-store lancedb
python eval_retrieval.py --queries ./tests/rag-mini.yaml
--vector-store lancedb --top-k 5
```

Contoh keluaran:

```
[INFO] Documents ingested: 18
[INFO] Chunks created: 49
[INFO] Top-1 hit rate: 70.00%
[INFO] Top-5 hit rate: 100.00%
```

## Uji di Open WebUI

Setelah *knowledge base* aktif, ajukan pertanyaan seperti:

- “Kapan saya perlu reindex?”
- “Apa beda Focused Retrieval dan Full Context?”
- “Mengapa chunking memengaruhi retrieval?”

Lalu audit:

- apakah sumber yang diambil tepat?
- apakah jawaban menyebutkan bukti yang benar?
- apakah jawaban terasa terlalu umum?
- apakah ada query yang harusnya mudah tetapi tetap gagal?

Kalau proyek kecil ini berhasil, pembaca akan merasakan sesuatu yang jauh lebih berharga daripada demo: **mereka punya basis pengetahuan lokal yang benar-benar bisa dipakai bekerja.**

## Penutup Bab

Bab ini mengajarkan satu hal yang sering hilang di banyak tutorial RAG: **Open WebUI bukan sekadar tempat bertanya ke file; ia adalah tempat mengoperasikan pipeline pengetahuan.** Dokumen dibersihkan, diunggah, di-embed, dicari, lalu diuji. *Focused Retrieval* dan *Full Context* bukan tombol kosmetik, melainkan keputusan arsitektur. *Hybrid retrieval*, metadata filtering, dan reranking bukan fitur mewah, melainkan alat untuk membuat retrieval lebih jujur. *Recall@k*, MRR, *groundedness*, akurasi jawaban, dan jejak sumber bukan sekadar metrik, tetapi cara kita membedakan sistem yang hanya tampak pintar dari sistem yang sungguh bisa dipercaya. ([Open WebUI](#))

Dengan menyelesaikan bab ini, pembaca seharusnya sudah sampai pada titik yang sangat penting: mereka bukan lagi sekadar “memiliki Open WebUI”, tetapi **mulai memiliki RAG yang bisa diaudit.** Itulah fondasi sebelum kita masuk ke keputusan arsitektur yang lebih besar: **kapan RAG sudah cukup, dan kapan PEFT mulai layak dipertimbangkan**

## Bab 10. *Hybrid RAG + FT*: Kapan RAG Cukup, Kapan PEFT Wajib

Di dunia AI praktis, ada dua godaan yang sama-sama berbahaya. Godaan pertama adalah percaya bahwa **RAG saja pasti cukup untuk semua hal**. Godaan kedua adalah percaya bahwa **setiap masalah harus diselesaikan dengan *fine-tuning***. Keduanya salah bila dipakai sebagai dogma. Sistem yang sehat justru lahir dari diagnosis yang benar: **apakah masalah kita ada pada pengetahuan, pada ranking retrieval, pada representasi embedding, atau pada perilaku model?** Bab ini membantu pembaca menjawab pertanyaan itu dengan kepala dingin.

### 10.1 Empat keputusan arsitektur

Saya sarankan pembaca buku ini selalu mulai dari empat pilihan arsitektur yang sederhana tetapi sangat kuat.

#### **Pertama, RAG saja.**

Ini jalur paling hemat dan sering kali sudah cukup. Dokumen dibersihkan, di-*chunk*, di-*embed*, disimpan, lalu dicari saat query datang. Kalau retrieval sehat, model lokal seperti yang berjalan di atas Ollama bisa menjawab dengan cukup baik berdasarkan konteks yang ditemukan.

#### **Kedua, RAG + *reranker*.**

Ini dipakai ketika retrieval awal sudah lumayan, tetapi urutannya belum tajam. Open WebUI sendiri mendukung **hybrid search BM25 + vector search** dan **cross-encoder reranking**, sehingga secara praktis jalur ini memang tersedia dan sangat masuk akal sebelum melompat ke *fine-tuning*. ([Open WebUI](#))

#### **Ketiga, RAG + *fine-tuned embedder*.**

Ini dipakai ketika masalah utama bukan jawaban model, melainkan pencarian konteks. Kalau query yang benar sering gagal menemukan *chunk* yang tepat, atau dokumen yang tepat kalah oleh dokumen yang “mirip tapi salah”, maka mengubah representasi *embedding* sering lebih masuk akal daripada mengubah LLM.

#### **Keempat, *Hybrid RAG + FT*.**

Ini dipakai ketika retrieval sudah sehat, tetapi model dasar masih gagal menghasilkan keluaran yang konsisten, gaya yang tepat, format yang disiplin, atau *reasoning* yang khas domain. Pada titik ini, RAG tetap dipakai untuk memberi fakta dan sumber, sementara PEFT dipakai untuk mengubah perilaku model. Ini sangat sejalan dengan pembagian kerja yang disiratkan oleh bahan-bahan pendukung Anda: RAG untuk **pengetahuan yang dinamis dan terlacak**, FT untuk **perilaku model yang lebih spesifik**.

Cara berpikir cepat

Sebelum menyentuh *fine-tuning*, tanyakan:

- apakah dokumen yang benar sudah berhasil ditemukan?
- apakah masalahnya urutan retrieval?
- apakah masalahnya query dan *embedding*?
- atau memang model dasar tidak bisa mengikuti kebiasaan domain?

## 10.2 Kapan RAG cukup

Saya akan tegas di sini: **dalam banyak kasus, RAG sudah cukup**. Bahkan sering jauh lebih sehat daripada *fine-tuning*.

RAG cukup ketika masalah Anda dominan **faktual**. Misalnya:

- menjawab isi SOP,
- menjelaskan kebijakan,
- mencari langkah konfigurasi,
- mengutip definisi,
- membandingkan dua dokumen,
- merangkum bagian tertentu dari textbook.

RAG juga sangat cocok ketika **kutipan sumber** itu penting. Dokumentasi Open WebUI untuk Knowledge secara eksplisit menekankan bahwa AI dapat mencari dokumen “by meaning” dan **cites where it found the answer**. Bila kebutuhan Anda adalah jawaban yang bisa ditelusuri balik ke sumber, RAG lebih alami daripada *fine-tuning*. ([Open WebUI](#))

RAG juga unggul ketika **data berubah cepat**. SOP direvisi, kebijakan diperbarui, dokumentasi API berubah, *runbook* diperbaiki, dan textbook kerja internal ditambah lampiran baru. Dalam kondisi seperti ini, memperbarui corpus dan melakukan *reindex* biasanya lebih murah, lebih cepat, dan lebih jujur daripada melatih ulang model. Dokumentasi Open WebUI juga menegaskan bahwa ketika model *embedding* atau representasi RAG berubah, *knowledge base* dapat di-*reindex* tanpa harus “mendidik ulang” model generatif. ([Open WebUI](#))

Yang tak kalah penting, RAG cocok ketika **update harus murah**. Menambah dokumen baru ke *knowledge base* jauh lebih ringan daripada membangun pipeline *fine-tuning* lengkap. Karena itu, untuk mahasiswa, praktisi, dan tim kecil, prinsip awal yang paling waras biasanya adalah: **benahi corpus dulu, retrieval dulu, evaluasi dulu, baru pertimbangkan FT bila memang ada gejala kuat bahwa masalahnya ada di perilaku model**.

## Contoh kasus RAG sudah cukup

- chatbot internal untuk SOP operasional
- pencarian kebijakan organisasi
- asisten dokumentasi API
- ringkasan textbook teknis
- penelusuran log dan catatan troubleshooting

## How-to keputusan cepat

Gunakan RAG dulu bila:

- jawaban perlu bukti
- data sering berubah
- domain pengetahuan utama ada di dokumen
- anggaran komputasi terbatas
- Anda ingin sistem yang mudah dirawat

## 10.3 Kapan PEFT wajib mulai dipertimbangkan

Sekarang kita masuk ke sisi lain. **Kapan RAG saja tidak cukup?** Jawabannya: ketika masalah utama tidak lagi ada di fakta, tetapi di **perilaku model**.

PEFT mulai layak—bahkan kadang wajib—dipertimbangkan bila **gaya keluaran harus konsisten**. Misalnya Anda butuh model selalu menjawab dalam format audit tertentu, selalu memberi struktur laporan tertentu, atau selalu menulis sesuai pola komunikasi organisasi.

PEFT juga mulai relevan ketika **format keluaran sangat khusus**. Misalnya:

- hasil harus berbentuk JSON yang sangat disiplin,
- format tiket insiden harus baku,
- keluaran harus berupa template teknis tertentu,
- model harus selalu mematuhi urutan bagian yang spesifik.

Selain itu, PEFT menjadi masuk akal ketika **reasoning domain sangat khas**. Bukan fakta yang bermasalah, tetapi cara menjawab. Contohnya:

- model harus selalu menilai log dengan kerangka insiden tertentu,
- model harus selalu membaca dokumen hukum internal dengan pola penalaran tertentu,
- model harus menulis *memo* teknis dengan struktur yang sangat khas organisasi.

Yang terakhir, PEFT layak dipertimbangkan ketika **model dasar sering gagal mengikuti kebiasaan domain**, bahkan setelah retrieval sudah sehat. Di titik ini, Anda mungkin sudah berhasil menemukan dokumen yang benar, tetapi jawaban model tetap:

- terlalu umum,
- tidak disiplin format,
- salah gaya,
- atau tidak mampu menyusun jawaban sesuai kebiasaan kerja domain.

Di sinilah banyak orang keliru. Mereka melihat hasil jelek, lalu langsung menyalahkan model. Padahal diagnosis yang benar adalah: **apakah retrieval sudah sehat?** Kalau belum, kembali ke RAG. Kalau sudah sehat dan masalah tetap ada pada perilaku, baru PEFT masuk akal.

## 10.4 LoRA, QLoRA, DoRA dalam bahasa praktis

Sekarang mari kita turunkan jargon ke bahasa yang lebih membumi.

**LoRA** adalah metode PEFT yang menambahkan matriks kecil berperingkat rendah untuk menyesuaikan bobot model, alih-alih melatih ulang seluruh model. Dokumentasi resmi PEFT menjelaskannya sebagai **Low-Rank Adaptation** yang mengurangi jumlah parameter yang perlu dilatih secara drastis. Dalam bahasa praktis: **LoRA itu seperti menempelkan “tambalan cerdas” ke model**, bukan membongkar seluruh isi model. ([Hugging Face](#))

**QLoRA** adalah pendekatan yang menggabungkan **kuantisasi 4-bit** dengan adapter LoRA agar model besar bisa dilatih jauh lebih hemat memori. Dokumentasi quantization Transformers menjelaskan QLoRA sebagai fine-tuning LoRA di atas model yang diquantize 4-bit, sehingga biaya VRAM turun tajam dibanding melatih model penuh. Dalam bahasa praktis: **QLoRA adalah cara membuat LoRA lebih murah secara memori**. ([huggingface.co](#))

**DoRA** kini juga didukung dalam dokumentasi LoRA PEFT sebagai opsi `use_dora`, dan idenya adalah memisahkan pembaruan **direction** dan **magnitude** dari bobot. Dalam bahasa praktis: **DoRA mencoba membuat adaptasi lebih halus dan kadang lebih kuat daripada LoRA biasa**, tanpa harus jatuh ke *full fine-tuning*. Dokumentasi LoRA PEFT juga mencatat bahwa DoRA hanya mendukung beberapa jenis layer tertentu, sehingga pembaca perlu sadar bahwa ini bukan saklar universal untuk semua hal. ([Hugging Face](#))



## Ringkas dalam bahasa lapangan

- **LoRA**: paling mudah dipahami, ringan, cocok untuk banyak kebutuhan PEFT awal
- **QLoRA**: cocok saat memori terbatas dan model besar ingin tetap dilatih
- **DoRA**: opsi lebih lanjut saat ingin adaptasi yang lebih halus, tetapi kompleksitasnya naik

## How-to mental model

Bayangkan:

- RAG = memperbaiki **apa yang model baca**
- LoRA / QLoRA / DoRA = memperbaiki **cara model bereaksi terhadap apa yang dibaca**

## 10.5 Menyiapkan dataset training dari corpus

Kalau akhirnya Anda memang perlu PEFT, pertanyaan berikutnya adalah: **dataset training-nya datang dari mana?** Jawaban terbaik untuk buku ini adalah: **dari corpus yang sudah sehat.**

Ini alasan pentingnya seluruh buku ini dibangun dari corpus terlebih dahulu. Dataset training yang baik bukan lahir dari copy-paste acak, melainkan dari:

- dokumen yang sudah dibersihkan,
- metadata yang jelas,
- label yang konsisten,
- *chunking* yang sehat,
- dan query/jawaban evaluasi yang sudah teruji.

## Bentuk dataset yang masuk akal

Untuk kebutuhan PEFT, dataset bisa berbentuk:

- **instruction-response**
- **question-answer**
- **task-output**
- **format-constrained examples**

Misalnya:

- pertanyaan tentang reindex + jawaban yang mengikuti format tertentu
- potongan log + label insiden + justifikasi singkat
- potongan SOP + ringkasan prosedur yang baku
- dokumentasi API + jawaban dalam format JSON yang disiplin

## How-to membangun dataset awal

Mulai dari hal kecil:

- ambil 50–200 contoh yang benar-benar sehat
- jangan kejar jumlah besar dulu
- pastikan setiap contoh relevan dengan perilaku yang ingin diubah
- pisahkan *train* dan *validation*
- simpan metadata asal contoh

Contoh struktur file JSONL:

```
{ "instruction": "Kapan knowledge base perlu
di-reindex?", "input": "Dokumen menjelaskan bahwa perubahan
embedding model memerlukan reindex.", "output": "Knowledge
base perlu di-reindex ketika model embedding berubah atau
struktur representasi retrieval diubah." }
{ "instruction": "Ringkas SOP ini dalam 3
langkah.", "input": "...isi SOP...", "output": "1. Cadangkan
dokumen. 2. Jalankan reindex. 3. Uji query pembandingan." }
```

## CLI contoh membuat dataset ringkas

```
python - <<'PY'
import json

rows = [
 {
 "instruction": "Kapan knowledge base perlu
di-reindex?",
 "input": "Dokumen internal menjelaskan bahwa
perubahan embedding model memerlukan reindex.",
 "output": "Knowledge base perlu di-reindex ketika
model embedding berubah."
 }
]

with open("train.jsonl", "w", encoding="utf-8") as f:
 for row in rows:
 f.write(json.dumps(row, ensure_ascii=False) +
"\n")

print("Saved train.jsonl")
PY
```

Contoh keluaran:

```
Saved train.jsonl
```

## 10.6 Kapan *fine-tune embedder* lebih masuk akal daripada *fine-tune LLM*

Ini salah satu keputusan yang paling sering diabaikan. Banyak orang langsung berpikir untuk melatih LLM, padahal masalahnya ada di **retrieval**. Kalau query yang benar tidak berhasil menemukan dokumen yang benar, maka mengubah perilaku model generatif tidak akan menyelesaikan masalah.

Di titik seperti itu, **fine-tune embedder** bisa lebih masuk akal. Tanda-tandanya:

- query domain sering salah arah
- istilah lokal atau teknis tidak tertangkap baik
- dokumen yang tepat sering kalah oleh dokumen yang mirip tetapi salah
- *top-k* retrieval buruk, bahkan sebelum jawaban dihasilkan

Dalam kondisi seperti ini, memperbaiki representasi embedding sering lebih hemat dan lebih tepat sasaran daripada melatih LLM. Ini juga sejalan dengan arsitektur yang disarankan: **RAG + fine-tuned embedder** dipisahkan sebagai keputusan ketiga, berbeda dari *Hybrid RAG + FT*. Artinya masalah retrieval dan masalah perilaku memang harus dipisahkan.

### Ringkasan sederhana

- kalau **dokumen yang benar belum ketemu** → pikirkan embedder
- kalau **dokumen yang benar sudah ketemu tapi jawaban masih jelek** → pikirkan LLM / PEFT

### How-to diagnosis praktis

Gunakan tiga tahap:

- cek *top-k hit rate*
- cek kualitas *reranking*
- baru cek kualitas generation

Kalau retrieval awal masih buruk, jangan lompat ke PEFT untuk LLM.

## 10.7 Teknik synthetic data: kapan berguna, kapan berbahaya

*Synthetic data* sangat menggoda. Ia bisa mempercepat pembuatan dataset, memperbanyak contoh, dan membantu menutup kekurangan data manual. Tetapi ia juga bisa menambah **noise yang tampak rapi**. Jadi bagian ini harus ditulis dengan nada dewasa: **synthetic data berguna, tetapi tidak boleh dipuja**.

### Kapan berguna

- untuk memperkaya variasi pertanyaan
- untuk menghasilkan parafrase
- untuk membuat contoh format keluaran
- untuk memperbanyak skenario yang strukturnya jelas
- untuk data latihan awal saat contoh manual masih sedikit

### Kapan berbahaya

- bila dijadikan pengganti verifikasi manusia
- bila sumber aslinya sendiri belum sehat
- bila model pembuat data sintetik membawa bias atau kesalahan besar
- bila semua variasi terasa “berbeda di permukaan, sama di inti”
- bila contoh sintetik lebih dominan daripada data nyata tanpa kontrol kualitas

### Aturan sehat

- mulai dari data nyata
- tambahkan sintetik sebagai pelengkap
- selalu beri label mana data nyata, mana sintetik
- evaluasi performa model pada data nyata, bukan hanya data sintetik

### How-to membuat sintetik secara hati-hati

Misalnya untuk memperbanyak variasi pertanyaan:

```
Tugas:
Buat 5 variasi pertanyaan dari kalimat berikut tanpa mengubah
makna teknisnya.
Pertanyaan asli:
"Kapan knowledge base perlu di-reindex?"
Batasi hasil pada Bahasa Indonesia formal, singkat, dan tetap
relevan untuk domain RAG.
```

Lalu simpan hasilnya dengan penanda:

```
{"instruction": "Dalam kondisi apa basis pengetahuan harus
dibangun ulang?", "source_type": "synthetic", "derived_from": "q01"}
```

Dengan cara ini, *synthetic data* menjadi alat bantu, bukan sumber kebenaran palsu.

## 10.8 Penutup: corpus sebagai infrastruktur pengetahuan jangka panjang

Kalau seluruh buku ini dibaca dengan benar, maka pembaca seharusnya sampai pada satu perubahan cara pandang yang sangat penting: **corpus bukan tahap persiapan kecil sebelum AI, tetapi infrastruktur pengetahuan jangka panjang**. RAG yang sehat berdiri di atas corpus. Retrieval yang tajam berdiri di atas corpus. *Fine-tuned embedder* pun berdiri di atas corpus. Bahkan dataset training untuk PEFT yang sehat seharusnya lahir dari corpus yang sudah dibersihkan, dilabeli, diuji, dan dipahami.

Itulah sebabnya bab ini saya tutup bukan dengan ajakan untuk buru-buru melatih model, melainkan dengan ajakan yang lebih tenang: **diagnosis dulu sebelum tuning**. Bila masalahnya faktual, dinamis, dan butuh kutipan, RAG sering sudah cukup. Bila masalahnya ranking, tambahkan *reranker*. Bila masalahnya representasi retrieval, pertimbangkan *fine-tuned embedder*. Bila masalahnya perilaku model, format, atau *reasoning* domain, barulah PEFT layak dipertimbangkan. Dengan urutan seperti ini, pembaca tidak akan mudah tertipu oleh ilusi bahwa semakin banyak pelatihan pasti semakin baik. Sering justru yang dibutuhkan adalah **corpus yang lebih sehat, retrieval yang lebih jujur, dan evaluasi yang lebih disiplin**. ([Hugging Face](#))

### Ringkasan keputusan praktis

Agar mudah diingat pembaca, gunakan peta keputusan berikut:

- **RAG saja**
  - masalah faktual
  - perlu sumber
  - data cepat berubah
  - update murah lebih penting
- **RAG + reranker**
  - retrieval awal lumayan
  - urutan hasil belum tajam
  - banyak istilah teknis atau *keyword* penting
- **RAG + fine-tuned embedder**
  - dokumen yang benar sering gagal ditemukan
  - istilah domain tidak tertangkap baik
  - *top-k* retrieval buruk
- **Hybrid RAG + FT**
  - dokumen yang benar sudah ketemu
  - tetapi gaya, format, dan *reasoning* model masih tidak sesuai domain

## CLI penutup yang sehat

Sebelum memutuskan FT, biasakan cek retrieval dulu:

```
python eval_retrieval.py \
 --queries ./tests/queries.yaml \
 --vector-store lancedb \
 --db-path ./vectordb \
 --table-name pilot_corpus \
 --top-k 5
```

Contoh keluaran:

```
[INFO] Queries tested: 20
[INFO] Top-1 hit rate: 55.00%
[INFO] Top-5 hit rate: 90.00%
```

Kalau *top-1* dan *top-k* masih jelek, **jangan buru-buru FT LLM**. Benahi retrieval dulu. Itu sering jauh lebih murah, lebih cepat, dan lebih benar.

# Lampiran A. Checklist Instalasi Cepat Ubuntu 26.04 + Ollama + Open WebUI

Lampiran ini dibuat untuk pembaca yang ingin **segera punya lingkungan kerja RAG lokal yang hidup** di atas **Ubuntu 26.04 + Ollama + Open WebUI**. Fokusnya bukan konfigurasi paling mewah, tetapi **jalur tercepat yang tetap sehat**. Perlu dicatat, dokumentasi resmi Ubuntu masih menandai **Ubuntu 26.04 LTS** sebagai rilis yang **sedang dalam pengembangan**, dengan **rilis final dijadwalkan 23 April 2026**. Karena itu, pembaca perlu siap bila ada perubahan kecil pada nama paket atau perilaku repository. ([Dokumentasi Ubuntu](#))

## A.1 Prinsip sebelum mulai

Gunakan tiga prinsip sederhana ini:

- **pin versi seperlunya**
- **pisahkan host, model, dan UI**
- **uji setiap lapisan sebelum lanjut**

Dokumentasi Open WebUI secara eksplisit menyarankan **memakai tag versi spesifik** untuk lingkungan produksi, bukan *floating tags* seperti `main` atau `latest`. ([Open WebUI](#))

## A.2 Checklist cepat

### 1. Verifikasi sistem operasi

Pastikan mesin benar-benar memakai Ubuntu 26.04.

```
lsb_release -a
uname -r
```

Contoh output:

```
Distributor ID: Ubuntu
Description: Ubuntu 26.04 LTS
Release: 26.04
Codename: resolute
```

#### Checklist

- Ubuntu terpasang
- user punya akses `sudo`
- koneksi internet aktif
- ruang disk cukup untuk model dan container

## 2. Update sistem

```
sudo apt update
sudo apt -y upgrade
sudo apt -y autoremove
```

Contoh output:

```
Reading package lists... Done
Building dependency tree... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not
upgraded.
```

### Checklist

- indeks paket terbaru
- paket dasar sudah diperbarui
- reboot dilakukan bila kernel berubah

## 3. Pasang paket dasar host

```
sudo apt install -y \
 curl wget ca-certificates gnupg lsb-release \
 git build-essential python3 python3-pip python3-venv
```

Contoh output:

```
The following NEW packages will be installed:
build-essential git python3-pip python3-venv ...
```

### Checklist

- `curl` tersedia
- `git` tersedia
- `python3` tersedia
- `venv` tersedia

## 4. Verifikasi Python

```
python3 --version
pip3 --version
```

Contoh output:

```
Python 3.12.3
pip 24.0 from /usr/lib/python3/dist-packages/pip (python
3.12)
```

### Checklist

- Python jalan
- pip jalan



## 5. Instal Docker Engine + Compose plugin

Dokumentasi resmi Docker untuk Ubuntu merekomendasikan pemasangan dari repositori resmi Docker, lalu memasang `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin`, dan `docker-compose-plugin`. Plugin Compose resmi diverifikasi dengan `docker compose version`. ([Docker Documentation](#))

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL
https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
 "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc] \
 https://download.docker.com/linux/ubuntu \
 $(. /etc/os-release && echo
${UBUNTU_CODENAME:-$VERSION_CODENAME}) stable" | \
 sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

Verifikasi:

```
sudo systemctl status docker --no-pager
docker compose version
```

Contoh output:

```
• docker.service - Docker Application Container Engine
 Active: active (running)
```

```
Docker Compose version v2.39.4
```

### Checklist

- Docker aktif
- Compose plugin aktif
- service `docker` berjalan

## 6. Tambahkan user ke grup docker

```
sudo usermod -aG docker $USER
newgrp docker
docker version
```

Contoh output:

```
Client: Docker Engine - Community
Version: 29.4.0
```

### Checklist

- user bisa menjalankan **docker** tanpa **sudo**

## 7. Instal Ollama di Linux

Dokumentasi resmi Ollama untuk Linux menyediakan *installer* cepat dan juga mendukung pemasangan versi spesifik melalui **OLLAMA\_VERSION**.

Dokumentasi yang sama juga merekomendasikan menjalankan Ollama sebagai **startup service** dengan **systemd**. ([Ollama Docs](#))

Cara cepat:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama -v
```

Contoh output:

```
ollama version is 0.6.5
```

### Checklist

- Ollama terpasang
- versi tercatat

## 8. Aktifkan Ollama sebagai service

```
sudo useradd -r -s /bin/false -U -m -d /usr/share/ollama
ollama || true
sudo usermod -a -G ollama $(whoami)

sudo tee /etc/systemd/system/ollama.service > /dev/null
<<'EOF'
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="PATH=/usr/bin:/usr/local/bin"

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable ollama
sudo systemctl start ollama
sudo systemctl status ollama --no-pager
```

Contoh output:

- ollama.service - Ollama Service  
Active: active (running)

### Checklist

- **ollama.service** aktif
- service otomatis berjalan saat boot

## 9. Bila perlu, set `OLLAMA_HOST`

Dokumentasi FAQ Ollama menjelaskan bahwa bila Ollama dijalankan dengan `systemd`, variabel lingkungan seperti `OLLAMA_HOST` sebaiknya diatur lewat `systemctl edit ollama.service`. ([Ollama Docs](#))

```
sudo systemctl edit ollama.service
```

Isi override:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0:11434"
```

Lalu:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Cek:

```
curl http://localhost:11434/api/tags
```

Contoh output:

```
{"models": []}
```

### Checklist

- API Ollama merespons
- host dan port sesuai kebutuhan

## 10. Tarik satu model uji

```
ollama pull qwen3:8b
ollama list
```

Contoh output:

```
pulling manifest
writing manifest
success
```

NAME	ID	SIZE
qwen3:8b	1a2b3c4d5e6f	5.2 GB

### Checklist

- minimal satu model tersedia
- `ollama list` menampilkan model

## 11. Siapkan folder proyek

```
mkdir -p ~/rag-lab/openwebui-stack/data/open-webui
cd ~/rag-lab/openwebui-stack
```

### Checklist

- folder proyek ada
- volume data persisten sudah disiapkan

## 12. Buat docker-compose.yml untuk Open WebUI

Dokumentasi Open WebUI menampilkan jalur *Quick Start* berbasis Docker dan menegaskan bahwa untuk penggunaan yang lebih stabil, **tag versi spesifik** sebaiknya dipin. ([Open WebUI](#))

Contoh **docker-compose.yml** minimum yang direkomendasikan:

```
services:
 openwebui:
 image: ghcr.io/open-webui/open-webui:v0.8.6
 container_name: open-webui
 restart: always
 ports:
 - "3000:8080"
 environment:
 - OLLAMA_BASE_URL=http://host.docker.internal:11434
 extra_hosts:
 - "host.docker.internal:host-gateway"
 volumes:
 - ./data/open-webui:/app/backend/data
```

### Checklist

- image memakai tag versi spesifik
- volume data persisten aktif
- **OLLAMA\_BASE\_URL** diarahkan ke host Ollama

### 13. Jalankan Open WebUI

```
docker compose up -d
docker ps
```

Contoh output:

NAME	IMAGE	STATUS
open-webui	open-webui:v0.8.6	Up 10 seconds

#### Checklist

- container **open-webui** hidup
- port **3000** terbuka

### 14. Cek UI Open WebUI

```
curl -I http://localhost:3000
```

Contoh output:

```
HTTP/1.1 200 OK
server: uvicorn
```

Buka di browser:

```
http://localhost:3000
```

#### Checklist

- halaman login muncul
- akun admin pertama berhasil dibuat

### 15. Uji koneksi Open WebUI ke Ollama

Cek dari host:

```
curl http://localhost:11434/api/tags
```

Cek dari container:

```
docker exec -it open-webui sh -c "wget -qO-
http://host.docker.internal:11434/api/tags"
```

Contoh output:

```
{"models":[{"name":"qwen3:8b","size":5583457484}]}
```

#### Checklist

- model tampil dari host
- model juga terlihat dari container
- Open WebUI bisa membaca Ollama

## 16. Uji *knowledge* pertama

Dokumentasi Knowledge Open WebUI menjelaskan bahwa dokumen yang diunggah dapat dipakai dengan **Focused Retrieval** atau **Full Context**. ([Open WebUI](#))

Langkah cepat:

- buat satu file Markdown kecil
- unggah ke **Knowledge**
- pilih model dari Ollama
- ajukan 2–3 pertanyaan sederhana

Contoh file:

```
cat > sample-note.md <<'EOF'
Apa itu Reindex

Reindex diperlukan ketika model embedding berubah atau
struktur representasi retrieval berubah besar.
EOF
```

### Checklist

- file bisa diunggah
- model bisa menjawab berdasarkan file
- sumber jawaban terlihat masuk akal

## A.3 Checklist sanity check akhir

Gunakan daftar ini sebelum Anda menganggap instalasi selesai:

- Ubuntu 26.04 aktif dan stabil
- Docker aktif
- Docker Compose aktif
- Ollama aktif sebagai service
- satu model sudah di-*pull*
- Open WebUI aktif di port **3000**
- Open WebUI bisa menjangkau Ollama
- satu file uji berhasil diunggah ke *Knowledge*
- satu pertanyaan uji berhasil dijawab
- semua versi penting sudah dicatat

## A.4 Perintah cek cepat yang paling berguna

```
lsb_release -a
python3 --version
docker compose version
systemctl status docker --no-pager
systemctl status ollama --no-pager
ollama list
curl http://localhost:11434/api/tags
curl -I http://localhost:3000
docker logs open-webui --tail 50
journalctl -u ollama --no-pager --follow --pager-end
```

Dokumentasi troubleshooting Ollama sendiri merekomendasikan melihat log di Linux `systemd` melalui `journalctl -u ollama --no-pager --follow --pager-end`. ([Ollama Docs](#))

## A.5 Masalah umum dan solusi cepat

**Masalah:** Open WebUI hidup, tetapi model kosong

**Cek:**

```
curl http://localhost:11434/api/tags
docker exec -it open-webui sh -c "wget -qO-
http://host.docker.internal:11434/api/tags"
```

**Arah solusi:** periksa `OLLAMA_BASE_URL`, `extra_hosts`, dan status service Ollama. ([Ollama Docs](#))

**Masalah:** update Open WebUI membuat perilaku berubah

**Arah solusi:** jangan pakai `main` atau `latest`; pakai tag versi spesifik dan *rollback* bila perlu. Dokumentasi Open WebUI menyediakan panduan *rolling back* dengan pin tag versi. ([Open WebUI](#))

**Masalah:** Open WebUI terasa berat

**Arah solusi:** kecilkan skala model, mulai dari *knowledge base* kecil, dan tinjau panduan optimasi RAM/performa Open WebUI. ([Open WebUI](#))



## A.6 Catatan penutup

Checklist ini sengaja dibuat **cepat, lurus, dan tidak cerewet**. Tujuannya satu: membuat pembaca secepat mungkin sampai ke titik di mana:

- **Ollama hidup**
- **Open WebUI hidup**
- **model lokal bisa dipakai**
- **dokumen pertama bisa diuji**

Setelah titik itu tercapai, pekerjaan yang lebih penting baru dimulai: **membangun corpus yang sehat, melakukan retrieval yang jujur, dan mengevaluasi hasilnya dengan disiplin**. Itulah inti buku ini.

# Lampiran B. Template DOC Markdown + YAML untuk RAG

Lampiran ini berisi template dokumen kerja yang bisa dipakai untuk:

- *knowledge note*
- SOP
- FAQ
- catatan teknis
- tutorial singkat
- dokumen logis lain yang ingin dimasukkan ke corpus RAG

Prinsip dasarnya sederhana:

- gunakan Markdown untuk isi
- gunakan YAML *front matter* untuk metadata
- jaga satu dokumen tetap fokus
- buat judul dan subjudul jelas
- simpan kode dalam *code block*
- hindari paragraf terlalu panjang

## B.1 Aturan minimum template

Setiap dokumen sebaiknya punya tiga lapisan:

### Lapisan 1. Metadata YAML

Berisi identitas dokumen.

### Lapisan 2. Struktur isi Markdown

Berisi judul, tujuan, isi utama, langkah, risiko, atau FAQ.

### Lapisan 3. Penanda konteks

Berisi label seperti topik, prioritas, *freshness*, dan sensitivitas.

## B.2 Template umum paling aman

Gunakan ini sebagai template dasar untuk hampir semua dokumen:

```

title: "Judul Dokumen"
source: "asal-dokumen"
date: "2026-04-13"
category: "knowledge-note"
domain: "rag"
language: "id"
```

```
sensitivity: "internal"
quality: "reviewed"
topic: "rag"
subtopic: "metadata"
entities: ["Open WebUI", "Ollama"]
confidence: "high"
priority: "medium"
freshness: "current"
tags: ["rag", "metadata", "yaml", "markdown"]
version: "1.0"
author: "Nama Penulis atau Tim"
status: "active"

```

# Judul Dokumen

## Tujuan

Tuliskan tujuan dokumen ini dalam 1-3 kalimat singkat.

## Ringkasan

Tuliskan ringkasan inti dokumen dalam paragraf pendek.

## Isi Utama

Tuliskan isi utama dokumen secara ringkas, jelas, dan fokus.

## Langkah atau Penjelasan

1. Langkah pertama
2. Langkah kedua
3. Langkah ketiga

## Catatan Penting

- Poin penting 1
- Poin penting 2
- Poin penting 3

## Risiko atau Batasan

Tuliskan keterbatasan, asumsi, atau hal yang perlu diwaspadai.

## Referensi Internal

- Nama file lain
- SOP terkait
- Catatan teknis terkait

Template ini cocok untuk kebanyakan kebutuhan awal.

## B.3 Arti field YAML yang paling penting

Berikut field yang paling berguna dan sebaiknya dipertahankan konsisten:

- **title**  
Judul dokumen. Harus jelas dan spesifik.
- **source**  
Asal dokumen. Misalnya: `lab-rag`, `tim-ai-lokal`, `sop-internal`, `hasil-uji`.
- **date**  
Tanggal dokumen dibuat atau diperbarui.
- **category**  
Jenis dokumen. Misalnya: `knowledge-note`, `SOP`, `FAQ`, `technical-note`, `tutorial`.
- **domain**  
Domain utama. Misalnya: `rag`, `security`, `python`, `programming`.
- **language**  
Bahasa dokumen, misalnya `id` atau `en`.
- **sensitivity**  
Tingkat sensitivitas: `public`, `internal`, `restricted`.
- **quality**  
Mutu dokumen: `draft`, `reviewed`, `approved`.
- **topic**  
Topik utama.
- **subtopic**  
Fokus yang lebih sempit.
- **entities**  
Nama sistem, tool, layanan, istilah penting.
- **confidence**  
Tingkat keyakinan terhadap isi: `low`, `medium`, `high`.
- **priority**  
Prioritas pemakaian dokumen: `low`, `medium`, `high`.
- **freshness**  
Kondisi kebaruan: `current`, `review-soon`, `stale`.
- **tags**  
Kata kunci tambahan.
- **version**  
Versi dokumen.
- **status**  
Status hidup dokumen: `active`, `deprecated`, `archived`.

## B.4 Template untuk knowledge note

Cocok untuk penjelasan satu konsep.

```

title: "Apa itu Focused Retrieval di Open WebUI"
source: "catatan-belajar"
date: "2026-04-13"
category: "knowledge-note"
domain: "rag"
language: "id"
sensitivity: "public"
quality: "reviewed"
topic: "rag"
subtopic: "focused-retrieval"
entities: ["Open WebUI"]
confidence: "high"
priority: "medium"
freshness: "current"
tags: ["focused-retrieval", "knowledge-base"]
version: "1.0"
status: "active"

Apa itu Focused Retrieval di Open WebUI

Definisi
Focused Retrieval adalah mode RAG yang mengambil *chunk* paling
relevan dari kumpulan dokumen.

Kapan Dipakai
Gunakan saat corpus cukup besar dan tidak semua isi dokumen perlu
dimasukkan ke konteks.

Kelebihan
- lebih hemat konteks
- lebih cocok untuk banyak dokumen
- lebih tepat untuk pertanyaan spesifik

Risiko
Jika dokumen kotor atau *chunking* buruk, hasil retrieval bisa
meleset.
```

## B.5 Template untuk SOP

Cocok untuk prosedur operasional.

```

title: "SOP Reindex Knowledge Base"
source: "tim-ai-lokal"
date: "2026-04-13"
category: "SOP"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "approved"
topic: "rag"
subtopic: "reindex"
entities: ["Open WebUI", "embedding"]
confidence: "high"
priority: "high"
freshness: "current"
tags: ["sop", "reindex", "knowledge-base"]
version: "1.2"
status: "active"

SOP Reindex Knowledge Base

Tujuan
Menjelaskan langkah reindex saat model *embedding* berubah.

Prasyarat
- akses admin tersedia
- dokumen sumber masih ada
- model *embedding* baru sudah ditentukan

Langkah
1. Catat model *embedding* lama.
2. Ganti konfigurasi model *embedding*.
3. Jalankan *reindex knowledge base*.
4. Uji 10 query pembandingan.
5. Catat hasil sebelum dan sesudah.

Output yang Diharapkan
- dokumen lama berhasil diindeks ulang
- retrieval membaik atau tetap stabil
- tidak ada error dimensi *embedding*

Risiko
- hasil retrieval memburuk jika corpus lama bermasalah
- waktu indeks ulang bisa lama untuk corpus besar
```

## B.6 Template untuk FAQ

Cocok untuk pertanyaan yang sering muncul.

```

title: "FAQ Knowledge Base RAG"
source: "lab-rag"
date: "2026-04-13"
category: "FAQ"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "reviewed"
topic: "rag"
subtopic: "faq"
entities: ["Open WebUI", "Ollama"]
confidence: "high"
priority: "medium"
freshness: "current"
tags: ["faq", "rag", "knowledge-base"]
version: "1.0"
status: "active"

FAQ Knowledge Base RAG

Pertanyaan
Kapan saya harus melakukan *reindex*?

Jawaban
Lakukan *reindex* ketika model *embedding* berubah atau struktur
representasi retrieval berubah besar.

Pertanyaan
Kapan memakai *Focused Retrieval*?

Jawaban
Gunakan saat corpus besar dan pertanyaan cenderung spesifik.

Pertanyaan
Kapan memakai *Full Context*?

Jawaban
Gunakan untuk dokumen pendek yang perlu dibaca utuh oleh model.
```

## B.7 Template untuk catatan teknis

Cocok untuk hasil uji, observasi, dan troubleshooting.

```

title: "Catatan Uji Koneksi Open WebUI ke Ollama"
source: "lab-rag"
date: "2026-04-13"
category: "technical-note"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "draft"
topic: "deployment"
subtopic: "connectivity"
entities: ["Open WebUI", "Ollama", "Docker"]
confidence: "medium"
priority: "medium"
freshness: "current"
tags: ["connectivity", "docker", "ollama"]
version: "0.9"
status: "active"

Catatan Uji Koneksi Open WebUI ke Ollama

Tujuan
Mencatat hasil uji koneksi antara container Open WebUI dan layanan Ollama di host.

Hasil
- UI hidup di port 3000
- API Ollama aktif di port 11434
- koneksi berhasil dengan `host.docker.internal`

Masalah
Pada percobaan awal, model tidak muncul karena `OLLAMA_BASE_URL` salah.

Solusi
Gunakan:
curl http://localhost:11434/api/tags
docker exec -it open-webui sh -c "wget -qO- http://host.docker.internal:11434/api/tags"

Kesimpulan
Masalah ada pada alamat host, bukan pada model atau container.
```



## B.8 Template untuk tutorial singkat

Cocok untuk \*how-to\* satu tujuan.

```

title: "Cara Menarik Model di Ollama"
source: "lab-rag"
date: "2026-04-13"
category: "tutorial"
domain: "llm"
language: "id"
sensitivity: "public"
quality: "reviewed"
topic: "ollama"
subtopic: "model-pull"
entities: ["Ollama"]
confidence: "high"
priority: "medium"
freshness: "current"
tags: ["ollama", "model", "pull"]
version: "1.0"
status: "active"

Cara Menarik Model di Ollama

Tujuan
Menjelaskan langkah dasar untuk mengambil model lokal.

Langkah
```bash
ollama pull qwen3:8b
ollama list

## Output yang Diharapkan
pulling manifest
writing manifest
success

## Catatan
Pastikan layanan Ollama aktif sebelum menjalankan perintah ini.
```

B.9 Template untuk dokumen log yang sudah diperkaya

Cocok untuk corpus log yang sudah dinormalisasi.

```
{
  "title": "SSH Failed Login Event",
  "source": "auth.log",
  "date": "2026-04-13T02:14:22Z",
  "category": "log-event",
  "domain": "security",
  "language": "system",
  "sensitivity": "internal",
  "quality": "reviewed",
  "topic": "authentication",
  "subtopic": "failed-login",
  "entities": ["sshd", "web01"],
  "confidence": "medium",
  "priority": "high",
  "freshness": "current",
  "tags": ["ssh", "failed-login", "security"],
  "status": "active",
  "timestamp": "2026-04-13T02:14:22Z",
  "host": "web01",
  "service": "sshd",
  "user": "admin",
  "ip": "203.0.113.8",
  "severity": "warning",
  "event": "failed_login",
  "outcome": "failure",
  "label": "suspicious",
  "text": "Failed password for invalid user admin from 203.0.113.8
port 54422 ssh2"
}
```

B.10 Template untuk dokumen programming

Cocok untuk README, API docs, atau catatan fungsi.

```
---
title: "README Instalasi Service RAG"
source: "repo-internal"
date: "2026-04-13"
category: "README"
domain: "programming"
language: "en"
sensitivity: "internal"
quality: "reviewed"
topic: "deployment"
subtopic: "installation"
entities: ["Docker", "Open WebUI", "Ollama"]
confidence: "high"
priority: "medium"
freshness: "current"
tags: ["deployment", "docker", "rag"]
version: "1.0"
status: "active"
repo: "rag-lab"
component: "deploy"
---

# README Instalasi Service RAG

## Purpose
Describe the minimum steps to launch the local RAG stack.

## Steps
docker compose up -d
ollama list
curl http://localhost:11434/api/tags

## Notes
Make sure Ollama is active.
Make sure the Open WebUI container can reach the Ollama API.
```

B.11 Aturan isi dokumen yang disarankan

Gunakan checklist ini setiap kali menulis dokumen baru:

- **Judul harus jelas**
- **Setiap dokumen hanya membahas satu topik utama**
- **Metadata minimum harus lengkap**
- **Gunakan paragraf pendek**
- **Gunakan daftar untuk menjelaskan langkah**
- **Code *block* harus utuh**
- **Istilah harus konsisten**
- **Jangan mencampur versi draft dan final tanpa penanda yang jelas**
- **Tingkat sensitivitas dokumen harus ditulis dengan jelas**
- **Status dokumen harus jelas**

B.12 Howto membuat dokumen baru dengan cepat

Buat folder kerja

```
mkdir -p ./docs/manual  
cd ./docs/manual
```

Buat file baru

```
nano faq-reindex.md
```

Isi dengan template minimum

```
---  
title: "FAQ Reindex"  
source: "lab-rag"  
date: "2026-04-13"  
category: "FAQ"  
domain: "rag"  
language: "id"  
sensitivity: "internal"  
quality: "reviewed"  
topic: "rag"  
subtopic: "reindex"  
entities: ["Open WebUI"]  
confidence: "high"  
priority: "medium"  
freshness: "current"  
tags: ["faq", "reindex"]  
version: "1.0"  
status: "active"  
---  
  
# FAQ Reindex  
  
## Pertanyaan  
Kapan reindex dilakukan?
```

```
## Jawaban
Saat model embedding berubah atau struktur retrieval berubah
besar.

## Cek hasil
sed -n '1,30p' faq-reindex.md
```

Contoh output:

```
---
title: "FAQ Reindex"
source: "lab-rag"
date: "2026-04-13"
category: "FAQ"
...
```

B.13 Cek metadata minimum dengan CLI

Gunakan pemeriksaan sederhana ini:

```
grep -E
'^ (title|source|date|category|domain|language|sensitivity|qualit
y|topic|subtopic|confidence|priority|freshness|version|status) : '
./*.md
```

Contoh output:

```
./faq-reindex.md:title: "FAQ Reindex"
./faq-reindex.md:source: "lab-rag"
./faq-reindex.md:date: "2026-04-13"
...
```

Kalau ada field yang hilang, dokumen belum boleh masuk corpus.

B.14 Template ringkas paling minimum

Kalau pembaca ingin versi sangat cepat, gunakan ini:

```
---
title: "Judul"
source: "asal"
date: "2026-04-13"
category: "knowledge-note"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "reviewed"
topic: "rag"
subtopic: "umum"
confidence: "medium"
priority: "medium"
freshness: "current"
version: "1.0"
status: "active"
---

# Judul

## Ringkasan
Isi singkat dokumen.

## Isi Utama
Tuliskan isi utama dengan jelas.

## Catatan
Tuliskan hal penting atau langkah singkat.
```

Penutup Lampiran

Template ini sengaja dibuat **tidak mewah, tetapi kuat**. Tujuannya bukan membuat dokumen terlihat cantik, melainkan membuat dokumen **mudah dibaca manusia, mudah dipotong mesin, mudah dicari oleh retrieval, dan mudah diaudit**. Dalam semangat buku ini, **dokumen yang baik lebih berharga daripada file yang banyak tetapi kacau**. Lampiran ini bisa dipakai sebagai fondasi seluruh corpus manual Anda.

Lampiran C. Template Labeling Log ATTACK / NON-ATTACK / SUSPICIOUS / UNKNOWN

Lampiran ini dibuat untuk membantu pembaca mengubah log mentah menjadi **dokumen log yang lebih bermakna, lebih konsisten, dan lebih mudah dicari**. Fokusnya bukan membuat sistem deteksi ancaman yang sempurna, tetapi membangun **label kerja** yang cukup sehat untuk:

- analisis retrieval
- RAG keamanan
- *triage* awal
- latihan klasifikasi
- pembangunan corpus log lokal

Empat label utama yang dipakai adalah:

- **ATTACK**
- **NON-ATTACK**
- **SUSPICIOUS**
- **UNKNOWN**

Prinsip terpentingnya sederhana:

- **jangan terlalu cepat memberi label ATTACK**
- **jangan terlalu malas memakai UNKNOWN**
- **lebih baik jujur daripada terlalu percaya diri**
- **label harus konsisten, bukan dramatis**

C.1 Definisi empat label utama

ATTACK

Gunakan label ini bila ada **indikasi kuat** bahwa event atau rangkaian event merupakan bagian dari aktivitas berbahaya.

Contoh:

- eksploitasi yang jelas
- *payload* berbahaya yang dikenal
- IOC yang tervalidasi
- *brute force* yang kuat dan berulang
- *web shell* atau eksekusi perintah mencurigakan yang sangat jelas
- akses ilegal yang sudah cukup bukti

NON-ATTACK

Gunakan label ini bila event jelas merupakan:

- aktivitas normal
- aktivitas administratif yang sah
- *health check*
- *scanner* internal yang diketahui
- kegagalan biasa yang tidak mengandung indikasi ancaman
- *noise* operasional yang sudah dipahami

SUSPICIOUS

Gunakan label ini bila ada gejala aneh atau berisiko, tetapi **belum cukup bukti** untuk menyebutnya serangan.

Contoh:

- banyak *failed login* dari IP asing tetapi belum cukup pola
- *user agent* aneh
- request ke endpoint sensitif
- lonjakan error yang tidak wajar
- akses ke jalur administratif yang belum jelas niatnya

UNKNOWN

Gunakan label ini bila:

- data belum cukup
- konteks belum cukup
- log terpotong
- sumber log belum jelas
- event masih perlu korelasi tambahan
- analisis belum yakin

Catatan penting:

UNKNOWN bukan kegagalan. UNKNOWN adalah kejujuran analitik.

C.2 Template skema label minimum

Gunakan skema ini sebagai fondasi.

```
log_labels:
  allowed: ["ATTACK", "NON-ATTACK", "SUSPICIOUS", "UNKNOWN"]

definitions:
  ATTACK: "indikasi kuat aktivitas berbahaya atau eksploitasi"
  NON-ATTACK: "aktivitas normal atau terverifikasi aman"
  SUSPICIOUS: "ada gejala aneh, tetapi bukti belum cukup"
  UNKNOWN: "konteks belum cukup atau data belum lengkap"

confidence:
  allowed: ["low", "medium", "high"]

sensitivity:
  allowed: ["internal", "restricted"]

priority:
  allowed: ["low", "medium", "high", "critical"]

freshness:
  allowed: ["current", "review-soon", "stale"]
```

C.3 Field minimum yang saya rekomendasikan

Setiap log yang akan dimasukkan ke corpus sebaiknya punya field berikut:

- **timestamp**
- **host**
- **service**
- **user**
- **ip**
- **severity**
- **event**
- **outcome**
- **label**
- **confidence**
- **source**
- **priority**
- **freshness**
- **notes**

Kalau beberapa field tidak ada, tetap simpan dengan nilai kosong atau `null`, tetapi **jangan menghilangkan struktur**.

C.4 Template JSON paling dasar

Gunakan ini untuk satu event log.

```
{
  "timestamp": "2026-04-13T02:14:22Z",
  "host": "web01",
  "service": "sshd",
  "user": "admin",
  "ip": "203.0.113.8",
  "severity": "warning",
  "event": "failed_login",
  "outcome": "failure",
  "label": "SUSPICIOUS",
  "confidence": "medium",
  "source": "auth.log",
  "priority": "high",
  "freshness": "current",
  "notes": "Failed login dari user invalid, perlu cek frekuensi dan IP lain terkait."
}
```

C.5 Template YAML untuk anotasi manual

Kalau ingin bentuk yang lebih mudah dibaca manusia, gunakan YAML seperti ini:

```
timestamp: "2026-04-13T02:14:22Z"
host: "web01"
service: "sshd"
user: "admin"
ip: "203.0.113.8"
severity: "warning"
event: "failed_login"
outcome: "failure"
label: "SUSPICIOUS"
confidence: "medium"
source: "auth.log"
priority: "high"
freshness: "current"
notes: "Login gagal untuk invalid user. Belum cukup bukti untuk ATTACK."
raw_text: "2026-04-13T02:14:22Z web01 sshd[8831]: Failed password for invalid user admin from 203.0.113.8 port 54422 ssh2"
```

C.6 Template CSV untuk pekerjaan cepat

Kalau pembaca ingin bekerja cepat di spreadsheet atau editor CSV, gunakan kolom berikut:

```
timestamp,host,service,user,ip,severity,event,outcome,label,confidence
,source,priority,freshness,notes
2026-04-13T02:14:22Z,web01,sshd,admin,203.0.113.8,warning,failed_login
,failure,SUSPICIOUS,medium,auth.log,high,current,"invalid user, perlu
korelasi"
```

C.7 Contoh kasus dan label yang disarankan

Kasus 1. Login gagal satu kali

```
2026-04-13T02:14:22Z web01 sshd[8831]: Failed password for
invalid user admin from 203.0.113.8 port 54422 ssh2
```

Label yang disarankan: **SUSPICIOUS**

Alasan: ada gejala aneh, tetapi satu event saja belum cukup untuk menyebut ATTACK.

Kasus 2. Banyak login gagal beruntun dari IP sama

```
2026-04-13T02:14:22Z web01 sshd[8831]: Failed password...
2026-04-13T02:14:25Z web01 sshd[8834]: Failed password...
2026-04-13T02:14:27Z web01 sshd[8837]: Failed password...
2026-04-13T02:14:30Z web01 sshd[8842]: Failed password...
```

Label yang disarankan: **ATTACK** atau **SUSPICIOUS**, tergantung konteks

Alasan: kalau frekuensi tinggi, user tidak valid, IP asing, dan pola jelas, label **ATTACK** mulai masuk akal. Kalau konteks belum cukup, tetap **SUSPICIOUS**.

Kasus 3. Health check internal

```
2026-04-13T03:00:01Z web01 nginx: GET /healthz 200 from
10.0.0.5
```

Label yang disarankan: **NON-ATTACK**

Alasan: event normal, internal, dan expected.

Kasus 4. Akses ke endpoint sensitif

```
2026-04-13T03:12:44Z web01 nginx: GET /.env 404 from
198.51.100.22
```

Label yang disarankan: **SUSPICIOUS**

Alasan: pola eksplorasi umum terhadap file sensitif, tetapi satu request belum cukup untuk ATTACK final.

Kasus 5. Log tidak lengkap

2026-04-13 web01 something failed from unknown source

Label yang disarankan: UNKNOWN

Alasan: field penting hilang, konteks tidak cukup.

C.8 Aturan keputusan cepat

Gunakan aturan singkat ini saat memberi label:

- **ATTACK**
 - ada pola kuat
 - ada bukti eksploitasi
 - ada IOC
 - ada rangkaian event yang jelas berbahaya
- **NON-ATTACK**
 - event normal
 - aktivitas sah
 - ada konteks operasional yang jelas
- **SUSPICIOUS**
 - ada keanehan
 - indikasi awal ada
 - bukti belum cukup
- **UNKNOWN**
 - data kurang
 - konteks kurang
 - analisis belum yakin

C.9 Template keputusan manual per event

Gunakan lembar kerja ini untuk anotasi cepat.

```
# Lembar Label Event Log

## Event ID
auth-2026-04-13-0001

## Raw Text
2026-04-13T02:14:22Z web01 sshd[8831]: Failed password for invalid
user admin from 203.0.113.8 port 54422 ssh2

## Field Penting
- timestamp: 2026-04-13T02:14:22Z
- host: web01
- service: sshd
- user: admin
- ip: 203.0.113.8
- event: failed_login
- outcome: failure

## Label
SUSPICIOUS

## Confidence
medium

## Alasan
Login gagal untuk invalid user. Ada indikasi probing, tetapi belum
cukup bukti untuk menyebut ATTACK.

## Tindak Lanjut
- cek frekuensi dari IP yang sama
- cek host lain
- cek rentang waktu 5 menit sebelum dan sesudah
```

C.10 Howto kerja labeling log yang sehat

Langkah 1. Simpan log mentah

```
mkdir -p ./data/logs/{raw,parsed,labeled}  
cp auth.log ./data/logs/raw/
```

Langkah 2. Ambil contoh event kecil

```
head -n 20 ./data/logs/raw/auth.log
```

Contoh output:

```
2026-04-13T02:14:22Z web01 sshd[8831]: Failed password for  
invalid user admin from 203.0.113.8 port 54422 ssh2  
2026-04-13T02:15:01Z web01 sshd[8839]: Accepted password  
for onno from 10.0.0.15 port 51234 ssh2
```

Langkah 3. Parse field penting

Pisahkan:

- waktu
- host
- service
- user
- IP
- event
- outcome

Langkah 4. Beri label manual

Gunakan hanya empat label utama:

- ATTACK
- NON-ATTACK
- SUSPICIOUS
- UNKNOWN

Langkah 5. Tambahkan confidence

- low
- medium
- high

Langkah 6. Simpan hasil ke file baru

Misalnya:

```
cat > ./data/logs/labeled/auth-0001.json <<'EOF'
{
  "timestamp": "2026-04-13T02:14:22Z",
  "host": "web01",
  "service": "sshd",
  "user": "admin",
  "ip": "203.0.113.8",
  "severity": "warning",
  "event": "failed_login",
  "outcome": "failure",
  "label": "SUSPICIOUS",
  "confidence": "medium",
  "source": "auth.log",
  "priority": "high",
  "freshness": "current",
  "notes": "invalid user, perlu korelasi lebih lanjut"
}
EOF
```

Langkah 7. Cek hasil

```
cat ./data/logs/labeled/auth-0001.json
```

Contoh output:

```
{
  "timestamp": "2026-04-13T02:14:22Z",
  "host": "web01",
  "service": "sshd",
  "user": "admin",
  "ip": "203.0.113.8",
  "severity": "warning",
  "event": "failed_login",
  "outcome": "failure",
  "label": "SUSPICIOUS",
  "confidence": "medium",
  "source": "auth.log",
  "priority": "high",
  "freshness": "current",
  "notes": "invalid user, perlu korelasi lebih lanjut"
}
```

C.11 Template batch labeling sederhana

Kalau ingin memulai dari banyak file kecil, gunakan pola ini:

```
python - <<'PY'
import json
from pathlib import Path

records = [
    {
        "timestamp": "2026-04-13T02:14:22Z",
        "host": "web01",
        "service": "sshd",
        "user": "admin",
        "ip": "203.0.113.8",
        "severity": "warning",
        "event": "failed_login",
        "outcome": "failure",
        "label": "SUSPICIOUS",
        "confidence": "medium",
        "source": "auth.log",
        "priority": "high",
        "freshness": "current",
        "notes": "invalid user"
    },
    {
        "timestamp": "2026-04-13T03:00:01Z",
        "host": "web01",
        "service": "nginx",
        "user": "",
        "ip": "10.0.0.5",
        "severity": "info",
        "event": "health_check",
        "outcome": "success",
        "label": "NON-ATTACK",
        "confidence": "high",
        "source": "access.log",
        "priority": "low",
        "freshness": "current",
        "notes": "internal health check"
    }
]

outdir = Path("./data/logs/labeled")
outdir.mkdir(parents=True, exist_ok=True)

for i, rec in enumerate(records, start=1):
    path = outdir / f"log-{i:04d}.json"
    path.write_text(json.dumps(rec, indent=2), encoding="utf-8")
    print(f"saved: {path}")
PY
```

Contoh output:


```
saved: data/logs/labeled/log-0001.json
saved: data/logs/labeled/log-0002.json
```

C.12 Checklist mutu sebelum log di-embed

Gunakan daftar ini:

- timestamp ada
- host ada
- service ada
- IP ada kalau tersedia
- event dan outcome jelas
- label diisi dengan salah satu dari 4 label utama
- confidence ada
- source log jelas
- notes singkat menjelaskan alasan label
- tidak ada nilai label liar di luar skema

Cek cepat dengan CLI

```
grep -R '"label"' ./data/logs/labeled | head
```

Validasi nilai label

```
python - <<'PY'
import json
from pathlib import Path

allowed = {"ATTACK", "NON-ATTACK", "SUSPICIOUS", "UNKNOWN"}

for path in Path("./data/logs/labeled").glob("*.json"):
    obj = json.loads(path.read_text())
    label = obj.get("label")
    if label not in allowed:
        print(f"[WARN] {path} label invalid: {label}")
    else:
        print(f"[OK]    {path}")
PY
```

Contoh output:

```
[OK]    data/logs/labeled/log-0001.json
[OK]    data/logs/labeled/log-0002.json
```

C.13 Anti-kesalahan umum

Hindari kebiasaan ini:

- semua event aneh langsung diberi **ATTACK**
- semua event yang tidak dipahami diberi **NON-ATTACK**
- tidak memakai **UNKNOWN**
- tidak menulis alasan label
- tidak menyimpan sumber log
- tidak membedakan event tunggal dan rangkaian event

Golden Rule:

lebih baik sedikit konservatif tetapi konsisten, daripada agresif tetapi kacau.

C.14 Template ringkas paling minimum

Kalau pembaca ingin versi super singkat, gunakan ini:

```
{
  "timestamp": "2026-04-13T02:14:22Z",
  "host": "web01",
  "service": "sshd",
  "ip": "203.0.113.8",
  "event": "failed_login",
  "outcome": "failure",
  "label": "SUSPICIOUS",
  "confidence": "medium",
  "source": "auth.log",
  "notes": "perlu korelasi lebih lanjut"
}
```

Penutup Lampiran

Template ini sengaja dibuat **tidak rumit, tetapi disiplin**. Tujuannya bukan membuat klasifikasi keamanan yang sempurna, melainkan memberi **struktur labeling yang cukup sehat** agar log bisa menjadi corpus yang berguna untuk retrieval, audit, dan analisis awal. Dalam semangat buku ini, **label yang jujur dan konsisten lebih berharga daripada label yang sok pasti tetapi salah**.

Lampiran D. Contoh alur PDF → Markdown → *cleaning* → *ingest*

Lampiran ini menunjukkan satu alur kerja sederhana tetapi kuat:

- **PDF mentah**
- ekstraksi ke **Markdown**
- pembersihan hasil ekstraksi
- pemecahan menjadi dokumen yang lebih sehat
- validasi cepat
- *ingest* ke corpus / vector store
- siap dipakai di Open WebUI

Tujuan lampiran ini bukan membuat pipeline tercanggih, tetapi memberi pembaca **jalur minimum yang benar**.

D.1 Struktur folder kerja

Buat struktur folder seperti ini:

```
mkdir -p ./work/{input,output/{raw,cleaned,chapters},logs}
tree -L 3 ./work
```

Contoh output:

```
./work
├── input
├── logs
└── output
    ├── chapters
    ├── cleaned
    └── raw
```

Checklist

- folder input siap
- folder output mentah ada
- folder hasil bersih ada
- folder bab/pecahan dokumen ada

D.2 Letakkan PDF mentah

Misalnya file textbook Anda bernama `textbook.pdf`.

```
cp textbook.pdf ./work/input/  
ls -lh ./work/input/
```

Contoh output:

```
-rw-r--r-- 1 user user 18M Apr 13 10:10 textbook.pdf
```

Checklist

- file PDF sudah ada di folder input
- nama file jelas
- ukuran file masuk akal

D.3 Instal library ekstraksi

Dokumentasi resmi PyMuPDF4LLM menempatkan `to_markdown()` sebagai pintu masuk utama untuk mengubah PDF menjadi teks terstruktur yang ramah untuk LLM/RAG. ([PyMuPDF](#))

```
python3 -m venv .venv  
source .venv/bin/activate  
pip install pymupdf pymupdf4llm
```

Cek:

```
python - <<'PY'  
import fitz  
import pymupdf4llm  
print("PyMuPDF OK")  
print("PyMuPDF4LLM OK")  
PY
```

Contoh output:

```
PyMuPDF OK  
PyMuPDF4LLM OK
```

Checklist

- virtual environment aktif
- `pymupdf` terpasang
- `pymupdf4llm` terpasang

D.4 Tahap 1 — PDF → Markdown

Sekarang kita ekstrak PDF ke Markdown.

```
python - <<'PY'
from pathlib import Path
import pymupdf4llm

src = Path("./work/input/textbook.pdf")
dst = Path("./work/output/raw/textbook.raw.md")

md = pymupdf4llm.to_markdown(str(src))
dst.write_text(md, encoding="utf-8")

print(f"Saved: {dst}")
print("-" * 60)
print(md[:1000])
PY
```

Contoh output:

```
Saved: work/output/raw/textbook.raw.md
-----
# Chapter 1 Introduction
Retrieval-Augmented Generation ...
```

Checklist

- file Markdown mentah berhasil dibuat
- heading mulai terlihat
- teks tidak kosong

D.5 Tahap 2 — inspeksi awal hasil ekstraksi

Sebelum membersihkan, lihat dulu apakah hasil ekstraksi cukup masuk akal.

```
sed -n '1,80p' ./work/output/raw/textbook.raw.md
```

Atau cek heading:

```
grep -n '^#' ./work/output/raw/textbook.raw.md | head -20
```

Contoh output:

```
1:# Chapter 1 Introduction
18:## What is RAG?
42:## Why retrieval matters
```

Checklist

- struktur bab terlihat
- subbab terlihat
- tidak terlalu banyak baris aneh di bagian awal

D.6 Tahap 3 — cleaning dasar

Ini tahap yang sangat penting. Hasil ekstraksi mentah biasanya masih mengandung:

- nomor halaman
- baris kosong berlebihan
- paragraf yang pecah aneh
- *header/footer*
- sisa tabel atau caption yang berantakan

Dokumentasi Open WebUI tentang RAG mengingatkan bahwa ketika hasil terasa “halusinatif” atau melewati hal penting, masalahnya sering adalah **konteks**, bukan model. Itu sebabnya *cleaning* ini tidak boleh dilewatkan. ([Open WebUI](#))

Contoh skrip *cleaning* dasar:

```
python - <<'PY'
import re
from pathlib import Path

src = Path("./work/output/raw/textbook.raw.md")
dst = Path("./work/output/cleaned/textbook.clean.md")

text = src.read_text(encoding="utf-8")

# 1. Buang nomor halaman yang berdiri sendiri
text = re.sub(r'(?m)^\s*\d+\s*$', '', text)

# 2. Buang baris kosong berlebihan
text = re.sub(r'\n{3,}', '\n\n', text)

# 3. Gabungkan baris pecah sederhana
text = re.sub(r'(?<\n)\n(?:[#\-\*\d])', ' ', text)

# 4. Rapikan spasi berlebihan
text = re.sub(r'[ \t]{2,}', ' ', text)

dst.write_text(text.strip() + "\n", encoding="utf-8")
print(f"Saved cleaned file to: {dst}")
PY
```

Contoh output:

```
Saved cleaned file to:
work/output/cleaned/textbook.clean.md
```

Checklist

- nomor halaman berdiri sendiri berkurang
- paragraf lebih rapi
- baris kosong tidak berlebihan

D.7 Tahap 4 — inspeksi hasil bersih

Lihat hasil *cleaning*:

```
sed -n '1,120p' ./work/output/cleaned/textbook.clean.md
```

Bandingkan jumlah heading:

```
grep -c '^#' ./work/output/raw/textbook.raw.md  
grep -c '^#' ./work/output/cleaned/textbook.clean.md
```

Contoh output:

```
8  
8
```

Kalau jumlah heading tetap, itu biasanya pertanda baik: struktur utama masih terjaga.

D.8 Tahap 5 — pecah per bab atau per subbab

Untuk textbook, satu file besar sering **terlalu gemuk** untuk retrieval. Lebih sehat bila dipecah ke unit yang lebih masuk akal, misalnya per bab. Ini juga selaras dengan arah bab *chunking* dan struktur *parent-child* yang Anda bangun di buku.

Contoh pecah per heading utama #:

```
python - <<'PY'
from pathlib import Path
import re

src = Path("./work/output/cleaned/textbook.clean.md")
text = src.read_text(encoding="utf-8")

parts = re.split(r'(?m)^# ', text)
outdir = Path("./work/output/chapters")
outdir.mkdir(parents=True, exist_ok=True)

count = 0
for part in parts:
    part = part.strip()
    if not part:
        continue
    lines = part.splitlines()
    title = lines[0].strip()
    slug = re.sub(r'^a-zA-Z0-9+', '-',
title.lower()).strip('-')[:60]
    content = "# " + part + "\n"
    path = outdir / f"{count+1:02d}-{slug}.md"
    path.write_text(content, encoding="utf-8")
    print(f"saved: {path}")
    count += 1

print(f"Created {count} chapter files.")
PY
```

Contoh output:

```
saved: work/output/chapters/01-chapter-1-introduction.md
saved: work/output/chapters/02-chapter-2-vector-stores.md
saved: work/output/chapters/03-chapter-3-embedding-strategies.md
Created 8 chapter files.
```

Checklist

- tiap bab jadi file terpisah
- nama file jelas
- jumlah file masuk akal

D.9 Tahap 6 — tambah YAML metadata

Sekarang ubah hasil bab menjadi dokumen kerja yang siap untuk corpus.

Contoh metadata minimum:

```
---
title: "Chapter 3 Embedding Strategies"
source: "textbook-rag-2026"
date: "2026-04-13"
category: "book-chapter"
domain: "rag"
language: "en"
sensitivity: "public"
quality: "reviewed"
topic: "embedding"
subtopic: "strategies"
entities: ["embedding", "retrieval"]
confidence: "high"
priority: "medium"
freshness: "current"
version: "1.0"
status: "active"
---
```

```
# Chapter 3 Embedding Strategies
...
```

Cara cepat menambahkan metadata ke satu file:

```
python - <<'PY'
from pathlib import Path

path =
Path("./work/output/chapters/03-chapter-3-embedding-strategies.m
d")
body = path.read_text(encoding="utf-8")

yaml = """---
title: "Chapter 3 Embedding Strategies"
source: "textbook-rag-2026"
date: "2026-04-13"
category: "book-chapter"
domain: "rag"
language: "en"
sensitivity: "public"
quality: "reviewed"
topic: "embedding"
subtopic: "strategies"
entities: ["embedding", "retrieval"]
confidence: "high"
priority: "medium"
freshness: "current"
version: "1.0"
status: "active"
---
```

```

"""

path.write_text(yaml + body, encoding="utf-8")
print(f"updated: {path}")
PY

```

Contoh output:

```

updated:
work/output/chapters/03-chapter-3-embedding-strategies.md

```

D.10 Tahap 7 — validasi cepat sebelum ingest

Sebelum masuk ke corpus, cek:

- heading masih ada
- metadata ada
- tidak ada nomor halaman liar
- paragraf tidak rusak
- file tidak kosong

Cek metadata:

```

grep -E
'^ (title|source|date|category|domain|language|sensitivity|qualit
y|topic|subtopic|confidence|priority|freshness|version|status): '
./work/output/chapters/*.md | head -30

```

Cek heading:

```

grep -n '^#'
./work/output/chapters/03-chapter-3-embedding-strategies.md |
head

```

Checklist

- metadata minimum lengkap
- heading sehat
- file terbaca enak oleh manusia

D.11 Tahap 8 — salin ke folder corpus bersih

Sekarang dokumen hasil ekstraksi dan pembersihan dipindah ke corpus yang siap di-*ingest*.

```
mkdir -p ./data/cleaned/textbook
cp ./work/output/chapters/*.md ./data/cleaned/textbook/
ls -l ./data/cleaned/textbook | head
```

Contoh output:

```
01-chapter-1-introduction.md
02-chapter-2-vector-stores.md
03-chapter-3-embedding-strategies.md
```

D.12 Tahap 9 — ingest pilot ke vector store

Gunakan script pilot yang sudah kita siapkan sebelumnya.

```
python ingest_pilot.py \
  --input ./data/cleaned/textbook \
  --limit 20 \
  --vector-store lancedb \
  --db-path ./vectordb \
  --table-name textbook_corpus
```

Contoh output:

```
[INFO] Loading embedding model:
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
[INFO] Documents ingested: 8
[INFO] Chunks created: 31
[INFO] Vector store: lancedb
[INFO] DB path: ./vectordb
```

Checklist

- file berhasil di-*ingest*
- chunk terbentuk
- tidak ada error metadata besar

D.13 Tahap 10 — evaluasi retrieval cepat

Setelah ingest, jangan langsung percaya. Uji retrieval.

Contoh queries.yaml:

```
queries:
  - id: q01
    question: "Apa itu embedding strategies?"
    expected_titles_contains:
      - "Embedding Strategies"

  - id: q02
    question: "Mengapa retrieval penting?"
    expected_titles_contains:
      - "Introduction"
```

Jalankan evaluasi:

```
python eval_retrieval.py \
  --queries ./tests/queries.yaml \
  --vector-store lancedb \
  --db-path ./vectordb \
  --table-name textbook_corpus \
  --top-k 5
```

Contoh output:

```
[INFO] Queries tested: 2
[INFO] Top-1 hit rate: 50.00%
[INFO] Top-5 hit rate: 100.00%
```

Kalau hasil ini belum memuaskan, jangan langsung menyalahkan model. Cek lagi:

- *chunking*
- *metadata*
- *heading*
- kebersihan hasil ekstraksi

D.14 Tahap 11 — unggah ke Open WebUI

Setelah corpus kecil terasa sehat, masuk ke Open WebUI.

Langkah:

- buka **Workspace > Knowledge**
- buat *knowledge base* baru, misalnya `textbook-rag-2026`
- unggah file dari `./data/cleaned/textbook`
- pilih **Focused Retrieval** untuk koleksi besar
- uji beberapa pertanyaan

Dokumentasi Open WebUI memang menganjurkan **Focused Retrieval** untuk koleksi besar yang ingin dicari efisien, sedangkan **Full Context** lebih cocok untuk dokumen pendek yang perlu dimasukkan utuh. ([Open WebUI](#))

D.15 Alur ringkas satu halaman

Kalau pembaca ingin versi super singkat, gunakan alur ini:

- taruh PDF di `./work/input`
- ekstrak ke Markdown dengan PyMuPDF4LLM
- simpan hasil mentah di `./work/output/raw`
- lakukan *cleaning*
- pecah per bab atau subbab
- tambah YAML metadata
- pindahkan ke `./data/cleaned`
- *ingest* dengan `ingest_pilot.py`
- evaluasi dengan `eval_retrieval.py`
- unggah ke Open WebUI dan uji query nyata

D.16 Contoh file akhir yang siap masuk corpus

```
---
title: "Chapter 3 Embedding Strategies"
source: "textbook-rag-2026"
date: "2026-04-13"
category: "book-chapter"
domain: "rag"
language: "en"
sensitivity: "public"
quality: "reviewed"
topic: "embedding"
subtopic: "strategies"
entities: ["embedding", "retrieval"]
confidence: "high"
priority: "medium"
freshness: "current"
version: "1.0"
status: "active"
---

# Chapter 3 Embedding Strategies

## Why embeddings matter
Embeddings are vector representations of text that preserve semantic
relationships.

## Choosing an embedding model
A good embedding model should match the language, domain, and query
behavior of the corpus.
```

D.17 Kesalahan umum yang harus dihindari

- langsung mengunggah PDF mentah ke RAG
- tidak menyimpan hasil ekstraksi mentah
- membersihkan terlalu agresif sampai heading hilang
- tidak memecah textbook besar
- tidak menambah metadata
- tidak mengevaluasi retrieval sebelum unggah ke Open WebUI
- mengganti model *embedding* tanpa rencana *reindex*

Dokumentasi Open WebUI sendiri menekankan bahwa banyak masalah RAG adalah **masalah konteks**, bukan salah model. ([Open WebUI](#))

Penutup Lampiran

Lampiran ini mengajarkan satu kebiasaan yang sangat penting: **PDF bukan corpus**. PDF hanyalah bahan mentah. Corpus baru lahir setelah PDF:

- diekstrak,
- dibersihkan,
- dipulihkan strukturnya,
- dipecah ke unit sehat,
- diberi metadata,
- diuji,
- lalu baru di-*ingest*.

Dengan alur ini, pembaca tidak lagi sekadar “punya file PDF”, tetapi mulai punya **dokumen kerja yang layak dipercaya oleh retrieval**. PyMuPDF4LLM memberi jalur yang kuat untuk ekstraksi ke **Markdown/JSON/TXT**, dan Open WebUI memberi tempat untuk mengoperasikan hasil akhirnya sebagai *knowledge base*. ([PyMuPDF](#))

Lampiran E. Checklist Evaluasi Kualitas Corpus

Lampiran ini dibuat untuk menjawab satu pertanyaan yang sangat penting:

“Bagaimana saya tahu bahwa corpus saya sudah cukup sehat untuk dipakai di RAG?”

Pertanyaan ini jauh lebih penting daripada kelihatannya. Banyak orang terlalu cepat masuk ke tahap *embedding*, *vector database*, dan chat UI, padahal corpus-nya sendiri belum siap. Akibatnya:

- retrieval terasa “agak nyambung”, tetapi sering meleset,
- dokumen penting kalah oleh dokumen yang lebih umum,
- sumber tidak jelas,
- model tampak halusinatif padahal konteksnya memang jelek.

Karena itu, sebelum dokumen masuk ke sistem, biasanya memeriksa kualitas corpus dengan disiplin.

E.1 Prinsip dasar evaluasi corpus

Gunakan tiga prinsip ini:

- **lebih baik corpus kecil tetapi sehat**
- **evaluasi corpus sebelum evaluasi model**
- **uji corpus dengan query nyata, bukan hanya rasa percaya diri**

Corpus yang sehat harus lolos minimal pada lima lapisan:

- kualitas dokumen
- kualitas metadata
- kualitas label
- kualitas *chunking*
- kualitas retrieval

E.2 Checklist tingkat 1 — kualitas dokumen sumber

Gunakan checklist ini untuk memeriksa kualitas dasar dokumen.

A. Struktur dokumen

- **judul jelas**
- **satu dokumen punya fokus utama yang tegas**
- heading dan subheading masuk akal
- paragraf tidak terlalu panjang
- daftar dan langkah ditulis rapi
- tabel seperlunya, tidak berlebihan

- *code block* utuh, tidak terpotong

B. Kebersihan isi

- tidak ada nomor halaman liar
- *header/footer* berulang sudah dibuang
- catatan kaki tidak menempel liar
- paragraf hasil ekstraksi PDF sudah dirapikan
- caption gambar tidak merusak alur utama
- tidak ada baris kosong berlebihan
- tidak ada potongan teks rusak atau karakter aneh

C. Kelayakan isi

- isi dokumen benar-benar berguna untuk query
- bukan duplikasi mentah dari file lain
- bukan draft usang yang tak diberi tanda
- bukan campuran banyak topik tanpa batas jelas

Cek cepat dengan CLI

```
grep -R '^#' ./data/cleaned | head -20
grep -R '^[0-9]\+$' ./data/cleaned | head
```

Contoh output:

```
./data/cleaned/sop-reindex.md:# SOP Reindex Knowledge Base
./data/cleaned/faq-focused-retrieval.md:# FAQ Focused
Retrieval
```

Kalau banyak angka halaman liar muncul, hasil ekstraksi PDF masih perlu dibersihkan.

E.3 Checklist tingkat 2 — kualitas metadata

Metadata adalah tulang punggung retrieval yang bisa difilter dan diaudit. Minimal periksa field berikut:

- `title`
- `source`
- `date`
- `category`
- `domain`
- `language`
- `sensitivity`
- `quality`
- `topic`
- `subtopic`
- `confidence`
- `priority`
- `freshness`
- `version`
- `status`

Pertanyaan evaluasi metadata

- apakah semua dokumen memakai nama field yang konsisten?
- apakah nilai field ditulis konsisten?
- apakah ada field penting yang kosong?
- apakah metadata benar-benar berguna untuk filter dan audit?
- apakah sumber dokumen dapat ditelusuri?

Cek cepat dengan CLI

```
grep -E  
'^(title|source|date|category|domain|language|sensitivity|  
quality|topic|subtopic|confidence|priority|freshness|versi  
on|status):' ./data/cleaned/*.md
```

Kalau hasilnya tidak konsisten, retrieval berbasis metadata akan lemah.

E.4 Checklist tingkat 3 — kualitas label

Label membantu retrieval menjadi lebih cerdas daripada sekadar kemiripan makna.

Untuk corpus umum

- topik jelas
- subtopik jelas
- entitas penting dicatat
- tingkat kepercayaan ditulis
- sensitivitas ditulis
- prioritas ditulis
- *freshness* ditulis

Untuk corpus log

- label memakai skema yang konsisten
- hanya memakai:
 - ATTACK
 - NON-ATTACK
 - SUSPICIOUS
 - UNKNOWN
- confidence ada
- notes/alasan label ada

Pertanyaan evaluasi label

- apakah label terlalu sedikit sehingga retrieval tidak terarah?
- apakah label terlalu banyak sehingga membingungkan?
- apakah ada sinonim liar untuk label yang sama?
- apakah skema label didokumentasikan?

Cek cepat nilai label log

```
python - <<'PY'
import json
from pathlib import Path

allowed = {"ATTACK", "NON-ATTACK", "SUSPICIOUS",
"UNKNOWN"}
for path in Path("./data/logs/labeled").glob("*.json"):
    obj = json.loads(path.read_text())
    label = obj.get("label")
    if label not in allowed:
        print(f"[WARN] {path} -> label invalid: {label}")
    else:
        print(f"[OK]    {path}")
PY
```

E.5 Checklist tingkat 4 — kualitas *chunking*

Di buku ini, *chunking* diposisikan sebagai **jantung retrieval**. Jadi kualitas corpus tidak bisa dinilai tanpa memeriksa kualitas *chunk*.

Checklist chunking

- ukuran *chunk* tidak terlalu kecil
- ukuran *chunk* tidak terlalu besar
- *overlap* tidak terlalu kecil
- *overlap* tidak terlalu besar
- heading tidak terputus dari isi
- definisi tidak terpisah dari penjelasan penting
- langkah prosedural tidak terbelah di tengah
- *code block* tidak terpotong
- *chunk* tidak terlalu duplikatif

Tanda chunking buruk

- satu query sering menemukan banyak *chunk* yang hampir sama
- jawaban terasa “hampir benar” tetapi kekurangan konteks
- *top-k* retrieval diisi *chunk* dari bagian yang sama terus
- kalimat penting tersebar di beberapa *chunk* terpisah

Uji cepat dengan script yang sudah dibuat

```
python compare_chunking.py \  
  --input ./data/cleaned \  
  --queries ./tests/queries.yaml \  
  --embedding-model \  
sentence-transformers/paraphrase-multilingual-MiniLM-  
-L12-v2 \  
  --chunk-configs 800:100 1200:150 1600:200 \  
  --top-k 5
```

Contoh output:

```
[INFO] Konfigurasi paling menjanjikan saat ini:  
heading:1200:150
```

E.6 Checklist tingkat 5 — kualitas retrieval

Pada akhirnya corpus harus diuji lewat retrieval. Ini titik yang paling jujur.

Metrik minimum yang disarankan

- **Top-1 hit rate**
- **Top-K hit rate**
- **recall@k**
- **MRR**
- **groundedness**
- **ketepatan jawaban**
- **jejak sumber**

Pertanyaan evaluasi retrieval

- apakah dokumen yang benar masuk *top-k*?
- apakah hasil pertama sering langsung benar?
- apakah dokumen generik mengalahkan dokumen resmi?
- apakah jawaban benar-benar memakai bukti dari corpus?
- apakah pengguna bisa menelusuri sumber?

Cek retrieval minimum

```
python eval_retrieval.py \  
  --queries ./tests/queries.yaml \  
  --vector-store lancedb \  
  --db-path ./vectordb \  
  --table-name pilot_corpus \  
  --top-k 5
```

Contoh output:

```
[INFO] Queries tested: 20  
[INFO] Top-1 hit rate: 60.00%  
[INFO] Top-5 hit rate: 95.00%
```

Interpretasi cepat:

- **Top-5 bagus, Top-1 jelek** → retrieval lumayan, urutan perlu dibantu *reranker*
- **Top-5 jelek** → masalah ada di corpus, *embedding*, atau *chunking*

E.7 Checklist kualitas per domain

Karena buku ini menekankan tiga dunia berbeda, gunakan evaluasi yang sadar domain.

A. Corpus *knowledge/text*

- judul dan heading rapi
- SOP final dibedakan dari draft
- FAQ dipisah jelas
- dokumen kebijakan diberi sumber dan versi
- memo tidak bercampur dengan dokumen final
- query konseptual dan prosedural sama-sama diuji

B. Corpus log

- *timestamp* ada
- host ada
- service ada
- event dan outcome jelas
- label log konsisten
- korelasi waktu diuji
- *attack/non-attack* tidak ditentukan terlalu agresif

C. Corpus programming

- file code tidak dipotong di tengah fungsi
- README dipisah dari source code
- API docs dipisah per endpoint atau bagian
- changelog disimpan
- commit message yang penting disimpan
- query tentang fungsi, instalasi, dan perubahan diuji terpisah

E.8 Tabel penilaian cepat corpus

Gunakan tabel ini untuk audit manual.

Aspek	Pertanyaan	Nilai 1–5	Catatan
Struktur dokumen	Apakah dokumen rapi dan fokus?		
Metadata	Apakah metadata lengkap dan konsisten?		
Label	Apakah label bermakna dan tidak kacau?		
Chunking	Apakah unit makna tetap utuh?		
Retrieval	Apakah dokumen benar muncul di top-k?		
Groundedness	Apakah jawaban didukung konteks?		
Sumber	Apakah sumber bisa ditelusuri?		
Kesiapan operasional	Apakah corpus siap dipakai harian?		

Interpretasi

- **36–40** = sangat sehat
- **28–35** = cukup sehat, bisa dipakai dengan hati-hati
- **20–27** = perlu perbaikan sebelum diandalkan
- **< 20** = jangan masuk ke produksi, jangan buru-buru *fine-tuning*

E.9 Lembar audit mini sebelum *ingest*

Gunakan lembar ini sebelum *ingest* besar-besaran.

```
# Audit Mini Corpus

## Nama corpus
rag-lab-2026

## Tanggal audit
2026-04-13

## Jumlah dokumen
24

## Pemeriksaan cepat
- Dokumen sehat: ya / tidak
- Metadata lengkap: ya / tidak
- Label konsisten: ya / tidak
- Chunking diuji: ya / tidak
- Query uji siap: ya / tidak
- Top-5 retrieval sehat: ya / tidak
- Groundedness diperiksa: ya / tidak

## Temuan utama
- ...
- ...
- ...

## Keputusan
- [ ] siap ingest
- [ ] ingest pilot dulu
- [ ] bersihkan ulang corpus
- [ ] ganti strategi chunking
- [ ] uji embedding lain
```


E.10 Howto audit corpus dari nol

Langkah 1. Audit dokumen

- cek 10 file pertama secara manual
- cek heading
- cek kebersihan hasil ekstraksi
- cek apakah file benar-benar relevan

Langkah 2. Audit metadata

- jalankan `grep`
- pastikan field minimum ada
- cek konsistensi nilai

Langkah 3. Audit label

- cek beberapa dokumen dari tiap kategori
- pastikan tidak ada label liar
- pastikan label benar-benar punya fungsi

Langkah 4. Audit chunking

- bandingkan 2–3 konfigurasi
- lihat *top-1* dan *top-k*

Langkah 5. Audit retrieval

- jalankan query uji
- cek sumber
- cek apakah jawaban benar-benar berdasar konteks

E.11 CLI audit cepat yang sangat berguna

```
# Cek file yang tersedia
find ./data/cleaned -type f | head -20

# Cek heading
grep -R '^#' ./data/cleaned | head -20

# Cek metadata penting
grep -R '^title:|^source:|^date:|^domain:|^topic:' ./data/cleaned
| head -50

# Cek duplikasi kasar berdasarkan nama
find ./data/cleaned -type f | sed 's#.#/##' | sort | uniq -d

# Jalankan evaluasi retrieval
python eval_retrieval.py \
  --queries ./tests/queries.yaml \
  --vector-store lancedb \
  --db-path ./vectordb \
  --table-name pilot_corpus \
  --top-k 5
```

E.12 Tanda corpus sudah cukup sehat

Corpus biasanya sudah cukup sehat bila:

- dokumen yang benar sering muncul di *top-k*
- metadata konsisten
- label tidak liar
- query uji stabil
- jawaban bisa ditelusuri ke sumber
- perubahan dokumen bisa dikelola
- tim tidak bingung membaca corpus itu sendiri

Tanda corpus belum sehat

- dokumen yang benar sering kalah oleh yang generik
- dokumen lama dan baru bercampur
- metadata kosong atau berantakan
- *chunking* membuat konteks pecah
- *top-k* penuh duplikasi
- sumber sulit ditelusuri
- orang mulai tergoda menyalahkan model padahal corpus kacau

E.13 Golden Rule

Sebelum Anda menyalahkan model, tanyakan lima hal ini:

- apakah dokumen yang benar memang ada?
- apakah dokumen itu bersih?
- apakah metadata dan labelnya sehat?
- apakah *chunking*-nya masuk akal?
- apakah retrieval benar-benar menemukan konteks yang tepat?

Kalau lima hal itu belum sehat, **jangan buru-buru *fine-tuning***.

Penutup Lampiran

Checklist ini dibuat untuk menanamkan satu kebiasaan yang sangat penting: **corpus harus diaudit sebelum dipercaya**. Dalam banyak proyek RAG, yang rusak bukan model, melainkan **bahan bacaan model**. Dengan checklist ini, pembaca tidak lagi sekadar mengumpulkan file, tetapi belajar menilai corpus sebagai **aset pengetahuan yang harus bersih, tertib, teruji, dan bisa ditelusuri**. Itu jauh lebih berharga daripada sekadar demo AI yang kelihatan pintar.

Lampiran F. Source Code ingest_pilot.py

Berikut **source code** `ingest_pilot.py` yang bisa langsung dipakai untuk uji pilot corpus kecil dari folder `./data/cleaned`.

Fungsinya:

- membaca file `.md`, `.txt`, `.json`
- mengambil **YAML front matter** jika ada
- membuat *chunk* sederhana
- membuat **embedding lokal**
- menyimpan hasil ke **LanceDB** atau **Chroma**
- menampilkan ringkasan ingest

Contoh pakai:

```
python ingest_pilot.py --input ./data/cleaned --limit 50
--vector-store lancedb
```

Kalau ingin Chroma:

```
python ingest_pilot.py --input ./data/cleaned --limit 50
--vector-store chroma
```

Source Code ingest_pilot.py

```
#!/usr/bin/env python3
"""
ingest_pilot.py

Pilot ingest corpus kecil untuk RAG.
- Baca file Markdown/TXT/JSON dari folder input
- Ambil metadata YAML front matter jika ada
- Chunk teks
- Buat embedding lokal
- Simpan ke LanceDB atau Chroma
- Tampilkan ringkasan ingest

Contoh:
    python ingest_pilot.py --input ./data/cleaned --limit 50
    --vector-store lancedb
"""

from __future__ import annotations

import argparse
import dataclasses
import hashlib
import json
```

```

import os
import re
import sys
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Tuple

# -----
# Utility data structures
# -----

@dataclasses.dataclass
class Document:
    source_path: str
    text: str
    metadata: Dict[str, Any]

@dataclasses.dataclass
class Chunk:
    chunk_id: str
    parent_doc_id: str
    text: str
    metadata: Dict[str, Any]

# -----
# Basic file readers
# -----

YAML_FRONT_MATTER_RE = re.compile(r"^---\n(.*?)\n---\n",
re.DOTALL)
HEADER_RE = re.compile(r"^\s{0,3}#{1,6}\s+", re.MULTILINE)

def sha1_text(text: str) -> str:
    return hashlib.sha1(text.encode("utf-8")).hexdigest()

def load_text_file(path: Path) -> str:
    return path.read_text(encoding="utf-8", errors="ignore")

def parse_simple_yaml(yaml_text: str) -> Dict[str, Any]:
    """
    Parser YAML sederhana agar script tetap ringan.
    Cocok untuk front matter dengan pola key: value dan list
    sederhana [a, b].
    Untuk YAML kompleks, sebaiknya install pyyaml dan ganti
    parser ini.
    """
    result: Dict[str, Any] = {}
    for raw_line in yaml_text.splitlines():
        line = raw_line.strip()

```

```

        if not line or line.startswith("#"):
            continue
        if ":" not in line:
            continue
        key, value = line.split(":", 1)
        key = key.strip()
        value = value.strip()

        if value.startswith("[") and value.endswith("]"):
            items = [x.strip().strip('\'').strip('"') for x in
value[1:-1].split(",") if x.strip()]
            result[key] = items
        else:
            value = value.strip('\'').strip('"')
            result[key] = value
    return result

def parse_markdown_with_front_matter(path: Path) -> Document:
    raw = load_text_file(path)
    metadata: Dict[str, Any] = {}

    match = YAML_FRONT_MATTER_RE.match(raw)
    if match:
        yaml_block = match.group(1)
        metadata = parse_simple_yaml(yaml_block)
        raw = raw[match.end():]

    metadata.setdefault("title", path.stem)
    metadata.setdefault("source", "unknown")
    metadata.setdefault("date", "")
    metadata.setdefault("domain", "unknown")
    metadata.setdefault("language", "unknown")
    metadata.setdefault("sensitivity", "unknown")
    metadata.setdefault("quality", "unknown")
    metadata.setdefault("topic_labels", [])

    metadata["file_type"] = "markdown"
    metadata["source_path"] = str(path)

    return Document(source_path=str(path), text=raw.strip(),
metadata=metadata)

def parse_txt(path: Path) -> Document:
    text = load_text_file(path)
    metadata = {
        "title": path.stem,
        "source": "unknown",
        "date": "",
        "domain": "unknown",
        "language": "unknown",
        "sensitivity": "unknown",

```

```

        "quality": "unknown",
        "topic_labels": [],
        "file_type": "text",
        "source_path": str(path),
    }
    return Document(source_path=str(path), text=text.strip(),
                    metadata=metadata)

def parse_json(path: Path) -> Document:
    raw = load_text_file(path)
    try:
        obj = json.loads(raw)
    except json.JSONDecodeError:
        raise ValueError(f"JSON tidak valid: {path}")

    if isinstance(obj, dict):
        text = obj.get("text", "")
        metadata = obj.get("metadata", {})
        if not isinstance(metadata, dict):
            metadata = {}
    else:
        text = raw
        metadata = {}

    metadata.setdefault("title", path.stem)
    metadata.setdefault("source", "unknown")
    metadata.setdefault("date", "")
    metadata.setdefault("domain", "unknown")
    metadata.setdefault("language", "unknown")
    metadata.setdefault("sensitivity", "unknown")
    metadata.setdefault("quality", "unknown")
    metadata.setdefault("topic_labels", [])
    metadata["file_type"] = "json"
    metadata["source_path"] = str(path)

    return Document(source_path=str(path),
                    text=str(text).strip(), metadata=metadata)

def load_documents(input_dir: Path, limit: int) ->
List[Document]:
    supported = {".md", ".txt", ".json"}
    files = sorted([p for p in input_dir.rglob("*") if
                    p.is_file() and p.suffix.lower() in supported])

    if limit > 0:
        files = files[:limit]

    documents: List[Document] = []
    for path in files:
        try:
            if path.suffix.lower() == ".md":

```

```

        doc = parse_markdown_with_front_matter(path)
    elif path.suffix.lower() == ".txt":
        doc = parse_txt(path)
    elif path.suffix.lower() == ".json":
        doc = parse_json(path)
    else:
        continue

    if doc.text.strip():
        documents.append(doc)
except Exception as exc:
    print(f"[WARN] Gagal baca {path}: {exc}",
file=sys.stderr)

return documents

# -----
# Text chunking
# -----

def normalize_text(text: str) -> str:
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = re.sub(r"\n{3,}", "\n\n", text)
    text = re.sub(r"[ \t]{2,}", " ", text)
    return text.strip()

def split_by_headers(markdown_text: str) -> List[str]:
    """
    Pecah kasar berdasarkan heading Markdown.
    Jika tidak ada heading, fallback ke paragraf.
    """
    text = normalize_text(markdown_text)
    if not HEADER_RE.search(text):
        return [x.strip() for x in text.split("\n\n") if
x.strip()]

    parts: List[str] = []
    current: List[str] = []
    for line in text.splitlines():
        if HEADER_RE.match(line) and current:
            parts.append("\n".join(current).strip())
            current = [line]
        else:
            current.append(line)
    if current:
        parts.append("\n".join(current).strip())
    return [p for p in parts if p]

def chunk_text(
    text: str,

```



```

        target_chars: int = 1200,
        overlap_chars: int = 150,
    ) -> List[str]:
        """
        Chunk sederhana berbasis karakter + hormati
        heading/paragraf.
        Tidak sempurna, tetapi cukup bagus untuk pilot kecil.
        """
        sections = split_by_headers(text)
        chunks: List[str] = []

        buffer = ""
        for sec in sections:
            sec = sec.strip()
            if not sec:
                continue

            if len(sec) > target_chars * 1.5:
                paras = [p.strip() for p in sec.split("\n\n") if
p.strip()]
                for para in paras:
                    if not buffer:
                        buffer = para
                    elif len(buffer) + 2 + len(para) <=
target_chars:
                        buffer += "\n\n" + para
                    else:
                        chunks.append(buffer.strip())
                        tail = buffer[-overlap_chars:] if
overlap_chars > 0 else ""
                        buffer = (tail + "\n\n" + para).strip()
                else:
                    if not buffer:
                        buffer = sec
                    elif len(buffer) + 2 + len(sec) <= target_chars:
                        buffer += "\n\n" + sec
                    else:
                        chunks.append(buffer.strip())
                        tail = buffer[-overlap_chars:] if overlap_chars
> 0 else ""
                        buffer = (tail + "\n\n" + sec).strip()

            if buffer.strip():
                chunks.append(buffer.strip())

        # Bersihkan chunk terlalu kecil
        cleaned = [c for c in chunks if len(c.strip()) >= 80]
        return cleaned

def build_chunks(
    docs: List[Document],
    chunk_size: int,

```

```

        overlap: int,
    ) -> List[Chunk]:
        out: List[Chunk] = []

        for doc in docs:
            parent_doc_id = sha1_text(doc.source_path)[:16]
            chunks = chunk_text(doc.text, target_chars=chunk_size,
                                overlap_chars=overlap)

            for idx, ch in enumerate(chunks, start=1):
                chunk_id = f"{parent_doc_id}-{idx:04d}"
                meta = dict(doc.metadata)
                meta["chunk_index"] = idx
                meta["parent_doc_id"] = parent_doc_id
                meta["chunk_id"] = chunk_id
                meta["char_len"] = len(ch)

                out.append(
                    Chunk(
                        chunk_id=chunk_id,
                        parent_doc_id=parent_doc_id,
                        text=ch,
                        metadata=meta,
                    )
                )
        return out

# -----
# Metadata validation
# -----

REQUIRED_METADATA = [
    "title",
    "source",
    "date",
    "domain",
    "language",
    "sensitivity",
    "quality",
    "topic_labels",
]

def validate_metadata(metadata: Dict[str, Any]) -> List[str]:
    missing: List[str] = []
    for key in REQUIRED_METADATA:
        if key not in metadata:
            missing.append(key)
        elif metadata[key] in ("", None):
            missing.append(key)
    return missing

```

```

# -----
# Embedding
# -----

def load_embedder(model_name: str):
    try:
        from sentence_transformers import SentenceTransformer
    except ImportError as exc:
        raise RuntimeError(
            "sentence-transformers belum terpasang. "
            "Install dengan: pip install sentence-transformers"
        ) from exc

    return SentenceTransformer(model_name)

def embed_texts(embedder, texts: List[str], batch_size: int =
32) -> List[List[float]]:
    vectors = embedder.encode(
        texts,
        batch_size=batch_size,
        show_progress_bar=True,
        convert_to_numpy=True,
        normalize_embeddings=True,
    )
    return [v.tolist() for v in vectors]

# -----
# Vector store writers
# -----

def write_lancedb(chunks: List[Chunk], vectors:
List[List[float]], db_path: str, table_name: str) -> None:
    try:
        import lancedb
        import pyarrow as pa
    except ImportError as exc:
        raise RuntimeError(
            "LanceDB/pyarrow belum terpasang. "
            "Install dengan: pip install lancedb pyarrow"
        ) from exc

    db = lancedb.connect(db_path)

    records: List[Dict[str, Any]] = []
    for chunk, vec in zip(chunks, vectors):
        rec = {
            "id": chunk.chunk_id,
            "vector": vec,
            "text": chunk.text,
            "title": chunk.metadata.get("title", ""),

```

```

        "source": chunk.metadata.get("source", ""),
        "date": chunk.metadata.get("date", ""),
        "domain": chunk.metadata.get("domain", ""),
        "language": chunk.metadata.get("language", ""),
        "sensitivity": chunk.metadata.get("sensitivity",
""),
        "quality": chunk.metadata.get("quality", ""),
        "topic_labels":
json.dumps(chunk.metadata.get("topic_labels", []),
ensure_ascii=False),
        "source_path": chunk.metadata.get("source_path",
""),
        "parent_doc_id": chunk.metadata.get("parent_doc_id",
""),
        "chunk_index": int(chunk.metadata.get("chunk_index",
0)),
        "char_len": int(chunk.metadata.get("char_len", 0)),
    }
    records.append(rec)

if not records:
    return

# Buat schema eksplisit agar stabil
dim = len(records[0]["vector"])
schema = pa.schema([
    pa.field("id", pa.string()),
    pa.field("vector", pa.list_(pa.float32(), dim)),
    pa.field("text", pa.string()),
    pa.field("title", pa.string()),
    pa.field("source", pa.string()),
    pa.field("date", pa.string()),
    pa.field("domain", pa.string()),
    pa.field("language", pa.string()),
    pa.field("sensitivity", pa.string()),
    pa.field("quality", pa.string()),
    pa.field("topic_labels", pa.string()),
    pa.field("source_path", pa.string()),
    pa.field("parent_doc_id", pa.string()),
    pa.field("chunk_index", pa.int32()),
    pa.field("char_len", pa.int32()),
])

if table_name in db.table_names():
    table = db.open_table(table_name)
    table.add(records)
else:
    db.create_table(table_name, data=records, schema=schema)

def write_chroma(chunks: List[Chunk], vectors:
List[List[float]], db_path: str, collection_name: str) -> None:
    try:

```

```

import chromadb
except ImportError as exc:
    raise RuntimeError(
        "chromadb belum terpasang. "
        "Install dengan: pip install chromadb"
    ) from exc

client = chromadb.PersistentClient(path=db_path)
collection =
client.get_or_create_collection(name=collection_name)

ids: List[str] = []
docs: List[str] = []
metadatas: List[Dict[str, Any]] = []

for chunk in chunks:
    md = {
        "title": str(chunk.metadata.get("title", "")),
        "source": str(chunk.metadata.get("source", "")),
        "date": str(chunk.metadata.get("date", "")),
        "domain": str(chunk.metadata.get("domain", "")),
        "language": str(chunk.metadata.get("language", "")),
        "sensitivity": str(chunk.metadata.get("sensitivity",
"")),
        "quality": str(chunk.metadata.get("quality", "")),
        "topic_labels":
", ".join(chunk.metadata.get("topic_labels", [])) if
isinstance(chunk.metadata.get("topic_labels"), list) else
str(chunk.metadata.get("topic_labels", "")),
        "source_path": str(chunk.metadata.get("source_path",
"")),
        "parent_doc_id":
str(chunk.metadata.get("parent_doc_id", "")),
        "chunk_index": int(chunk.metadata.get("chunk_index",
0)),
        "char_len": int(chunk.metadata.get("char_len", 0)),
    }
    ids.append(chunk.chunk_id)
    docs.append(chunk.text)
    metadatas.append(md)

collection.add(
    ids=ids,
    documents=docs,
    metadatas=metadatas,
    embeddings=vectors,
)

# -----
# Reporting
# -----

```

```

def print_summary(
    docs: List[Document],
    chunks: List[Chunk],
    vector_store: str,
    embed_model: str,
    db_path: str,
    invalid_docs: List[Tuple[str, List[str]]],
) -> None:
    avg_chunks = (len(chunks) / len(docs)) if docs else 0.0

    print()
    print(f"[INFO] Documents ingested: {len(docs)}")
    print(f"[INFO] Chunks created: {len(chunks)}")
    print(f"[INFO] Avg chunks/doc: {avg_chunks:.2f}")
    print(f"[INFO] Embedding model: {embed_model}")
    print(f"[INFO] Vector store: {vector_store}")
    print(f"[INFO] DB path: {db_path}")

    if invalid_docs:
        print(f"[WARN] Documents with incomplete metadata:
{len(invalid_docs)}")
        for path, missing in invalid_docs[:10]:
            print(f"        - {path} | missing: {'
'.join(missing)}")
    else:
        print("[INFO] Semua dokumen memiliki metadata minimum.")

# -----
# Main
# -----

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Pilot ingest
corpus kecil ke vector store.")
    parser.add_argument("--input", required=True, help="Folder
input dokumen cleaned.")
    parser.add_argument("--limit", type=int, default=50,
help="Batas jumlah file. 0 = semua.")
    parser.add_argument("--chunk-size", type=int, default=1200,
help="Ukuran chunk berbasis karakter.")
    parser.add_argument("--overlap", type=int, default=150,
help="Overlap chunk berbasis karakter.")
    parser.add_argument(
        "--embedding-model",
        default="sentence-transformers/paraphrase-multilingual-MiniLM-L1
2-v2",
        help="Nama model embedding lokal/Hugging Face.",
    )
    parser.add_argument(
        "--vector-store",
        choices=["lancedb", "chroma"],

```

```

        default="lancedb",
        help="Vector store target.",
    )
    parser.add_argument("--db-path", default="./vectordb",
        help="Path penyimpanan vector DB.")
    parser.add_argument("--table-name", default="pilot_corpus",
        help="Nama table/collection.")
    return parser.parse_args()

def main() -> int:
    args = parse_args()
    input_dir = Path(args.input)

    if not input_dir.exists() or not input_dir.is_dir():
        print(f"[ERROR] Folder input tidak ditemukan:
        {input_dir}", file=sys.stderr)
        return 1

    docs = load_documents(input_dir, args.limit)
    if not docs:
        print("[ERROR] Tidak ada dokumen valid untuk diproses.",
        file=sys.stderr)
        return 1

    invalid_docs: List[Tuple[str, List[str]]] = []
    for doc in docs:
        missing = validate_metadata(doc.metadata)
        if missing:
            invalid_docs.append((doc.source_path, missing))

    chunks = build_chunks(docs, chunk_size=args.chunk_size,
        overlap=args.overlap)
    if not chunks:
        print("[ERROR] Tidak ada chunk yang berhasil dibuat.",
        file=sys.stderr)
        return 1

    print(f"[INFO] Loading embedding model:
    {args.embedding_model}")
    embedder = load_embedder(args.embedding_model)

    texts = [c.text for c in chunks]
    vectors = embed_texts(embedder, texts)

    os.makedirs(args.db_path, exist_ok=True)

    if args.vector_store == "lancedb":
        write_lancedb(chunks, vectors, db_path=args.db_path,
            table_name=args.table_name)
    elif args.vector_store == "chroma":
        write_chroma(chunks, vectors, db_path=args.db_path,
            collection_name=args.table_name)

```

```

    else:
        print(f"[ERROR] Vector store tidak dikenal:
{args.vector_store}", file=sys.stderr)
        return 1

    print_summary(
        docs=docs,
        chunks=chunks,
        vector_store=args.vector_store,
        embed_model=args.embedding_model,
        db_path=args.db_path,
        invalid_docs=invalid_docs,
    )
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Dependensi yang dibutuhkan

Versi minimum praktis:

```
pip install sentence-transformers lancedb pyarrow chromadb
```

Kalau hanya mau pakai LanceDB:

```
pip install sentence-transformers lancedb pyarrow
```

Kalau hanya mau pakai Chroma:

```
pip install sentence-transformers chromadb
```


Contoh struktur file input

Misalnya file `./data/cleaned/reindex-openwebui.md`:

```
---
title: "Cara Reindex Knowledge Base Open WebUI"
source: "lab-rag"
date: "2026-04-13"
domain: "rag"
language: "id"
sensitivity: "internal"
quality: "reviewed"
topic_labels: ["open-webui", "embedding", "reindex"]
---

# Cara Reindex Knowledge Base Open WebUI

## Tujuan
Menjelaskan kapan *reindex* diperlukan.

## Ringkasan
Lakukan *reindex* saat model *embedding* berubah.
```

Contoh menjalankan

```
python ingest_pilot.py \
  --input ./data/cleaned \
  --limit 50 \
  --vector-store lancedb \
  --db-path ./vectordb \
  --table-name pilot_corpus
```

Contoh output:

```
[INFO] Loading embedding model:
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Batches: 100%|████████████████████████████████████████| 4/4 [00:01<00:00,
3.12it/s]

[INFO] Documents ingested: 12
[INFO] Chunks created: 38
[INFO] Avg chunks/doc: 3.17
[INFO] Embedding model:
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
[INFO] Vector store: lancedb
[INFO] DB path: ./vectordb
[INFO] Semua dokumen memiliki metadata minimum.
```

Kalau Anda ingin, saya bisa lanjutkan dengan **source code** `eval_retrieval.py` untuk pasangan dari script ini.

Lampiran G. Source Code eval_retrieval.py

Berikut lanjutan yang paling pas: **source code** `eval_retrieval.py` untuk mengevaluasi hasil retrieval dari pilot corpus yang sudah di-*ingest*.

Script ini dibuat supaya:

- membaca **query uji** dari file YAML
- mencari ke **LanceDB** atau **Chroma**
- menampilkan **Top-1** dan **Top-K hit rate**
- memeriksa apakah dokumen yang diharapkan muncul
- memberi laporan sederhana yang mudah dibaca mahasiswa dan praktisi

Tujuannya bukan membuat evaluator akademik yang rumit, tetapi **alat cek sehat yang jujur**. Dengan alat ini, pembaca bisa tahu: retrieval-nya benar-benar membaik atau hanya terasa “kayaknya lebih bagus”.

Contoh file query uji

Simpan sebagai `./tests/queries.yaml`:

```
queries:
  - id: q01
    question: "Kapan saya harus melakukan reindex knowledge
base?"
    expected_sources:
      - "lab-rag"
    expected_titles_contains:
      - "Reindex"

  - id: q02
    question: "Apa fungsi Focused Retrieval di Open WebUI?"
    expected_titles_contains:
      - "Focused Retrieval"

  - id: q03
    question: "Bagaimana cara menarik model di Ollama?"
    expected_titles_contains:
      - "Menarik Model"
      - "Cara Menarik Model"
```

Source code eval_retrieval.py

```
#!/usr/bin/env python3
"""
eval_retrieval.py

Evaluator retrieval sederhana untuk corpus pilot.
- Baca query uji dari YAML
- Ambil top-k hasil dari LanceDB atau Chroma
- Hitung Top-1 hit rate dan Top-K hit rate
- Cetak laporan ringkas dan detail kegagalan

Contoh:
    python eval_retrieval.py --queries ./tests/queries.yaml
    --vector-store lancedb
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Any, Dict, List, Tuple

def load_yaml_file(path: Path) -> Dict[str, Any]:
    try:
        import yaml
    except ImportError as exc:
        raise RuntimeError(
            "pyyaml belum terpasang. Install dengan: pip install pyyaml"
        ) from exc

    with path.open("r", encoding="utf-8") as f:
        data = yaml.safe_load(f)

    if not isinstance(data, dict):
        raise ValueError("Format YAML tidak valid: root harus object/dictionary.")
    return data

def load_embedder(model_name: str):
    try:
        from sentence_transformers import SentenceTransformer
    except ImportError as exc:
        raise RuntimeError(
            "sentence-transformers belum terpasang. "
            "Install dengan: pip install sentence-transformers"
        ) from exc
```

```

return SentenceTransformer(model_name)

def embed_query(embedder, text: str) -> List[float]:
    vec = embedder.encode(
        [text],
        show_progress_bar=False,
        convert_to_numpy=True,
        normalize_embeddings=True,
    )[0]
    return vec.tolist()

def normalize_str(value: Any) -> str:
    return str(value or "").strip().lower()

def match_expected(result: Dict[str, Any], query_spec: Dict[str, Any]) -> bool:
    """
    Aturan match sederhana:
    - Jika expected_sources ada, salah satu harus cocok persis
    (case-insensitive)
    - Jika expected_titles_contains ada, title harus mengandung
    salah satu substring
    - Jika keduanya ada, cukup salah satu cocok? Tidak.
    Kita buat logika lebih ketat:
    - jika keduanya ada, salah satu kondisi sumber ATAU judul
    yang cocok sudah dianggap cukup?
    Untuk evaluasi ringan, ya, agar tidak terlalu rapuh.
    """
    title = normalize_str(result.get("title", ""))
    source = normalize_str(result.get("source", ""))

    expected_sources = [normalize_str(x) for x in
        query_spec.get("expected_sources", [])]
    expected_titles_contains = [normalize_str(x) for x in
        query_spec.get("expected_titles_contains", [])]

    source_match = False
    title_match = False

    if expected_sources:
        source_match = source in expected_sources

    if expected_titles_contains:
        title_match = any(fragment in title for fragment in
            expected_titles_contains)

    if expected_sources and expected_titles_contains:
        return source_match or title_match
    if expected_sources:
        return source_match

```

```

        if expected_titles_contains:
            return title_match

    return False

def search_lancedb(
    db_path: str,
    table_name: str,
    query_vector: List[float],
    top_k: int,
) -> List[Dict[str, Any]]:
    try:
        import lancedb
    except ImportError as exc:
        raise RuntimeError(
            "lancedb belum terpasang. Install dengan: pip
install lancedb pyarrow"
        ) from exc

    db = lancedb.connect(db_path)
    table = db.open_table(table_name)

    rows = (
        table.search(query_vector)
        .limit(top_k)
        .to_list()
    )

    results: List[Dict[str, Any]] = []
    for row in rows:
        results.append(
            {
                "id": row.get("id", ""),
                "title": row.get("title", ""),
                "source": row.get("source", ""),
                "text": row.get("text", ""),
                "domain": row.get("domain", ""),
                "source_path": row.get("source_path", ""),
                "chunk_index": row.get("chunk_index", 0),
                "score": row.get("_distance", None),
            }
        )
    return results

def search_chroma(
    db_path: str,
    collection_name: str,
    query_vector: List[float],
    top_k: int,
) -> List[Dict[str, Any]]:
    try:

```

```

        import chromadb
    except ImportError as exc:
        raise RuntimeError(
            "chromadb belum terpasang. Install dengan: pip
install chromadb"
        ) from exc

    client = chromadb.PersistentClient(path=db_path)
    collection = client.get_collection(name=collection_name)

    res = collection.query(
        query_embeddings=[query_vector],
        n_results=top_k,
    )

    ids = res.get("ids", [[]])[0]
    docs = res.get("documents", [[]])[0]
    metas = res.get("metadatas", [[]])[0]
    distances = res.get("distances", [[]])[0] if "distances" in
res else [None] * len(ids)

    results: List[Dict[str, Any]] = []
    for idx, doc_id in enumerate(ids):
        md = metas[idx] if idx < len(metas) and metas[idx] else
{}
        results.append(
            {
                "id": doc_id,
                "title": md.get("title", ""),
                "source": md.get("source", ""),
                "text": docs[idx] if idx < len(docs) else "",
                "domain": md.get("domain", ""),
                "source_path": md.get("source_path", ""),
                "chunk_index": md.get("chunk_index", 0),
                "score": distances[idx] if idx < len(distances)
else None,
            }
        )
    return results

def evaluate_queries(
    queries: List[Dict[str, Any]],
    vector_store: str,
    db_path: str,
    table_name: str,
    embedder,
    top_k: int,
) -> Dict[str, Any]:
    total = 0
    top1_hits = 0
    topk_hits = 0
    failures: List[Dict[str, Any]] = []

```

```

details: List[Dict[str, Any]] = []

for item in queries:
    qid = item.get("id", f"q{total+1:03d}")
    question = item.get("question", "").strip()

    if not question:
        continue

    total += 1
    qvec = embed_query(embedder, question)

    if vector_store == "lancedb":
        results = search_lancedb(db_path=db_path,
table_name=table_name, query_vector=qvec, top_k=top_k)
    elif vector_store == "chroma":
        results = search_chroma(db_path=db_path,
collection_name=table_name, query_vector=qvec, top_k=top_k)
    else:
        raise ValueError(f"Vector store tidak dikenal:
{vector_store}")

    hit_positions: List[int] = []
    for idx, result in enumerate(results, start=1):
        if match_expected(result, item):
            hit_positions.append(idx)

    top1 = len(hit_positions) > 0 and hit_positions[0] == 1
    topk = len(hit_positions) > 0

    if top1:
        top1_hits += 1
    if topk:
        topk_hits += 1

    detail = {
        "id": qid,
        "question": question,
        "top1_hit": top1,
        "topk_hit": topk,
        "hit_positions": hit_positions,
        "results": results,
    }
    details.append(detail)

    if not topk:
        failures.append(detail)

return {
    "total": total,
    "top1_hits": top1_hits,
    "topk_hits": topk_hits,

```

```

        "top1_rate": (top1_hits / total * 100.0) if total else
0.0,
        "topk_rate": (topk_hits / total * 100.0) if total else
0.0,
        "failures": failures,
        "details": details,
    }

def print_summary(report: Dict[str, Any], top_k: int) -> None:
    print()
    print(f"[INFO] Queries tested: {report['total']}")
    print(f"[INFO] Top-1 hits: {report['top1_hits']}")
    print(f"[INFO] Top-{top_k} hits: {report['topk_hits']}")
    print(f"[INFO] Top-1 hit rate: {report['top1_rate']:.2f}%")
    print(f"[INFO] Top-{top_k} hit rate:
{report['topk_rate']:.2f}%")

    if report["failures"]:
        print(f"[WARN] Query gagal masuk Top-{top_k}:
{len(report['failures'])}")
        for failure in report["failures"][:10]:
            print(f"        - {failure['id']}:
{failure['question']}")
        else:
            print(f"[INFO] Semua query berhasil masuk Top-{top_k}.")

def print_verbose_failures(report: Dict[str, Any], top_k: int)
-> None:
    failures = report.get("failures", [])
    if not failures:
        return

    print()
    print("=" * 80)
    print("DETAIL KEGAGALAN")
    print("=" * 80)

    for failure in failures:
        print(f"\n[FAIL] {failure['id']}:
{failure['question']}")
        print(f"Top-{top_k} tidak menemukan hasil yang cocok.")
        print("Hasil teratas yang dikembalikan:")

        for idx, result in enumerate(failure["results"],
start=1):
            title = result.get("title", "")
            source = result.get("source", "")
            domain = result.get("domain", "")
            source_path = result.get("source_path", "")
            text_preview = (result.get("text", "") or
"".replace("\n", " "))

```



```

        text_preview = text_preview[:140] + ("..." if
len(text_preview) > 140 else "")

        print(f" {idx}. title={title!r} | source={source!r}
| domain={domain!r}")
        print(f"      source_path={source_path}")
        print(f"      preview={text_preview}")

def save_json_report(report: Dict[str, Any], output_path: Path)
-> None:
    with output_path.open("w", encoding="utf-8") as f:
        json.dump(report, f, indent=2, ensure_ascii=False)

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluasi
retrieval corpus pilot.")
    parser.add_argument("--queries", required=True, help="Path
file YAML query uji.")
    parser.add_argument(
        "--embedding-model",
        default="sentence-transformers/paraphrase-multilingual-MiniLM-L1
2-v2",
        help="Nama model embedding yang sama dengan saat
ingest.",
    )
    parser.add_argument(
        "--vector-store",
        choices=["lancedb", "chroma"],
        default="lancedb",
        help="Vector store target.",
    )
    parser.add_argument("--db-path", default="./vectordb",
help="Path vector DB.")
    parser.add_argument("--table-name", default="pilot_corpus",
help="Nama table/collection.")
    parser.add_argument("--top-k", type=int, default=5,
help="Jumlah hasil retrieval yang diuji.")
    parser.add_argument("--verbose", action="store_true",
help="Tampilkan detail kegagalan.")
    parser.add_argument("--save-json", default="", help="Simpan
laporan JSON ke file.")
    return parser.parse_args()

def main() -> int:
    args = parse_args()

    query_path = Path(args.queries)
    if not query_path.exists():

```

```

        print(f"[ERROR] File query tidak ditemukan:
{query_path}", file=sys.stderr)
        return 1

    data = load_yaml_file(query_path)
    queries = data.get("queries", [])
    if not isinstance(queries, list) or not queries:
        print("[ERROR] File query YAML tidak berisi daftar
'queries' yang valid.", file=sys.stderr)
        return 1

    print(f"[INFO] Loading embedding model:
{args.embedding_model}")
    embedder = load_embedder(args.embedding_model)

    report = evaluate_queries(
        queries=queries,
        vector_store=args.vector_store,
        db_path=args.db_path,
        table_name=args.table_name,
        embedder=embedder,
        top_k=args.top_k,
    )

    print_summary(report, top_k=args.top_k)

    if args.verbose:
        print_verbose_failures(report, top_k=args.top_k)

    if args.save_json:
        output_path = Path(args.save_json)
        save_json_report(report, output_path)
        print(f"[INFO] JSON report saved: {output_path}")

    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Dependensi

Tambahkan **pyyaml** karena script ini membaca file YAML:

```

pip install pyyaml sentence-transformers lancedb pyarrow
chromadb

```

Kalau hanya LanceDB:

```

pip install pyyaml sentence-transformers lancedb pyarrow

```

Contoh menjalankan

Evaluasi LanceDB

```
python eval_retrieval.py \  
  --queries ./tests/queries.yaml \  
  --vector-store lancedb \  
  --db-path ./vectordb \  
  --table-name pilot_corpus \  
  --top-k 5 \  
  --verbose
```

Evaluasi Chroma

```
python eval_retrieval.py \  
  --queries ./tests/queries.yaml \  
  --vector-store chroma \  
  --db-path ./vectordb \  
  --table-name pilot_corpus \  
  --top-k 5
```

Contoh output

```
[INFO] Loading embedding model:  
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

```
[INFO] Queries tested: 3  
[INFO] Top-1 hits: 2  
[INFO] Top-5 hits: 3  
[INFO] Top-1 hit rate: 66.67%  
[INFO] Top-5 hit rate: 100.00%  
[INFO] Semua query berhasil masuk Top-5.
```

Kalau ada yang gagal:

```
[INFO] Queries tested: 3  
[INFO] Top-1 hits: 1  
[INFO] Top-5 hits: 2  
[INFO] Top-1 hit rate: 33.33%  
[INFO] Top-5 hit rate: 66.67%  
[WARN] Query gagal masuk Top-5: 1  
  - q02: Apa fungsi Focused Retrieval di Open WebUI?
```

```
=====
=====
DETAIL KEGAGALAN
=====
=====
```

```
[FAIL] q02: Apa fungsi Focused Retrieval di Open WebUI?  
Top-5 tidak menemukan hasil yang cocok.  
Hasil teratas yang dikembalikan:
```

```
1. title='Cara Menarik Model di Ollama' | source='lab-rag' |  
domain='llm'  
    source_path=./data/cleaned/tutorial-ollama.md  
    preview=# Cara Menarik Model di Ollama ## Tujuan  
Menjelaskan langkah dasar untuk mengambil model lokal...
```

Pasangan workflow yang sehat

Urutan kerja yang saya sarankan untuk buku ini:

- `ingest_pilot.py` untuk memasukkan corpus kecil dulu
- `eval_retrieval.py` untuk menguji kualitas retrieval
- baru setelah itu ubah:
 - *embedding model*
 - ukuran *chunk*
 - overlap
 - metadata
 - vector database
 - *reranker*

Dengan begitu, pembaca tidak sekadar “merasa sistemnya bagus”, tetapi **punya angka sederhana untuk menilai retrieval**.

Lampiran H. Source Code `clean_markdown.py`

Berikut **source code Python `clean_markdown.py`** untuk **memeriksa dan membersihkan file Markdown hasil generasi otomatis**. Script ini cocok untuk kasus seperti:

- hasil ekstraksi PDF → Markdown
- hasil konversi otomatis dari dokumen lain
- hasil AI yang masih berantakan
- file `.md` yang ingin dicek sebelum masuk ke corpus RAG

Fungsi utamanya:

- membaca file `.md`
- membersihkan **nomor halaman liar**
- merapikan **baris kosong berlebihan**
- menggabungkan **baris paragraf yang pecah**
- menjaga **heading** dan **code block**
- mendeteksi beberapa masalah umum
- menulis hasil bersih ke file output
- menampilkan **laporan check sederhana**

Contoh pakai

Bersihkan satu file

```
python clean_markdown.py --input
./work/output/raw/textbook.raw.md --output
./work/output/cleaned/textbook.clean.md
```

Mode check saja, tanpa menulis file

```
python clean_markdown.py --input
./work/output/raw/textbook.raw.md --check-only
```

Bersihkan semua file `.md` dalam satu folder

```
python clean_markdown.py --input-dir ./work/output/raw
--output-dir ./work/output/cleaned
```

Source code `clean_markdown.py`

```
#!/usr/bin/env python3
"""
clean_markdown.py

Membersihkan dan memeriksa file Markdown hasil generasi otomatis
agar lebih siap dipakai untuk corpus RAG.

Fitur:
- baca satu file atau satu folder
- bersihkan nomor halaman liar
- rapikan baris kosong berlebihan
- gabungkan paragraf pecah sederhana
- pertahankan heading dan code block
- deteksi masalah umum
- tampilkan laporan ringkas

Contoh:
    python clean_markdown.py --input input.md --output output.md
    python clean_markdown.py --input input.md --check-only
    python clean_markdown.py --input-dir ./raw --output-dir ./cleaned
"""

from __future__ import annotations

import argparse
import dataclasses
import re
import sys
from pathlib import Path
from typing import List, Tuple

@dataclasses.dataclass
class CheckReport:
    path: str
    total_lines: int
    empty_lines: int
    heading_count: int
    code_block_count: int
    standalone_page_numbers: int
    suspicious_short_lines: int
    broken_paragraph_candidates: int
    changed: bool
    warnings: List[str]

PAGE_NUMBER_RE = re.compile(r"^\s*\d+\s*$")
HEADING_RE = re.compile(r"^\s{0,3}#{1,6}\s+")
LIST_RE = re.compile(r"^\s*([-+]|\\d+\\.)\s+")
CODE_FENCE_RE = re.compile(r"^\s*```")
TABLE_RE = re.compile(r"^\s*\|.*\\s*$")
```

```

HORIZONTAL_RULE_RE = re.compile(r"^\\s*([-*_]\\s*){3,}\\$")

def read_text(path: Path) -> str:
    return path.read_text(encoding="utf-8", errors="ignore")

def write_text(path: Path, content: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content, encoding="utf-8")

def analyze_markdown(text: str, path: str) -> CheckReport:
    lines = text.splitlines()
    warnings: List[str] = []

    heading_count = 0
    code_block_count = 0
    empty_lines = 0
    standalone_page_numbers = 0
    suspicious_short_lines = 0
    broken_paragraph_candidates = 0

    in_code_block = False

    for i, line in enumerate(lines):
        stripped = line.strip()

        if not stripped:
            empty_lines += 1

        if CODE_FENCE_RE.match(line):
            in_code_block = not in_code_block
            code_block_count += 1

        if in_code_block:
            continue

        if HEADING_RE.match(line):
            heading_count += 1

        if PAGE_NUMBER_RE.match(line):
            standalone_page_numbers += 1

        if (
            stripped
            and len(stripped) < 20
            and not HEADING_RE.match(line)
            and not LIST_RE.match(line)
            and not TABLE_RE.match(line)
            and not HORIZONTAL_RULE_RE.match(line)
        ):
            suspicious_short_lines += 1

```

```

# heuristik kandidat paragraf pecah:
# baris sekarang tidak kosong, bukan heading/list/table/code
# baris berikutnya juga tidak kosong, bukan
heading/list/table/code
# dan baris sekarang tidak berakhir dengan tanda baca kuat
if i < len(lines) - 1:
    next_line = lines[i + 1]
    if (
        stripped
        and next_line.strip()
        and not HEADING_RE.match(line)
        and not HEADING_RE.match(next_line)
        and not LIST_RE.match(line)
        and not LIST_RE.match(next_line)
        and not TABLE_RE.match(line)
        and not TABLE_RE.match(next_line)
        and not CODE_FENCE_RE.match(line)
        and not CODE_FENCE_RE.match(next_line)
        and not HORIZONTAL_RULE_RE.match(line)
        and not HORIZONTAL_RULE_RE.match(next_line)
        and not stripped.endswith((".", "!", "?", ":", ";"))
    ):
        broken_paragraph_candidates += 1

if standalone_page_numbers > 0:
    warnings.append(f"Ditemukan {standalone_page_numbers} nomor
halaman liar.")
if suspicious_short_lines > 10:
    warnings.append(f"Banyak baris pendek mencurigakan:
{suspicious_short_lines}.")
if broken_paragraph_candidates > 10:
    warnings.append(
        f"Banyak kandidat paragraf pecah:
{broken_paragraph_candidates}."
    )
if code_block_count % 2 != 0:
    warnings.append("Jumlah fence code block ganjil. Ada
kemungkinan code block tidak tertutup.")

return CheckReport(
    path=path,
    total_lines=len(lines),
    empty_lines=empty_lines,
    heading_count=heading_count,
    code_block_count=code_block_count,
    standalone_page_numbers=standalone_page_numbers,
    suspicious_short_lines=suspicious_short_lines,
    broken_paragraph_candidates=broken_paragraph_candidates,
    changed=False,
    warnings=warnings,
)

```



```

def clean_markdown_text(text: str) -> Tuple[str, bool]:
    """
    Membersihkan markdown dengan tetap menjaga heading, list, table,
    dan code block.
    """
    lines = text.replace("\r\n", "\n").replace("\r", "\n").split("\n")
    cleaned_lines: List[str] = []
    changed = False

    in_code_block = False
    paragraph_buffer: List[str] = []

    def flush_paragraph_buffer():
        nonlocal changed
        if not paragraph_buffer:
            return

        if len(paragraph_buffer) == 1:
            cleaned_lines.append(paragraph_buffer[0].rstrip())
        else:
            merged = " ".join(x.strip() for x in paragraph_buffer if
x.strip())
            merged = re.sub(r"\s{2,}", " ", merged).strip()
            cleaned_lines.append(merged)
            changed = True
            paragraph_buffer.clear()

    def is_structural(line: str) -> bool:
        return (
            not line.strip()
            or HEADING_RE.match(line) is not None
            or LIST_RE.match(line) is not None
            or TABLE_RE.match(line) is not None
            or HORIZONTAL_RULE_RE.match(line) is not None
            or CODE_FENCE_RE.match(line) is not None
        )

    for line in lines:
        raw = line.rstrip()
        stripped = raw.strip()

        if CODE_FENCE_RE.match(raw):
            flush_paragraph_buffer()
            cleaned_lines.append(raw)
            in_code_block = not in_code_block
            continue

        if in_code_block:
            cleaned_lines.append(raw)
            continue

        # Buang nomor halaman yang berdiri sendiri

```

```

        if PAGE_NUMBER_RE.match(raw):
            changed = True
            continue

        # Buang baris yang hanya spasi
        if not stripped:
            flush_paragraph_buffer()
            # cegah banyak baris kosong berturut-turut
            if not cleaned_lines or cleaned_lines[-1] != "":
                cleaned_lines.append("")
            else:
                changed = True
                continue

        # Heading / list / table / hr harus dipertahankan sebagai unit
        if is_structural(raw):
            flush_paragraph_buffer()
            cleaned_lines.append(raw)
            continue

        # kandidat paragraf biasa
        paragraph_buffer.append(raw)

    flush_paragraph_buffer()

    cleaned = "\n".join(cleaned_lines)

    # rapikan baris kosong berlebihan lagi
    before = cleaned
    cleaned = re.sub(r"\n{3,}", "\n\n", cleaned).strip() + "\n"
    if cleaned != before:
        changed = True

    return cleaned, changed

def process_file(input_path: Path, output_path: Path | None,
check_only: bool) -> int:
    original = read_text(input_path)
    report_before = analyze_markdown(original, str(input_path))
    cleaned, changed = clean_markdown_text(original)
    report_after = analyze_markdown(cleaned, str(input_path))
    report_after.changed = changed

    print("=" * 72)
    print(f"FILE      : {input_path}")
    if output_path:
        print(f"OUTPUT   : {output_path}")
    print(f"LINES    : {report_before.total_lines} ->
{report_after.total_lines}")
    print(f"HEAD     : {report_before.heading_count}")
    print(f"CODE     : {report_before.code_block_count}")

```

```

    print(f"PAGE#   : {report_before.standalone_page_numbers} ->
{report_after.standalone_page_numbers}")
    print(f"SHORT   : {report_before.suspicious_short_lines} ->
{report_after.suspicious_short_lines}")
    print(f"BROKEN   : {report_before.broken_paragraph_candidates} ->
{report_after.broken_paragraph_candidates}")
    print(f"CHANGED: {'yes' if changed else 'no'}")

    if report_before.warnings:
        print("WARN   :")
        for w in report_before.warnings:
            print(f"    - {w}")

    if check_only:
        print("STATUS : CHECK ONLY")
        return 0

    if output_path is None:
        print("STATUS : SKIP (tidak ada output path)")
        return 1

    write_text(output_path, cleaned)
    print("STATUS : OK")
    return 0

def collect_md_files(input_dir: Path) -> List[Path]:
    return sorted(
        p for p in input_dir.rglob("*.md")
        if p.is_file()
    )

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Check dan clean file Markdown hasil generasi
otomatis."
    )

    group = parser.add_mutually_exclusive_group(required=True)
    group.add_argument("--input", help="Path file Markdown tunggal.")
    group.add_argument("--input-dir", help="Folder berisi file
Markdown.")

    parser.add_argument("--output", help="Path output untuk satu
file.")
    parser.add_argument("--output-dir", help="Folder output untuk mode
folder.")
    parser.add_argument(
        "--check-only",
        action="store_true",
        help="Hanya cek file, tanpa menulis hasil bersih."
    )

```

```

    return parser.parse_args()

def main() -> int:
    args = parse_args()

    if args.input:
        input_path = Path(args.input)
        if not input_path.exists():
            print(f"[ERROR] File tidak ditemukan: {input_path}",
file=sys.stderr)
            return 1

        output_path = Path(args.output) if args.output else None
        if not args.check_only and output_path is None:
            print("[ERROR] Untuk mode file tunggal, --output wajib
diisi jika bukan --check-only.", file=sys.stderr)
            return 1

        return process_file(input_path, output_path, args.check_only)

    if args.input_dir:
        input_dir = Path(args.input_dir)
        if not input_dir.exists() or not input_dir.is_dir():
            print(f"[ERROR] Folder tidak ditemukan: {input_dir}",
file=sys.stderr)
            return 1

        files = collect_md_files(input_dir)
        if not files:
            print(f"[ERROR] Tidak ada file .md di folder:
{input_dir}", file=sys.stderr)
            return 1

        if not args.check_only and not args.output_dir:
            print("[ERROR] Untuk mode folder, --output-dir wajib diisi
jika bukan --check-only.", file=sys.stderr)
            return 1

        rc = 0
        output_dir = Path(args.output_dir) if args.output_dir else
None

        for path in files:
            if args.check_only:
                out = None
            else:
                rel = path.relative_to(input_dir)
                out = output_dir / rel # type: ignore[operator]
            result = process_file(path, out, args.check_only)
            if result != 0:
                rc = result

```

```
        return rc

    return 1

if __name__ == "__main__":
    raise SystemExit(main())
```

Apa yang dibersihkan oleh script ini

Script ini fokus ke masalah yang paling sering muncul pada file Markdown hasil otomatis:

- **nomor halaman liar**
- **baris kosong berlebihan**
- **paragraf yang pecah jadi banyak baris**
- **spasi berlebihan**
- **laporan potensi masalah** seperti:
 - terlalu banyak baris pendek mencurigakan
 - terlalu banyak kandidat paragraf pecah
 - *code fence* tidak seimbang

Script ini **tidak** mencoba:

- memperbaiki tabel rusak yang kompleks
- memperbaiki OCR buruk
- memahami struktur semantik yang rumit
- mengubah isi substantif dokumen

Jadi, posisinya tepat sebagai **alat check dan cleaning awal**, sebelum validasi manual.

Contoh 1: check saja

```
python clean_markdown.py --input
./work/output/raw/textbook.raw.md --check-only
```

Contoh output:

```
=====
FILE      : work/output/raw/textbook.raw.md
LINES     : 842 -> 801
HEAD      : 27
CODE      : 4
PAGE#     : 18 -> 0
SHORT     : 39 -> 12
BROKEN    : 117 -> 9
CHANGED   : yes
WARN      :
  - Ditemukan 18 nomor halaman liar.
  - Banyak baris pendek mencurigakan: 39.
  - Banyak kandidat paragraf pecah: 117.
STATUS    : CHECK ONLY
```

Contoh 2: bersihkan satu file

```
python clean_markdown.py \
--input ./work/output/raw/textbook.raw.md \
--output ./work/output/cleaned/textbook.clean.md
```

Contoh output:

```
=====
FILE      : work/output/raw/textbook.raw.md
OUTPUT    : work/output/cleaned/textbook.clean.md
LINES     : 842 -> 801
HEAD      : 27
CODE      : 4
PAGE#     : 18 -> 0
SHORT     : 39 -> 12
BROKEN    : 117 -> 9
CHANGED   : yes
WARN      :
  - Ditemukan 18 nomor halaman liar.
  - Banyak baris pendek mencurigakan: 39.
  - Banyak kandidat paragraf pecah: 117.
STATUS    : OK
```

Contoh 3: bersihkan satu folder

```
python clean_markdown.py \
--input-dir ./work/output/raw \
--output-dir ./work/output/cleaned
```

Catatan penggunaan yang sehat

Gunakan script ini dengan pola kerja seperti ini:

- ekstrak PDF ke Markdown
- jalankan `clean_markdown.py`
- baca cepat hasil bersih
- cek heading, metadata, dan *code block*
- baru lanjut ke:
 - pecah per bab/subbab
 - tambah YAML metadata
 - *ingest* ke vector store
 - uji retrieval

Jangan menganggap hasil script ini sebagai akhir. Ini adalah **langkah pembersihan awal**, bukan pengganti validasi manusia.

Tambahan kecil yang bisa diajarkan di buku

Kalau pembaca ingin alur paling cepat:

```
python clean_markdown.py \  
  --input ./work/output/raw/textbook.raw.md \  
  --output ./work/output/cleaned/textbook.clean.md  
  
grep -n '^#' ./work/output/cleaned/textbook.clean.md | head  
sed -n '1,80p' ./work/output/cleaned/textbook.clean.md
```

Itu sudah cukup untuk memberi mereka satu kebiasaan yang sangat penting: **cek dulu kualitas Markdown hasil otomatis sebelum dipakai sebagai corpus**.

Kalau Anda mau, saya bisa lanjutkan membuat **versi `clean_markdown.py` yang lebih canggih**, misalnya:

- menambahkan **YAML metadata otomatis**
- memecah **per bab/subbab**
- membuat **laporan CSV/JSON**
- atau mendeteksi **tabel rusak dan heading lemah**.

Lampiran I. test_embedding_retrieval.py

Berikut lanjutan yang paling logis: **source code** `test_embedding_retrieval.py` untuk **membandingkan beberapa model *embedding*** pada corpus dan query uji yang sama.

Script ini berguna untuk menjawab pertanyaan praktis seperti:

- model *embedding* mana yang paling cocok untuk corpus Anda;
- apakah model A lebih bagus untuk Bahasa Indonesia;
- apakah model B lebih bagus untuk dokumen teknis atau *code-aware*;
- apakah model yang lebih berat benar-benar memberi hasil lebih baik.

Jadi, ini bukan sekadar script tambahan. Ini alat untuk membuat keputusan **waras**, bukan keputusan ikut tren.

Fungsi script

Script ini akan:

- membaca dokumen dari folder corpus kecil
- membuat *chunk*
- membaca query uji dari YAML
- mencoba **beberapa model *embedding***
- melakukan retrieval lokal di memori
- menghitung:
 - **Top-1 hit rate**
 - **Top-K hit rate**
- menampilkan perbandingan hasil antar-model

Format file query uji

Gunakan file yang sama seperti `tests/queries.yaml`:

```
queries:
  - id: q01
    question: "Kapan saya harus melakukan reindex knowledge base?"
    expected_titles_contains:
      - "Reindex"

  - id: q02
    question: "Apa fungsi Focused Retrieval di Open WebUI?"
    expected_titles_contains:
      - "Focused Retrieval"

  - id: q03
    question: "Bagaimana cara menarik model di Ollama?"
    expected_titles_contains:
      - "Menarik Model"
      - "Cara Menarik Model"
```

Source code test_embedding_retrieval.py

```
#!/usr/bin/env python3
"""
test_embedding_retrieval.py

Bandingkan beberapa model embedding pada corpus dan query uji yang
sama.

Fitur:
- baca file .md, .txt, .json
- ambil YAML front matter jika ada
- chunk teks sederhana
- embed dokumen dan query
- retrieval cosine similarity di memori
- hitung Top-1 dan Top-K hit rate
- bandingkan beberapa model embedding

Contoh:
python test_embedding_retrieval.py \
  --input ./data/cleaned \
  --queries ./tests/queries.yaml \
  --models
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 \
  intfloat/multilingual-e5-base \
  --limit 50 \
  --top-k 5
"""

from __future__ import annotations
```

```

import argparse
import dataclasses
import json
import math
import re
import sys
from pathlib import Path
from typing import Any, Dict, List, Tuple

# =====
# Data structures
# =====

@dataclasses.dataclass
class Document:
    source_path: str
    text: str
    metadata: Dict[str, Any]

@dataclasses.dataclass
class Chunk:
    chunk_id: str
    parent_doc_id: str
    text: str
    metadata: Dict[str, Any]

# =====
# File parsing
# =====

YAML_FRONT_MATTER_RE = re.compile(r"^---\n(.*?)\n---\n", re.DOTALL)
HEADER_RE = re.compile(r"^\s{0,3}#{1,6}\s+", re.MULTILINE)

def sha1_text(text: str) -> str:
    import hashlib
    return hashlib.sha1(text.encode("utf-8")).hexdigest()

def load_text_file(path: Path) -> str:
    return path.read_text(encoding="utf-8", errors="ignore")

def parse_simple_yaml(yaml_text: str) -> Dict[str, Any]:
    result: Dict[str, Any] = {}
    for raw_line in yaml_text.splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#"):
            continue
        if ":" not in line:

```

```

        continue
    key, value = line.split(":", 1)
    key = key.strip()
    value = value.strip()

    if value.startswith("[") and value.endswith("]"):
        items = [x.strip().strip('"').strip("'") for x in
value[1:-1].split(",") if x.strip()]
        result[key] = items
    else:
        result[key] = value.strip('"').strip("'")
return result

def parse_markdown_with_front_matter(path: Path) -> Document:
    raw = load_text_file(path)
    metadata: Dict[str, Any] = {}

    match = YAML_FRONT_MATTER_RE.match(raw)
    if match:
        metadata = parse_simple_yaml(match.group(1))
        raw = raw[match.end():]

    metadata.setdefault("title", path.stem)
    metadata.setdefault("source", "unknown")
    metadata.setdefault("date", "")
    metadata.setdefault("domain", "unknown")
    metadata.setdefault("language", "unknown")
    metadata.setdefault("sensitivity", "unknown")
    metadata.setdefault("quality", "unknown")
    metadata.setdefault("topic_labels", [])
    metadata["file_type"] = "markdown"
    metadata["source_path"] = str(path)

    return Document(source_path=str(path), text=raw.strip(),
metadata=metadata)

def parse_txt(path: Path) -> Document:
    text = load_text_file(path)
    metadata = {
        "title": path.stem,
        "source": "unknown",
        "date": "",
        "domain": "unknown",
        "language": "unknown",
        "sensitivity": "unknown",
        "quality": "unknown",
        "topic_labels": [],
        "file_type": "text",
        "source_path": str(path),
    }

```

```

        return Document(source_path=str(path), text=text.strip(),
metadata=metadata)

def parse_json(path: Path) -> Document:
    raw = load_text_file(path)
    obj = json.loads(raw)

    if isinstance(obj, dict):
        text = obj.get("text", "")
        metadata = obj.get("metadata", {})
        if not isinstance(metadata, dict):
            metadata = {}
    else:
        text = raw
        metadata = {}

    metadata.setdefault("title", path.stem)
    metadata.setdefault("source", "unknown")
    metadata.setdefault("date", "")
    metadata.setdefault("domain", "unknown")
    metadata.setdefault("language", "unknown")
    metadata.setdefault("sensitivity", "unknown")
    metadata.setdefault("quality", "unknown")
    metadata.setdefault("topic_labels", [])
    metadata["file_type"] = "json"
    metadata["source_path"] = str(path)

    return Document(source_path=str(path), text=str(text).strip(),
metadata=metadata)

def load_documents(input_dir: Path, limit: int) -> List[Document]:
    supported = {".md", ".txt", ".json"}
    files = sorted([p for p in input_dir.rglob("*") if p.is_file() and
p.suffix.lower() in supported])

    if limit > 0:
        files = files[:limit]

    docs: List[Document] = []
    for path in files:
        try:
            if path.suffix.lower() == ".md":
                doc = parse_markdown_with_front_matter(path)
            elif path.suffix.lower() == ".txt":
                doc = parse_txt(path)
            elif path.suffix.lower() == ".json":
                doc = parse_json(path)
            else:
                continue

            if doc.text.strip():

```

```

        docs.append(doc)
    except Exception as exc:
        print(f"[WARN] Gagal baca {path}: {exc}", file=sys.stderr)
    return docs

# =====
# Chunking
# =====

def normalize_text(text: str) -> str:
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = re.sub(r"\n{3,}", "\n\n", text)
    text = re.sub(r"[ \t]{2,}", " ", text)
    return text.strip()

def split_by_headers(markdown_text: str) -> List[str]:
    text = normalize_text(markdown_text)
    if not HEADER_RE.search(text):
        return [x.strip() for x in text.split("\n\n") if x.strip()]

    parts: List[str] = []
    current: List[str] = []
    for line in text.splitlines():
        if HEADER_RE.match(line) and current:
            parts.append("\n".join(current).strip())
            current = [line]
        else:
            current.append(line)
    if current:
        parts.append("\n".join(current).strip())
    return [p for p in parts if p]

def chunk_text(text: str, target_chars: int = 1200, overlap_chars: int
= 150) -> List[str]:
    sections = split_by_headers(text)
    chunks: List[str] = []
    buffer = ""

    for sec in sections:
        if len(sec) > target_chars * 1.5:
            paras = [p.strip() for p in sec.split("\n\n") if
p.strip()]
            for para in paras:
                if not buffer:
                    buffer = para
                elif len(buffer) + 2 + len(para) <= target_chars:
                    buffer += "\n\n" + para
                else:
                    chunks.append(buffer.strip())

```

```

        tail = buffer[-overlap_chars:] if overlap_chars >
0 else ""
        buffer = (tail + "\n\n" + para).strip()
    else:
        if not buffer:
            buffer = sec
        elif len(buffer) + 2 + len(sec) <= target_chars:
            buffer += "\n\n" + sec
        else:
            chunks.append(buffer.strip())
            tail = buffer[-overlap_chars:] if overlap_chars > 0
    else ""
        buffer = (tail + "\n\n" + sec).strip()

    if buffer.strip():
        chunks.append(buffer.strip())

    return [c for c in chunks if len(c.strip()) >= 80]

def build_chunks(docs: List[Document], chunk_size: int, overlap: int)
-> List[Chunk]:
    out: List[Chunk] = []
    for doc in docs:
        parent_doc_id = sha1_text(doc.source_path)[:16]
        doc_chunks = chunk_text(doc.text, target_chars=chunk_size,
overlap_chars=overlap)

        for idx, ch in enumerate(doc_chunks, start=1):
            chunk_id = f"{parent_doc_id}-{idx:04d}"
            meta = dict(doc.metadata)
            meta["chunk_index"] = idx
            meta["parent_doc_id"] = parent_doc_id
            meta["chunk_id"] = chunk_id
            meta["char_len"] = len(ch)

            out.append(
                Chunk(
                    chunk_id=chunk_id,
                    parent_doc_id=parent_doc_id,
                    text=ch,
                    metadata=meta,
                )
            )
    return out

# =====
# Query loading
# =====

def load_yaml_file(path: Path) -> Dict[str, Any]:
    try:

```

```

        import yaml
    except ImportError as exc:
        raise RuntimeError("pyyaml belum terpasang. Install dengan:
pip install pyyaml") from exc

    with path.open("r", encoding="utf-8") as f:
        data = yaml.safe_load(f)

    if not isinstance(data, dict):
        raise ValueError("Format YAML tidak valid.")
    return data

# =====
# Embedding + retrieval
# =====

def load_embedder(model_name: str):
    try:
        from sentence_transformers import SentenceTransformer
    except ImportError as exc:
        raise RuntimeError(
            "sentence-transformers belum terpasang. Install dengan:
pip install sentence-transformers"
        ) from exc

    return SentenceTransformer(model_name)

def embed_texts(embedder, texts: List[str], batch_size: int = 32):
    vectors = embedder.encode(
        texts,
        batch_size=batch_size,
        show_progress_bar=True,
        convert_to_numpy=True,
        normalize_embeddings=True,
    )
    return vectors

def cosine_similarity_dense(query_vec, doc_matrix) -> List[float]:
    """
    Karena kita sudah normalize_embeddings=True,
    dot product = cosine similarity.
    """
    scores = doc_matrix @ query_vec
    return scores.tolist()

def top_k_indices(scores: List[float], k: int) -> List[int]:
    return sorted(range(len(scores)), key=lambda i: scores[i],
reverse=True)[:k]

```

```

# =====
# Matching logic
# =====

def normalize_str(value: Any) -> str:
    return str(value or "").strip().lower()

def match_expected(result: Dict[str, Any], query_spec: Dict[str, Any])
-> bool:
    title = normalize_str(result.get("title", ""))
    source = normalize_str(result.get("source", ""))

    expected_sources = [normalize_str(x) for x in
query_spec.get("expected_sources", [])]
    expected_titles_contains = [normalize_str(x) for x in
query_spec.get("expected_titles_contains", [])]

    source_match = False
    title_match = False

    if expected_sources:
        source_match = source in expected_sources

    if expected_titles_contains:
        title_match = any(fragment in title for fragment in
expected_titles_contains)

    if expected_sources and expected_titles_contains:
        return source_match or title_match
    if expected_sources:
        return source_match
    if expected_titles_contains:
        return title_match

    return False

# =====
# Evaluation
# =====

def evaluate_model(
    model_name: str,
    chunks: List[Chunk],
    queries: List[Dict[str, Any]],
    top_k: int,
) -> Dict[str, Any]:
    print(f"\n[INFO] Loading embedding model: {model_name}")
    embedder = load_embedder(model_name)

    doc_texts = [c.text for c in chunks]

```



```

doc_vectors = embed_texts(embedder, doc_texts)

total = 0
top1_hits = 0
topk_hits = 0
failures: List[Dict[str, Any]] = []

for item in queries:
    qid = item.get("id", f"q{total+1:03d}")
    question = str(item.get("question", "")).strip()
    if not question:
        continue

    total += 1
    query_vec = embed_texts(embedder, [question])[0]
    scores = cosine_similarity_dense(query_vec, doc_vectors)
    indices = top_k_indices(scores, top_k)

    results: List[Dict[str, Any]] = []
    hit_positions: List[int] = []

    for rank, idx in enumerate(indices, start=1):
        ch = chunks[idx]
        result = {
            "id": ch.chunk_id,
            "title": ch.metadata.get("title", ""),
            "source": ch.metadata.get("source", ""),
            "domain": ch.metadata.get("domain", ""),
            "source_path": ch.metadata.get("source_path", ""),
            "chunk_index": ch.metadata.get("chunk_index", 0),
            "score": float(scores[idx]),
            "text": ch.text,
        }
        results.append(result)
        if match_expected(result, item):
            hit_positions.append(rank)

    top1 = len(hit_positions) > 0 and hit_positions[0] == 1
    topk_hit = len(hit_positions) > 0

    if top1:
        top1_hits += 1
    if topk_hit:
        topk_hits += 1
    else:
        failures.append(
            {
                "id": qid,
                "question": question,
                "results": results,
            }
        )

```

```

    return {
        "model": model_name,
        "total": total,
        "top1_hits": top1_hits,
        "topk_hits": topk_hits,
        "top1_rate": (top1_hits / total * 100.0) if total else 0.0,
        "topk_rate": (topk_hits / total * 100.0) if total else 0.0,
        "failures": failures,
    }

def print_model_summary(report: Dict[str, Any], top_k: int) -> None:
    print(f"\n=== MODEL: {report['model']} ===")
    print(f"[INFO] Queries tested: {report['total']}")
    print(f"[INFO] Top-1 hits: {report['top1_hits']}")
    print(f"[INFO] Top-{top_k} hits: {report['topk_hits']}")
    print(f"[INFO] Top-1 hit rate: {report['top1_rate']:.2f}%")
    print(f"[INFO] Top-{top_k} hit rate: {report['topk_rate']:.2f}%")

    if report["failures"]:
        print(f"[WARN] Query gagal masuk Top-{top_k}: {len(report['failures'])}")
        for failure in report["failures"][:5]:
            print(f"    - {failure['id']}: {failure['question']}")
    else:
        print(f"[INFO] Semua query berhasil masuk Top-{top_k}.")

def print_ranking_table(reports: List[Dict[str, Any]], top_k: int) -> None:
    ranked = sorted(reports, key=lambda x: (x["topk_rate"], x["top1_rate"]), reverse=True)

    print("\n" + "=" * 90)
    print("PERBANDINGAN MODEL EMBEDDING")
    print("=" * 90)
    print(f"{'No':<4} {'Model':<55} {'Top-1':>10} {'Top-'+str(top_k):>10}")
    print("-" * 90)

    for idx, rep in enumerate(ranked, start=1):
        print(f"{idx:<4} {rep['model']:<55} {rep['top1_rate']:>9.2f}% {rep['topk_rate']:>9.2f}%")

    print("-" * 90)
    if ranked:
        print(f"[INFO] Model paling menjanjikan saat ini: {ranked[0]['model']}")

def print_verbose_failures(reports: List[Dict[str, Any]], top_k: int) -> None:
    print("\n" + "=" * 90)

```

```

print("DETAIL KEGAGALAN PER MODEL")
print("=" * 90)

for report in reports:
    if not report["failures"]:
        continue

    print(f"\n### MODEL: {report['model']}")
    for failure in report["failures"][:3]:
        print(f"\n[FAIL] {failure['id']}: {failure['question']}")
        for idx, result in enumerate(failure["results"], start=1):
            preview = result["text"].replace("\n", " ")
            preview = preview[:120] + ("..." if len(preview) > 120
else "")

            print(
                f"    {idx}. title={result['title']!r} |
source={result['source']!r} "
                f"| score={result['score']:.4f}"
            )
            print(f"        source_path={result['source_path']}")
            print(f"        preview={preview}")

# =====
# Main
# =====

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Bandingkan beberapa
model embedding untuk retrieval.")
    parser.add_argument("--input", required=True, help="Folder dokumen
cleaned.")
    parser.add_argument("--queries", required=True, help="File YAML
query uji.")
    parser.add_argument(
        "--models",
        nargs="+",
        required=True,
        help="Daftar model embedding yang ingin dibandingkan.",
    )
    parser.add_argument("--limit", type=int, default=50, help="Batas
jumlah file. 0 = semua.")
    parser.add_argument("--chunk-size", type=int, default=1200,
help="Ukuran chunk berbasis karakter.")
    parser.add_argument("--overlap", type=int, default=150,
help="Overlap chunk berbasis karakter.")
    parser.add_argument("--top-k", type=int, default=5, help="Top-K
retrieval.")
    parser.add_argument("--verbose", action="store_true",
help="Tampilkan detail kegagalan.")
    parser.add_argument("--save-json", default="", help="Simpan hasil
ke JSON.")
    return parser.parse_args()

```

```

def main() -> int:
    args = parse_args()

    input_dir = Path(args.input)
    query_path = Path(args.queries)

    if not input_dir.exists() or not input_dir.is_dir():
        print(f"[ERROR] Folder input tidak ditemukan: {input_dir}",
file=sys.stderr)
        return 1

    if not query_path.exists():
        print(f"[ERROR] File query tidak ditemukan: {query_path}",
file=sys.stderr)
        return 1

    docs = load_documents(input_dir, args.limit)
    if not docs:
        print("[ERROR] Tidak ada dokumen valid.", file=sys.stderr)
        return 1

    chunks = build_chunks(docs, chunk_size=args.chunk_size,
overlap=args.overlap)
    if not chunks:
        print("[ERROR] Tidak ada chunk yang berhasil dibuat.",
file=sys.stderr)
        return 1

    query_data = load_yaml_file(query_path)
    queries = query_data.get("queries", [])
    if not isinstance(queries, list) or not queries:
        print("[ERROR] File YAML tidak berisi daftar 'queries' yang
valid.", file=sys.stderr)
        return 1

    print(f"[INFO] Documents loaded: {len(docs)}")
    print(f"[INFO] Chunks created: {len(chunks)}")
    print(f"[INFO] Models to compare: {len(args.models)}")

    reports: List[Dict[str, Any]] = []
    for model_name in args.models:
        report = evaluate_model(
            model_name=model_name,
            chunks=chunks,
            queries=queries,
            top_k=args.top_k,
        )
        reports.append(report)
        print_model_summary(report, top_k=args.top_k)

    print_ranking_table(reports, top_k=args.top_k)

```

```

if args.verbose:
    print_verbose_failures(reports, top_k=args.top_k)

if args.save_json:
    out_path = Path(args.save_json)
    with out_path.open("w", encoding="utf-8") as f:
        json.dump(reports, f, indent=2, ensure_ascii=False)
    print(f"[INFO] JSON report saved: {out_path}")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Dependensi

Script ini perlu:

```
pip install sentence-transformers pyyaml
```

Kalau belum ada **torch**, biasanya akan ikut ditarik sebagai dependensi **sentence-transformers**.

Contoh menjalankan

Uji dua model

```

python test_embedding_retrieval.py \
    --input ./data/cleaned \
    --queries ./tests/queries.yaml \
    --models \
        sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 \
    \
        intfloat/multilingual-e5-base \
    --limit 50 \
    --top-k 5

```

Uji tiga model dan simpan laporan JSON

```

python test_embedding_retrieval.py \
    --input ./data/cleaned \
    --queries ./tests/queries.yaml \
    --models \
        sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 \
    \
        intfloat/multilingual-e5-base \
        BAAI/bge-m3 \
    --limit 50 \

```

```
--top-k 5 \
--verbose \
--save-json ./reports/embedding-comparison.json
```

Contoh output

```
[INFO] Documents loaded: 12
[INFO] Chunks created: 38
[INFO] Models to compare: 3

[INFO] Loading embedding model:
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

=== MODEL: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
===
[INFO] Queries tested: 3
[INFO] Top-1 hits: 2
[INFO] Top-5 hits: 3
[INFO] Top-1 hit rate: 66.67%
[INFO] Top-5 hit rate: 100.00%
[INFO] Semua query berhasil masuk Top-5.

[INFO] Loading embedding model: intfloat/multilingual-e5-base

=== MODEL: intfloat/multilingual-e5-base ===
[INFO] Queries tested: 3
[INFO] Top-1 hits: 3
[INFO] Top-5 hits: 3
[INFO] Top-1 hit rate: 100.00%
[INFO] Top-5 hit rate: 100.00%
[INFO] Semua query berhasil masuk Top-5.

=====
PERBANDINGAN MODEL EMBEDDING
=====
No    Model
Top-1    Top-5
-----
1      intfloat/multilingual-e5-base
100.00%    100.00%
2      sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
66.67%    100.00%
-----
[INFO] Model paling menjanjikan saat ini:
intfloat/multilingual-e5-base
```

Cara membaca hasilnya

Yang perlu diperhatikan:

- **Top-1 hit rate**
Seberapa sering hasil pertama langsung benar. Ini penting untuk pengalaman pengguna yang ingin jawaban cepat dan tepat.
- **Top-K hit rate**
Seberapa sering dokumen yang benar masih muncul di daftar atas. Ini penting kalau nanti Anda menambahkan *reranker*.
- **Detail kegagalan**
Sangat berguna untuk melihat apakah model gagal karena:
 - lemah di Bahasa Indonesia
 - lemah di istilah teknis
 - lemah di *code-aware retrieval*
 - kalah oleh dokumen yang terlalu umum

Urutan kerja yang sehat

Untuk alur buku ini, saya sarankan urutannya seperti ini:

- tulis dan rapikan dokumen manual
- jalankan `ingest_pilot.py`
- jalankan `eval_retrieval.py`
- jalankan `test_embedding_retrieval.py`
- baru setelah itu putuskan:
 - model *embedding* final
 - *vector database* yang dipakai
 - perlu *reranker* atau belum
 - perlu *hybrid retrieval* atau belum

Dengan urutan itu, pembaca tidak menebak-nebak. Mereka **menguji sebelum memutuskan**.

Lampiran J. compare_chunking.py

Berikut lanjutan yang paling pas: **source code** `compare_chunking.py` untuk membandingkan beberapa strategi *chunking* pada corpus dan query uji yang sama.

Ini penting karena seringkali masalah retrieval **bukan** ada di model *embedding*, melainkan di cara dokumen dipotong. *Chunk* yang terlalu besar membuat konteks kabur. *Chunk* yang terlalu kecil membuat makna pecah. *Overlap* yang salah membuat informasi penting hilang di batas potongan. Jadi script ini membantu pembaca melihat dengan lebih jujur:

- ukuran *chunk* berapa yang paling masuk akal;
- apakah *overlap* membantu atau justru menambah noise;
- strategi sederhana mana yang cukup bagus untuk pilot awal.

Script ini membandingkan beberapa konfigurasi:

- `chunk_size`
- `overlap`
- mode pemotongan sederhana berbasis heading/paragraf

Lalu menghitung:

- **Top-1 hit rate**
- **Top-K hit rate**
- jumlah *chunk* yang dihasilkan

Contoh file query uji

Tetap bisa memakai `./tests/queries.yaml` yang sama:

```
queries:
  - id: q01
    question: "Kapan saya harus melakukan reindex knowledge
base?"
    expected_titles_contains:
      - "Reindex"

  - id: q02
    question: "Apa fungsi Focused Retrieval di Open WebUI?"
    expected_titles_contains:
      - "Focused Retrieval"

  - id: q03
    question: "Bagaimana cara menarik model di Ollama?"
    expected_titles_contains:
      - "Menarik Model"
      - "Cara Menarik Model"
```


Source code compare_chunking.py

```
#!/usr/bin/env python3
"""
compare_chunking.py

Membandingkan beberapa konfigurasi chunking pada corpus dan query uji
yang sama.

Fitur:
- baca file .md, .txt, .json
- ambil YAML front matter jika ada
- buat chunk dengan beberapa konfigurasi
- embed dokumen dan query
- retrieval cosine similarity di memori
- hitung Top-1 dan Top-K hit rate
- bandingkan hasil antar konfigurasi chunking

Contoh:
    python compare_chunking.py \
        --input ./data/cleaned \
        --queries ./tests/queries.yaml \
        --embedding-model
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 \
        --chunk-configs 800:100 1200:150 1600:200 \
        --top-k 5
"""

from __future__ import annotations

import argparse
import dataclasses
import json
import re
import sys
from pathlib import Path
from typing import Any, Dict, List

# =====
# Data structures
# =====

@dataclasses.dataclass
class Document:
    source_path: str
    text: str
    metadata: Dict[str, Any]

@dataclasses.dataclass
class Chunk:
    chunk_id: str
```

```

    parent_doc_id: str
    text: str
    metadata: Dict[str, Any]

# =====
# File parsing
# =====

YAML_FRONT_MATTER_RE = re.compile(r"^---\n(.*?)\n---\n", re.DOTALL)
HEADER_RE = re.compile(r"^\s{0,3}#{1,6}\s+", re.MULTILINE)

def sha1_text(text: str) -> str:
    import hashlib
    return hashlib.sha1(text.encode("utf-8")).hexdigest()

def load_text_file(path: Path) -> str:
    return path.read_text(encoding="utf-8", errors="ignore")

def parse_simple_yaml(yaml_text: str) -> Dict[str, Any]:
    result: Dict[str, Any] = {}
    for raw_line in yaml_text.splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#"):
            continue
        if ":" not in line:
            continue
        key, value = line.split(":", 1)
        key = key.strip()
        value = value.strip()

        if value.startswith("[") and value.endswith("]"):
            items = [x.strip().strip('"').strip("'") for x in
value[1:-1].split(",") if x.strip()]
            result[key] = items
        else:
            result[key] = value.strip('"').strip("'")
    return result

def parse_markdown_with_front_matter(path: Path) -> Document:
    raw = load_text_file(path)
    metadata: Dict[str, Any] = {}

    match = YAML_FRONT_MATTER_RE.match(raw)
    if match:
        metadata = parse_simple_yaml(match.group(1))
        raw = raw[match.end():]

    metadata.setdefault("title", path.stem)

```

```

        metadata.setdefault("source", "unknown")
        metadata.setdefault("date", "")
        metadata.setdefault("domain", "unknown")
        metadata.setdefault("language", "unknown")
        metadata.setdefault("sensitivity", "unknown")
        metadata.setdefault("quality", "unknown")
        metadata.setdefault("topic_labels", [])
        metadata["file_type"] = "markdown"
        metadata["source_path"] = str(path)

    return Document(source_path=str(path), text=raw.strip(),
                    metadata=metadata)

def parse_txt(path: Path) -> Document:
    text = load_text_file(path)
    metadata = {
        "title": path.stem,
        "source": "unknown",
        "date": "",
        "domain": "unknown",
        "language": "unknown",
        "sensitivity": "unknown",
        "quality": "unknown",
        "topic_labels": [],
        "file_type": "text",
        "source_path": str(path),
    }
    return Document(source_path=str(path), text=text.strip(),
                    metadata=metadata)

def parse_json(path: Path) -> Document:
    raw = load_text_file(path)
    obj = json.loads(raw)

    if isinstance(obj, dict):
        text = obj.get("text", "")
        metadata = obj.get("metadata", {})
        if not isinstance(metadata, dict):
            metadata = {}
    else:
        text = raw
        metadata = {}

    metadata.setdefault("title", path.stem)
    metadata.setdefault("source", "unknown")
    metadata.setdefault("date", "")
    metadata.setdefault("domain", "unknown")
    metadata.setdefault("language", "unknown")
    metadata.setdefault("sensitivity", "unknown")
    metadata.setdefault("quality", "unknown")
    metadata.setdefault("topic_labels", [])

```

```

        metadata["file_type"] = "json"
        metadata["source_path"] = str(path)

    return Document(source_path=str(path), text=str(text).strip(),
                    metadata=metadata)

def load_documents(input_dir: Path, limit: int) -> List[Document]:
    supported = {".md", ".txt", ".json"}
    files = sorted([p for p in input_dir.rglob("*") if p.is_file() and
                    p.suffix.lower() in supported])

    if limit > 0:
        files = files[:limit]

    docs: List[Document] = []
    for path in files:
        try:
            if path.suffix.lower() == ".md":
                doc = parse_markdown_with_front_matter(path)
            elif path.suffix.lower() == ".txt":
                doc = parse_txt(path)
            elif path.suffix.lower() == ".json":
                doc = parse_json(path)
            else:
                continue

            if doc.text.strip():
                docs.append(doc)
        except Exception as exc:
            print(f"[WARN] Gagal baca {path}: {exc}", file=sys.stderr)
    return docs

# =====
# Chunking
# =====

def normalize_text(text: str) -> str:
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = re.sub(r"\n{3,}", "\n\n", text)
    text = re.sub(r"[ \t]{2,}", " ", text)
    return text.strip()

def split_by_headers(text: str) -> List[str]:
    text = normalize_text(text)
    if not HEADER_RE.search(text):
        return [x.strip() for x in text.split("\n\n") if x.strip()]

    parts: List[str] = []
    current: List[str] = []
    for line in text.splitlines():

```

```

        if HEADER_RE.match(line) and current:
            parts.append("\n".join(current).strip())
            current = [line]
        else:
            current.append(line)
    if current:
        parts.append("\n".join(current).strip())
    return [p for p in parts if p]

def split_by_paragraphs(text: str) -> List[str]:
    text = normalize_text(text)
    return [x.strip() for x in text.split("\n\n") if x.strip()]

def chunk_from_units(units: List[str], target_chars: int,
overlap_chars: int) -> List[str]:
    chunks: List[str] = []
    buffer = ""

    for unit in units:
        unit = unit.strip()
        if not unit:
            continue

        if len(unit) > target_chars * 1.5:
            subparts = [p.strip() for p in unit.split("\n\n") if
p.strip()]
            for sub in subparts:
                if not buffer:
                    buffer = sub
                elif len(buffer) + 2 + len(sub) <= target_chars:
                    buffer += "\n\n" + sub
                else:
                    chunks.append(buffer.strip())
                    tail = buffer[-overlap_chars:] if overlap_chars >
0 else ""
                    buffer = (tail + "\n\n" + sub).strip()
            else:
                if not buffer:
                    buffer = unit
                elif len(buffer) + 2 + len(unit) <= target_chars:
                    buffer += "\n\n" + unit
                else:
                    chunks.append(buffer.strip())
                    tail = buffer[-overlap_chars:] if overlap_chars > 0
else ""
                    buffer = (tail + "\n\n" + unit).strip()

        if buffer.strip():
            chunks.append(buffer.strip())

    return [c for c in chunks if len(c.strip()) >= 80]

```

```

def chunk_text(text: str, mode: str, target_chars: int, overlap_chars:
int) -> List[str]:
    if mode == "heading":
        units = split_by_headers(text)
    elif mode == "paragraph":
        units = split_by_paragraphs(text)
    else:
        raise ValueError(f"Mode chunking tidak dikenal: {mode}")

    return chunk_from_units(units, target_chars=target_chars,
overlap_chars=overlap_chars)

def build_chunks(docs: List[Document], chunk_size: int, overlap: int,
mode: str) -> List[Chunk]:
    out: List[Chunk] = []
    for doc in docs:
        parent_doc_id = sha1_text(doc.source_path)[:16]
        doc_chunks = chunk_text(doc.text, mode=mode,
target_chars=chunk_size, overlap_chars=overlap)

        for idx, ch in enumerate(doc_chunks, start=1):
            chunk_id = f"{parent_doc_id}-{idx:04d}"
            meta = dict(doc.metadata)
            meta["chunk_index"] = idx
            meta["parent_doc_id"] = parent_doc_id
            meta["chunk_id"] = chunk_id
            meta["char_len"] = len(ch)
            meta["chunk_mode"] = mode
            meta["chunk_size"] = chunk_size
            meta["overlap"] = overlap

            out.append(
                Chunk(
                    chunk_id=chunk_id,
                    parent_doc_id=parent_doc_id,
                    text=ch,
                    metadata=meta,
                )
            )
    return out

# =====
# Query loading
# =====

def load_yaml_file(path: Path) -> Dict[str, Any]:
    try:
        import yaml
    except ImportError as exc:

```

```

        raise RuntimeError("pyyaml belum terpasang. Install dengan:
pip install pyyaml") from exc

    with path.open("r", encoding="utf-8") as f:
        data = yaml.safe_load(f)

    if not isinstance(data, dict):
        raise ValueError("Format YAML tidak valid.")
    return data

# =====
# Embedding and retrieval
# =====

def load_embedder(model_name: str):
    try:
        from sentence_transformers import SentenceTransformer
    except ImportError as exc:
        raise RuntimeError(
            "sentence-transformers belum terpasang. Install dengan:
pip install sentence-transformers"
        ) from exc

    return SentenceTransformer(model_name)

def embed_texts(embedder, texts: List[str], batch_size: int = 32):
    vectors = embedder.encode(
        texts,
        batch_size=batch_size,
        show_progress_bar=True,
        convert_to_numpy=True,
        normalize_embeddings=True,
    )
    return vectors

def cosine_similarity_dense(query_vec, doc_matrix) -> List[float]:
    scores = doc_matrix @ query_vec
    return scores.tolist()

def top_k_indices(scores: List[float], k: int) -> List[int]:
    return sorted(range(len(scores)), key=lambda i: scores[i],
reverse=True)[:k]

# =====
# Matching logic
# =====

def normalize_str(value: Any) -> str:

```

```

        return str(value or "").strip().lower()

def match_expected(result: Dict[str, Any], query_spec: Dict[str, Any])
-> bool:
    title = normalize_str(result.get("title", ""))
    source = normalize_str(result.get("source", ""))

    expected_sources = [normalize_str(x) for x in
query_spec.get("expected_sources", [])]
    expected_titles_contains = [normalize_str(x) for x in
query_spec.get("expected_titles_contains", [])]

    source_match = False
    title_match = False

    if expected_sources:
        source_match = source in expected_sources

    if expected_titles_contains:
        title_match = any(fragment in title for fragment in
expected_titles_contains)

    if expected_sources and expected_titles_contains:
        return source_match or title_match
    if expected_sources:
        return source_match
    if expected_titles_contains:
        return title_match

    return False

# =====
# Evaluation
# =====

def evaluate_chunk_config(
    chunks: List[Chunk],
    queries: List[Dict[str, Any]],
    embedder,
    top_k: int,
    label: str,
) -> Dict[str, Any]:
    doc_texts = [c.text for c in chunks]
    doc_vectors = embed_texts(embedder, doc_texts)

    total = 0
    top1_hits = 0
    topk_hits = 0
    failures: List[Dict[str, Any]] = []

    for item in queries:

```



```

qid = item.get("id", f"q{total+1:03d}")
question = str(item.get("question", "")).strip()
if not question:
    continue

total += 1
query_vec = embed_texts(embedder, [question])[0]
scores = cosine_similarity_dense(query_vec, doc_vectors)
indices = top_k_indices(scores, top_k)

results: List[Dict[str, Any]] = []
hit_positions: List[int] = []

for rank, idx in enumerate(indices, start=1):
    ch = chunks[idx]
    result = {
        "id": ch.chunk_id,
        "title": ch.metadata.get("title", ""),
        "source": ch.metadata.get("source", ""),
        "domain": ch.metadata.get("domain", ""),
        "source_path": ch.metadata.get("source_path", ""),
        "chunk_index": ch.metadata.get("chunk_index", 0),
        "score": float(scores[idx]),
        "text": ch.text,
    }
    results.append(result)
    if match_expected(result, item):
        hit_positions.append(rank)

top1 = len(hit_positions) > 0 and hit_positions[0] == 1
topk = len(hit_positions) > 0

if top1:
    top1_hits += 1
if topk:
    topk_hits += 1
else:
    failures.append(
        {
            "id": qid,
            "question": question,
            "results": results,
        }
    )

return {
    "config": label,
    "chunks_created": len(chunks),
    "total": total,
    "top1_hits": top1_hits,
    "topk_hits": topk_hits,
    "top1_rate": (top1_hits / total * 100.0) if total else 0.0,
    "topk_rate": (topk_hits / total * 100.0) if total else 0.0,

```

```

        "failures": failures,
    }

def print_config_summary(report: Dict[str, Any], top_k: int) -> None:
    print(f"\n=== CHUNK CONFIG: {report['config']} ===")
    print(f"[INFO] Chunks created: {report['chunks_created']}")
    print(f"[INFO] Queries tested: {report['total']}")
    print(f"[INFO] Top-1 hits: {report['top1_hits']}")
    print(f"[INFO] Top-{top_k} hits: {report['topk_hits']}")
    print(f"[INFO] Top-1 hit rate: {report['top1_rate']:.2f}%")
    print(f"[INFO] Top-{top_k} hit rate: {report['topk_rate']:.2f}%")

    if report["failures"]:
        print(f"[WARN] Query gagal masuk Top-{top_k}: {len(report['failures'])}")
        for failure in report["failures"][:5]:
            print(f"    - {failure['id']}: {failure['question']}")
    else:
        print(f"[INFO] Semua query berhasil masuk Top-{top_k}.")

def print_ranking_table(reports: List[Dict[str, Any]], top_k: int) -> None:
    ranked = sorted(
        reports,
        key=lambda x: (x["topk_rate"], x["top1_rate"],
        -x["chunks_created"]),
        reverse=True,
    )

    print("\n" + "=" * 96)
    print("PERBANDINGAN KONFIGURASI CHUNKING")
    print("=" * 96)
    print(f"{'No':<4} {'Config':<32} {'Chunks':>10} {'Top-1':>10} {'Top-'+str(top_k):>10}")
    print(f"{'-' * 96}")

    for idx, rep in enumerate(ranked, start=1):
        print(
            f"{'idx':<4} {rep['config']:<32} {rep['chunks_created']:>10} "
            f"{'rep['top1_rate']:>9.2f}% {rep['topk_rate']:>9.2f}%"
        )

    print(f"{'-' * 96}")
    if ranked:
        print(f"[INFO] Konfigurasi paling menjanjikan saat ini: {ranked[0]['config']}")

def print_verbose_failures(reports: List[Dict[str, Any]], top_k: int) -> None:

```

```

print("\n" + "=" * 96)
print("DETAIL KEGAGALAN PER KONFIGURASI CHUNKING")
print("=" * 96)

for report in reports:
    if not report["failures"]:
        continue

    print(f"\n### CONFIG: {report['config']}")
    for failure in report["failures"][:3]:
        print(f"\n[FAIL] {failure['id']}: {failure['question']}")
        for idx, result in enumerate(failure["results"], start=1):
            preview = result["text"].replace("\n", " ")
            preview = preview[:120] + ("..." if len(preview) > 120
else "")
            print(
                f"    {idx}. title={result['title']!r} |
source={result['source']!r} "
                f"| score={result['score']:.4f}"
            )
            print(f"        source_path={result['source_path']}")
            print(f"        preview={preview}")

# =====
# Main
# =====

def parse_chunk_configs(values: List[str], default_mode: str) ->
List[Dict[str, Any]]:
    configs: List[Dict[str, Any]] = []
    for value in values:
        # format:
        # 1200:150
        # heading:1200:150
        parts = value.split(":")
        if len(parts) == 2:
            mode = default_mode
            chunk_size = int(parts[0])
            overlap = int(parts[1])
        elif len(parts) == 3:
            mode = parts[0].strip()
            chunk_size = int(parts[1])
            overlap = int(parts[2])
        else:
            raise ValueError(
                f"Format chunk-config salah: {value}. "
                f"Gunakan '1200:150' atau 'heading:1200:150'"
            )

        configs.append(
            {
                "mode": mode,

```

```

        "chunk_size": chunk_size,
        "overlap": overlap,
        "label": f"{mode}:{chunk_size}:{overlap}",
    }
)
return configs

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Bandingkan beberapa konfigurasi chunking.")
    parser.add_argument("--input", required=True, help="Folder dokumen cleaned.")
    parser.add_argument("--queries", required=True, help="File YAML query uji.")
    parser.add_argument(
        "--embedding-model",

default="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2",
        help="Model embedding untuk evaluasi.",
    )
    parser.add_argument(
        "--chunk-configs",
        nargs="+",
        required=True,
        help="Daftar konfigurasi chunking. Contoh: 800:100 1200:150 heading:1600:200",
    )
    parser.add_argument(
        "--default-mode",
        choices=["heading", "paragraph"],
        default="heading",
        help="Mode default jika config hanya berisi ukuran:overlap.",
    )
    parser.add_argument("--limit", type=int, default=50, help="Batas jumlah file. 0 = semua.")
    parser.add_argument("--top-k", type=int, default=5, help="Top-K retrieval.")
    parser.add_argument("--verbose", action="store_true", help="Tampilkan detail kegagalan.")
    parser.add_argument("--save-json", default="", help="Simpan hasil ke JSON.")
    return parser.parse_args()

def main() -> int:
    args = parse_args()

    input_dir = Path(args.input)
    query_path = Path(args.queries)

    if not input_dir.exists() or not input_dir.is_dir():

```

```

        print(f"[ERROR] Folder input tidak ditemukan: {input_dir}",
file=sys.stderr)
        return 1

    if not query_path.exists():
        print(f"[ERROR] File query tidak ditemukan: {query_path}",
file=sys.stderr)
        return 1

    docs = load_documents(input_dir, args.limit)
    if not docs:
        print("[ERROR] Tidak ada dokumen valid.", file=sys.stderr)
        return 1

    query_data = load_yaml_file(query_path)
    queries = query_data.get("queries", [])
    if not isinstance(queries, list) or not queries:
        print("[ERROR] File YAML tidak berisi daftar 'queries' yang
valid.", file=sys.stderr)
        return 1

    configs = parse_chunk_configs(args.chunk_configs,
default_mode=args.default_mode)

    print(f"[INFO] Documents loaded: {len(docs)}")
    print(f"[INFO] Chunk configs to compare: {len(configs)}")
    print(f"[INFO] Loading embedding model: {args.embedding_model}")
    embedder = load_embedder(args.embedding_model)

    reports: List[Dict[str, Any]] = []
    for cfg in configs:
        chunks = build_chunks(
            docs,
            chunk_size=cfg["chunk_size"],
            overlap=cfg["overlap"],
            mode=cfg["mode"],
        )
        report = evaluate_chunk_config(
            chunks=chunks,
            queries=queries,
            embedder=embedder,
            top_k=args.top_k,
            label=cfg["label"],
        )
        reports.append(report)
        print_config_summary(report, top_k=args.top_k)

    print_ranking_table(reports, top_k=args.top_k)

    if args.verbose:
        print_verbose_failures(reports, top_k=args.top_k)

    if args.save_json:

```

```

    out_path = Path(args.save_json)
    with out_path.open("w", encoding="utf-8") as f:
        json.dump(reports, f, indent=2, ensure_ascii=False)
    print(f"[INFO] JSON report saved: {out_path}")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Dependensi

Script ini perlu:

```
pip install sentence-transformers pyyaml
```

Contoh menjalankan

Bandingkan tiga ukuran chunk dengan mode default heading

```

python compare_chunking.py \
    --input ./data/cleaned \
    --queries ./tests/queries.yaml \
    --embedding-model
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 \
    --chunk-configs 800:100 1200:150 1600:200 \
    --top-k 5

```

Bandingkan mode heading dan paragraph

```

python compare_chunking.py \
    --input ./data/cleaned \
    --queries ./tests/queries.yaml \
    --embedding-model
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 \
    --chunk-configs heading:800:100 heading:1200:150
paragraph:800:100 paragraph:1200:150 \
    --top-k 5 \
    --verbose

```

Simpan hasil JSON

```
python compare_chunking.py \  
  --input ./data/cleaned \  
  --queries ./tests/queries.yaml \  
  --embedding-model  
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 \  
  --chunk-configs 800:100 1200:150 1600:200 \  
  --save-json ./reports/chunking-comparison.json
```

Contoh output

```
[INFO] Documents loaded: 12  
[INFO] Chunk configs to compare: 3  
[INFO] Loading embedding model:  
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

```
=== CHUNK CONFIG: heading:800:100 ===  
[INFO] Chunks created: 51  
[INFO] Queries tested: 3  
[INFO] Top-1 hits: 2  
[INFO] Top-5 hits: 3  
[INFO] Top-1 hit rate: 66.67%  
[INFO] Top-5 hit rate: 100.00%  
[INFO] Semua query berhasil masuk Top-5.
```

```
=== CHUNK CONFIG: heading:1200:150 ===  
[INFO] Chunks created: 38  
[INFO] Queries tested: 3  
[INFO] Top-1 hits: 3  
[INFO] Top-5 hits: 3  
[INFO] Top-1 hit rate: 100.00%  
[INFO] Top-5 hit rate: 100.00%  
[INFO] Semua query berhasil masuk Top-5.
```

```
=== CHUNK CONFIG: heading:1600:200 ===  
[INFO] Chunks created: 27  
[INFO] Queries tested: 3  
[INFO] Top-1 hits: 2  
[INFO] Top-5 hits: 3  
[INFO] Top-1 hit rate: 66.67%  
[INFO] Top-5 hit rate: 100.00%  
[INFO] Semua query berhasil masuk Top-5.
```

PERBANDINGAN KONFIGURASI CHUNKING				
No	Config	Chunks	Top-1	Top-5
1	heading:1200:150 100.00%	38	100.00%	
2	heading:800:100 100.00%	51	66.67%	
3	heading:1600:200 100.00%	27	66.67%	
[INFO] Konfigurasi paling menjanjikan saat ini: heading:1200:150				

Cara membaca hasilnya

Ada tiga hal penting yang perlu diperhatikan:

- **Top-1 hit rate**
Ini menunjukkan seberapa sering potongan paling atas langsung benar. Untuk pengalaman pengguna, ini sangat penting.
- **Top-K hit rate**
Ini menunjukkan apakah dokumen yang benar masih muncul di daftar atas. Ini penting kalau nanti Anda ingin menambah *reranker*.
- **Jumlah chunk yang dihasilkan**
Konfigurasi yang terlalu kecil biasanya menghasilkan terlalu banyak *chunk*. Itu bisa membantu recall, tetapi juga menambah noise. Konfigurasi yang terlalu besar bisa membuat satu *chunk* mengandung terlalu banyak topik.

Jadi, **konfigurasi terbaik bukan selalu yang menghasilkan paling banyak chunk**, tetapi yang memberi retrieval paling stabil dengan biaya yang masih masuk akal.

Aturan praktis yang bisa diajarkan di buku

Dari script ini, pembaca bisa belajar satu prinsip penting:

- **chunk terlalu kecil**
sering menangkap detail, tetapi konteks mudah pecah
- **chunk terlalu besar**
konteks utuh, tetapi retrieval jadi kabur
- **overlap terlalu kecil**
informasi di batas bisa hilang
- **overlap terlalu besar**
duplikasi naik, noise naik, indeks membengkak

Karena itu, keputusan *chunking* harus diuji, bukan ditebak.

Workflow sehat setelah script ini

Urutan yang makin matang sekarang menjadi:

- `ingest_pilot.py`
- `eval_retrieval.py`
- `test_embedding_retrieval.py`
- `compare_chunking.py`

Setelah itu baru lebih masuk akal membahas:

- *metadata filtering*
- *hybrid retrieval*
- *reranker*
- *reindex*
- skenario Open WebUI RAG yang lebih serius

Glossary

Istilah	Definisi
API Docs	Dokumen yang menjelaskan cara memakai <i>Application Programming Interface</i> , termasuk endpoint, parameter, dan contoh respons.
Authority	Tingkat otoritas atau kewibawaan sumber dokumen. Dokumen resmi biasanya memiliki <i>authority</i> lebih tinggi daripada catatan pribadi.
BM25	Metode pencarian berbasis kata kunci yang sering dipakai dalam <i>information retrieval</i> untuk menilai relevansi dokumen terhadap query.
Chunk	Potongan teks yang lebih kecil dari dokumen asli dan dipakai sebagai unit dasar untuk <i>embedding</i> dan retrieval.
Chunking	Proses memecah dokumen menjadi potongan-potongan kecil agar lebih mudah dicari dan dipakai dalam RAG.
Chroma	<i>Open-source vector database</i> yang cocok untuk prototyping cepat dan eksperimen RAG.
Clean Document	Dokumen yang sudah dibersihkan dari gangguan seperti nomor halaman, <i>header/footer</i> , paragraf pecah, atau duplikasi.
Confidence	Tingkat keyakinan terhadap kebenaran isi dokumen atau label yang diberikan.
Context Enrichment	Teknik menambahkan konteks sekitar <i>chunk</i> yang ditemukan agar jawaban model tidak terlalu sempit atau kehilangan makna.
Corpus	Kumpulan dokumen yang sengaja disusun, dibersihkan, dilabeli, dan disiapkan untuk kebutuhan retrieval atau pelatihan model.
Cross-Encoder Reranker	Model yang menilai ulang pasangan query-dokumen untuk mengurutkan hasil retrieval agar lebih relevan.
Dense Retrieval	Pendekatan retrieval berbasis <i>dense embedding</i> yang mencari kemiripan makna, bukan hanya kecocokan kata.
DoRA	Varian PEFT yang memisahkan pembaruan arah dan magnitudo bobot model untuk adaptasi yang lebih halus.
Embedding	Representasi numerik dari teks yang memungkinkan sistem menghitung kedekatan makna antar-potongan dokumen.
Embedding Model	Model yang mengubah teks menjadi vektor agar bisa dipakai dalam pencarian semantik.

FAQ	<i>Frequently Asked Questions</i> , yaitu kumpulan pertanyaan dan jawaban yang sering muncul.
Fine-Tuning (FT)	Proses melatih ulang model dengan dataset tertentu agar perilaku atau kemampuannya lebih sesuai dengan domain yang diinginkan.
Focused Retrieval	Mode RAG di Open WebUI yang mengambil <i>chunk</i> paling relevan dari dokumen, bukan menyuntikkan seluruh isi dokumen.
Freshness	Tingkat kebaruan dokumen. Dokumen yang baru biasanya lebih relevan untuk pengetahuan yang cepat berubah.
Full Context	Mode yang memasukkan seluruh isi dokumen ke konteks model, cocok untuk dokumen pendek yang perlu dibaca utuh.
Groundedness	Tingkat keterikatan jawaban model pada bukti atau konteks yang benar-benar tersedia di dokumen sumber.
Hierarchical Chunking	Strategi <i>chunking</i> bertingkat, misalnya bab → subbab → paragraf, untuk menjaga konteks besar dan detail kecil sekaligus.
Hybrid Retrieval	Pendekatan retrieval yang menggabungkan pencarian semantik (<i>dense</i>) dan pencarian kata kunci (<i>sparse</i> atau BM25).
Index	Struktur data yang membantu pencarian dokumen atau vektor menjadi lebih cepat dan terarah.
Ingest	Proses memasukkan dokumen ke pipeline RAG, biasanya mencakup pembacaan, <i>chunking</i> , <i>embedding</i> , dan penyimpanan ke vector store.
JSON	Format data terstruktur yang sering dipakai untuk menyimpan dokumen, metadata, log, atau hasil anotasi.
Knowledge Base	Kumpulan dokumen atau pengetahuan yang sudah disiapkan agar bisa dicari dan dipakai oleh sistem AI.
Label	Penanda klasifikasi yang dipakai untuk membantu retrieval, filtering, atau analisis, misalnya ATTACK , SUSPICIOUS , atau approved .
LanceDB	<i>Open-source vector database</i> yang kuat untuk penyimpanan lokal tertanam serta mendukung <i>hybrid search</i> dan <i>metadata filtering</i> .
LoRA	<i>Low-Rank Adaptation</i> , metode PEFT yang melatih parameter tambahan kecil tanpa melatih ulang seluruh model.
Markdown	Format teks ringan yang mudah dibaca manusia dan mudah diproses mesin, cocok untuk dokumen corpus manual.

Metadata	Informasi terstruktur tentang dokumen, seperti judul, sumber, tanggal, domain, dan sensitivitas.
MRR	<i>Mean Reciprocal Rank</i> , metrik yang mengukur seberapa cepat dokumen relevan pertama muncul dalam hasil retrieval.
Node	Unit dokumen yang lebih kecil dalam beberapa framework RAG, biasanya hasil pemecahan dokumen menjadi potongan bermakna.
Open WebUI	Antarmuka <i>open-source</i> untuk mengakses model lokal, mengelola <i>knowledge base</i> , dan menjalankan workflow RAG.
Ollama	Runtime model lokal yang memudahkan menjalankan LLM di mesin sendiri.
Parent Document Retrieval	Teknik retrieval yang mencari <i>chunk</i> kecil terlebih dahulu lalu mengambil dokumen atau <i>chunk</i> induk agar konteks lebih utuh.
PDF Scan	PDF yang pada dasarnya berupa gambar hasil pemindaian, bukan teks digital yang mudah diekstrak.
PDF Teks	PDF yang berisi lapisan teks digital sehingga lebih mudah diproses menjadi Markdown, TXT, atau JSON.
PEFT	<i>Parameter-Efficient Fine-Tuning</i> , keluarga metode fine-tuning hemat parameter seperti LoRA, QLoRA, dan DoRA.
Priority	Tingkat prioritas dokumen dalam corpus, misalnya rendah, sedang, tinggi, atau kritis.
PyMuPDF4LLM	Library yang membantu mengekstrak PDF ke format yang lebih ramah untuk LLM dan RAG seperti Markdown, JSON, dan TXT.
QLoRA	Versi LoRA yang memakai kuantisasi 4-bit agar fine-tuning model besar lebih hemat memori.
Query	Pertanyaan atau permintaan pencarian yang diajukan pengguna ke sistem retrieval atau RAG.
Raw Document	Dokumen mentah yang belum dibersihkan, belum dilabeli, dan belum tentu siap untuk RAG.
Recall@k	Metrik yang mengukur apakah dokumen relevan berhasil muncul dalam hasil teratas sebanyak <i>k</i> .
Recursive Chunking	Strategi memecah dokumen secara bertahap, biasanya mengikuti heading, paragraf, lalu ukuran tertentu bila masih terlalu besar.
Reindex	Proses membangun ulang indeks atau representasi embedding, biasanya diperlukan saat model embedding berubah.

Reranker	Model atau tahap tambahan yang menyusun ulang hasil retrieval agar dokumen paling relevan naik ke posisi atas.
Retrieval	Proses mencari dan mengambil potongan dokumen yang relevan terhadap query.
RAG	<i>Retrieval-Augmented Generation</i> , pendekatan yang menggabungkan pencarian dokumen dan generasi jawaban oleh model bahasa.
Semantic Chunking	Strategi memecah dokumen berdasarkan perubahan makna, bukan hanya berdasarkan panjang karakter.
Sensitivity	Tingkat kerahasiaan dokumen, misalnya public , internal , atau restricted .
Sentence Window	Teknik yang mengambil beberapa kalimat sebelum dan sesudah kalimat relevan agar konteks tetap utuh.
Sparse Retrieval	Pendekatan retrieval yang menekankan kecocokan istilah atau token penting, bukan hanya kemiripan semantik.
SOP	<i>Standard Operating Procedure</i> , dokumen prosedur baku untuk menjalankan pekerjaan tertentu.
Synthetic Data	Data buatan yang dihasilkan untuk membantu pelatihan, pengujian, atau memperkaya variasi dataset.
Tag	Penanda ringan berupa kata atau frasa pendek untuk membantu pengelompokan atau pencarian dokumen.
Top-1 Hit Rate	Persentase query yang langsung menemukan hasil benar di posisi pertama.
Top-k Hit Rate	Persentase query yang berhasil menemukan hasil benar dalam k hasil teratas.
Vector Database	Basis data khusus untuk menyimpan <i>embedding</i> dan melakukan pencarian berbasis kemiripan vektor.
Vector Store	Istilah umum untuk tempat penyimpanan <i>embedding</i> dan metadata yang dipakai dalam retrieval semantik.
Versioning	Praktik memberi versi pada dokumen, skema label, model <i>embedding</i> , atau indeks agar perubahan bisa ditelusuri.
YAML	Format teks terstruktur yang mudah dibaca manusia, sering dipakai untuk metadata dokumen atau konfigurasi.

Referensi

Anserini. (n.d.). *Evaluation concepts*. Retrieved April 13, 2026, from <https://www.mintlify.com/castorini/anserini/concepts/evaluation> (Mintlify)

Anserini. (n.d.). *Information retrieval fundamentals*. Retrieved April 13, 2026, from <https://mintlify.com/castorini/anserini/concepts/information-retrieval> (Mintlify)

Astral. (n.d.). *Installation | uv*. Retrieved April 13, 2026, from <https://docs.astral.sh/uv/getting-started/installation/> (Astral Docs)

Chroma. (n.d.). *Metadata filtering*. Retrieved April 13, 2026, from <https://docs.trychroma.com/docs/querying-collections/metadata-filtering> (Chroma Docs)

Confluent, Inc. (2025). *The developer's guide to RAG with data streaming: How to build retrieval-augmented generation (RAG) for real-time generative AI applications using the Confluent Data Streaming Platform*. Confluent.

Devlin, M. S. (2025). *Building LLM agents with RAG, knowledge graphs & reflection: A practical guide to building intelligent, context-aware, and self-improving AI agents*.

Docker Inc. (n.d.). *Install Docker Engine on Ubuntu*. Retrieved April 13, 2026, from <https://docs.docker.com/engine/install/ubuntu/> (Open WebUI)

Gruber, J. (n.d.). *Markdown*. Retrieved April 13, 2026, from <https://daringfireball.net/projects/markdown/> (Daring Fireball)

Gruber, J. (n.d.). *Markdown syntax documentation*. Retrieved April 13, 2026, from <https://daringfireball.net/projects/markdown/syntax> (Daring Fireball)

Hugging Face. (n.d.). *Bitsandbytes*. Retrieved April 13, 2026, from <https://huggingface.co/docs/transformers/en/quantization/bitsandbytes> (Hugging Face)

Hugging Face. (n.d.). *LoRA*. Retrieved April 13, 2026, from https://huggingface.co/docs/peft/en/package_reference/lora (Hugging Face)

Hugging Face. (n.d.). *PEFT*. Retrieved April 13, 2026, from <https://huggingface.co/docs/peft/en/index> (Hugging Face)

Keep a Changelog. (n.d.). *Keep a changelog*. Retrieved April 13, 2026, from <https://keepachangelog.com/en/1.1.0/> (Mintlify)

LanceDB. (n.d.). *Hybrid search*. Retrieved April 13, 2026, from <https://docs.lancedb.com/search/hybrid-search> ([LanceDB](#))

LanceDB. (n.d.). *Metadata filtering in LanceDB*. Retrieved April 13, 2026, from <https://docs.lancedb.com/search/filtering> ([LanceDB](#))

LanceDB. (n.d.). *Search*. Retrieved April 13, 2026, from <https://docs.lancedb.com/search> ([LanceDB](#))

LangChain. (n.d.). *Evaluate a RAG application*. Retrieved April 13, 2026, from <https://docs.langchain.com/langsmith/evaluate-rag-tutorial> ([docs.langchain.com](#))

LlamaIndex. (n.d.). *Node parser modules*. Retrieved April 13, 2026, from https://developers.llamaindex.ai/python/framework/module_guides/loading/node_parsers/modules/ ([LlamaIndex OSS Documentation](#))

LlamaIndex. (2024, January 11). *Semantic chunker*. Retrieved April 13, 2026, from https://developers.llamaindex.ai/python/examples/node_parsers/semantic_chunking/ ([LlamaIndex OSS Documentation](#))

Open WebUI. (n.d.). *Features*. Retrieved April 13, 2026, from <https://docs.openwebui.com/features/> ([Open WebUI](#))

Open WebUI. (n.d.). *Getting started with Open WebUI*. Retrieved April 13, 2026, from <https://docs.openwebui.com/getting-started/> ([Open WebUI](#))

Open WebUI. (n.d.). *Knowledge*. Retrieved April 13, 2026, from <https://docs.openwebui.com/features/workspace/knowledge/> ([Open WebUI](#))

OpenTelemetry. (n.d.). *Logs data model*. Retrieved April 13, 2026, from <https://opentelemetry.io/docs/specs/otel/logs/data-model/> ([OpenTelemetry](#))

Ollama. (n.d.). *FAQ*. Retrieved April 13, 2026, from <https://docs.ollama.com/faq> ([Ollama Docs](#))

Ollama. (n.d.). *Linux*. Retrieved April 13, 2026, from <https://docs.ollama.com/linux> ([Ollama Docs](#))

Python Software Foundation. (n.d.). *venv — Creation of virtual environments*. Retrieved April 13, 2026, from <https://docs.python.org/3/library/venv.html> ([Python documentation](#))

PyMuPDF Documentation Team. (n.d.). *PyMuPDF4LLM*. Retrieved April 13, 2026, from <https://pymupdf.readthedocs.io/en/latest/pymupdf4llm/> ([PyMuPDF](#))

PyMuPDF Documentation Team. (n.d.). *The PyMuPDF4LLM API*. Retrieved April 13, 2026, from <https://pymupdf.readthedocs.io/en/latest/pymupdf4llm/api.html> (PyMuPDF)

Qdrant. (n.d.). *Filtering*. Retrieved April 13, 2026, from <https://qdrant.tech/documentation/search/filtering/> (Qdrant)

Qdrant. (n.d.). *Hybrid and multi-stage queries*. Retrieved April 13, 2026, from <https://qdrant.tech/documentation/search/hybrid-queries/> (Qdrant)

Qdrant. (n.d.). *Qdrant documentation*. Retrieved April 13, 2026, from <https://qdrant.tech/documentation/> (Qdrant)

Raieli, S., & Iuculano, G. (2025). *Building AI agents with LLMs, RAG, and knowledge graphs: A practical guide to autonomous and modern AI agents*. Packt Publishing.

Rothman, D. (2024). *RAG-driven generative AI: Build custom retrieval augmented generation pipelines with LlamaIndex, Deep Lake, and Pinecone*. Packt Publishing.

Sentence Transformers. (n.d.). *SentenceTransformers documentation*. Retrieved April 13, 2026, from <https://sbert.net/> (Sbert)

Sentence Transformers. (n.d.). *SparseEncoder*. Retrieved April 13, 2026, from https://sbert.net/docs/package_reference/sparse_encoder/SparseEncoder.html (Sbert)

Ubuntu. (2026). *Ubuntu 26.04 LTS release notes*. Retrieved April 13, 2026, from <https://documentation.ubuntu.com/release-notes/26.04/> (Open WebUI)

YAML Language Development Team. (2021). *YAML Ain't Markup Language (YAML™) revision 1.2.2*. <https://yaml.org/spec/1.2.2/> (yaml.org)

Tentang Penulis



Onno W. Purbo, saat ini bertugas sebagai rektor di Institut Teknologi Tangerang Selatan (ITTS). Onno memperoleh gelar Ph.D bidang Electrical Engineering dari University of Waterloo, Canada, adalah seorang copyleft, educator dan ICT evangelist. Dia sudah mempublikasikan 50++ buku, termasuk free ICT ebook untuk sekolah tahun 2008. Beberapa buku terakhirnya adalah "Internet-TCP/IP: Konsep Dan Implementasi", 2018; "Sistem Operasi, Konsep Dan Membuat Linux OpenWRT Dan ROM Android", 2019; "IPv6 Untuk Mendukung Operasi Jaringan Dan Domain Name System", 2019; "Kubernetes untuk Pemula", 2024 dan "Membuat Operator Seluler 5G Sendiri", 2024.

Dia memimpin sambungan pertama Internet di Institut Teknologi Bandung, tahun 1993-2000, dan menggunakannya untuk membuat jaringan Internet pendidikan yang pertama di Indonesia. Dia membebaskan frekuensi WiFi, memperkenalkan RT/RW-net, antenna Wajanbolic dan jaringan selular OpenBTS dan private 5G sendiri. Dia memimpin jaringan telepon pertama di atas Internet, VoIP Merdeka, yang kemudian hari dikenal sebagai VoIP Rakyat berbasis SIP dan menggunakan kode area +62520 dan +62521. Dia saat ini aktif memperkenalkan e-Learning, dan menjalankan server e-Learning di <http://lms.onnocenter.or.id/moodle/> 66,000++ siswa / mahasiswa dan <https://opencourse.itts.ac.id> 39.000+ mahasiswa secara gratis. Email: onno@indo.net.id